

机器人原理

— 芯片、大模型及云计算 (1.0)

作者 乔大大 Qisheng

说明

本文旨在通过对机器人可能涉及的芯片、大模型及云计算等技术及原理的评叙，来概括通用机器人发展的要素及方向！

作者水平所限，文理可能存在偏误，欢迎读者指正。

注，本文多用网络图例，以便概念说明。若及版权或不妥，请联系移去！

信箱， ThinkingLearning@163.com

2026 年 1 月

目录

目录	1
总体概要篇	1
一、 工程意义	1
二、 主旨概要	1
三、 分析概要	2
四、 硬件逆向	3
1. 机器人关节执行器	3
2. 机器人电路板	3
3. 系统芯片	4
4. 时序组合电路	5
5. 门电路	5
6. 晶体管	6
7. 载流子	6
五、 软件逆向	6
1. 云计算	6
2. 云设施	7
3. 网络通讯	7
4. 大模型	7
5. 机器人系统-ROS2	7
6. 操作系统-Linux	7
7. 板级支持包	8
六、 内容摘要	8
芯片篇	9
一、 机器人芯片选型	9
二、 半导体电路原理概要	10
三、 模拟电路设计原理概要	10
四、 数字电路设计原理概要	11
第一章 半导体物理	12
一、 能带理论与载流子	12
二、 半导体类型	14
1. N 型半导体,	14
2. P 型半导体	15
3. PN 结	15
三、 晶体二极管	22
四、 双极型三极管(BJT)	23
1. 放大作用	25
2. 工作点	27

	3. 偏置电路.....	28
五、	场效应三极管(FET).....	29
	1. 结型场效应管(JFET)	29
	2. 金属氧化场效应管(MOSFET).....	31
	3. 互补金属氧化场效应管(CMOS)	36
六、	晶体管电路.....	38
	1. 供电电压.....	38
	2. TTL 电路	39
	3. CMOS 电路.....	39
第二章	模拟集成电路.....	41
一、	晶体管放大电路.....	41
	1. 单级 BJT 放大电路	42
	2. 单级 MOSFET 放大电路	43
	3. CMOS 单级放大电路.....	45
	4. 多级放大电路.....	46
	5. 频率响应.....	47
	6. 负反馈.....	48
二、	功率放大电路.....	49
	1. A 类	49
	2. B 类	50
	3. AB 类	51
	4. C 类	51
	5. D 类	52
三、	运算放大电路.....	53
	1. 偏置电路.....	56
	2. 输入级电路.....	56
	3. 中间级.....	57
	4. 输出级.....	57
	5. 应用分析.....	57
四、	运算电路.....	64
	1. 比例运算电路.....	64
	2. 和运算电路(Summing Op. Amp.)	66
	3. 差分电路(Difference Op. Amp.)	67
	4. 积分电路(Integrator)	67
	5. 微分电路(Differentiator)	68
五、	比较器电路.....	69
	1. 电压比较器.....	69
	2. 过零检测器(Zero Crossing Detector).....	70
	3. 滞回比较器(施密特触发器)	71
	4. 双限比较器(窗口比较器)	71
六、	滤波电路.....	72
	1. 滤波分类.....	72
	2. 主动滤波器.....	73
	3. 滤波器构型.....	75

	4.	传输函数.....	76
	5.	音频应用.....	81
七、		电源电路.....	81
	1.	非稳压电路.....	82
	2.	直流稳压电源.....	82
	3.	可变稳压电源.....	83
	4.	开关稳压电源.....	83
八、		时钟电路.....	84
	1.	PLL(Phase Locked Loops).....	85
	2.	DLL(Delay Locked Loops).....	85
九、		ADC/DAC	86
	1.	ADC	86
	2.	DAC.....	88
第三章		数字集成电路.....	90
一、		组合电路.....	90
	1.	非门(NOT, 反相器).....	90
	2.	与非门(NAND)	91
	3.	或非门(NOR).....	93
	4.	传输门(Transmission Gate).....	94
	5.	三态门.....	95
	6.	多路选择器.....	95
	7.	计数器.....	96
	8.	全加器.....	96
	9.	全减法器.....	97
	10.	乘法器.....	98
	11.	除法器.....	98
	12.	数值比较器(Magnitude Comparator)	99
	13.	集电极开电路(OC, Open Collector).....	100
二、		时序电路.....	100
	1.	锁存器.....	100
	2.	触发器.....	101
	3.	寄存器.....	104
	4.	可编程逻辑阵列(PLA)	104
	5.	算术逻辑运算单元 (ALU)	105
	6.	时钟同步电路.....	107
第四章		系统电路.....	108
一、		计算系统.....	108
	1.	MCU	108
	2.	GPU	109
	3.	NPU	111
	4.	FPGA.....	112
	5.	SoC.....	113
二、		图像信号处理(ISP)	114
三、		存储系统.....	116

1.	DRAM	116
2.	SRAM	117
3.	Flash	118
四、	通信系统.....	120
1.	射频.....	120
2.	以太网.....	121
五、	总线系统.....	122
1.	总线类型.....	122
2.	串行总线.....	124
3.	并行总线.....	125
4.	PCIe	125
第五章	芯片设计.....	128
一、	前端设计.....	128
二、	后端设计.....	129
三、	智能设计.....	130
1.	Alpha Zero	130
2.	图神经网络.....	131
3.	AlphaChip	131
第六章	量子计算.....	133
一、	量子定义.....	133
1.	量子比特定义.....	134
2.	量子叠加态(Superposition State).....	136
3.	量子运算框架.....	137
二、	量子电路.....	137
1.	单量子比特门(Single Qubit Gate)	137
2.	多量子比特门(Multiple Qubit Gate)	138
三、	量子算法.....	140
四、	量子计算机.....	140
印刷电路板篇.....		142
第一章	印刷电路板.....	142
一、	原理设计.....	142
二、	布局布线.....	143
三、	生产制造.....	143
第二章	信号完整性.....	145
一、	物理基础.....	145
1.	信号模型.....	145
2.	阻抗模型.....	145
3.	电磁模型.....	147
4.	传输线.....	149
5.	EMI/EMC	150
二、	问题类型.....	151
1.	反射(Reflections)	151
2.	串扰(Crosstalk).....	152
3.	衰减/损耗(Attenuation/Losses).....	153

	4. 抖动(Jitter)	153
	5. 开关噪声	154
三、	测量分析	154
	1. 眼图分析	154
	2. 误码率测试	154
	3. 散射参数	154
	4. 时域反射	156
四、	解决方法	156
	1. 阻抗匹配	156
	2. 差分共模	156
	3. 优化回路	157
	4. 优化堆层(Layer Stackup)	157
	5. 防止电磁干扰	157
第三章	电源完整性(Power Integrity)	158
一、	问题类型	158
	1. 电压掉落	158
	2. 电源噪声	158
	3. 共振	158
	4. 地弹及轨道塌陷	159
二、	测量分析	160
	1. 网络阻抗分析	160
	2. 电压噪声测量	160
	3. 电热损失	160
三、	解决方法	160
	1. 去耦电容(Decoupling Capacitors)	161
	2. 正确电源面设计	161
	3. 低电感布局	161
第四章	智能设计	162
一、	Allegro X AI 框架	162
二、	端到端设计	163
关节执行器篇		164
第一章	线性执行器(Linear Actuator)	164
一、	行星滚珠丝杠	164
第二章	旋转执行器(Rotatory Actuator)	165
第三章	特斯拉执行器	166
一、	线性执行器	166
二、	旋转执行器	166
第四章	机器人灵巧手	168
一、	肌腱驱动(Tendon Driven)	168
二、	电缆驱动(Cable Driven)	168
三、	特斯拉机器人手	168
操作系统篇		170
一、	系统实质	170
1.	系统实质	170

	2. 基本逻辑.....	170
	3. 运行分层.....	171
	4. 中断异常.....	171
二、	ARM 架构.....	172
三、	操作系统框架.....	173
	1. 分层框架.....	173
	2. 列式框架.....	175
	3. 环形框架图.....	176
第一章	板级支持系统.....	177
一、	Bootloader-UBoot	177
	1. Start_armboot()	179
	2. main_loop().....	180
	3. netloop().....	181
	4. eth_rx().....	181
	5. nand	181
二、	系统内核.....	181
三、	设备驱动.....	182
四、	测试调试.....	182
五、	性能优化.....	182
第二章	操作系统-Linux.....	183
一、	内核启动.....	183
二、	中断/异常.....	183
三、	进程管理.....	186
四、	内存管理.....	187
五、	文件系统.....	188
六、	设备驱动.....	193
七、	系统调用.....	193
八、	电源管理.....	194
九、	虚拟化.....	194
	1. ARM 虚拟化.....	196
	2. 虚拟地址二级转译框架.....	196
	3. 内存映射 I/O 仿真.....	198
	4. 系统内存管理单元(System MMU 或 IOMMU).....	199
	5. 陷落仿真指令.....	199
	6. 虚拟中断.....	200
	7. 虚拟化通用定时器(Generic Timers).....	201
第三章	机器人系统-ROS2	203
模型篇		206
一、	模型要素.....	206
二、	大模型.....	207
三、	机器人模型.....	208
第一章	理解交互.....	211
一、	语音识别.....	211
	1. 数据处理.....	211

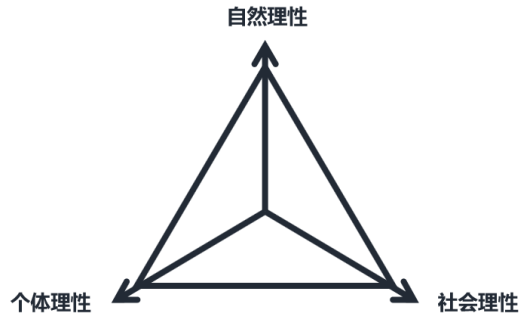
	2. 模型分析.....	214
	3. 多任务.....	217
二、	语音合成.....	218
	1. 多级管路框架.....	218
	2. 端到端框架.....	221
三、	语音翻译.....	222
四、	自然语言.....	223
	1. Transformer.....	224
	2. 模型优调(Fine-Tuning)	226
	3. 多模态.....	227
第二章	感知导航.....	229
一、	目标识别.....	229
二、	行为识别.....	230
三、	三维重建.....	231
四、	SLAM	238
第三章	规划决策.....	243
一、	路径规划.....	245
二、	运动规划.....	246
第四章	运动控制.....	249
一、	机械臂.....	250
	1. 机械臂类型.....	250
	2. 运动学.....	251
	3. 动力学.....	253
	4. 路径规划.....	256
	5. 轨迹规划.....	256
二、	传统机器人.....	257
	1. 线性倒立摆模型(LIPM, Linear Inverted Pendulum Model)	257
	2. 整体模型(Whole Body Model)	257
	3. 运动控制.....	258
三、	Nvidia GROOT.....	258
	1. 总体框架.....	259
	2. 未来隐表征对齐(FLARE)	259
	3. DreamGen	264
四、	Nvidia Allegro Hand.....	267
	1. 任务定义.....	268
	2. 环境定义.....	269
	3. 策略学习.....	269
	4. 奖励定义.....	271
	5. 仿真.....	272
五、	超视觉模态模型.....	272
	1. 硬件环境.....	273
	2. 训练环境及数据.....	273
	3. 机器学习框架.....	274
	4. 触觉传感器(DIGIT)	274

	5.	模型框架.....	274
	6.	损失函数.....	275
第五章		世界模型.....	277
第六章		机器人仿真.....	278
一、		仿真器.....	278
二、		MuJoCo	287
第七章		测试台(Benchmark)	293
一、		模型评估.....	293
二、		测试台.....	294
网络通讯篇.....			295
第一章		卫星网络.....	296
一、		卫星轨道.....	296
二、		网络架构.....	297
三、		卫星框架.....	299
四、		星间通信.....	299
五、		星地通信.....	299
六、		星链终端.....	300
	1.	天线阵列.....	300
	2.	主要芯片.....	301
	3.	固件.....	302
第二章		TCP/IP 网络	303
一、		物理层网络.....	303
二、		网络层网络.....	303
三、		传输层网络.....	304
四、		应用层网络.....	304
第三章		软件定义网络(SDN)	305
一、		VLAN.....	305
二、		VxLAN	306
云设施篇.....			307
一、		半导体芯片.....	308
二、		量子芯片.....	308
三、		虚拟机.....	308
四、		容器虚拟化.....	309
第一章		超算中心.....	310
一、		Tesla Dojo	310
二、		xAI Colossus.....	311
第二章		虚拟机.....	314
一、		Hypervisor	314
二、		KVM	315
三、		XEN	316
第三章		容器虚拟化.....	317
一、		LXC.....	317
二、		Dockers.....	318
	1.	Docker 架构	318

2.	镜像层(Image Layers)	319
3.	命令调用	321
第四章	虚拟机云设施	322
第五章	容器云设施	324
一、	Kubernetes	324
二、	Nomad	325
三、	Amazon ECS(Elastic Container Service)	326
云计算篇		328
第一章	Dify	329
第二章	RAG	330
第三章	分布训练	332
一、	Accelerate	332
二、	DeepSpeed	332
三、	FSDP	333
第四章	数据存储	334
一、	Parquet (拼木地板)存储	334
二、	Hadoop	336
综合篇		337
参考文献		339

总体概要篇

一、 工程意义



社会存在的意义在发展！这不只是哲学的隐义，更是避免社会陷于存量内乱、阶层固化剥削、规则朽化专制的要求，以及实现个体期冀的前提。

发展的方法在认识自然理性、社会理性、个体理性！发展的实质在引领科学发现、工业革命、走向星际文明；发展的基石在自主、自由、正义、保障、民主、理性，防止极权专制、官僚专制、阶层专制；发展的基本在促进个体理性与强大，实现大时空尺度健康。

商贸、殖民、科技、战争是人类历史发展中呈现的方式。可以看到，科技不只是商贸等优势
的支撑，更是实现社会增量发展的直接原因。

因此，科学发现、工程创新应是普遍、持续了解、分析、研究、实践的。

二、 主旨概要

今天，是星际时代、无限清洁能源时代及星际通讯计算时代的开端，同时也是人工智能、通用智能机器人时代的发轫。这些时代正在相互促进发展，形成前所未有的人类文明。

本文主要围绕通用智能机器人，分析相关的芯片、电路板、执行器、操作系统、大模型及云计算等要素的逻辑及共性，以尽量认识人工智能模型训练、推理以及机器人系统集成、规模化运行的原理，促进人工智能及通用智能机器人的科学研究、工程创新及商业化。

注，知识的问题，在概念多，逻辑少；逻辑多，共性少；共性多，意义少。

三、 分析概要

机器人模仿人组成，大致可分为能量系统、感知系统、思维系统、控制系统。相应的实现可分为电池电源充放电管理、视听触觉重力加速度传感器、训练推理大模型及关节电机执行器等。对应的基础设施包括集成电路、印刷电路板、操作系统、云计算等。

集成电路、印刷电路板、操作系统、大模型、云计算的设计实现流程一定程度上是相似的，一定层面来说都是软件。

- 1、需求。基于需求规制(Specification)进行总体架构设计；
- 2、设计。通过编程语言进行逻辑设计实现、验证；
- 3、适配。结合实际电路、元件或机器进行物理设计实现、验证；
- 4、生产。生产或部署。

当然，细节与可试性不同。比方，数字芯片设计时用 VHDL/Verilog 语言等进行前端逻辑设计，IC Compiler 等进行后端物理布局等适配设计，生成的图数据文件 GDSII 用于生产流片 (Tape-Out)；产品原型印刷电路板用 OrCAD 等设计原理图(Schematic)，PADS 等布局布线适配，生成光绘文件(Gerber)等制板贴片；Linux 操作系统用 C/ASM 语言等开发，实现板级支持包(BSP, Board Support Package)适配硬件平台，制作镜像以烧录；大模型基于 PyTorch/TensorFlow 等框架、用 Python 等语言开发，结合 GPU 等硬件进行优化，制作模型包以部署；云计算平台用 Java 语言等实现，虚拟机、容器等适配服务器平台，生成包文件及镜像以部署。

甚至，药物发现等同样可类此。比方，用 AlphaFold 设计，生化合成，制药。

从工程层面看，芯片、电路板、操作系统、大模型、云平台等的设计方法可概括为基于积木及俄罗斯套娃的整合开发。

从产品层面看，基于科学研究、技术整合的工作可称作工程产品设计，不同于基于交互的用户产品设计。工程产品设计是实现社会增量创新的本源，用户产品设计是存量的优化。

从计算层面看，今天计算的核心设施依然是基于半导体物理的大规模集成电路，同时也在融合着量子计算。

四、 硬件逆向

1. 机器人关节执行器

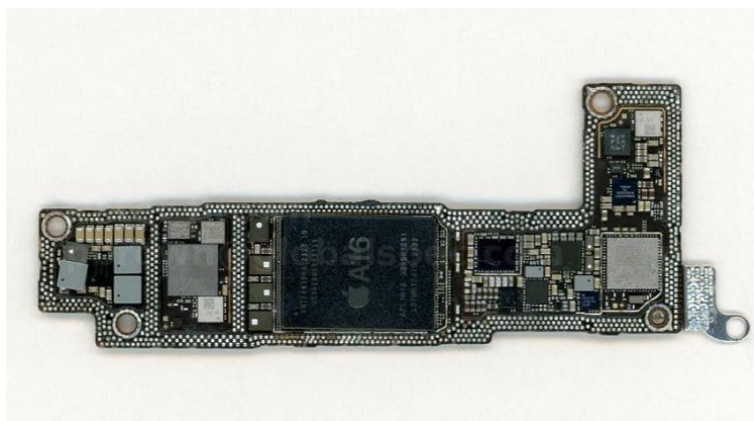
机器人外观主要的是材质及关节。关节的运动主要在于线性及旋转执行器。机器人手部综合不同执行器及肌腱线缆(Tendon Cables)等实现。

2. 机器人电路板

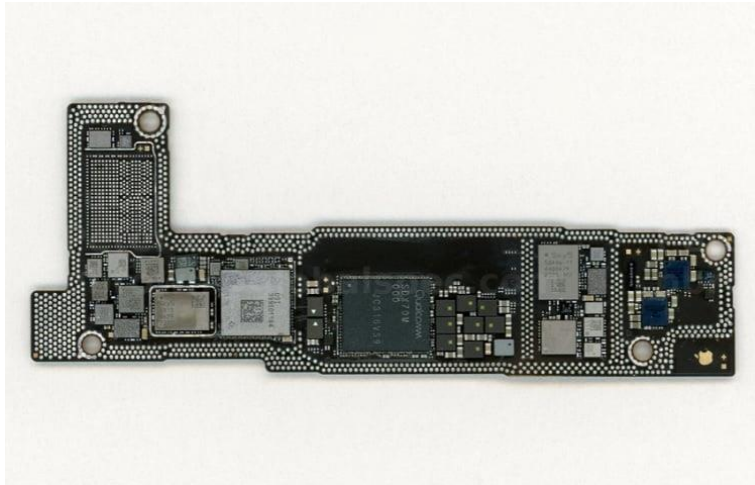
机器人电路可大致看作基于智能手机与智能汽车电路的融合改进。

智能手机电路通常分为应用主板(AP, Main Board)与射频板(RF Board)。大概总计数十个集成电路(IC)。

应用主板(Iphon15)，主要包括应用芯片等。

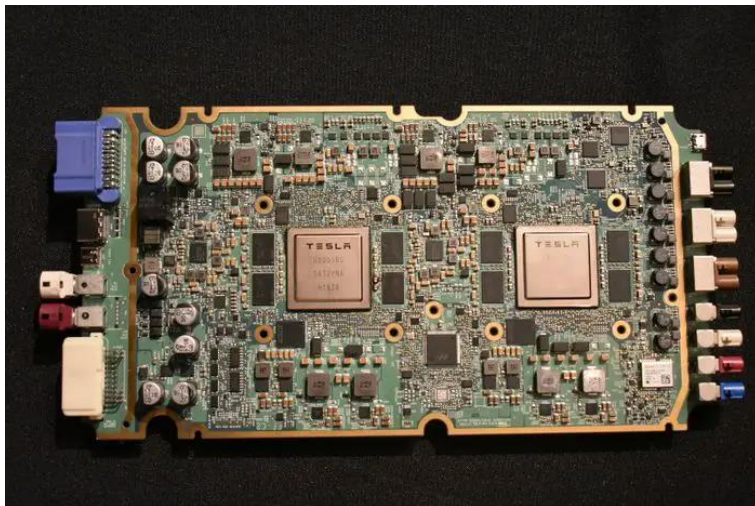


射频板(Iphon15)，主要包括蓝牙、WIFI、蜂窝通信系统等。



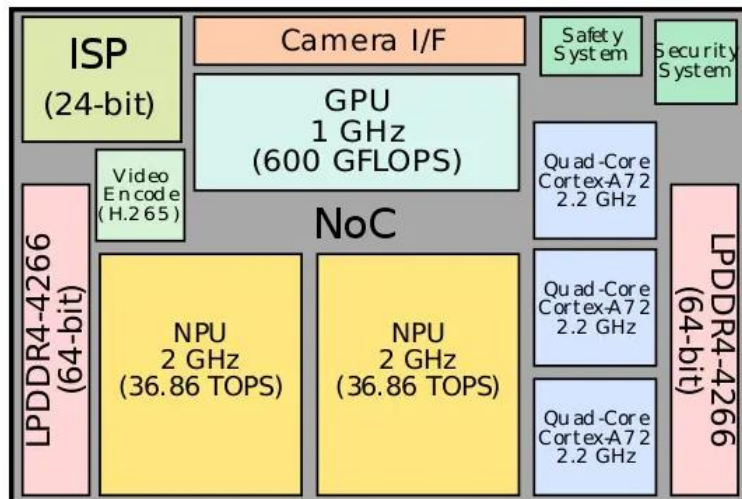
智能汽车电路通常分为智能驾驶、信息娱乐、通信、车身控制器、三电系统等十多个电路板。

智能驾驶板(Tesla HW3.0 Board)。



3. 系统芯片

智能驾驶芯片(Tesla HW3.0)。



芯片电路可分为模拟集成电路，处理连续信号，比方，放大器(Amplifiers)、滤波器、传感器、射频系统、电源管理系统等；数字集成电路，处理离散信号，比方，微处理器、内存、DSP等；混合集成电路，比方，模数/数模转换器(ADC/DAC)、锁相环(PLL)等。

4. 时序组合电路

数字集成电路作为数字逻辑系统，微观的看，内部可依功能分为基于多个时序逻辑电路(Sequential Logic Circuits)的数据通路(Data Path)及控制逻辑(Control Logic)两部分。这与网络通讯的控制、数据分离相似。

时序逻辑电路基于组合逻辑电路(Combinational Logic Circuits)及锁存器(Latch)、触发器(Flip-Flop)等定义的存储电路形成。

5. 门电路

更微观的看，这些逻辑电路可通过与(AND)、或(OR)、非(NOT)、非与(NAND)、非或(NOR)、异或(XOR)、异非或(XNOR)等布尔逻辑的门电路实现。

基于不同的门电路或元件(Components)数量，数字集成电路可分为小、中、大、超大、特大、极大规模集成电路。

6. 晶体管

再微观的看，这些门电路可基于双极型晶体管(Bi-Polar Transistor)或场效应晶体管(Field Effective Transistor)不同的电路结构(Circuit Structure)定义。

双极型晶体管分为 NPN 及 PNP 类型，通过电子(Electrons)及空穴(Holes)两类载流子(Carriers)导电。

场效应管分为结型(Junction FET)及金属氧化物型(MOSFET)等。MOS 场效应管只用一类载流子导电，用电子导电的称 NMOS 电路，用空穴导电的称 PMOS 电路；基于 NMOS 及 PMOS 电路的电路称互补金属氧化物半导体电路(CMOS Circuits)。

基于这些电路结构，数字集成电路可分为晶体管-晶体管逻辑(TTL)或双极型电路(Bipolar Circuits)及金属氧化物半导体(MOS)。

7. 载流子

基于能带理论了解载流子，电子、空穴。

五、 软件逆向

1. 云计算

云计算是当前人工智能时代的核心计算形式。比方，模型的训练、业务的部署等。

2. 云设施

超级计算机及云基础设施是支撑云计算的主要形式。超级计算机基于更高速连接及密集分布式实现。

3. 网络通讯

网络通讯是不同设备互联的方式。无线，特别是大规模卫星星座通讯，不只在改变着基于有线的通讯、计算格局，更可能在定义未来星际通信的方式。

4. 大模型

基于异构基函数，利用海量数据及奖励算法训练拟合人类思维函数，具备超大规模神经网络及参数的模型是今天人工智能、通用机器人的计算核心。

5. 机器人系统-ROS2

机器人操作系统是支撑机器人不同模块通信交互、控制管理的基本平台。

6. 操作系统-Linux

操作系统是对资源进行隔离，支持不同任务、不同环境并行运行的支撑。虚拟化技术的发展使得传统的操作系统更适用及促进着云计算的发展。

7. 板级支持包

板级支持包是操作系统与硬件资源进行连接的适配器。

六、 内容摘要

机器人工程云-端-半导体的金字塔。

一、	云计算-业务、模型训练	通用智能训练、大规模商用支撑
二、	云设施-超算中心	
三、	网络通讯	
四、	大模型	
五、	机器人操作系统-ROS2	
六、	操作系统-Linux	连通软硬件、应用调度运转的纽带
七、	板级支持包	
八、	芯片工作原理-ARM SoC、NPU、GPU、MEM、Flash	
九、	机器人关节执行器	
十、	信号完整性、功率完整性、EMI/EMC、时序、标准兼容、热管理	集成电路、板级电路产品化的前提
十一、	印刷电路板设计	芯片等电气元件集成装配的平台
十二、	芯片系统设计	
十三、	芯片系统电路	
十四、	组合电路、存储电路、时序电路	
十五、	门电路	数字集成电路的基本元素
十六、	PN 结、场效应管	
十七、	半导体、能带、载流子	信息工业、智能计算的基石

芯片篇

芯片是当前可重用火箭、星座通讯、人工智能、可控核聚变、药物发现等行业发展的基本要素。在越来越集成各模块向着片上系统(SoC)发展的同时，促进着机器学习及云计算虚拟化的软硬融合。

一、 机器人芯片选型

Tesla optimus, 2 块 FSD HW4.0(NoC,集成约 20 个 CPU 核,1 个 GPU,3 个 npu, 2 个 64bit LPDDR4 , 1 个 ISP , 1 个 H265 Video Encoder , 1 个 Safety system , 1 个 Security System)

优必选, 升级版 Walker X 主要是 Intel i7-8665U/（双路，1.9GHz）+NVIDIA GT1030 显卡，384 核心。

宇树科技, 四足 H1/H1-2 机器人, RK3588 芯片/Intel Core i5/i7 + 1-3 块 Nvidia Jetson Orin NX (100TOPS) ; G1 EDU 机器人, AI 部分用 Nvidia Jetson Orin 。

傅里叶, i7 13700H , 内置英特尔高斯神经加速器(GNA , Gaussian & Neural Accelerator), 核显, Iris Xe Graphics (96EU)。

众擎, Intel N97 + NVIDIA Jetson Orin 模组(通过 PCIe 4.0 接口与 N97 直连)。

整体来说，国产 CPU、GPU 相对落后，难点不只在芯片逻辑设计的功能、时序，物理设计的适配、制程、量子效应、热约束、热耗散、晶体管规模、异构计算等，同时也在第三方软硬件接口适配、应用健壮便利性、生态规模惯性、成本效率商业盈利等方面。

国产 SoC，瑞芯微、全志等。国产 GPU、AI ASIC，华为昇腾、寒武纪、海光、沐曦、摩尔线程、壁仞、燧原等。

当前，人工智能物理基石 GPU，早期只是图像编辑、渲染等作用。2006 年，Nvidia 发布基于 GPU 的 CUDA 中间件平台以支持通用计算(GPGPU)，比方，机器学习、深度学习、气象仿真、基因分析(Genome Analysis)、金融建模、物理仿真等。

二、 半导体电路原理概要

半导体电路原理，基于物理底层到应用上层，可分成六层。

第一层，发现形成用于导电的载流子(Carriers)。基于半导体原子核对电子的弱于绝缘体、高于导体的非稳态、弱约束力、不同类原子之间共价键对电子的非稳态、吐纳机制，掺杂不同价原子到半导体原子群中，形成自由电子(Electron)或原子内部的空穴(Hole)作导电载流子的N型半导体、P型半导体。

第二层，发现P型、N型半导体之间的结(Joint)效应。P型、N型半导体放到一起，不同载流子在电场作用下会进行漂移(Drift)、基于浓度不同进行扩散(Diffusion)，产生电流，同时载流子在内部电场作用下又会达成动态平衡。当在半导体结的外部施加正向偏置(Forward Bias)或反向偏置(Reverse Bias)时，又会使载流子流动。

第三层，基于PN结效应，发明不同构型的三极管，比方NPN、PNP、N衬、P衬等。实质就是怎样来控制在三极管中、串联的、非稳态导电特性的N型或P型半导体，以便使三极管在电路中起着导通或隔离作用等。基于三极管中间极连接导线，推拉空穴、自由电子载流子进行三极管导电，称双极型(Bipolar)晶体管。基于三极管中间极施加电场，推拉单类载流子进行导电，称场效应管。

第四层，基于不同三极管开关作用，对电流、电压的放大效应等，结合三极管共极特性、外部电源电路、时钟、并串、级联等，形成门、锁存器等逻辑计算、记忆，以及放大器等模拟运算、放大的半导体电路元件。

第五层，基于不同的半导体电路元件，设计模拟或数字集成电路。模拟电路，单级放大器、滤波器，运算放大器、比较器，锁相环(PLL, Phase Locked Loop)、ADC/DAC等。数字电路，门电路，锁存器、触发器、寄存器，组合电路、时序电路等。

第六层，不同的模拟、数字电路形成不同的系统。不同的系统形成不同的应用。模拟电路设计是数字电路设计的基础。一般数字电路在中央、模拟电路在外围。

三、 模拟电路设计原理概要

1、前端设计。规范定义(Specification Definition)；基于Virtuoso等EDA工具进行原理图设计(Schematic Entry)及原理图电路仿真。

2、后端设计。基于 Calibre 等 EDA 工具进行电路布局, 设计规则检验(DRC, Design Rule Check)、布局与原理检验(LVS, Layout vs. Schematic Check)、寄生参数计算(PEX, Parasitic Extraction), 布局仿真。

3、生成 GDSII 文件进行流片、封装、测试。

四、 数字电路设计原理概要

1、前端设计。规范定义; 基于 VHDL/Verilog 等语言进行数字电路逻辑设计; 利用 Modelsim 等 EDA 工具进行仿真功能验证测试。可在模块分区(Partition)后联合多块 FPGA 进行原型验证。

2、后端设计。基于 DC Compiler 等编译工具综合生成相关门表; 物理设计布局; 再进行时序、功耗、信号完整性及 DRC、LVS 等物理验证; 利用 Modelsim 等 EDA 工具进行仿真测试。

3、生成 GDSII 文件进行流片、封装、测试。

4、进行板极支持包调试以适配不同的操作系统或应用。

第一章 半导体物理

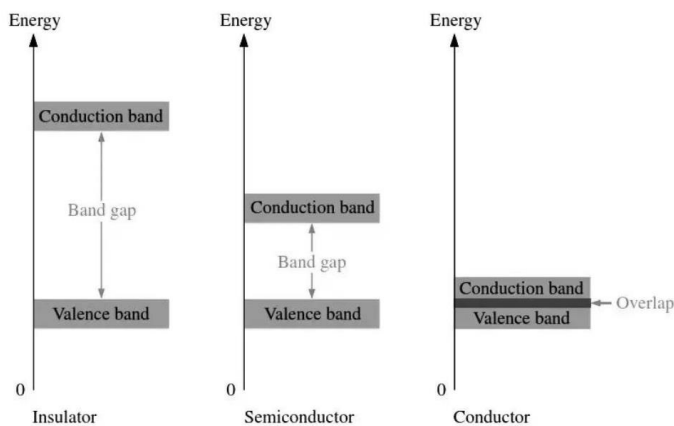
本篇半导体物理，基本对应半导体电路层级的第一到第三层。主要基于量子物理分析 N 型、P 型等不同类型半导体原子核层面、不同价原子核层面对电子的约束、结合特性；不同类型半导体成对结合或成三结合的构型；以及不同构型的不同极，在不连接导线，或连接导线、施加不同方向电压的载流子运动特性。

一、能带理论与载流子

绝缘体中无电子流动，体现绝缘性；导体基于电子流动产生电流。半导体导电性介于绝缘体与导体之间，可通过掺杂形成导电的载流子(电子、空穴)。

注，空穴不是真实的粒子。空穴的迁移实质是共价键之间的电子的迁移，不是自由电子的迁移。

基于能带理论分析半导体导电特性。



电子运动特性与电子能量紧密相关。

基于卢瑟福模型，电子绕着原子核旋转。电子的旋转可看作在离心力作用下的运动。离心半径越大，电子能量越高。

基于能带理论的泡利不相容原理。固体中运动电子的能量谱值通过一系列准连续的具有一定

宽度的能带组成。从原子核向核外方向，依次是价带(Valence Band)，禁带(Band Gap)和导带(Conductive Band)。

禁带，相邻的能带之间不能被电子占据的区域被称为禁带。禁带越宽，相邻能带之间发生越界的可能性越小。

导带，电子部分(Partially)占据，且其中的电子填充情况容易因电场作用改变的能带。导带中一般只有少量的电子，大多数能级空着。因此在外加作用下易发生能级的改变，故导带中电子导电，成为载流子。金属的导带一般都比较宽，因此表现出比较好的导电性。

价带，在绝对零度时，电子全部填满，是能级最高(Highest Energy Level)的满带。当电子从导带跃迁回价带空穴时辐射能量,可见光便是辐射能量的体现。

绝缘体的过渡禁带很宽，电场或光子很难改变其能带填充情况，同时导带是空着的，因此不能产生电流或光活性。

半导体的过渡禁带较窄，可借助杂质引入或光/热激发使少数电子跃迁到空着的导带。或者，当价带中出现少数空穴时，电子可从导带迁回价带，或者共价键中电子移动，使得空穴表现出在各原子价带间的运动，以获得一定的导电性和光热活性。

半导体原理相关概念。

电子势能，类比于重力势能、弹簧势能，电子相对于原子核，基于电荷作用力，因此具有势能，且距离原子核越远，势能越高。

本征激发，价带中电子吸收外部热能或光能等跃迁上来到导带中，形成自由电子，同时在价带中产生同样数量的空穴。

注意，自由电子与空穴成对发生。本征激发基于本身特征激发，相异于掺杂。

本征半导体，两类载流子浓度相等的半导体，可称本征半导体。

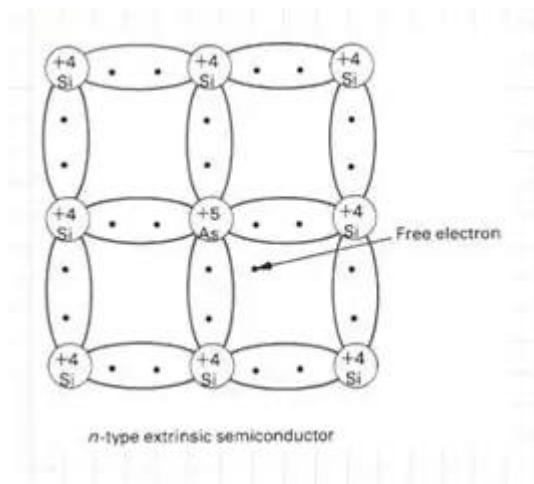
本征浓度，在一定温度下，本征半导体同时存在载流子产生与复合，二者达到动态平衡，使半导体载流子浓度保持一定值。温度变化，则达到新动态平衡。平衡状态下，本征半导体单位体积内自由电子或空穴数称本征浓度。

本征半导体本征浓度远小于原子密度，因此导电能力很弱，不适合制作半导体器件的材料。

二、 半导体类型

1. N 型半导体，

N 型，半导体主要基于电子导电。



N 型半导体中，掺杂是产生自由电子的主要方法。

半导体中掺杂一个 5 价原子，则不同原子基于共价带、形成共价键(Covalent Bond)后，多一个价电子。

基于原子核束缚力远小于共价键的特性，因此，多的价电子只要接受点能量，则很容易变成自由电子，使 5 价原子电离、形成+1 正离子。掺杂 5 价原子越多，自由电子越多。

同时，本征激发可产生自由电子及空穴。

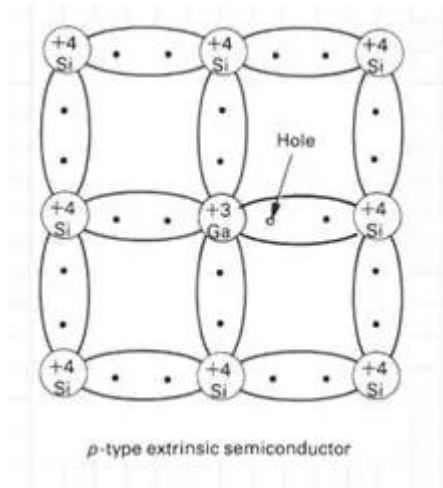
N 型半导体中，自由电子是多数载流子(多子)，空穴是少数载流子(少子)。

空穴数量少于本征激发的数量，因为空穴易与多的自由电子复合。

N 型半导体，本身和掺杂的都是原子，自由电子数等于空穴数加正离子数，因此是电中性。

2. P 型半导体

P 型，半导体主要基于空穴导电。



P 型半导体中，掺杂是产生空穴的主要方法。

半导体掺杂的价 3 原子缺少一个电子用于共价键，就会导致其他价 4 原子的共价电子来填补这个空穴。掺杂原子多了一个电子，因此成为负离子。总的来说，掺杂价 3 原子越多，空穴越多。

同时，本征激发可产生自由电子及空穴。

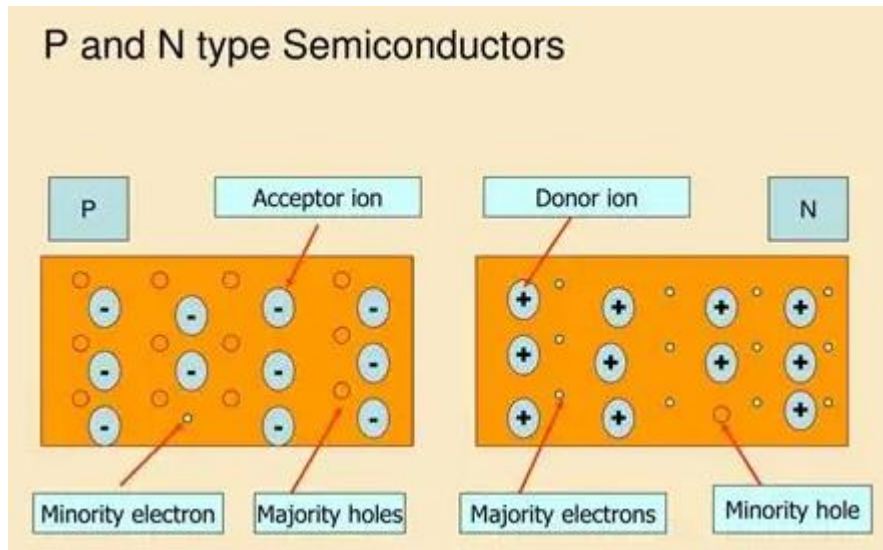
P 型半导体中，空穴是多数载流子，自由电子是少数载流子。自由电子数量少于本征激发数量，因为自由电子会与空穴复合掉一些。

P 型半导体，空穴数量等于自由电子数+负离子数，因此呈电中性。

3. PN 结

PN 结(Junction)，单晶体中 P 型半导体、N 型半导体材料之间的对接(Interface)。

杂质半导体(Extrinsic Semiconductor)电荷模型。



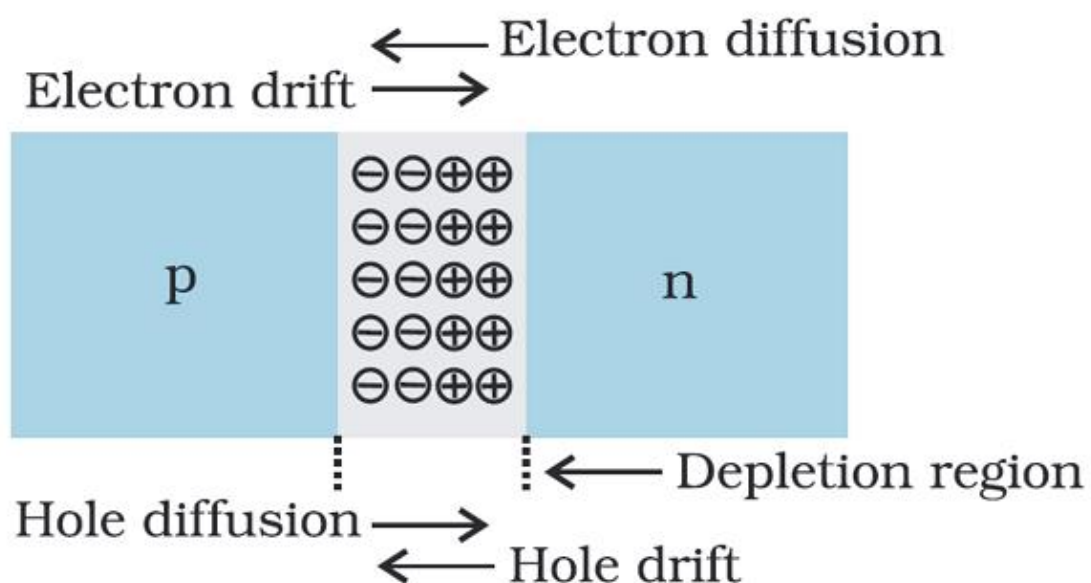
PN 结基于扩散与漂移使载流子定向运动形式，产生电流。

扩散运动，载流子浓度不同，导致浓度高的地区向低的地区扩散。扩散电流与浓度梯度相关。

在 PN 结中，P 型、N 型半导体多子相互扩散，在中部对接位置结合(Combination)、耗尽(Depleted)后，不动(Immobile)的离子(Donor + , Acceptor -)形成内电场。

漂移运动，在电场作用下形成载流子运动。漂移电流与电场强度，载流子浓度相关。

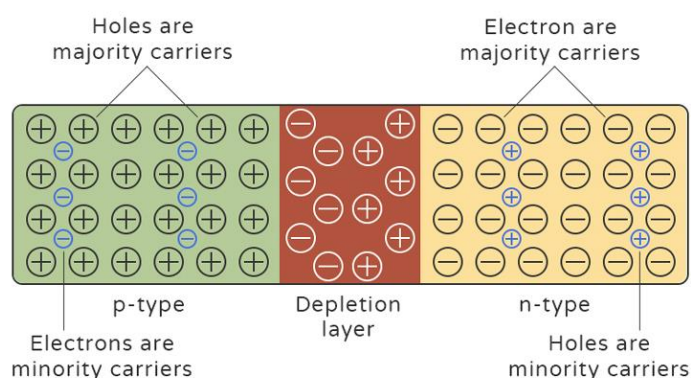
在 PN 结中，在内电场作用下，P 型、N 型半导体少子向中部集结。



PN 结扩散、漂移框架图，N 极多子是自由电子，浓度高，向 P 极扩散；P 极多子是空穴，浓度高，向 N 极扩散。N 极少子是空穴，在耗尽区(Depletion Region)电场作用下，向 P 极漂移，在中部集结；P 极少子是自由电子，在耗尽区电场作用下，向 N 极漂移，在中部集结。

PN 结耗尽区电场形成的同时，阻止 P 极、N 极多子相互扩散，最后形成平衡。

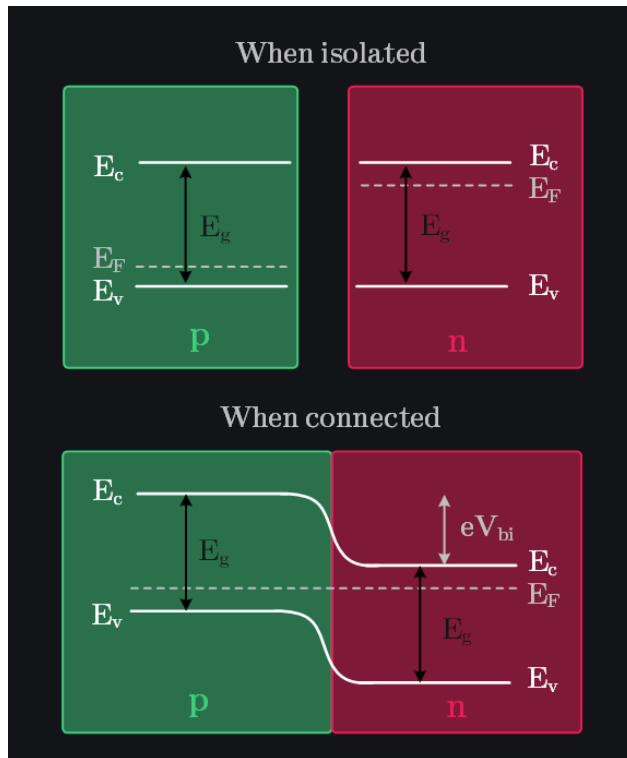
平衡状态下的 PN 结。



PN 结的平衡态框架图，空穴、电子结合或成为约束电荷(Bound Charges)，电离化的原子(Ionized Atom)形成中部(褐)空间电荷区(Space Charge Region)或耗尽层(Depletion Layer)、阻挡区(Barrier Region)、势垒区(Potential Barrier)内电场。

在多子扩散过程中，形成的内电场阻止扩散，使少子漂移，形成漂移电流；扩散电流与漂移电流反向，最后两者达到动态平衡，无电流，PN 结宽度固定。

基于能带理论分析 PN 结耗尽区载流子能带变化。



PN 结载流子能带关系变化框架图，当 P 型半导体与 N 型半导体连接，N 型半导体电子向 P 型半导体扩散，扩散力对电子做功，提高电子势能，P 型半导体电子导带向上弯曲(Conduction Band Upward Bending)或平移，禁带(Bandgap)宽度不变，因此，价带(Valence Band)同样向上弯曲或平移；扩散形成的内电场，创立势垒区(Potential Barrier)，阻止 N 型半导体电子向 P 型半导体扩散，因此，N 型半导体电子导带与价带向下弯曲(Downward Bending)或平移。

当 PN 结扩散平衡(Equilibrium)时，基于 P 型、N 型半导体电子费米能级对齐(Fermi Level Align)，P 型半导体价带能级 E_v 接近费米能级 E_f ，N 型半导体导带能级 E_c 接近费米能级 E_f ，可对应 P 型半导体具备高空穴浓度(High Hole Concentration)，N 型半导体具备高电子浓度(High Electron Concentration)。

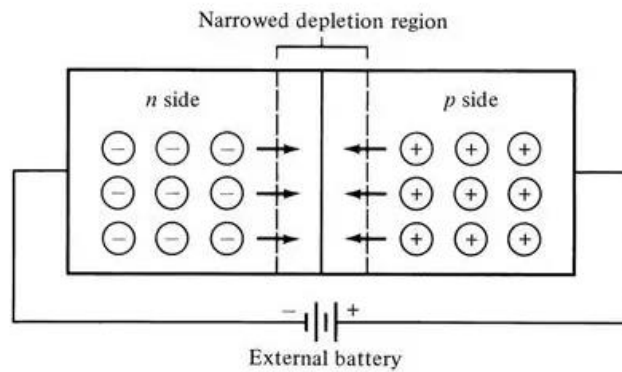
注，费米能级(Fermi Level)，绝对零度时，电子可以占据的最高能级。费米能量(Fermi Energy)，绝对零度时，费米能级对应的能量。

费米能级，在温度、掺杂时，费米能级可变化，可视作电子占据的最高能级与未占据最低能级的均值。在半导体中，未掺杂时，居于价带、导带之间的禁带中部；掺杂时，向价带或导带偏移。

费米能级，是电子占据能级的分水岭，费米级以下，能级几乎完全填满电子；费米级以上，能级几乎空。

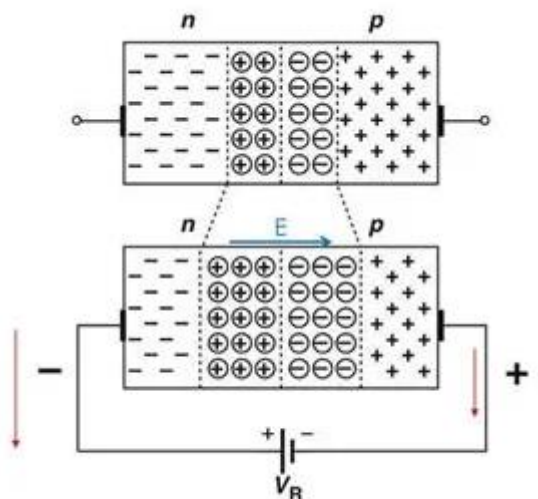
同固体(Solid)，费米能级可变化，费米能量不变。

1) PN 结正向偏置



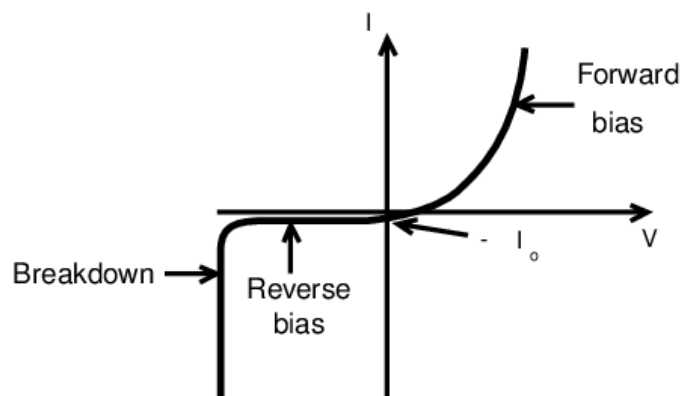
电源正极施加于 PN 结 P 极，使耗尽层宽度变窄，通过结区的电流变大。

2) PN 结反向偏置



电源正极施加于 PN 结 N 极，使耗尽层宽度变宽，阻止结区电流或使通过结区的电流变小。
当电源电压大到一定程度，击穿 PN 结的阻通效应，电流瞬时变大。

3) PN 结伏安特性



PN 结存在不同反向击穿类型。

齐纳击穿(Zener Breakdown), 基于高电场、量子隧穿(Quantum Tunneling)原理。PN 结窄, 载流子浓度高, 偏置电压使电场提高时, 使得共价键中价电子激发, 产生大量自由电子。击穿电压小于 5V, 电压不变化, 适用于调压(Voltage Regulation)等。

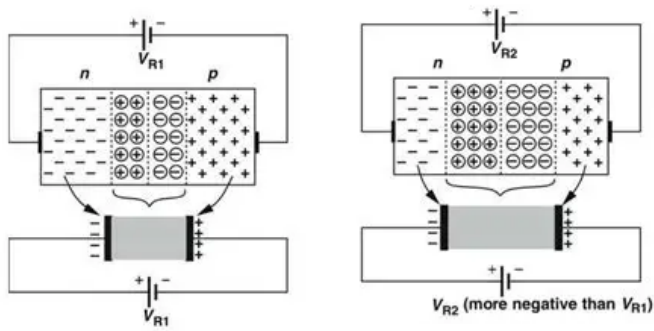
雪崩击穿(Avalanche Breakdown), 基于电子碰撞(Collision)。PN 结宽, 掺杂浓度低, 因此自由电子需要较强动能撞离共价键价电子、产生较大动能自由电子, 基于链式反应(Chain Reaction), 自由电子可撞离更多价电子, 因此漂移电流很快变大。击穿电压大于 5.6V, 适用于过压保护(Overvoltage Protection)。

在限制电流的情况下, 两者都是电击穿, 属于可逆击穿。电流不限制, 电流过大则功率过大, 会产生过热, 导致击穿, 此时击穿不可逆, PN 结不能恢复。

4) PN 结电容

结电容

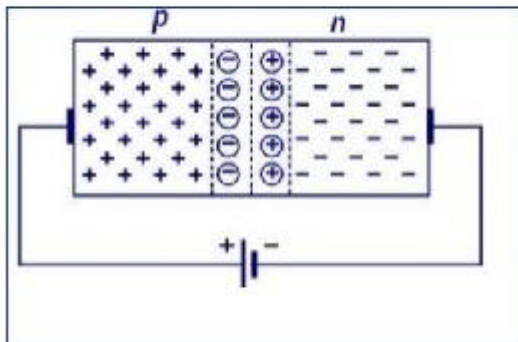
基于耗尽层或过渡区(Transition Region)的偶极子(Dipole)或离子对(Pair Of Positive And Negative Ions), 工作在反向偏置电压。又称势垒电容。



电荷存储电容

基于电荷存储效应，电压滞后于电流，工作在正向偏置电压。又称扩散电容。

当加载在 PN 结上的电压变化，则存储在 PN 结中的电荷变化。



存储电荷多了相当于电容充电，反之相当于放电。

PN 结总电容等于扩散电容+势垒电容 (单位 pf)。

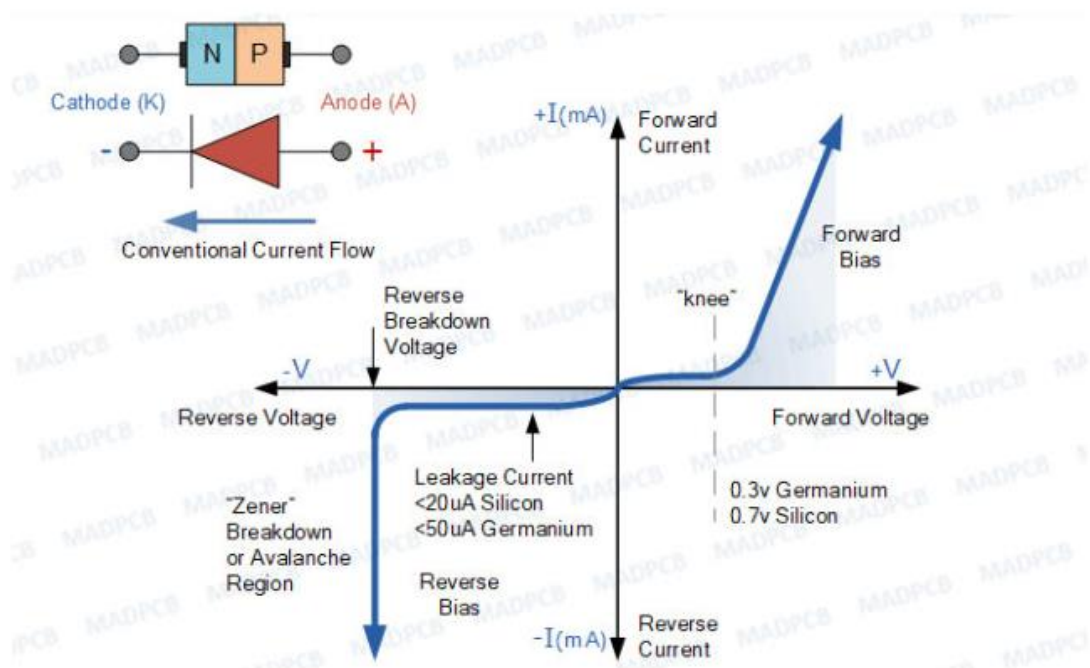
PN 结正偏时，扩散电容为主；反偏时，几乎无扩散，因此主要是势垒电容。

低频时，这些电容很小，可忽略。高频时，影响 PN 结单向导电性。

三、 晶体二极管

晶体二极管，基于 PN 结特性、具备阳极(Anode)及阴极(Cathode)电极、可限制电流单向导通的电子元器件。比方，齐纳二极管(Zener Diode)、肖特基二极管(Schottky Diode)等。

电压、电流特性。



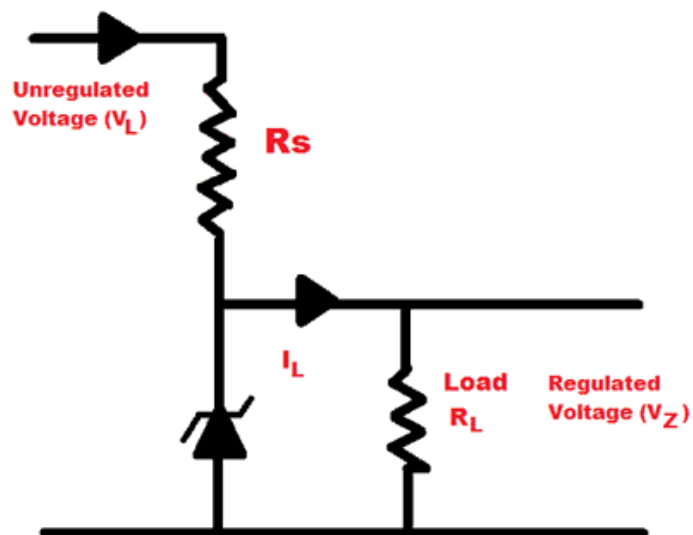
拐点(Knee)电压，正向电压未到该电压时，正向电流很小。因为要抵消内建电场。

基于二极管可创立不同模型。

开关模型，正偏短路，电压等于零；反偏开路，电流等于零。

恒压源模型，用理想二极管串联电压源。

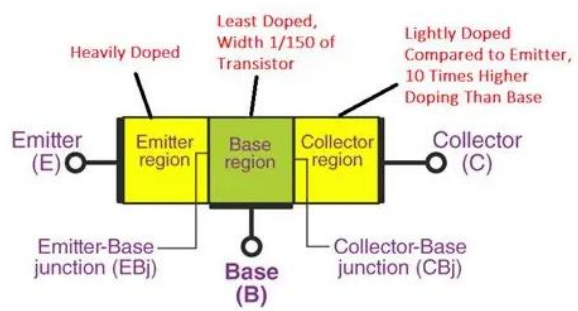
稳压器(Voltage Regulator)，二极管工作在击穿区。



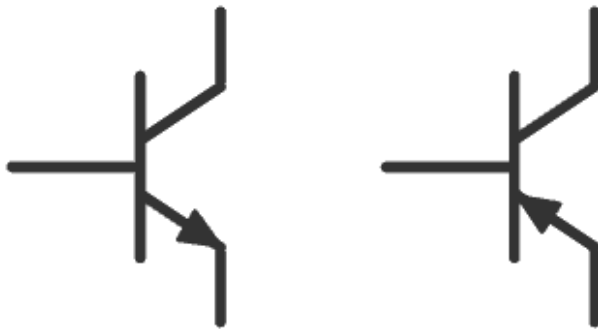
四、 双极型三极管(BJT)

双极型晶体管本质是基于相邻 PN 结不同的正向、反向偏置组合进行电流阻止、导通、放大控制的元件。

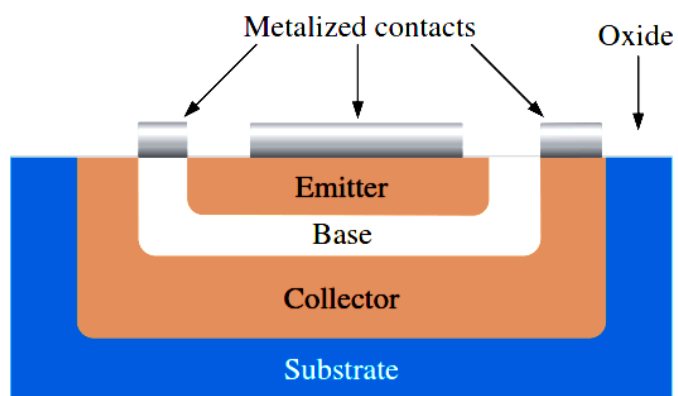
示例图。



标号。



构型图。



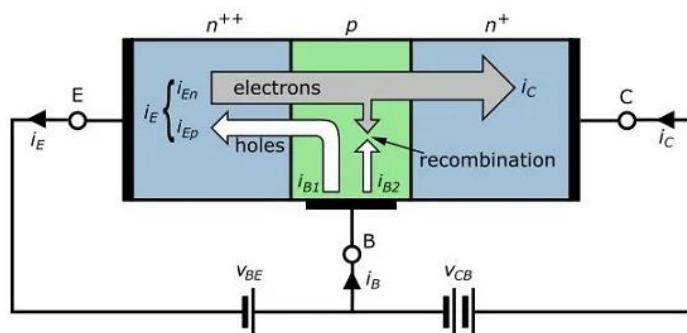
双极型三极管有三个端，射极(Emitter)、基极(Base)、收极(Collector)。基于结构分 NPN、PNP 类型。小电流输入基极，使得收极到射极有大电流。给基极电流相当于给基极开了一个导电通道，NPN 的 P 部分的空穴或 PNP 的 N 部分的电子沿着该导电通道流走，使得射极、收极导通电流。

截止区(Cut-Off Region)，射极-基极、收极-基极都是逆向偏置(reverse biased)，BJT 断开。

激活区(Active Region)，晶体管作电流放大器，射极-基极正向偏置(forward biased)，收极-基极逆向偏置(reverse biased)，射极、收极之间电流正比(Proportional)于基极电流。

饱和区(Saturation Region)，射极-基极、收极-基极都是正向偏置(forward biased)，相当于闭合电路，电流任意从收极流到射极。

双极型三极管载流子运动示例(NPN BJT, BE 正偏、BC 反偏)。



1. 放大作用

放大指 I_B 基于 PN 结伏安特性，利用 I_E 大电流放大到 I_C 。电流放大，可使电压放大。

可基于发射极电压 V_{BE} 的正向控制作用及 V_{CE} 的反向控制作用控制各极电流变化。

正向控制作用。

I_E 电流在 V_{BE} 控制下的变化情况类似于二极管，基于 I_E 与 I_C 、 I_B 是线性关系，因此 V_{BE} 可控制电流。 V_{BE} 达到一定程度时， I_E 显著增大。

反向控制作用。

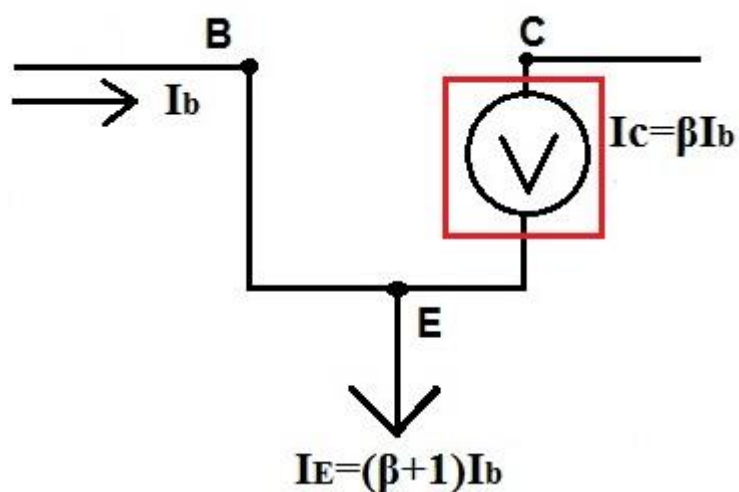
集电极变宽，使基区更窄，单位宽度内的载流子显著加大，基区载流子浓度梯度增大，扩散电流 I_E 变大，可使 I_C 变大。同时，基区宽度窄，载流子结合(Combination)机会小，电流 I_B 变大，可使 I_C 变大。

电流放大的要素。

基区很窄，使得电子容易运动；同时，发射区电子为主、浓度大，基区空穴为主、浓度小；使集电区电压 V_{CE} 大于基区电压 V_{BE} ，基区电压高于发射区电压，或者说集电极反偏，发射极正偏。

β (Beta(β))，双极型晶体管集电极电流对基极电流的比例，表示晶体管电流增益或放大因子(Amplification Factor)。

$$\beta = \frac{I_C}{I_B} = \frac{I_E}{I_B} - 1,$$

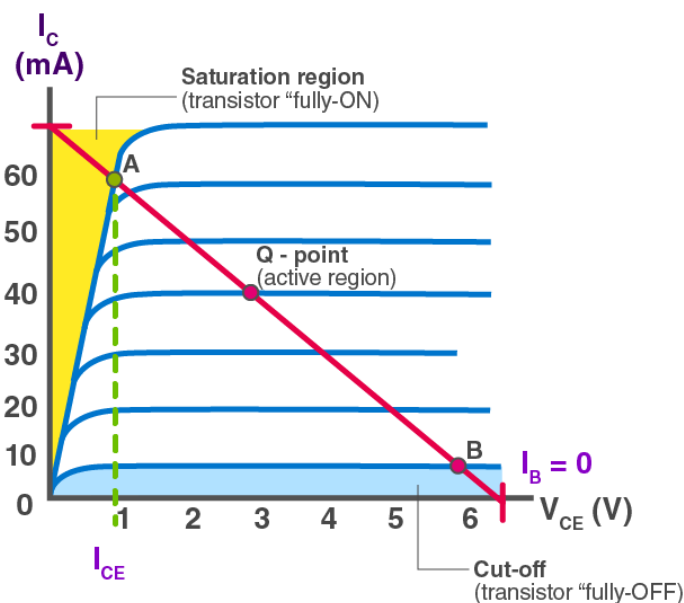


2. 工作点

双极型晶体管工作状态基于不同的基极直流电流或偏置电流可以划分成截止区、饱和区、激活区(Active Region)或线性区。在线性区/饱和区时，晶体管对输入信号具有放大作用。

要确保基极的偏置电流在合并输入的交流信号后，晶体管依然工作在线性区或饱和区，便需确定一个合适的偏置电流值。

确定的方法在于基于直流负载线(DC Load Line)找到静态工作点(Q-Point, Quiescent Operating Point)，使得该点可以同时满足晶体管饱和区、激活区特性。

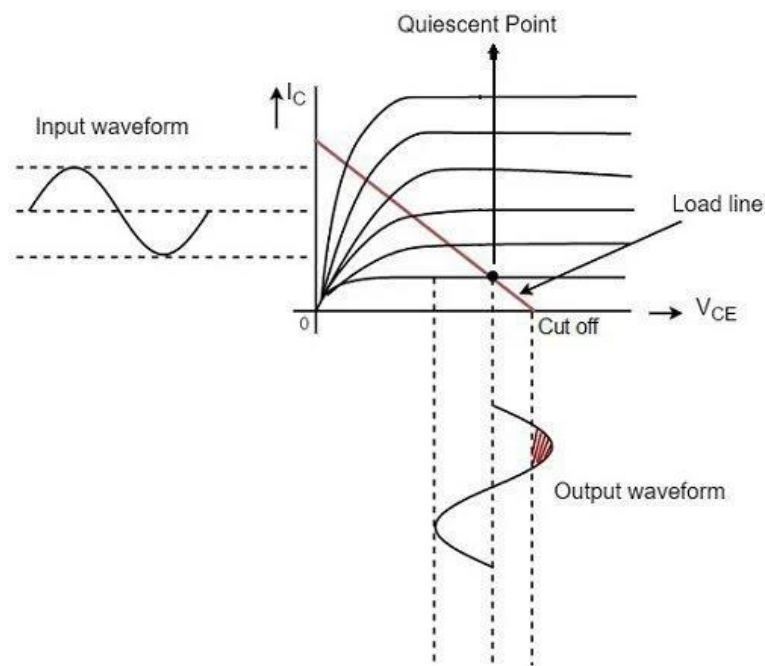
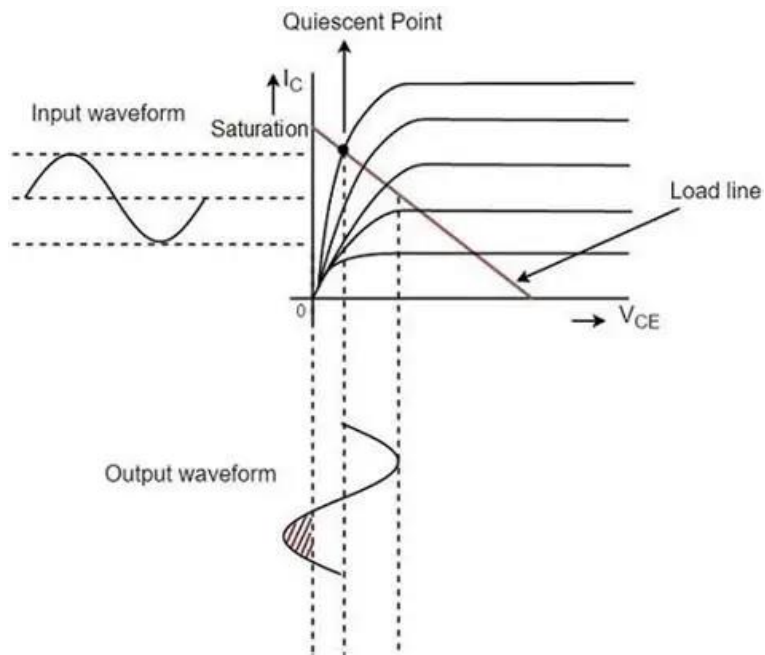


注，直流负载线，双极型晶体管电路在无输入信号、同时直流负载时的集电极-发射极电压、集电极电流关系曲线。该曲线点称工作点(Operating Points)。

注，静态工作点，直流负载线与晶体管特征曲线在饱和区、激活区的交点，

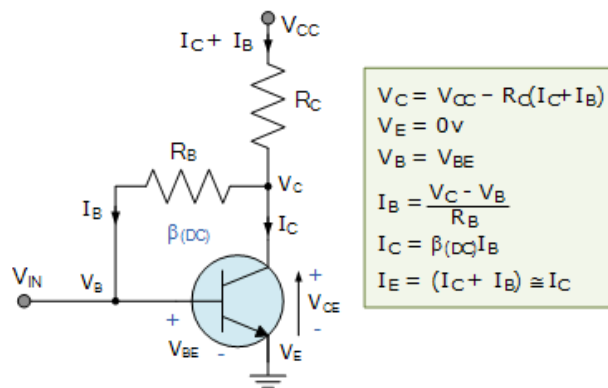
$$Q = \frac{V_{CC} - V_{CE}}{I_C}$$

不合适的静态工作点会使输出失真。



3. 偏置电路

常用的集电极反馈偏置(Collector Feedback Biasing)。

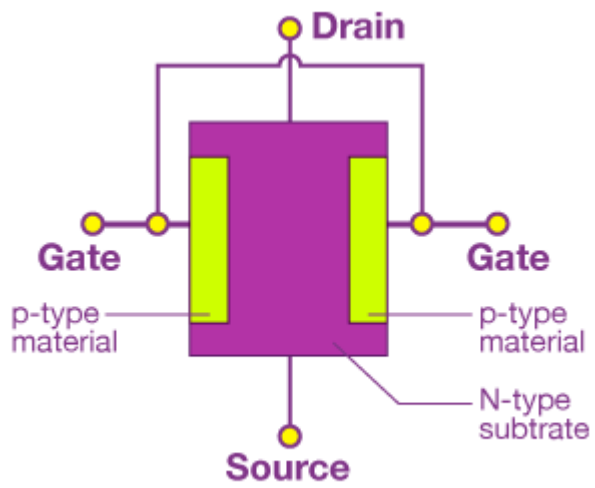


五、 场效应三极管(FET)

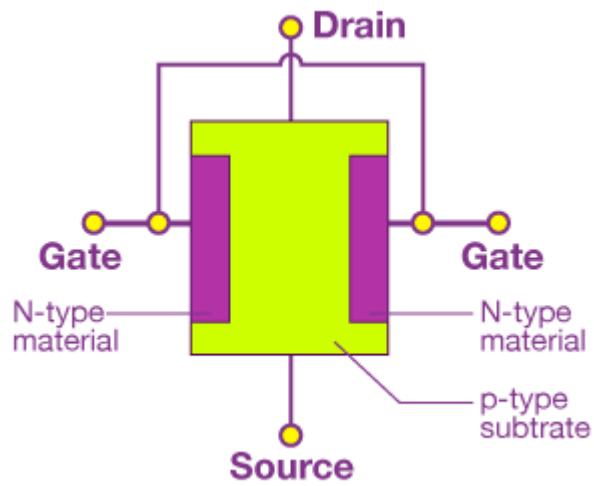
1. 结型场效应管(JFET)

结型场效应管实质基于偏置电压调控相对的 PN 结耗尽区宽窄，以门的形式控制 PN 结之间的电流通道开关。

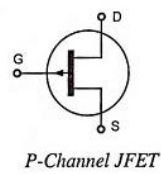
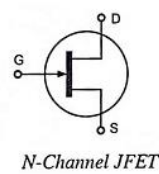
N 通道结型场效应管。



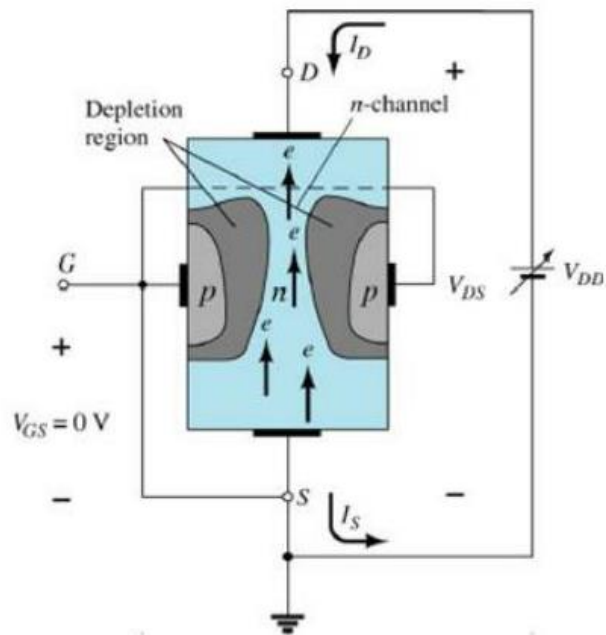
P 通道结型场效应管。



标记。



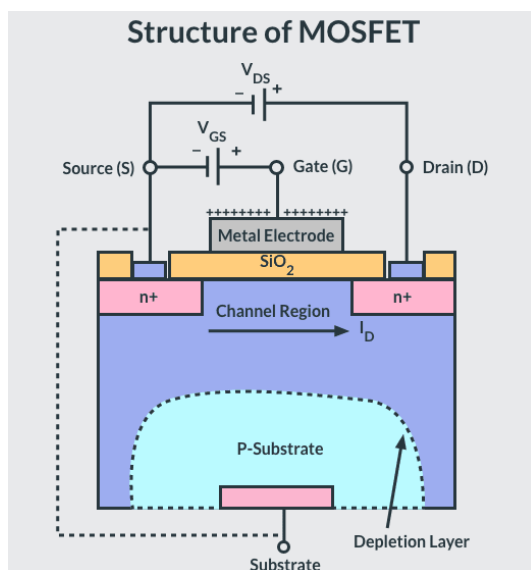
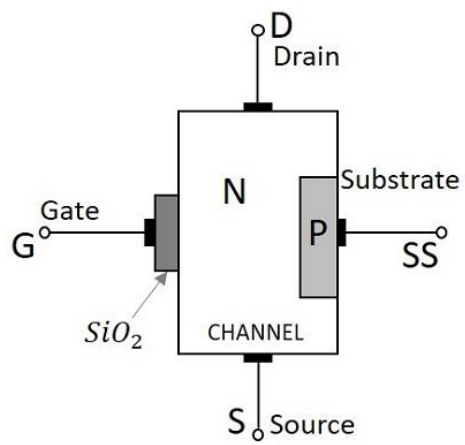
结型场效应管载流子运动示例(N-Channel JFET)。



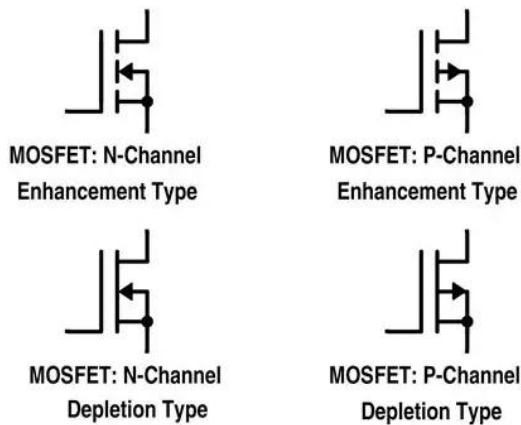
2. 金属氧化场效应管(MOSFET)

MOSFET 场效应管实质，相反于结型场效应管放式，基于电场调控相对的 PN 结之间的载流子浓度，以门的形式连通或关阻 PN 结。电流方向垂直于结型场效应管。

MOSFET 场效应管是电压驱动(Voltage Driven)的三端元件(3-Terminals Device)，包括栅极(Gate)、源极(Source)、漏极(Drain)。当然，也可以再包括衬底(Substrate)作第 4 端。



标记。

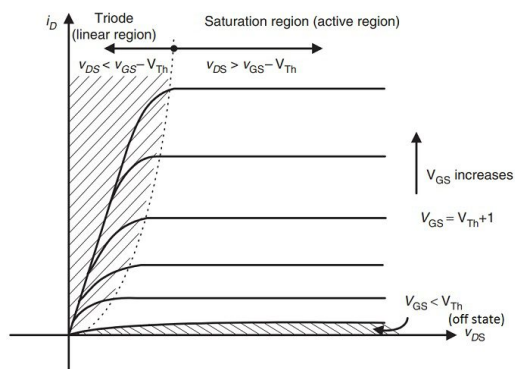


MOSFET 场效应管存在着栅极电压 V_{GS} 产生的电场及源极、漏极电压 V_{DS} 产生的电场，方便起可约看作十字型电场，或者是水平水管中部有个控制阀门，衬底就是需要通过栅极电压开合的阀门。

当栅极电压 V_{GS} 大于阈值电压 V_{TH} (最小的栅极-源极电压)时，垂直方向电场拉取半导体电子或空穴，导电沟道开通导电；当栅极电压小于阈值电压时，电场拉力不足，导电沟道关闭不导电(几乎)。

在导电沟道导电的情况下，基于源极、漏极电压 V_{DS} 的水平电场与垂直电场，栅极、阈值电压差 $V_{GS}-V_{TH}$ 的关系，导电沟道又分为线性区(Linear Region)，又称三极区(Triode Region)，或饱和区(Saturation Region)。 $V_{DS}<V_{GS}-V_{TH}$ ，对应线性区； $V_{DS}>V_{GS}-V_{TH}$ ，对应饱和区。

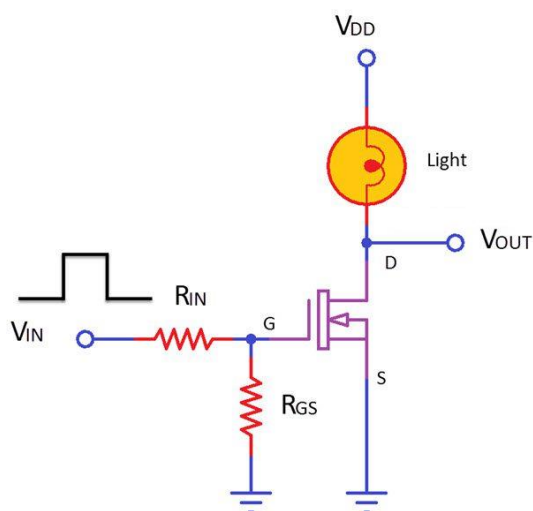
想像源极到漏极之间存在电场歧路。 V_{GS} 形成的电场大于 V_{DS} 形成的电场，则载流子大量流向栅极或衬底，导电沟道导电性很弱，源漏极电压大一些，流向漏极的电流就多一些，相应的电流大一些，因此源漏极电流与源漏极电压成线性关系； V_{GS} 形成的电场小于 V_{DS} 形成的电场，则载流子已大量流向漏极，此时再加大源漏极电压，也没有再变大的空间，电流基本不变。



相应于导电沟道不导电、线性导电、电流饱和，MOSFET 可以有截止区(Cut-Off Region)、线性区(Linear (or Triode) Region)、饱和区(Saturation Region)。

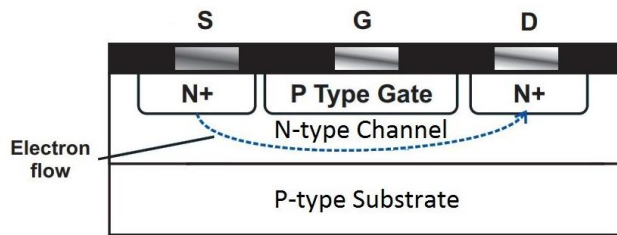
基于 MOSFET 的电压驱动的导电、不导电特性，MOSFET 可作开关。当然，局限于物理世界的约束，MOSFET 不可能瞬时开关，也无法彻底的关电流或无限导通电流。

MOSFET 开关电路示例。



MOSFET 可以分增强型(Enhancement)，开始时通道未形成，相当于常断开开关(NO Switch, Normally Open)；或耗尽型(Depletion)，开始时通道便已形成，相当于常闭合开关 (NC switch, Normally Closed)。

这两类 MOSFET 又分别分 n 通道(n-channel)，源漏极都是 N 型，栅极、衬底是 P 型；或 p 通道(p-channel)，源漏极是 P 型，栅极、衬底是 N 型。基于应用类型，MOSFET 通道结构可以是水平(IC 电路)或垂直的(电源应用)。



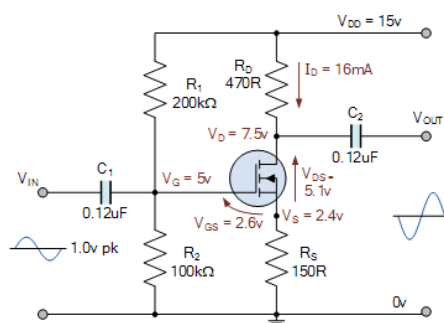
相对于双极型晶体管(Bipolar Junction Transistor)或结型晶体管(Junction Transistors)，MOSFET跨导(transconductance)低，电压增益(voltage gain)不大，因此，一般不合作放大器。不过，不是不可以，比方 N 型增强型 MOSFET。耗尽型 MOSFET 放大器与结型场效应管(JFET)相似。

注，跨导，主动设备的输出电流变化与输入电压变化的比例(Ratio)。可看作是电流对电压的

导数。比方，双极型晶体管，集电极电流变化对基极电压变化的比例， $\frac{\delta I_C}{\delta V_B}$ ；MOSFET，

漏极电流变化对栅极电压变化的比例， $\frac{\delta I_D}{\delta V_G}$ 。跨导越大，增益(Gain)或放大(Amplification)越大。

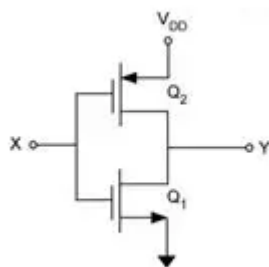
单阶(Single Stage)N 型 MOSFET 放大器。在饱和区，漏极电流(Drain Current)基于栅极-源极电压变换。



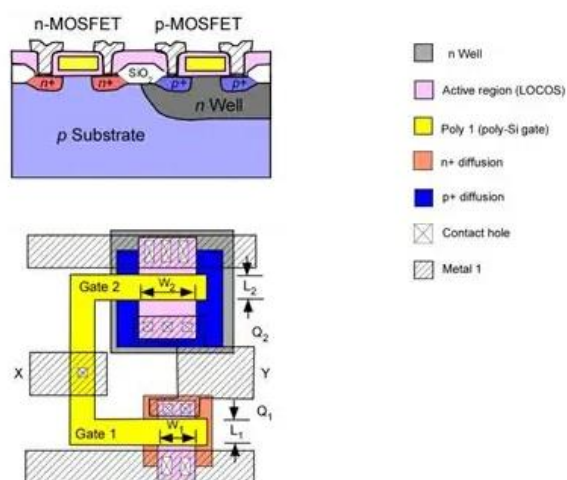
3. 互补金属氧化场效应管(CMOS)

CMOS 实质，基于不同类型 MOSFET 串联实现更准确的开关。

示例图。



构型图。



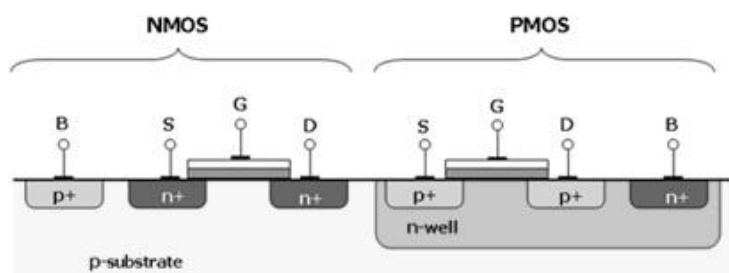
CMOS, 主流半导体技术.

应用场景。

微处理器(Micro Processors)、微控制器芯片(Micro Controller), 内存(RAM), 固存(ROM), EEPROM 及专用集成电路(ASICs)等。

结构。

多晶硅(polysilicon)与金属场效应管(metal FET)都被称作绝缘门场效应晶体管(IGFET, Insulated Gate FET), 因为在 GATE 下方的 SiO₂ 是绝缘体.



CMOS 基于 PMOS 及 NMOS 组成, 功耗小, 几乎无静态功耗, 只是在电路切换的时候存在功耗, 优于 NMOS 及双极型技术.

NMOS 就是 N 沟道(channel, source-drain 都是 N 型, 载荷是电子, 移动快于空穴)、P 衬底 (p-type substrate), source-substrate-drain 形成 NPN 结;

PMOS 就是 P 沟道(channel, source-drain 都是 P 型, 载荷是空穴, 对噪声免疫性好于电子)、N 型衬底(N-type substrate), source-substrate-drain 形成 PNP 结.

GATE 施加高/低电压(高电压吸引电子, 低电压吸引空穴), 形成垂直方向电场, 衬底的电子/空穴漂移到沟道层, 在 source-drain 水平电场的作用下形成电流。

注，衬底中加阱的做法是防止衬偏电压、防止耗尽层变大，导电沟道变窄甚至截止的问题。MOS 中源/漏极，阱和衬底形成的 NPN 或 PNP 晶体管结构可避免此情况。无阱模式，电子溢出等使源漏及衬底间的 NPN 或 PNP 结形成正向反馈的电子流动，使 MOS 功能的开关失效。

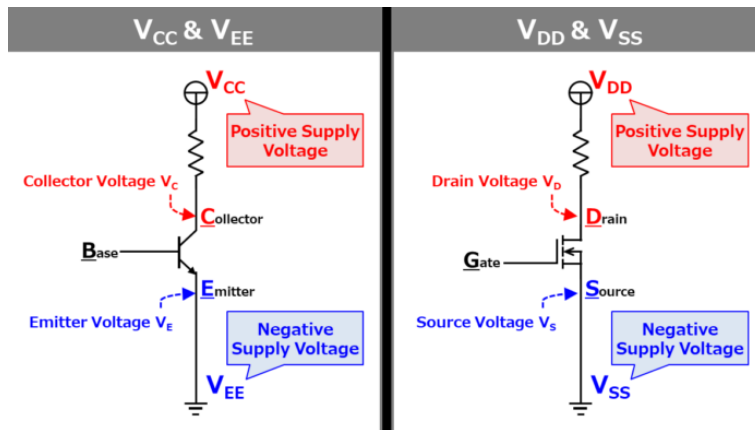
NPN 管比 PNP 管开关(Switching)快，在于 NPN 的多子是电子，PNP 的是空穴，电子的迁移率比空穴大。NMOS 比 PMOS 快，类似原因。

NPN 比 NMOS 快，在于 NPN 是体器件，其载流子的迁移率是半导体内的迁移率；NMOS 是表面器件，反形层在栅氧下的表面形成，其载流子的迁移率是表面迁移率；半导体的体迁移率大于表面迁移率。

六、 晶体管电路

晶体管电路，RTL(Resistor Transistor Logic)、TTL(Transistor-Transistor Logic)、CMOS(Complementary Metal Oxide Semiconductor Logic)电路等。

1. 供电电压



V_{CC} 、 V_{EE} 分别对应双极型晶体管的正供电电压、负供电电压。以区分于集电极、发射极电压 V_C 、 V_E 。

V_{DD} 、 V_{SS} 分别对应场效应管的正供电电压、负供电电压。以区分于栅极、源极电压 V_D 、 V_S 。

一般单电源电路， V_{EE} 、 V_{SS} 等同于地(Ground)。

2. TTL 电路

基于双极型晶体管的数字集成电路。具备较快的电压状态切换速度(Fast Switching Speed)及较高的输出电流驱动能力(High Output Current Drive Capability)。电压区间窄，功耗相对大。输入阻抗低，输出阻抗高。噪声敏感。

3. CMOS 电路

CMOS 电路，场效应集成电路，基于 PMOS、CMOS 的单极型数字集成电路。相对于 TTL 电路，具备宽电压区间及高输入阻抗。

优点，输入阻抗高、功耗低、抗干扰能力强且适合大规模集成。静态功耗几乎为零，输出逻辑电平可为 VDD 或 VSS，上升、下降时间同数量级等。

第二章 模拟集成电路

本篇模拟集成电路，基本对应半导体电路层级的第四、第五层。主要分析怎样利用不同晶体管、元器件设计放大、运算、比较、滤波、电源、时钟及模数转换电路等。

模拟集成电路，基于晶体管、二极管、电阻、电感(Inductors)、电容等处理连续信号。可实现信号放大(Amplification)、滤波(Filtering)、混合(Mixing)、调制(Modulation)、解调(Demodulation)、稳压及稳流器(Voltage and current regulators)等。功耗相对大，同时对噪声、外部温度变化等干扰敏感。主要应用有放大器、滤波器、数模/模数转换器、比较器、锁相环、稳压器等。

放大电路，基于晶体管类型可分为双极型放大器、场效应管放大器；基于工作点(Operating Points)及开关模型(Switching Mode)可分为A、B、AB、C、D、E等类型；基于输出可分为电压放大器(Voltage Amplifier)、电流放大器、功率放大器(Power Amplifiers)、运算放大器(OP, Operational Amplifier)；基于输入输出关系可分为跨导放大器(Transconductance Amplifier)、跨阻放大器(Transresistance Amplifier)；基于频率可分为低频、音频、射频放大器；基于级别可分为单级、多级等。

放大器主要参数，增益、频率像一个、带宽、截止频率(Cutoff Frequencies)、增益带宽积(Gain Bandwidth Product)、反馈、功耗、输入输出阻抗、噪声等。

一、 晶体管放大电路

放大电路，放大信号、控制能量输出的电路。可分为单级放大电路及多级放大电路。

单级双极型放大电路(Single Stage BJT Amplifiers)，基于晶体管类型及输入、输出共极(Terminal)类型，可分为共基极(Common Base)、共发射极(Common Emitter)、共集电极(Common Collector)等。

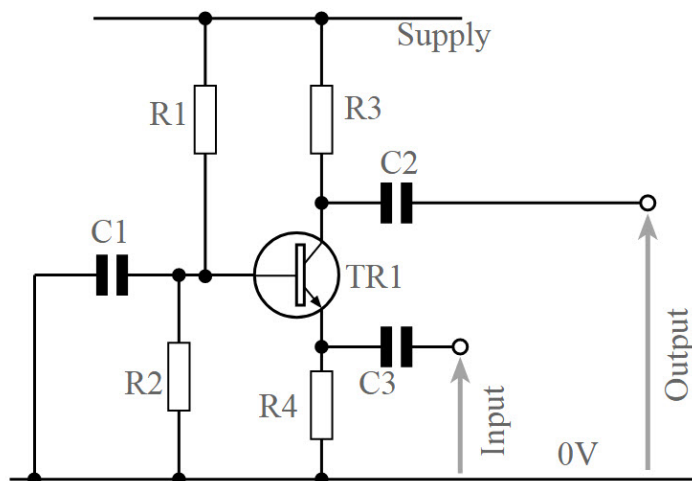
单级场效应管放大电路，相似方法可分为共源极(Common Source)、共漏极(Common Drain)、共栅极(Common Gate)等。

多级方法电路，基于单级放大电路级联。

1. 单级 BJT 放大电路

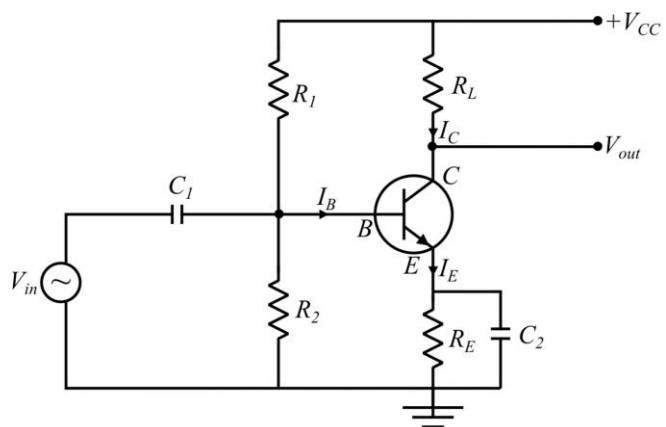
1) 共基极放大电路

基极(Base)相对于输入、输出电路是共同的。



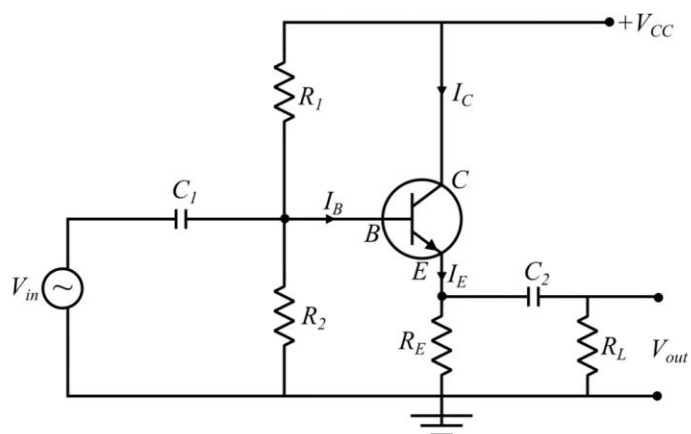
2) 共射极放大电路

发射极(Emitter)相对于输入、输出电路是共同的。



3) 共集极放大电路

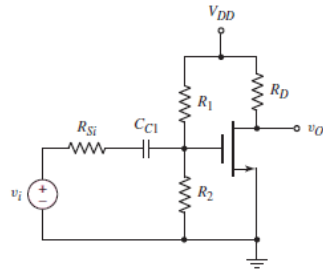
集极(Collector)相对于输入、输出电路是共同的。



2. 单级 MOSFET 放大电路

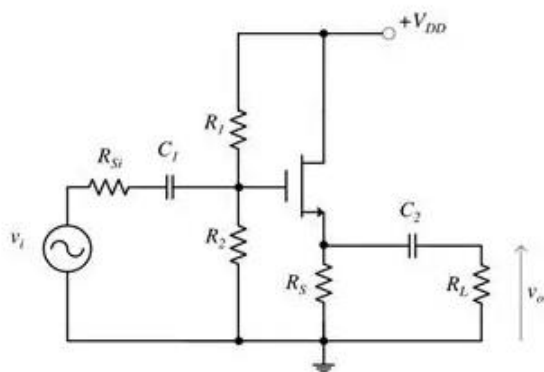
1) 共源极放大电路

栅极作输入，漏极作输出，共源极。



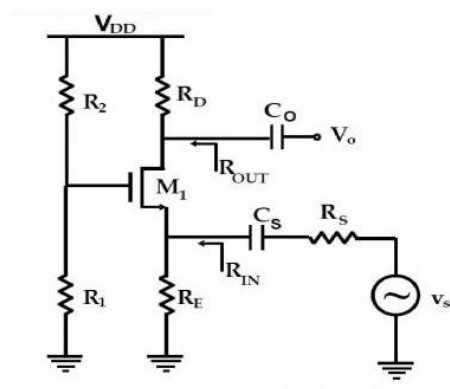
2) 共漏极放大电路

栅极作输入，源极作输出，共漏极。



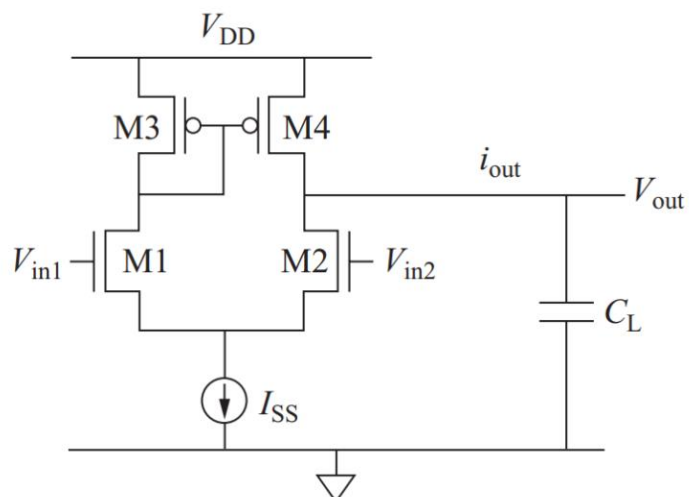
3) 共栅极放大电路

源极作输入，漏极作输出，共栅极。



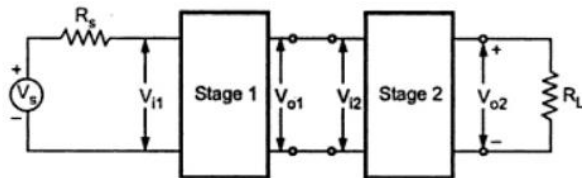
3. CMOS 单级放大电路

基于互补的 P 通道、N 通道场效应管，差分输入，单端输出(Single-Ended Output)。

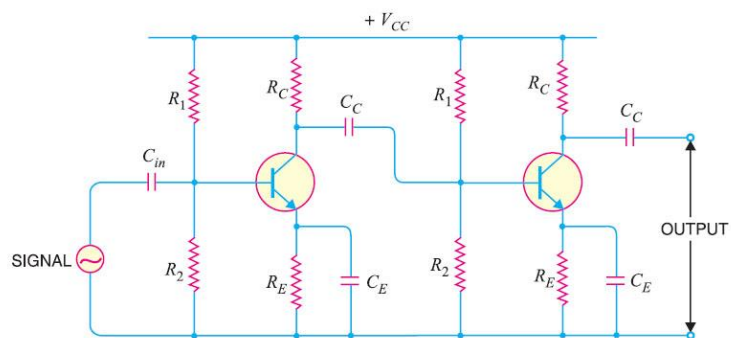


4. 多级放大电路

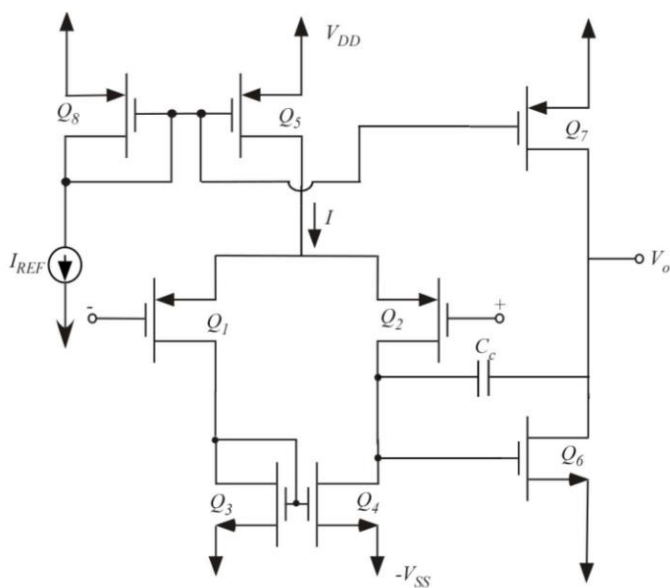
二级放大电路示例图。



二级阻容共射极放大电路(Two Stage RC Coupled CE Amplifier)。



二级 CMOS 运算放大电路



5. 频率响应

放大器频率响应，基于输入信号频率，放大电路的增益(Gain)、相位偏移(Phase Shift)的变化。

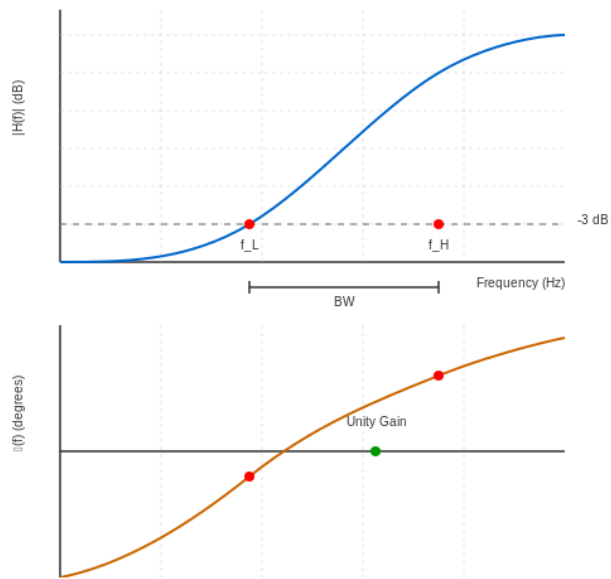
可基于传输函数定义，
$$H(f) = |H(f)|e^{j\phi(f)}$$
。

$|H(f)|$, 振幅响应(Magnitude Response)。

$\phi(f)$, 相位响应(Phase Response)。

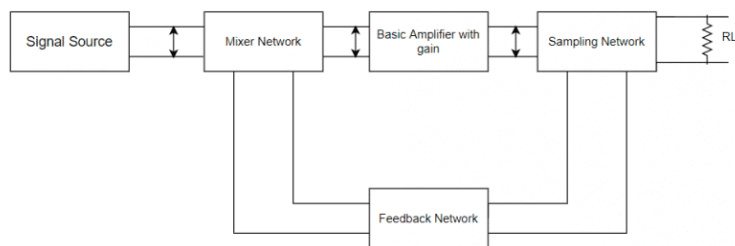
可基于波特图(Bode Plot)表示频率响应。

注，波特图，基于频率对数刻度的振幅图(Unit, Decibels)及相位图(Unit, Degree)。



在音频放大器中，用于在音频区间维持平坦频率响应，以避免变形(Distortion)；在射频放大器中，用于对带宽准确控制以避免相邻信道(Adjacent Channels)相互干扰；在运算放大器中，用于进行稳定性分析(Stability Analysis)以避免单位增益频率(Unit Gain Frequency)邻域相位偏移(Phase Shift)引发振荡(Oscillation)。

6. 负反馈



Signal Source， 电压或电流源。

Mixer Network， 混合网络或比较器(Comparator)。基于加减计算，混合源信号及反馈信号(Feedback Signal)，生成正(Positive)或负(Negative)反馈。

Amplifier， 增益放大器，放大电压或电流信号。

Sampling Network， 电流、电压采样网络。对输出电压进行采样。

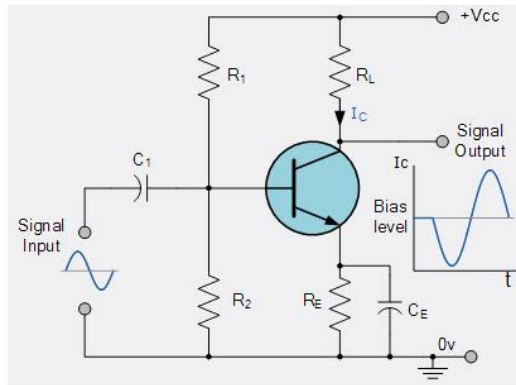
Feedback Network， 基于电阻、电感、电容的双端口反馈网络。反馈部分输出信号到输入。

二、 功率放大电路

功率放大器基于电路构型(Configuration)及运行方式(Operation Method)，分为 A、B、AB、C 类等基于导通角控制(Controlled Conduction Angle)的模拟放大器及基于数字电路、脉冲宽度调制(PWM, Pulse Width Modulation)开关(Switch)输出的 D、E、F 等数字类型放大器。

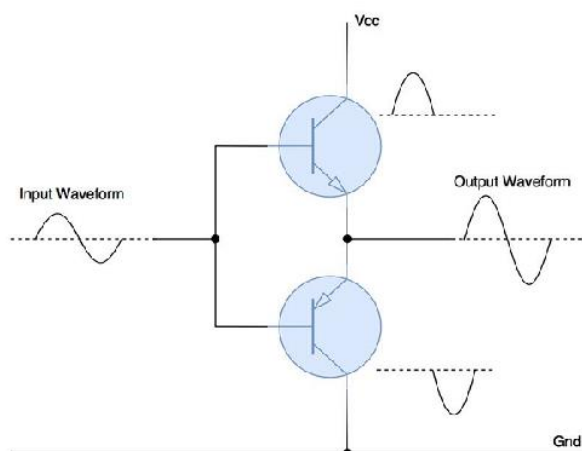
1. A 类

A 类，Q 点选在输入特性曲线直线部分，整个周期内，管子处于导通。效率低，因为没有信号时也有电流损耗。



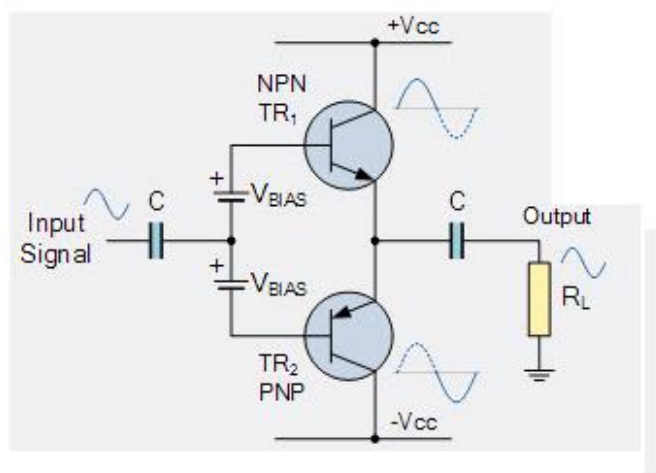
2. B 类

B 类，Q 点选在输入特性截止点，只在 U_i 正半周时管子导通，输出整半波。基于推挽电路，分别输出正、反半周期波形。不过，晶体管存在不工作区(Dead Band / Neutral Zone)，因此在零点邻域输出不理想。



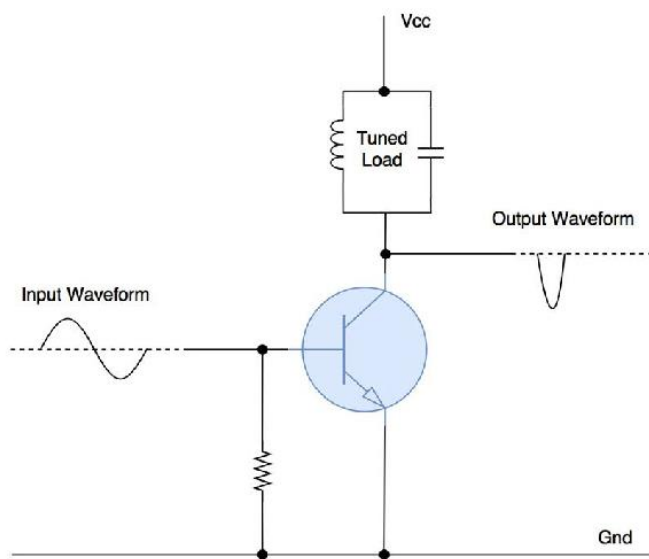
3. AB 类

AB 类，Q 点选择比 B 类稍高，低于 A 类位置，在 U_i 大半周期内管子导通，输出大半周期波形。基于推挽电路，分别输出大半周期波形。适用于音频放大等要信号精确的场景。



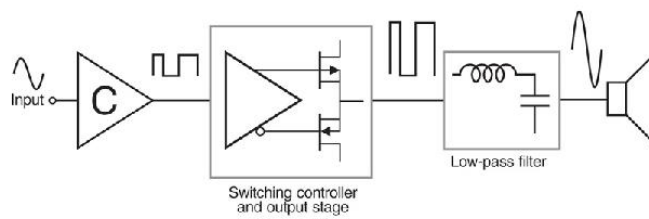
4. C 类

C 类，Q 点选在输入特性截止点左边，只在 U_i 正半周部分时间内管子导通，输出整半波形的顶部。基于推挽电路，分别输出正、反顶部、底部波形。适用于高频振荡器、射频放大器等场景。



5. D 类

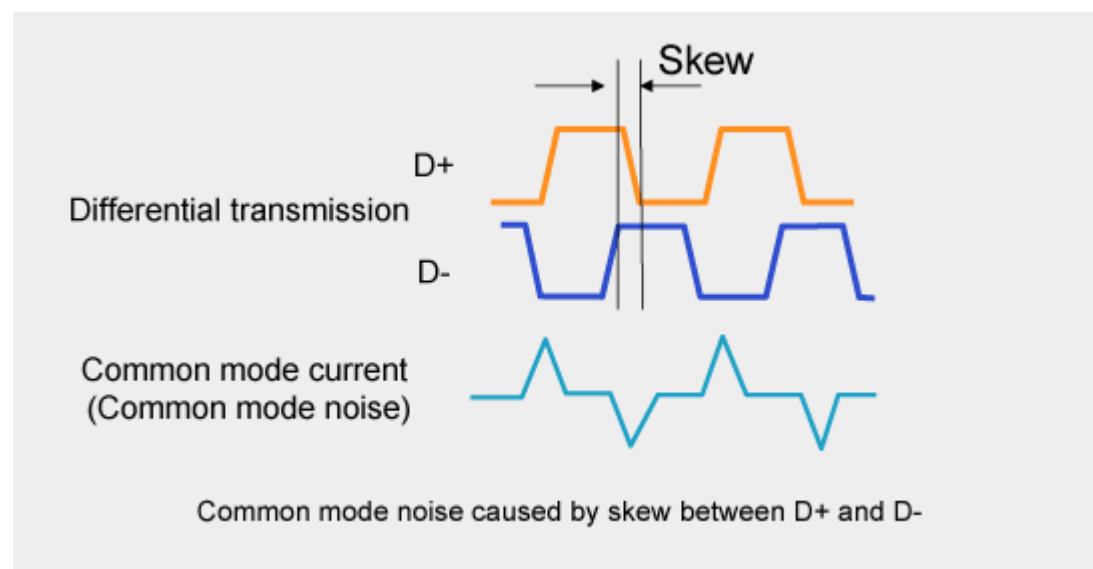
脉冲宽度调制(PWM)放大电路。



三、 运算放大电路

运算放大器，实质是差分计算、信号放大、交替输出。可放大电压差分的主动元件(Active Component)。

注，差分计算，对接收到的相互反向变化的电平相减，保持信号的同时，过滤掉共模噪声。



注， D_+ 与 D_- 的共模电平是两者的平均值，不一定是零。

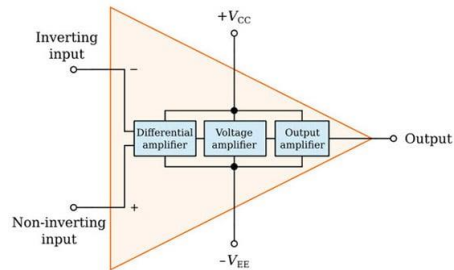
注，信号放大，利用晶体管进行电压放大。

注，交替输出，基于推挽电路交替输出、放大半个周期的信号。

注，主动元件，可产生能量、有增益、有方向的可控制电流或电压的元件。

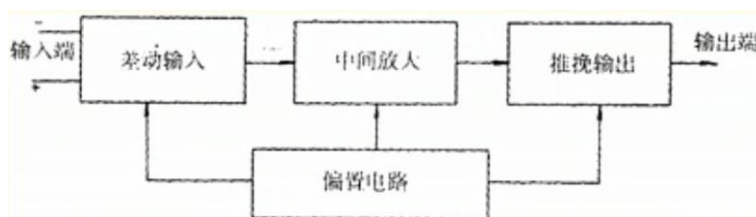
运算放大器可用于信号放大、主动滤波(Active Filtering)、非线性电路(Non-Linear Circuit)、振荡器(Oscillators)、数学运算(Mathematical Operations)等。可基于 PSPICE 仿真器验证运算放大器设计。

运算放大器基本框架。



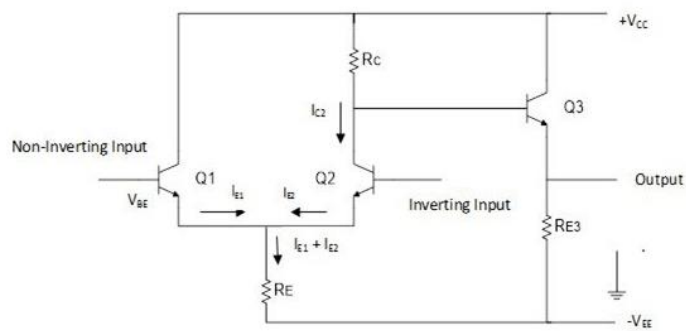
可以看到，运算放大器基本是差分放大器、电压放大器、输出放大器的组合。

同时，运算放大器基于偏置电路设置晶体管偏置电压(流)。

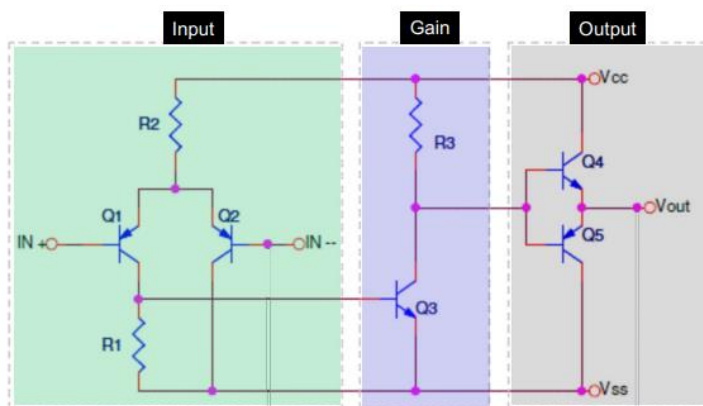


差分输入级电路，采用差分放大电路以抑制零点漂移；电压增益中间级电路，多级耦合电路，提供高增益；输出驱动级电路，互补对称推挽功放电路，低输出电阻；偏置电路，恒(电)流源电路。

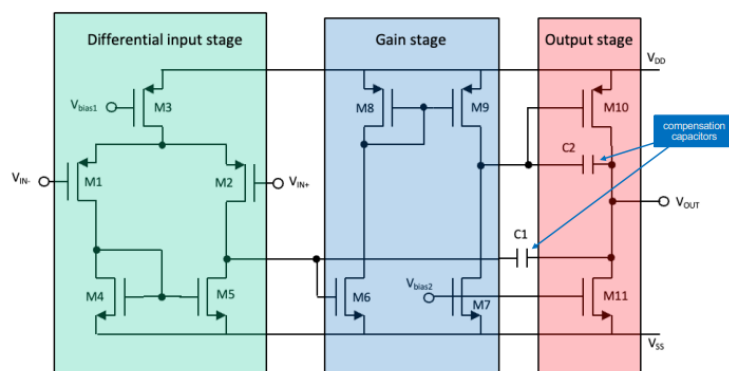
基本形式的运算放大器电路。



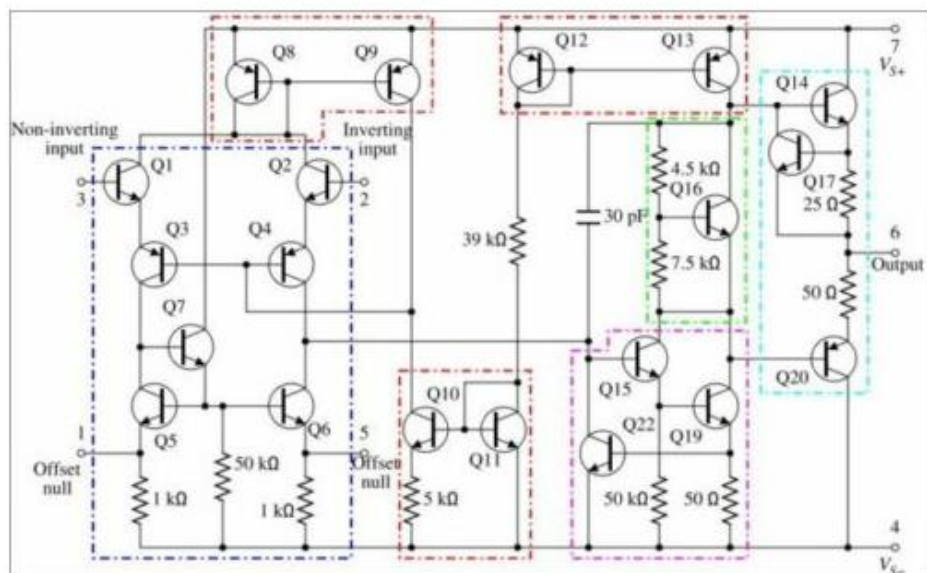
基于双极型晶体管的运算放大器。



基于 MOSFET 的运算放大器。



IC741 运算放大器电路。



1. 偏置电路

控制晶体管偏置电压/电流。

2. 输入级电路

输入信号差分计算。

3. 中间级

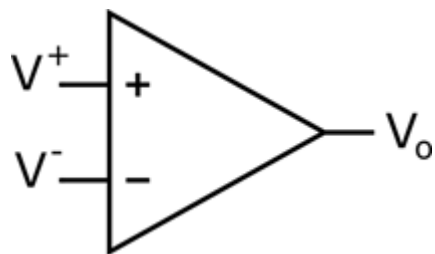
主要提供电压放大倍数。

4. 输出级

提供输出功率以满足负载要求，属于功率放大电路

5. 应用分析

开环状态(Open Loop)的理想运算放大器。

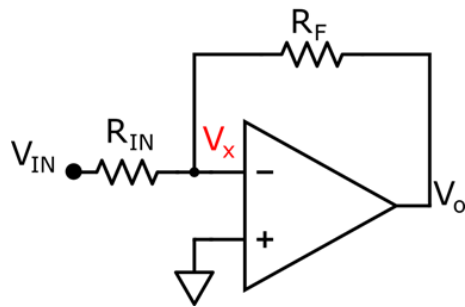


电压输出方程。

$$V_o = A_o(V^+ - V^-)$$

运算放大器工作在线性区时，需对放大器进行负反馈；工作在非线性区时，需保持开环状态或进行正反馈。

1) 理想负反馈运算放大器



特点，同相端电位与反相端之间无电流流动，但是电位相等，似乎联通，因此，称虚短电路 (Virtual Short Circuit)；同时，同相与反相输入端电流均为零，似乎断开，因此，称虚断电路 (Virtual Open Circuit)。

虚短电路推导过程。

电路的节点方程。

$$\frac{V_X - V_{IN}}{R_{IN}} + \frac{V_X - V_O}{R_F} = 0$$

移项可得。

$$\frac{V_X}{R_{IN}/R_F} = \frac{V_{IN}}{R_{IN}} + \frac{V_O}{R_F}$$

基于 $V_X = -(V^+ - V^-)$

联立运算放大器的输出方程，可得，

$$V_o = A_o(V^+ - V^-) = -A_o V_X$$

基于理想运算放大器， A_o 无限大；基于节点方程， V_o, V_{IN} 有限， V_X 不等于 V_o ，因此，

$$V_X = -\frac{V_o}{A_o} = -\frac{V_o}{\infty} = 0$$

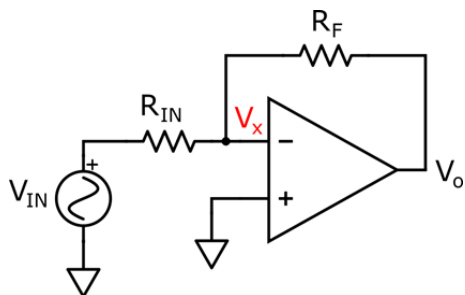
$$(V^+ - V^-) = \frac{V_o}{\infty} = 0$$

可知， $V^+ = V^- = 0$ ，同相电压(V^+ , Non-Inverting Input)等于反相电压(V^- , Inverting Input)。

基于负反馈的理想运算放大器的虚短电路特性，使得可通过同相电压，间接控制反相电压及输出电压。该方法可称作电压拷贝。

当同相节点连地时，反相节点等于 0，似乎同样连地，称虚地(Virtual Ground)。基于该特性，可简化节点方程的计算量。

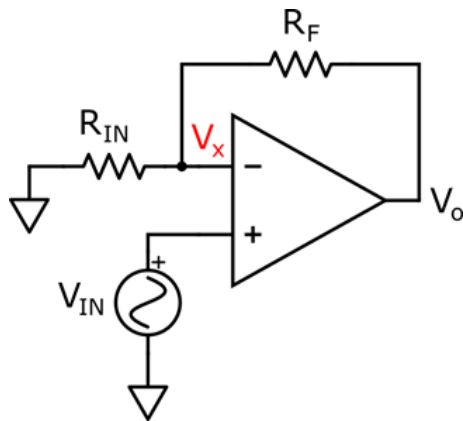
因此，反相放大器输出电压。



$$-\frac{V_{IN}}{R_{IN}} - \frac{V_o}{R_F} = 0$$

$$\frac{V_o}{V_{IN}} = -\frac{R_F}{R_{IN}}$$

同相放大器输出电压。



$$\frac{V_o}{V_{IN}} = 1 + \frac{R_F}{R_{IN}}$$

2) 实际负反馈运算放大电路

实际运算放大电路开环增益(Open Loop Gain)不是无限大，可能很小。

基于拉普拉斯变换(Laplace Transform)定义开环输出电压方程。

$$V_o = A(s) * (V^+ - V^-)$$

$A(s)$ ，一阶低通响应(First-Order Low Pass Response)，可视作开环增益。

$$A(s) = \frac{A_o}{1 + \frac{s}{2\pi f_c}}$$

注， A_o ，直流增益； f_c ，开环截止频率(Cut-Off Frequency)。

计算同相放大电路增益。

节点方程，

$$\frac{V_X}{R_{IN}} + \frac{V_X - V_o}{R_F} = 0$$

输出方程，

$$V_o = A(s) * (V_{IN} - V_X)$$

方程联立，可知增益方程，

$$\frac{V_o}{V_{IN}} = \frac{G_{CL}}{1 + \frac{G_{CL}}{A(s)}}$$

$$G_{CL} = 1 + \frac{R_F}{R_{IN}}$$

注，理想运算放大器闭环增益(Closed Loop Gain)，

当开环增益 $A(s)$ 无限大时，该方程等于理想同相放大器的增益方程。当 G_{CL} 相对 $A(s)$ 不可忽略时，电路实际增益相对于理想运算放大器增益会减少。

该方程与一阶低通响应或开环增益 $A(s)$ 联立，可得开环截止频率 f_c 与闭环截止频率 f_{cutoff} 关系。

$$f_{cut_{off}} = \frac{A_o}{G_{CL}} f_c$$

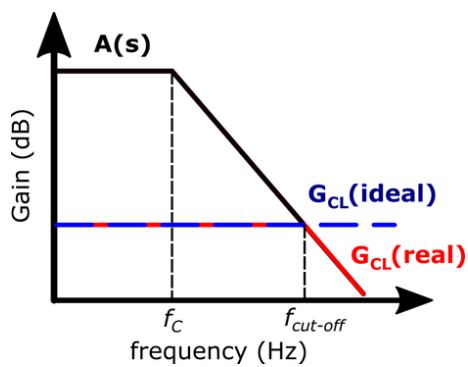
可知，同相放大器的闭环带宽(GBW, Gain Band Width)。

$$GBW = G_{CL} * f_{cut_{off}}$$

联立闭环截止频率 $f_{cut_{off}}$ 、开环增益 $A(s)$ 方程，可知开环增益 $A(s)$ 等于闭环增益 G_{CL} 时，对应闭环截止频率 $f_{cut_{off}}$ 。

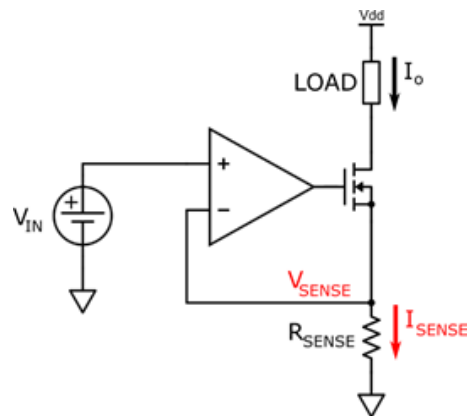
$$A(f_{cut_{off}}) = G_{CL}$$

开环增益 $A(s)$ 与同相运算放大器闭环理想增益 $G_{CL}(ideal)$ 、实际增益 $G_{CL}(real)$ 的波特图 (Bode Plot)。



因此，可用开环增益波特图预测运算放大器的闭环频率影响。

压控电流源。



$$I_{OUT} = \frac{V_{IN}}{R_{SENSE}}$$

3) 基于正反馈的运算放大器

$$V^+ > V^-, V_o = +V_{opp}$$

$$V^+ < V^-, V_o = -V_{opp}$$

$$I^+ = I^- = 0$$

注，共模抑制比(CMRR, Common Mode Rejection Ration)，测量(Measurement)运算放大器对共模信号(Common Mode Signals, Noise)的抑制能力。

$$CMRR = 20\log \frac{A_{OL}}{A_{CM}}$$

A_{OL} , 开环(Open Loop)电压差分增益。

A_{CM} , 共模增益(Common Mode Gain)。

四、 运算电路

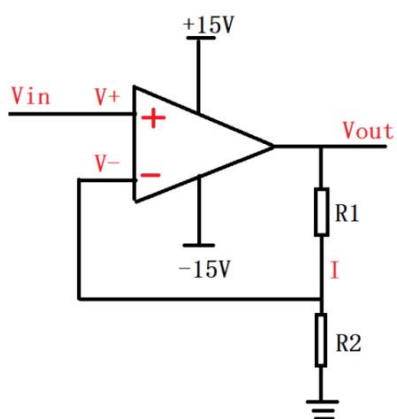
运算电路是运算放大器的线性应用，实质是基于运算放大器的正相、反相电位相等的虚短以及正相、反相输入电流等于零的虚断，设计运算电路。

注，线性应用，输出等于开路增益 * 正相和反相差。

运算放大器基于负反馈电路形式，工作在线性区，可实现比例、加减、积分、微分、对数、反对数等数学运算。

1. 比例运算电路

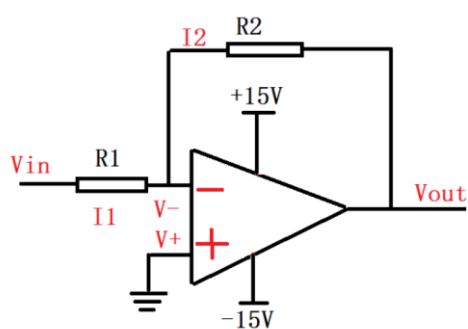
同相比例运算放大器(In-Phase Propotional Operational Amplifier)。



基于运算放大器虚短 (Virtual Short) , V_+, V_- 电位相等 , 可计算 ,

$$V_{out} = \frac{(R_1 + R_2)}{R_2} V_{in}.$$

反相(Inverting)比例运算放大器。

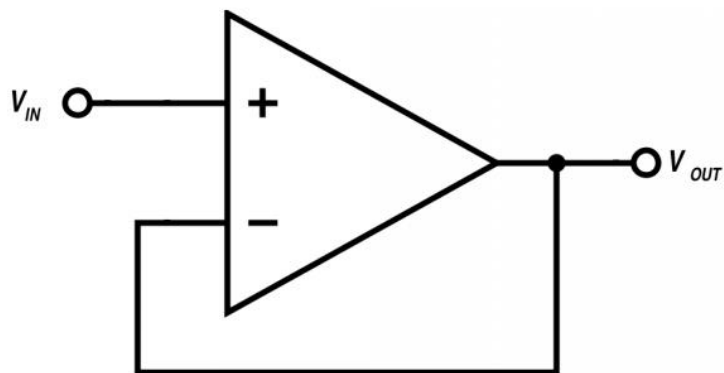


基于运算放大器虚断 (Virtual Break) , $I_- = I_+ = 0 = I_1 + I_2$, 可计算 ,

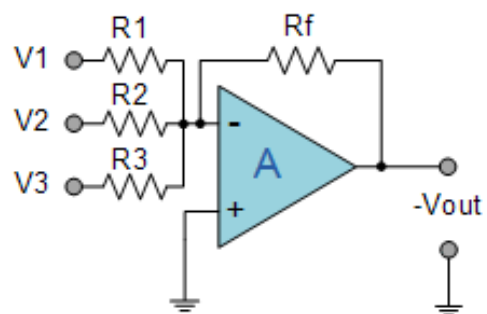
$$V_{out} = -V_{in} \frac{R_2}{R_1}.$$

当电阻 R_2 无限大时, 比例运算放大器变成电压跟随器(Voltage Follower)或缓冲放大器(Buffer

Amplifier)等。



2. 和运算电路(Summing Op. Amp.)

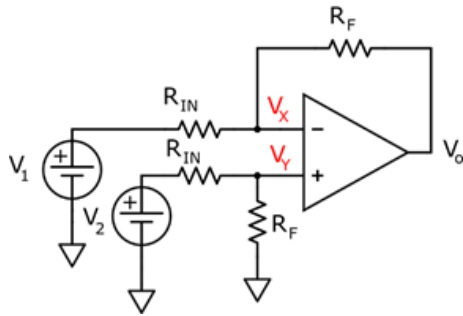


基于运算放大器虚短，可计算，

$$V_{out} = -\left(\frac{R_f}{R_1} V_1 + \frac{R_f}{R_2} V_2 + \frac{R_f}{R_3} V_3\right)$$

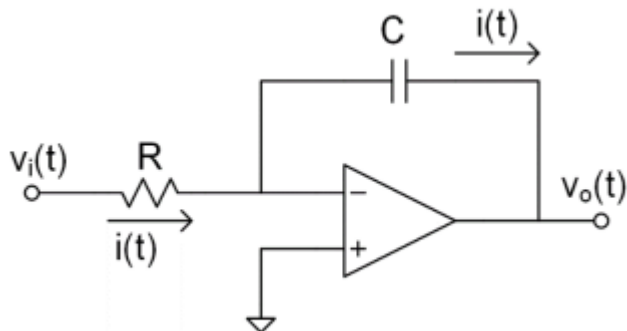
3. 差分电路(Difference Op. Amp.)

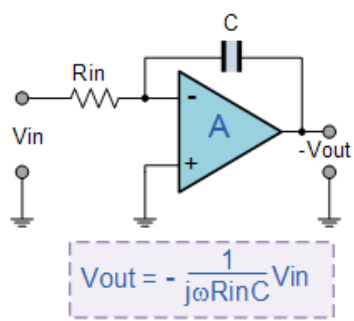
仪表中常用的差分放大器。



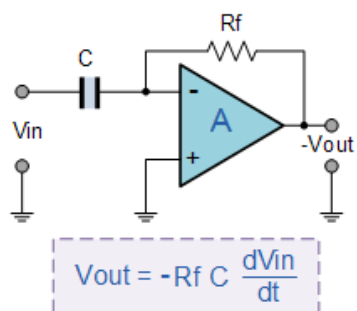
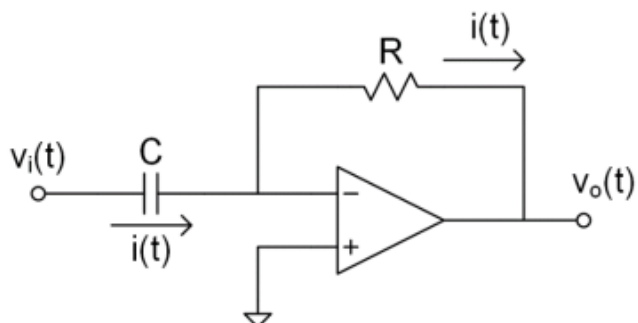
$$V_o = \frac{R_F}{R_{IN}(V_2 - V_1)}$$

4. 积分电路(Integrator)





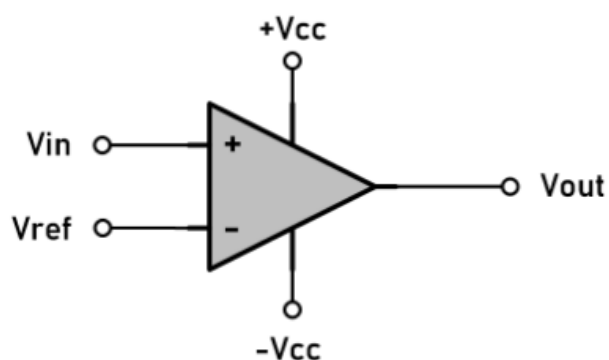
5. 微分电路(Differentiator)



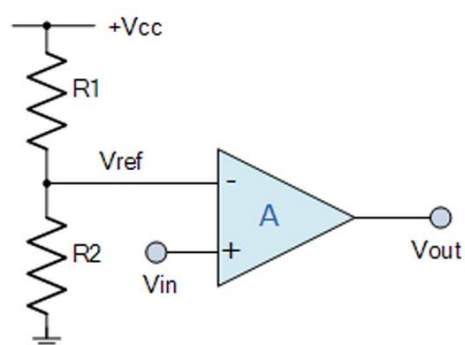
五、 比较器电路

比较器电路是运算放大器的非线性应用，用于比较电压或电流。实质是基于运算放大器正相或反相端的参考信号，利用导通角控制输出信号。可分为电压比较器、窗口比较器及滞回(Hyperesis)比较器。

注，非线性应用，输出与输入不成线性关系。



1. 电压比较器



当参考电压连运算放大器反相(Inverting)时，输入电压 V_{in} 大于参考电压 V_{ref} 时，产生高电平输出；连正相(Non-Inverting)时，产生低电平输出。

可基于不同形式设计参考电压电路，比方电阻、齐纳二极管等。

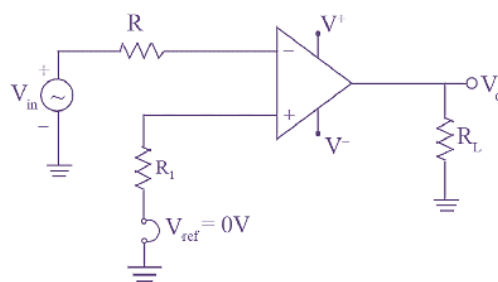
注，运算放大器在开环下，容易基于噪声或输入信号缓变(Slow Changing)引发振荡(Oscillation)。可通过正反馈(Positive Feedback)在比较电路中创立磁滞(Hysteresis)效应以阻止振荡，形成施密特触发器(Schmitt Trigger)。

电压比较器可用于形成模拟数字转换器(ADC)、脉冲宽度调制生成器(PWM Generation)等。

比方，PWM，控制电压与锯齿波形比较，控制电压高于锯齿波，输出高电平，低于锯齿波，输出低电平。

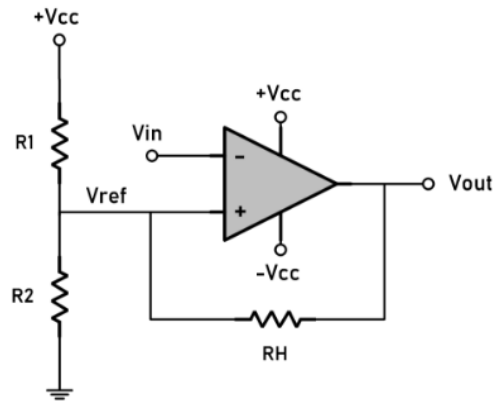
2. 过零检测器(Zero Crossing Detector)

检测交流信号过零点或极性变换点(Zero Crossing Point)。可应用于锁相环(PLL, Phase Locked Loop)、电机控制及频率检测等。



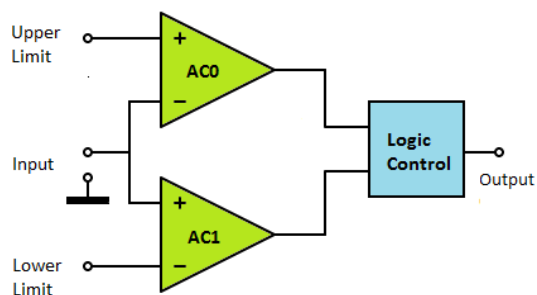
3. 滞回比较器(施密特触发器)

基于正反馈进行迟滞(Hysteresis)，在模数转换时移去模拟信号中的噪声。



4. 双限比较器(窗口比较器)

基于电压区间或窗口(Voltage Band Or Window)检测输入电压。



六、 滤波电路

滤波器可以分为数字滤波器、模拟滤波器。数字滤波器对离散信号过滤，分为非递归形式的有限脉冲响应(FIR, Finite Impulse Response)、递归形式的无限脉冲响应(IIR, Infinite Impulse Response)滤波器。基于数字信号处理器(DSP)实现。

模拟滤波电路的实质是基于晶体管、运算放大器的高频开关、高输入阻抗、低输出阻抗等特性替代传统的电感等元件，以实现低电平、高速电路场景对交流信号的过滤。核心部件是运算放大器积分器(Integrators)、运算放大器微分器(Differentiators)。

1. 滤波分类

基于频率整形(Frequency Shaping)可分为低通、高通、带通(Bandpass)、带阻(Bandstop)、全通滤波器。比方，音频可用低通滤波器过滤高频噪声，通讯可用带通滤波器筛选特定频带以防止不同信号的干扰，图像处理时可用低通滤波器模糊图像以消去噪声及细节

基于电路拓扑(Topologies)可分为被动(Passive)滤波电路、主动(Active)滤波电路。被动滤波电路，基于电容、电阻、电感等被动元件实现滤波电路。主动滤波器(Active Filters)，主要基于运算放大器或跨导放大器、电阻、电容实现。

基于滤波器传输函数(Transfer Function)多项式系数(Coefficients)，可分为 Butterworth、Chebyshev、Elliptic、Bessel 等滤波器。相应的传输函数可通过主动或被动滤波器实现。

基于滤波器构型(Configurations)或架构(Architectures)，可分为 Sallen-Key、多反馈(MFB, Multiple Feedback)等，用于实现不同的二阶传输函数。多反馈电路对元件变化敏感、高频特性更好，Sallen-Key 电路增益准确性(Gain Accuracy)更好。

注，阶数(Order)，传输函数(Transfer Function)表示式中频率的最高次数。

基于级联形式可分为串联、并联等。

基于应用可分为音频滤波器、射频滤波器等。

2. 主动滤波器

当前，改进的连续时间的主动滤波器(Continuous Time Active Filters)，有 Active-R(esistor)、Switched-R(esistor)、Ring Oscillator Based、PWM Based、Inverter Based OTA-C(apacitor)等类型。

注，OTA, Operational Transconductance Amplifier。

Active-RC 滤波器，用运算放大器(Op-Amp)、电阻、电感的电路替代被动滤波器中的电感。

Active-R 滤波器，利用运算放大器增益与频率相关的特性(Frequency Dependent Model)，去掉 Active-RC 滤波器中的电容，扩展了频域。

Active-C 滤波器，与 Active-R 相似，只是去掉的是 Active-RC 的电阻。

Switched-R 滤波器，用带开关的电阻(Switched-R)替代 Active-RC 滤波器中的电阻。

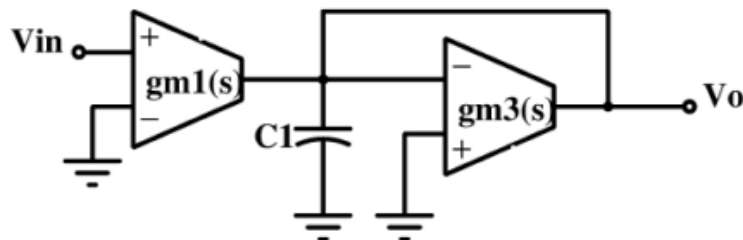
MOSFET-C 滤波器，用场效应管替代 Active-RC 滤波器中的电阻。

OTA-C 滤波器，运行在开环模式，设计参数是电容和跨导增益，等同于 Gm-C 滤波器。

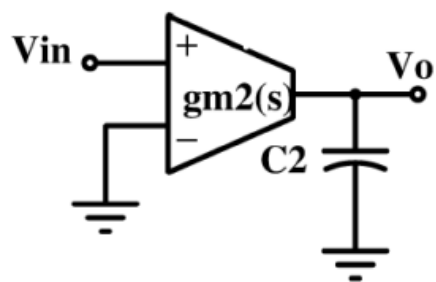
Switched-Capacitor 滤波器，基于 CMOS。

1) Gm-C 电路

有损形式。

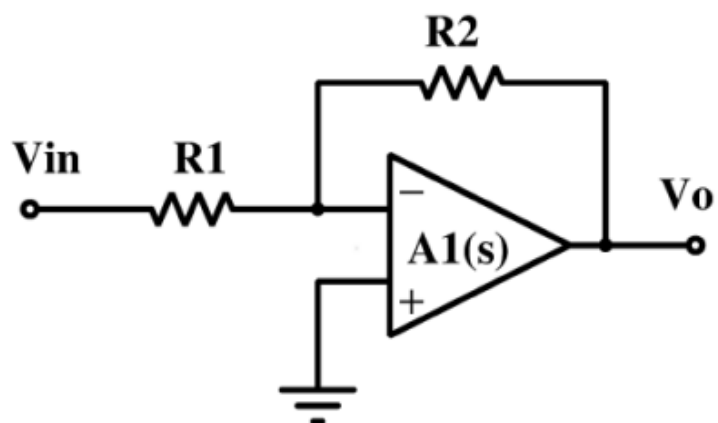


无损形式。

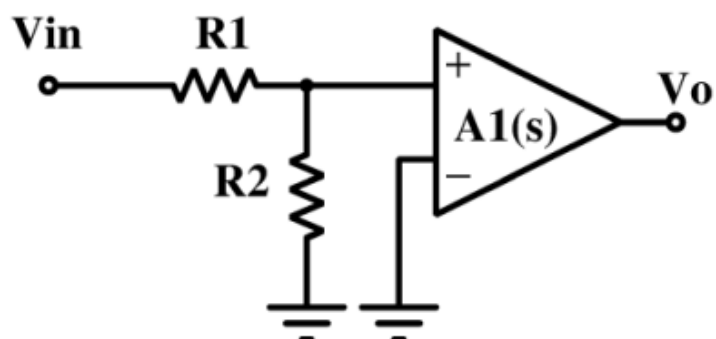


2) Active-R 滤波器

有损形式。



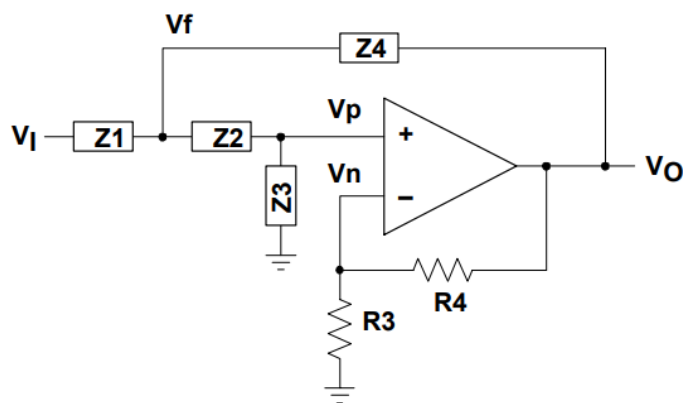
无损形式。



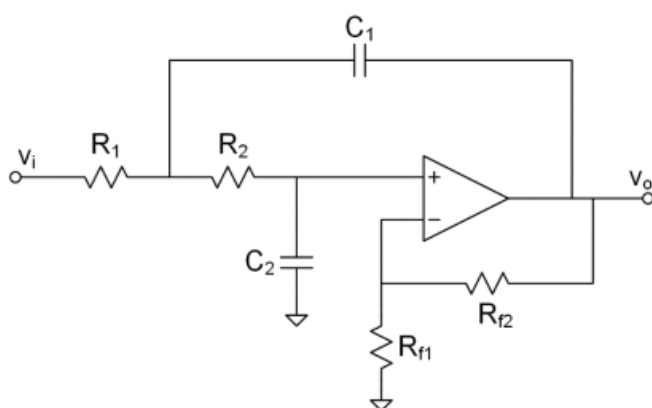
3. 滤波器构型

1) Sallen-Key

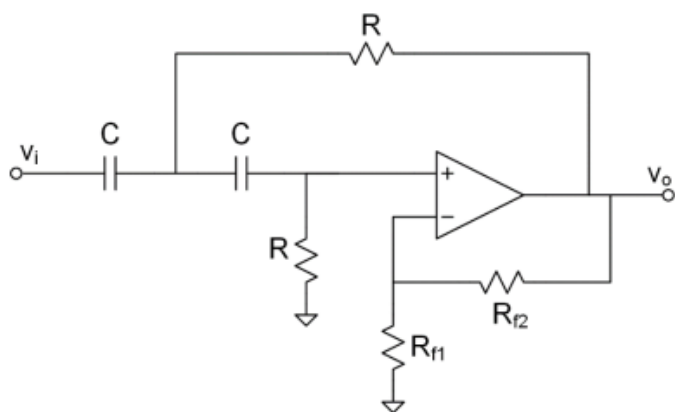
通用框架。



二阶低通(Second-Order Low Pass Filter)。

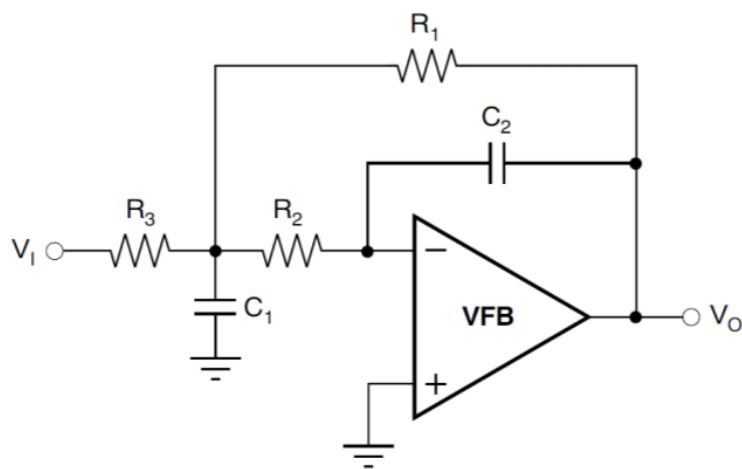


高通(High Pass Filter)。



注，高阶滤波器可以用 Butterworth、Chebyshev、Elliptic、Bessel 滤波器来设计。

2) 多路反馈



4. 传输函数

1) 产生背景

传输函数，大概来说就是系统的输出信号与输入信号的比值关系。

在控制系统、电路系统、信号处理等系统中，有时信号在时域的特征区分度不及在频域的区分度。因此，对系统的输出与输入信号的关系常转换到频率域进行分析。

这就是说，系统可用输出信号的频域形式与输入信号的频域形式的比值的对数来表示系统的增益(分贝)。

2) 函数定义

使函数定义具备通用及准确度的前提。

假定，系统是一个输入、输出关系不随时间改变的线性时不变系统(LTI, Linear Time Invariant)。

同时，基于傅里叶变换的通用形式，拉普拉斯变换实现信号时域到频域的转换。

可知，传输函数实质是线性时不变系统(LTI, Linear Time Invariant)的输出、输入信号在拉普拉斯频域的比值。

$$H(S) = \frac{N(S)}{D(S)}$$

S， 频率的复变量形式(Frequency Complex Variable)。

N(S)，输出的拉普拉斯变换函数多项式。

D(S)，输入的拉普拉斯变换函数多项式。

零点， 多项式 N(S)的解(Roots)。

极点， 多项式 D(S)的解(Roots)。

3) 函数作用

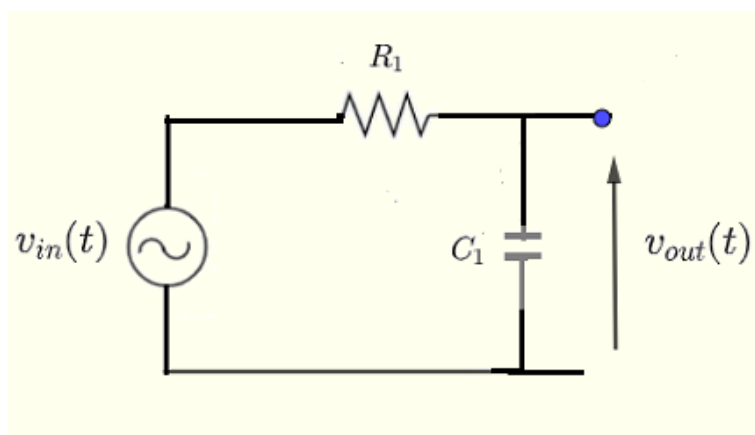
传输函数的作用，在滤波器设计时，可基于传输函数确定滤波器的频率响应(Frequency Response)、稳定性等特征，以便优化滤波器性能。

4) 函数实例

比方，阻、容低通滤波电路。

基于传输函数，确定输出功率降低 3 分贝(dB)或一半时的信号截止频率。

注，截止频率(Cutoff Frequency, Corner Frequency, Break Frequency)，系统频率响应的边界，系统能量在该点开始衰减(Attenuated)。在通带(Passband)场景，可分上截止频率(Upper)及下截止频率(Lower)。高通场景，低于截止频率，输出功率开始衰减；低通场景，高于截止频率，输出功率开始衰减。



滤波器基于电压的传输函数定义。

$$H(s) = \frac{V_{out}}{V_{in}} = \frac{Z_{C_1}}{Z_{C_1} + Z_{R_1}}$$

Z, 阻抗。

计算过程。

1、基于滤波器类型，替代传输函数中的变量。

知，

$$Z_{C_1} = \frac{1}{sC_1}, Z_{R_1} = R_1$$

可得拉普拉斯域的传输函数，

$$H(s) = \frac{1}{1+R_1C_1s}$$

$$s = j\omega$$

注，s 的指数是 1，因此滤波器是 1 阶。

因此，相位域(Phasor Domain)的传输函数。

$$H(\omega) = \frac{1}{1+jR_1C_1\omega}$$

2、基于传输函数，确定直流增益、截止频率等特征。

$$H(\omega) = \frac{1}{1+jR_1C_1\omega} = \frac{Z_{out}}{Z_{in}}$$

输出功率降低 3 分贝，意味着，

$$20\log_{10}|H(\omega)| = 20\log_{10}\frac{|Z_{out}|}{|Z_{in}|} = 20\log_{10}\frac{|1|}{|1+jR_1C_1\omega|} = -3dB$$

注，基于功率与电压的计算方程不同，因此增益不同。电压增益， $20\log A$ ； 功率增益， $10\log A$ 。

可知，

$$\frac{|1|}{|1+jR_1C_1\omega|} = 10^{\frac{-3}{20}}$$

$$|1+jR_1C_1\omega| = 10^{\frac{3}{20}} = \sqrt{10^{0.3}}$$

$$\sqrt{1^2 + R_1^2 C_1^2 \omega^2} = \sqrt{2}$$

$$1 + R_1^2 C_1^2 \omega^2 = 2$$

$$\omega_c = \frac{1}{R_1 C_1}$$

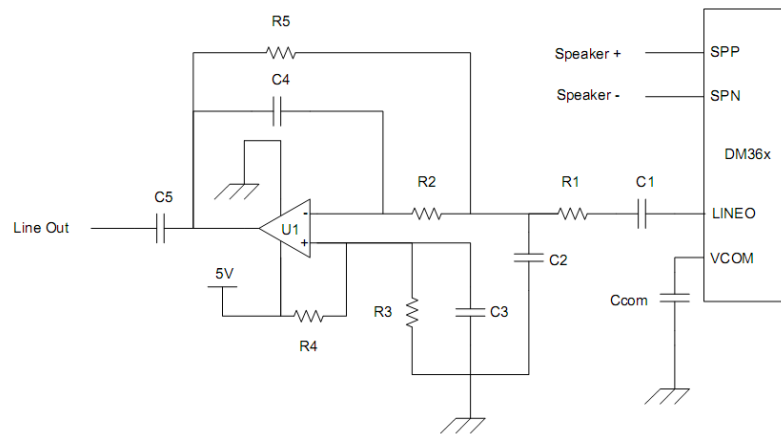
因此，对应的信号截止角频率，

设计滤波器电路时，角频率 ω 满足该关系，可实现电路输出功率降低 3 分贝。

注，基于运算放大器的电路，传输函数分析方法相似。区别在利用放大器的虚短等特点。

5. 音频应用

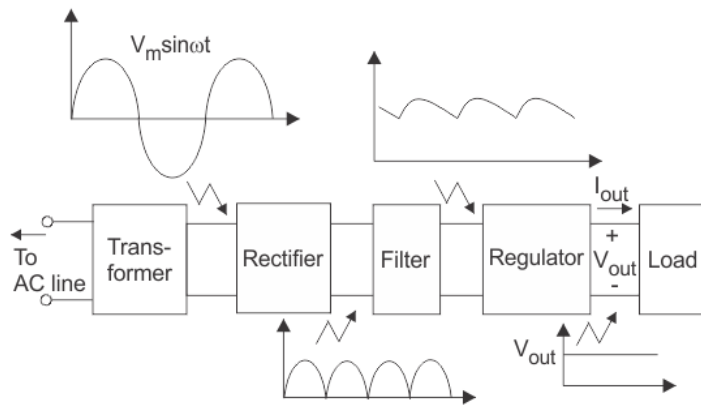
音频输出去噪电路。



七、 电源电路

电源电路实质，输入电源变换到输出电源。可分为非稳压电源(Unregulated Power Supplies)、线性稳压电源(Linear Regulated Power Supplies)、开关稳压电源(Switch Regulated Power Supplies)、纹波稳压电源(Ripple Regulated Power Supplies)。

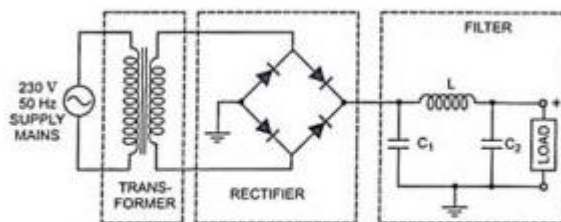
电源电路基本组成，变压器(Transformer)、整流器(Rectifier)、滤波器(Filter)、稳压器(Regulator)等。在输出时可设置过压保护电路。



具体电源电路设计类型很多，比方直流稳压电源(DC Power Supply)、可变直流稳压电源(Variable Power Supplies)、可变开关稳压电源(Variable Switching Power Supplies)等。

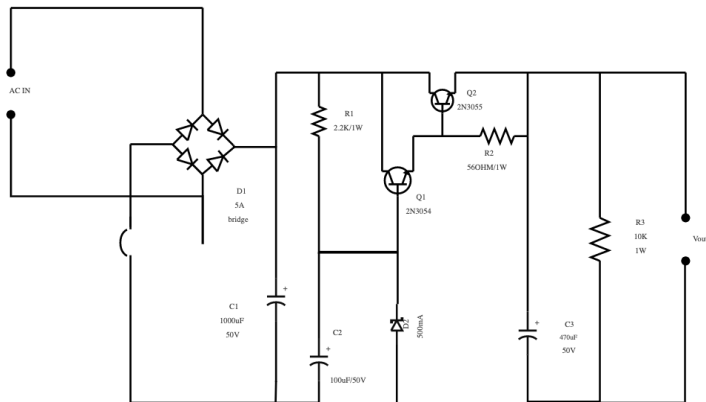
1. 非稳压电路

示例图。



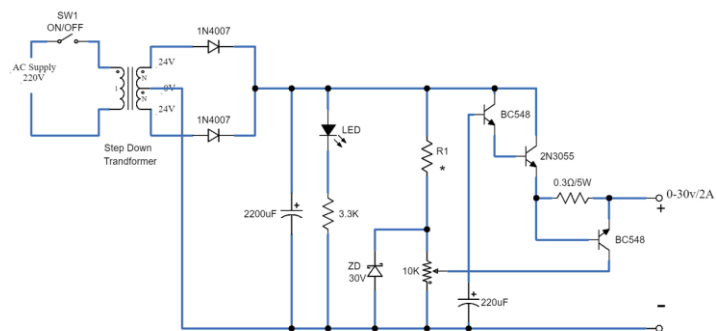
2. 直流稳压电源

示例图。



3. 可变稳压电源

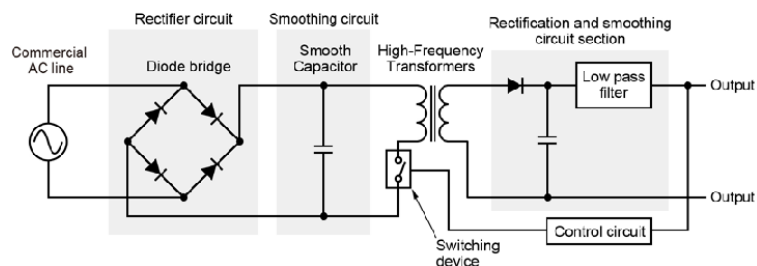
示例图。



4. 开关稳压电源

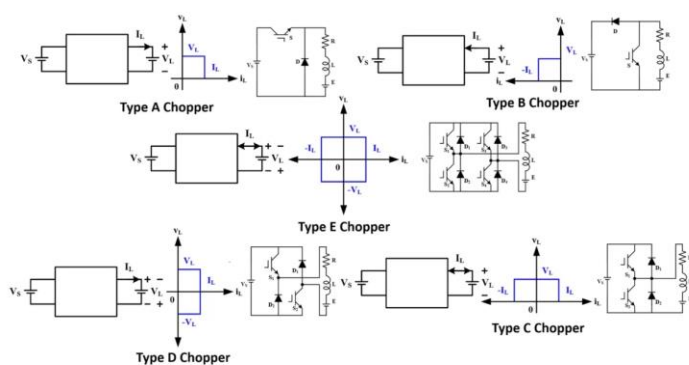
基于双极型晶体管或场效应管等开关器件设计。

基本示例图。



框架图，左侧，整流电路，基于整流电路转换交流到直流、平滑滤波；开关及转换电路(Converter/Transformer)，基于场效应管斩波电路(Chopper Circuit)进行电路高频开/关，转换直流到方波(Square Wave)；整流及滤波电路，整流、过滤纹波及噪声；控制电路(Control Circuit)，基于分压器(Voltage Divider)侦测输出电压，光耦隔离(Optocoupler Isolation)后，基于误差放大器(差分放大器/运算放大器)与参考信号比较，以调控脉冲宽度调制振荡器，控制转换器。

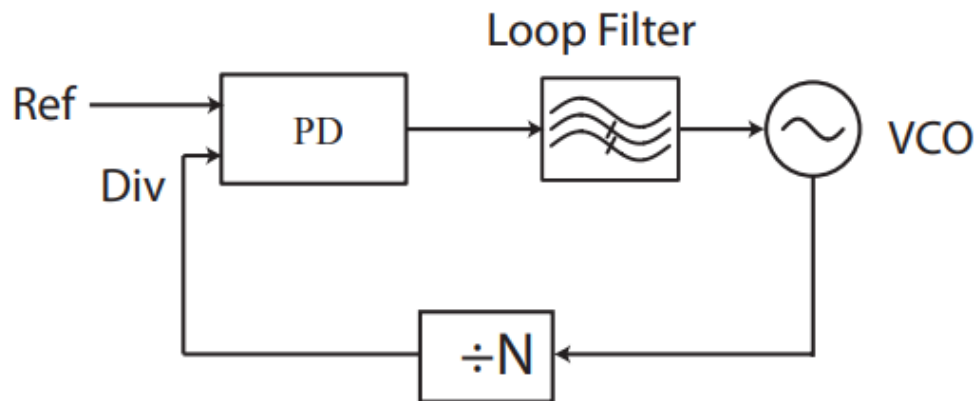
斩波电路。



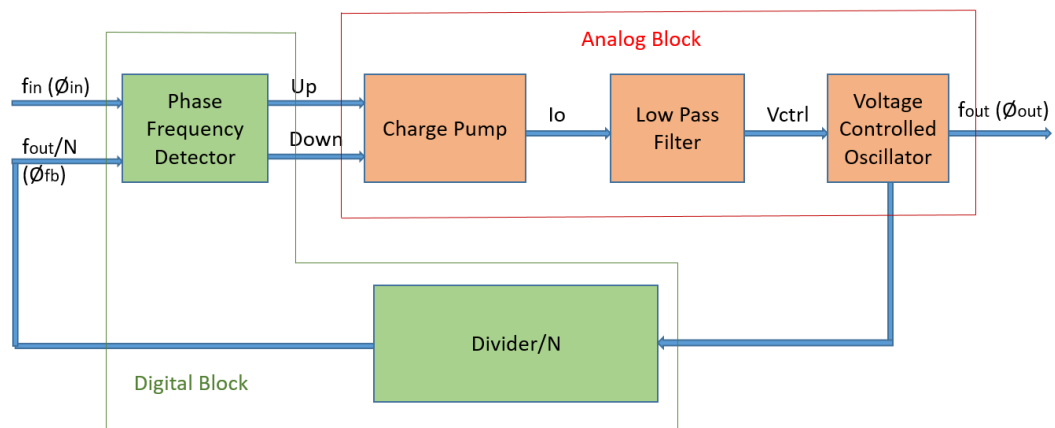
八、 时钟电路

1. PLL(Phase Locked Loops)

锁相环，用于倍乘晶体输出频率(Crystal Output Frequency)。

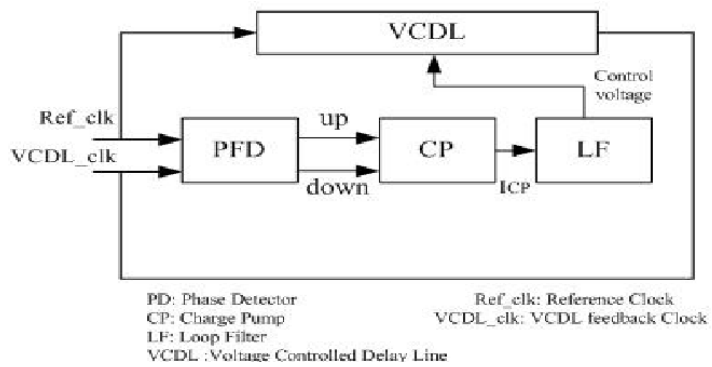


主要模块，鉴相器(PD, Phase Detector)，低通滤波器(LPF, Low Pass Filter)，压控振荡器(VCO, Voltage Controlled Oscillator)，分频器(Divider)。



2. DLL(Delay Locked Loops)

延迟锁定环，自动调节信号延迟，使不同路的信号相位一致。

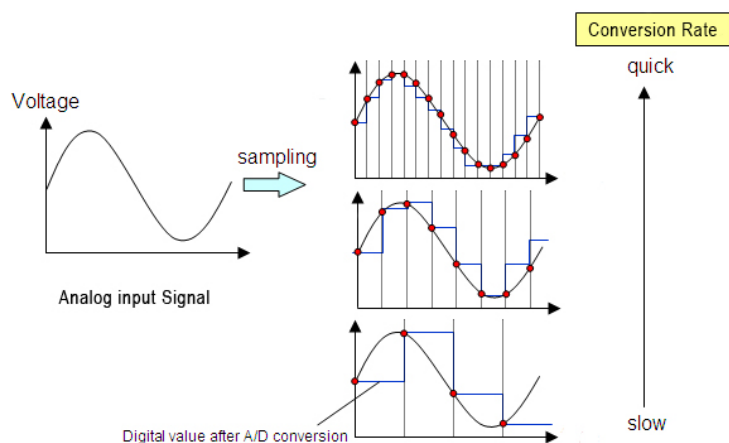


九、 ADC/DAC

1. ADC

模数转换器实质，基于奈奎斯特定律对模拟信号采样，采样的不同电平信息转换成数值。

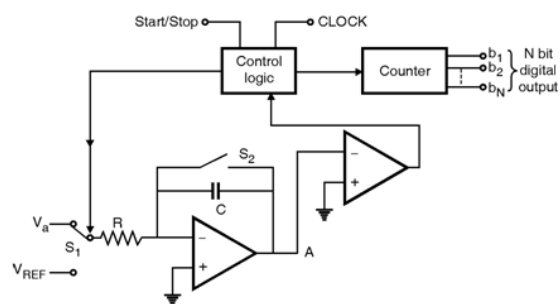
模拟信号采样示例。



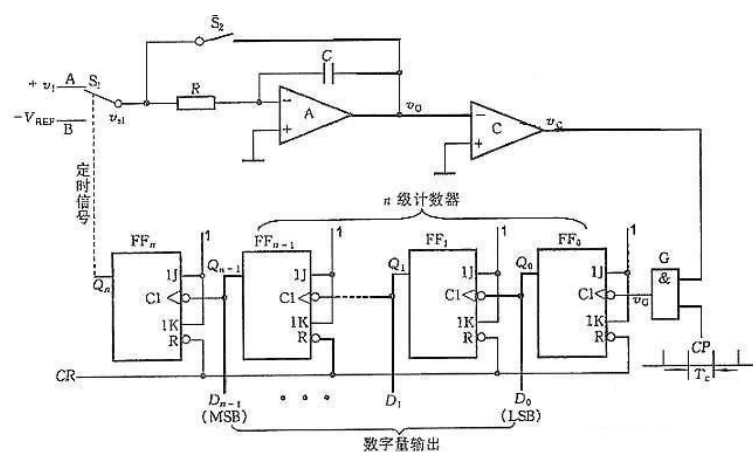
框架图，左侧，模拟信号；右侧，不同采样率对应不同采样信息；不同的采样电平/阶梯对应不同的数值。

模数转换器基于工作方式可分为计数器类型(Counter Type)、积分类型(Integrating)、逐次逼近类型(SAR, Successive Approximation)、闪存型(Flash)等。

1) Integrating ADC

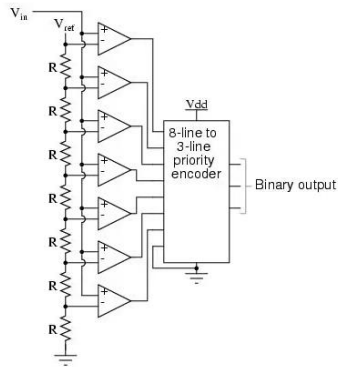


模数转换器框架图，左侧，开关(Switch)采样信号，积分电路保持(Hold)；居中，过零检测，时钟脉冲控制器同步采样开关及计数器；右侧，计数器输出数值。



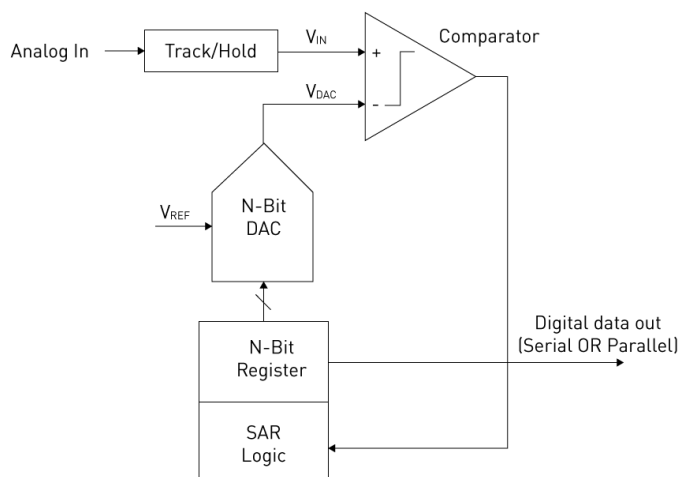
基于积分器 A、过零比较器 C、时钟脉冲控制门 G 及计数器 FF0-FFn 等几部分组成。

2) FLASH ADC



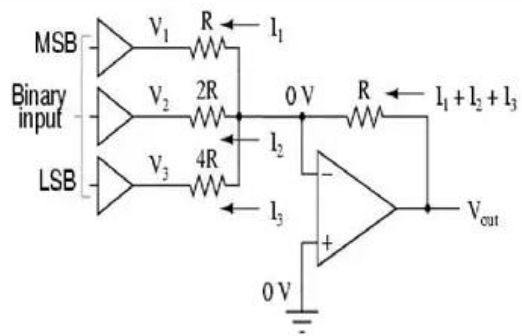
具备不同参考电压的多个比较器链式(Chain)连接，采样的多路输出，传到数字编码器。

3) SAR ADC



2. DAC

数模转换器。



第三章 数字集成电路

本篇数字集成电路，基本对应半导体电路层级的第四、第五层。主要分析怎样利用不同类型晶体管设计不同的门电路；基于门电路怎样设计存储、计算电路；基于组合、时序怎样设计算术逻辑运算单元等。

数字集成电路基本可看作基于门电路的数字逻辑电路。

基于场效应晶体管或双极型晶体管实现非门、与门、或门等基本门电路。

基于门电路的串并组合实现计算电路，加减法等。

基于门电路循环组合实现记忆维持电路，锁存器。

基于锁存器，结合时钟信号实现读存电路，触发器、寄存器、存储器等。

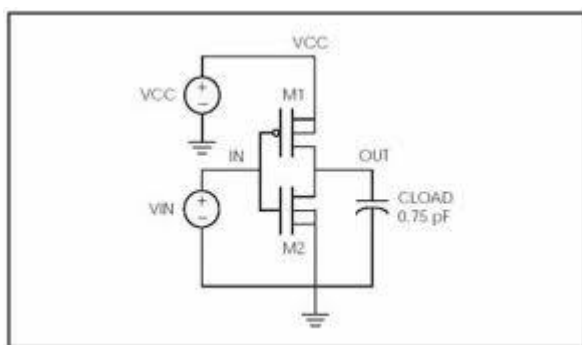
基于计算电路、存储电路实现算术逻辑运算电路。

一、 组合电路

编码器和译码器，数据选择器和分配器，加法器与数值比较器，算术逻辑运算单元（ALU），组合逻辑电路设计，组合逻辑电路的竞争冒险

1. 非门(NOT, 反相器)

利用 NMOS、PMOS 的门控电压相反的特性，可以将 NMOS、PMOS 的 source-drain-source-drain 串联，NMOS、PMOS 的门连到同一电压(作电路输入)，NMOS 管接下拉电路，PMOS 管接上拉电路，NMOS 与 PMOS 之间连接点作输出。该逻辑电路，当输入高电压时，将控制 NMOS 管连通地、PMOS 管断开，因此输出是零电位；当输入低电压时，将控制 NMOS 管断开、PMOS 管连通 Vdd，因此输出高电位。这就实现了取反逻辑。



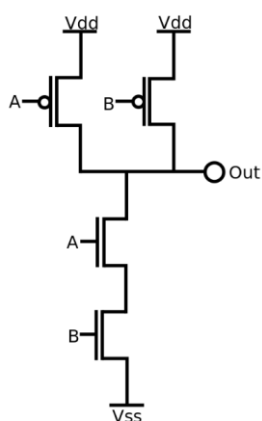
M1 为 PMOS 晶体管，M2 为 NMOS 晶体管，这是一个互补 CMOS 逻辑门。

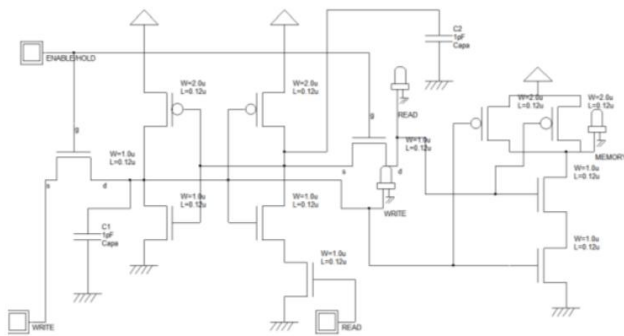
互补的 CMOS 逻辑门都包括一个控制上拉的 NMOS 上拉网络和一个控制下拉的 PMOS 网络，每个网络至少有一个同名的晶体管

当上拉和下拉都关断时，输出就会出现高阻或浮空的 z 状况。

2. 与非门(NAND)

基于 NMOS、PMOS 的门控电压相反的特性，可通过两 NMOS 串联、两 PMOS 并联，NMOS、PMOS 对的门分别共用不同的输入电压，可实现 NAND 门逻辑(输入都是 1，输出 0; 只要输入存在 0，则输出都是 1). 可以用来实现 Nand flash 或 Nand Logic.





NAND Flash

在 NAND 门的控制门或控制栅(control gate)极下、沟道上方之间添加一个绝缘体包裹的浮置门或浮置栅极(Floating gate)来实现电荷的存储.

源极及漏极连地, 控制门输入高电压, 基于 F-N 隧道效应(Fowler Nordheim tunneling)或量子隧穿效应, 沟道中的电子将穿过浮置栅极的绝缘层存入浮置栅极.

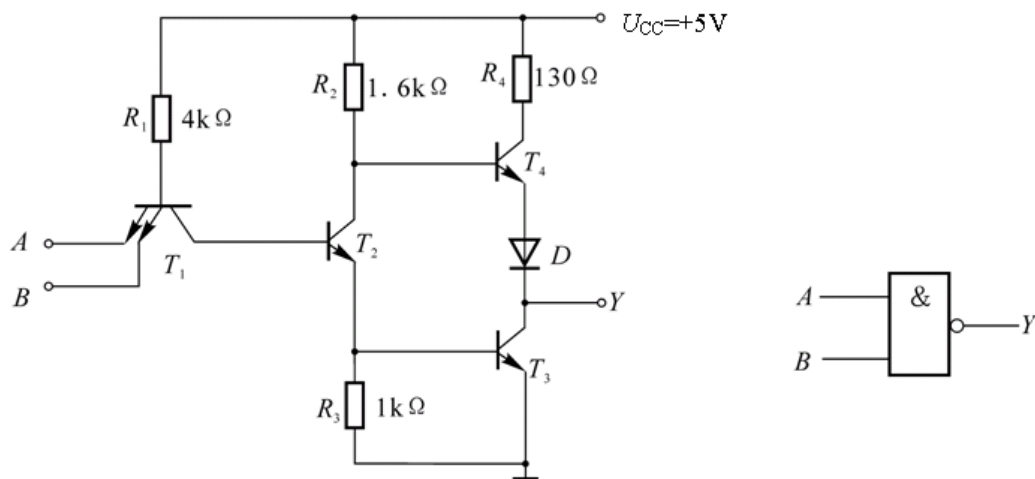
读取时, 控制门输入较低电压, 该电压与浮置栅极电荷抵消, 则 NAND 门开断, 输出 0, 否则 NAND 门闭合, 输出 1, 基于输出可判断浮置栅极存储状态.

进一步, 通过浮置栅极存储电荷数量的多少, 可将状态分为更多类型, 以表示更多 bit 数, 比方一个浮置栅极可以表示 2bit 信息.

形成 SLC(1bit)、MLC(2bit)、TLC(3bit)、QLC(4bit) 技术.

擦除, 在源极或漏极输入正高电压, 控制栅极输入负电压(或无电压), 通过形成的反向电场将浮置栅极的电荷拉走.

写及擦除时的量子隧穿效应都将损耗浮置栅极与沟道之间的氧化绝缘层, 使得电荷散逸, 数据错误.



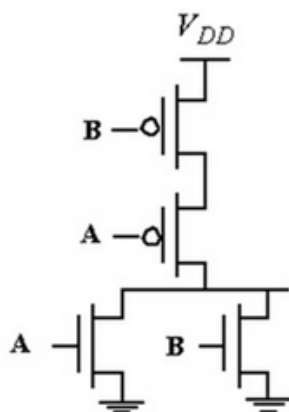
输入级由多发射极三极管 T_1 和电阻 R_1 构成。可以把 T_1 的集电结看成一个二极管，而把发射结看成与前者背靠背的两个二极管。这样， T_1 的作用和二极管与门的作用完全相同。

倒相级由三极管 T_2 和电阻 R_2 、 R_3 构成。通过 T_2 的集电极和发射极，提供两个相位相反的信号，以满足输出级互补工作的要求。

输出级是由三极管 T_3 、 T_4 ，二极管 D 和电阻 R_4 构成的“推拉式”电路。当 T_3 导通时， T_4 和 D 截止；反之 T_3 截止时， T_4 和 D 导通。倒相级和输出级的作用等效于逻辑非的功能。

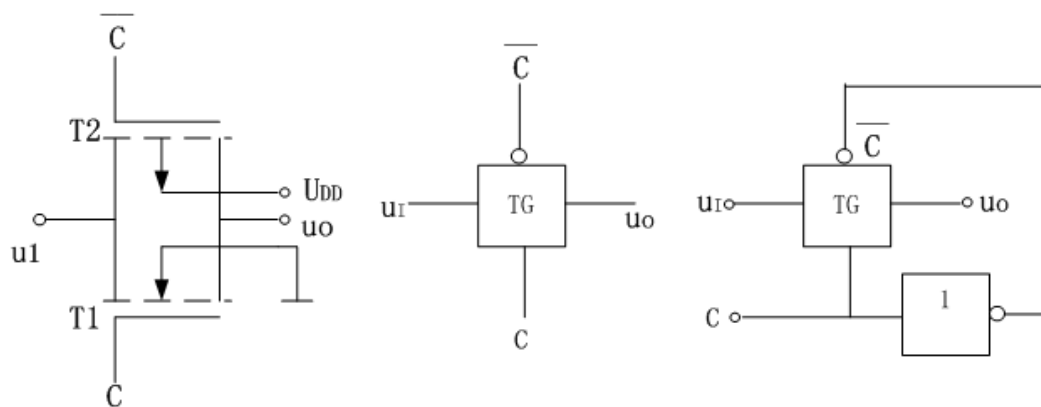
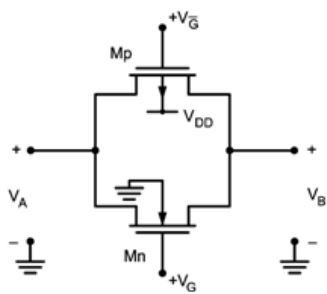
3. 或非门(NOR).

通过两 NMOS 并联、两 PMOS 串联，NMOS、PMOS 对的门分别共用不同的输入电压，可实现 NOR 门逻辑(输入都是 0, 输出 1; 只要输入存在 1, 则输出都是 0). 可以用来实现 NOR flash 或 NOR Logic.



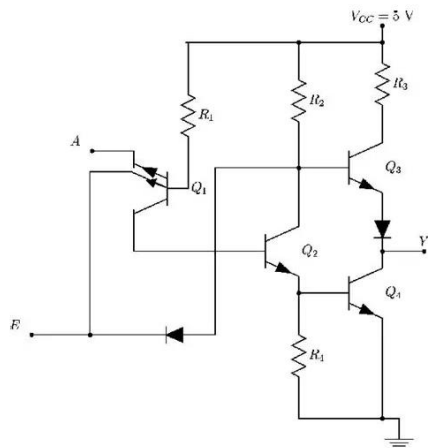
4. 传输门(Transmission Gate)

基于 CMOS 的传输门。



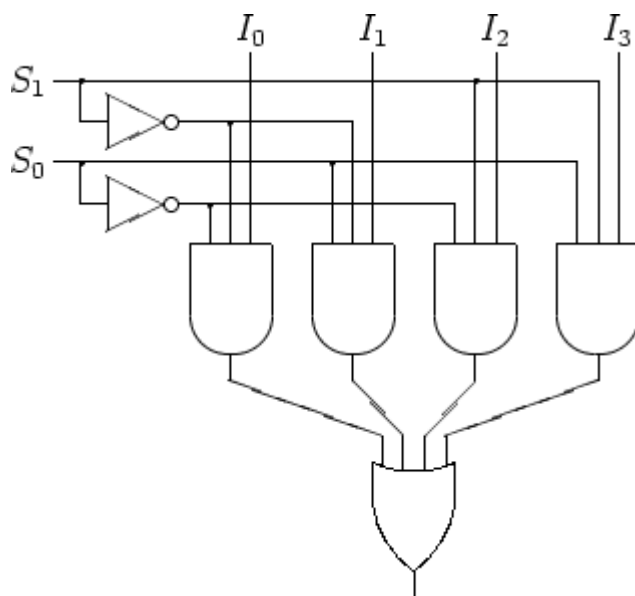
通过栅电压控制源漏的导通。对于明确要传输特定的电平的情况下，的确如此，但是对于既要传输 0 又要传 1 信号的情况就不同了。因为不同类型的 mos 管对不同信号的传输能力是不同的，如 nmos 传 0 信号是很完整的，传 1 信号，实际电压会低于 vdd；而 pmos 则相反。

5. 三态门



三态门基于输入信号控制电源及地信号的输出。屏蔽噪声，输出 0、1、高阻三态。

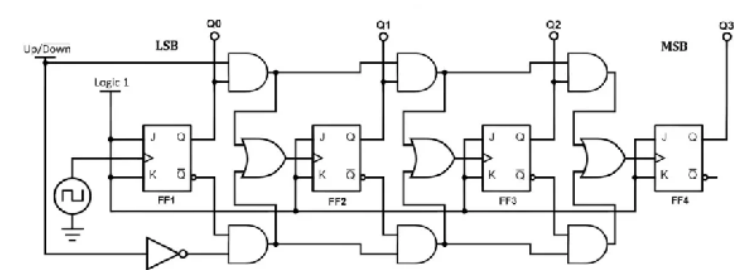
6. 多路选择器



可基于传输门或三态门设计。

7. 计数器

异步计数器。



8. 全加器

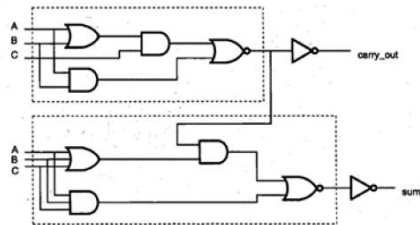
计算输入存在进位(Carry)的比特加法。

1bit 全加器。

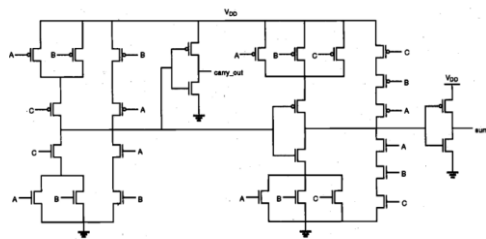
真值表。

INPUT			OUTPUT	
A	B	C	carry_out	sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

电路原理图。



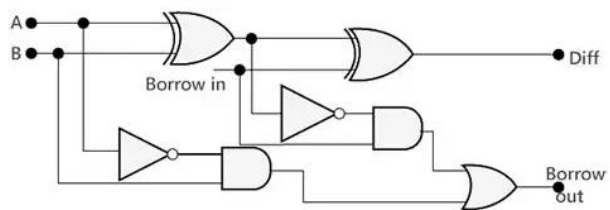
CMOS 电路。



注: 为防止 Vdd 与 Ground 之间短路, 可利用 栓锁(latch-up)效应保护机制进行防护.

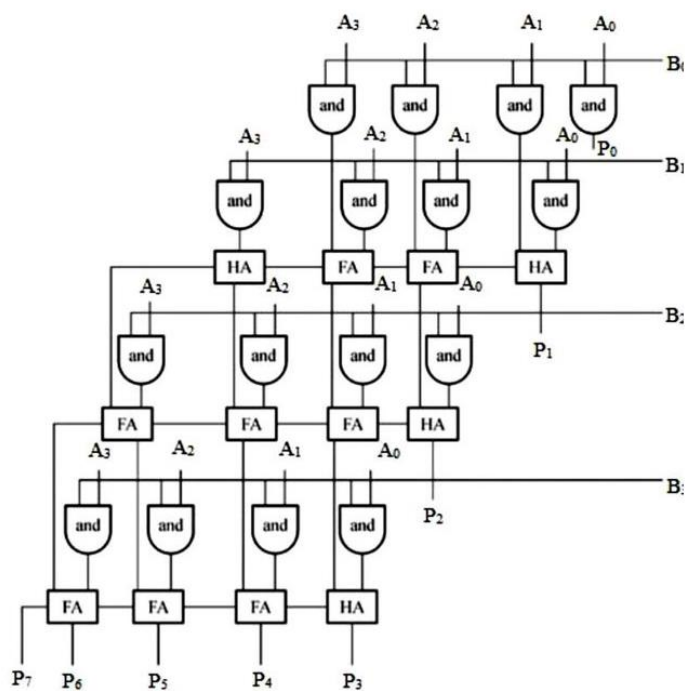
基于更多形式的串并联 NMOS、PMOS, 可定义全加器等. 将这些 NAND、NOR、ADDer 等结合起来, 通过控制信号控制, 可形成 ALU 模块.

9. 全减法器



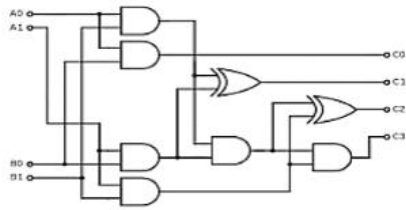
10.乘法器

4Bit 乘法器。

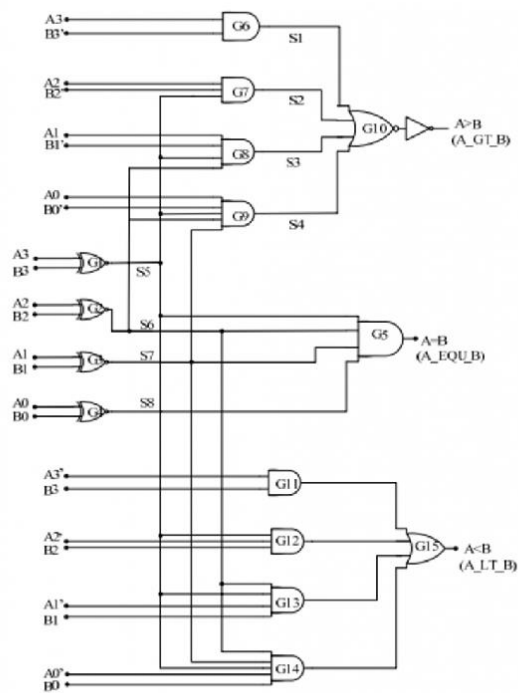


11.除法器

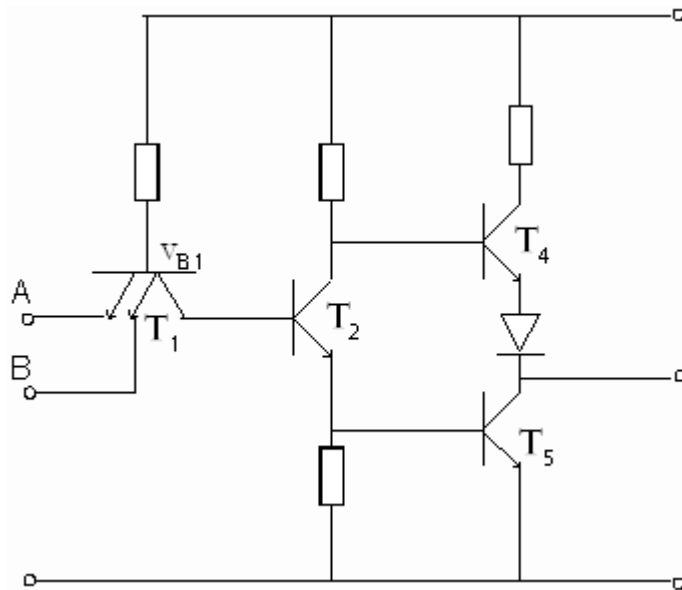
两比特二进制除法器。



12.数值比较器(Magnitude Comparator)



13.集电极开电路(OC, Open Collector)



二、 时序电路

时序电路基石，锁存器及时钟。

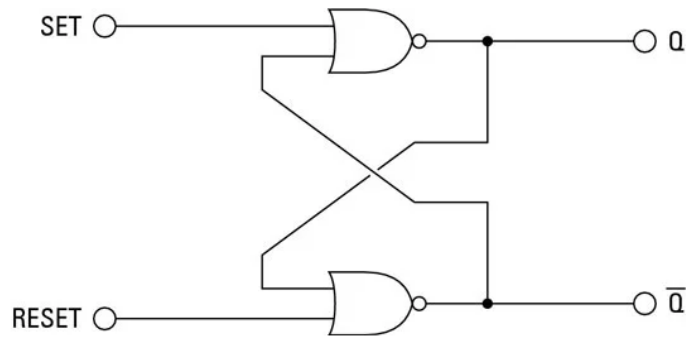
1. 锁存器

锁存器，基于两个或非(NOR)或者两个与非(NAND)门的置位-复位锁存器(Set-Reset Latch)，具

有置位(Set)、复位(Reset)两个输入，Q 及 $\overline{Q}$ 两个输出。当 S=0, R=0 时，输出维持不变，因此实现存储 1bit 的作用。

注，两个非门循环连接可以实现状态保持，但是无法进行输入输出。

基于与非门的锁存器。



2. 触发器

触发器基本类型。

R-S 触发器：具有置 0 、置 1 、保持功能。

D 触发器：具有置 0 、置 1 功能。

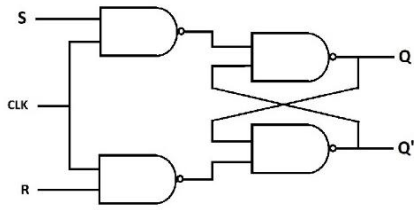
J-K 触发器：具有置 0 、置 1 、保持、翻转功能。

T 触发器：具有保持、翻转功能。

T' 触发器：具有翻转功能。

1) RS 触发器

RS 触发器(Flip-Flop)，时钟控制的 SR 锁存器。



电平锁存器(Level Sensitive Latch)、边沿锁存器(Edge Sensitive Latch)。

电平锁存器。

两个与非门 G1、G2 的输入、输出端交叉连接，即可构成基本 RS 触发器。所加的输入信号(低电平)称为触发信号，由它们导致的转换过程称为翻转。由于这里的触发信号是电平,因此这种触发器称为电平控制触发器。

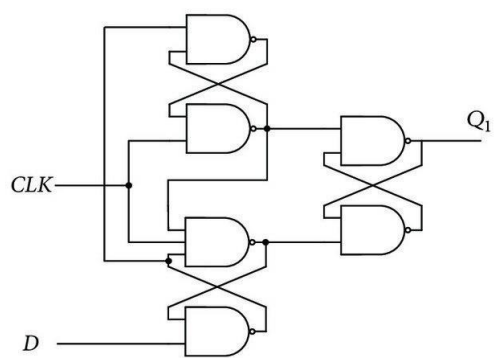
边沿锁存器。

两个锁存器串联构成。这样当 CLK 的前半个周期(高/低)来到时，必然是一个锁存器存了一个新的信号，而另一个锁存器保持着上一次存储的输入信号。当 clk 的后半个周期(低/高)来到时，之前刚存储过信号的则保持，而保持的则开始存储半个周期前的输入信号。

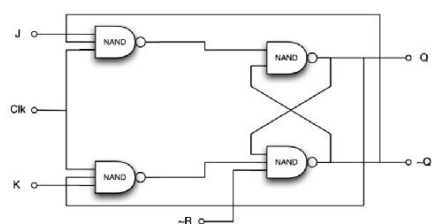
由于触发器的输出实际上是串联电路后面的锁存器的输出，因此从外部看起来就是当电平变化(半个周期)时，输入信号被捕获和输出了。

串联路径上的第一个锁存器是低电平控制使能或者相反，定义了边沿触发类型。比方，时钟或中断等信号高时，第一个锁存器更新存储，则意味着是下降沿沿触发，反之是上升沿触发。这同时定义了操作系统中控制中断边沿触发方式。

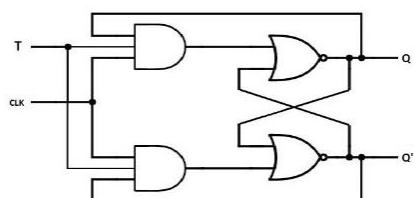
2) D 触发器



3) JK 触发器



4) T 触发器



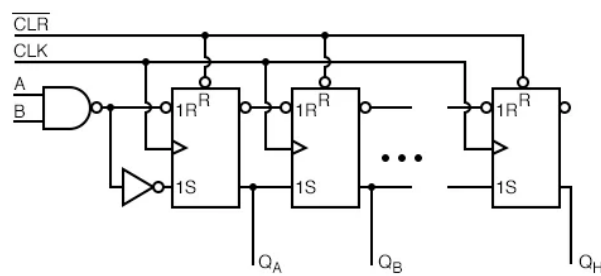
3. 寄存器

寄存器作用，访问控制多个值、存控数学运算的对象(Operands)、临时存记运算的中间结果。

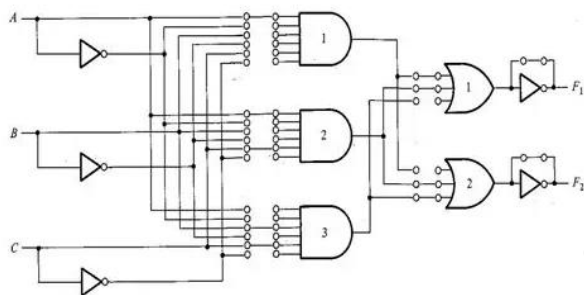
关键在保持状态记忆信息；同时，可进行输入输出。比方，利用使能信号，使得输入使能信号 1 时，寄存器存记输入；输出使能信号 1 时，输出状态到总线。

一个触发器能储存一位二进制代码。寄存器基于触发器组成的。一个触发器就是一个一位的寄存器，多个触发器就可以组成一个多位的寄存器。

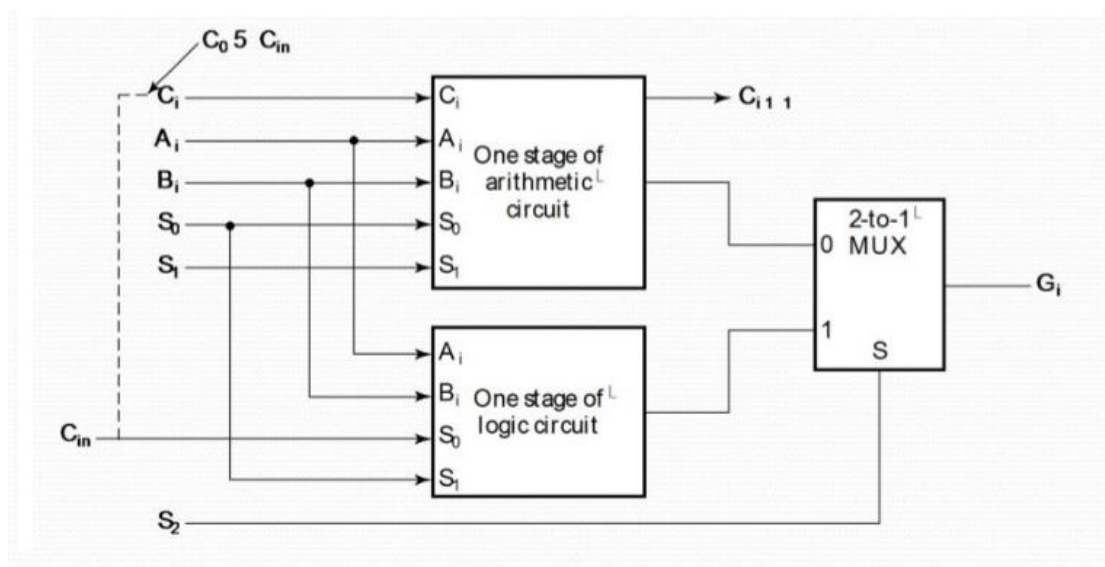
单入并出移位寄存器。



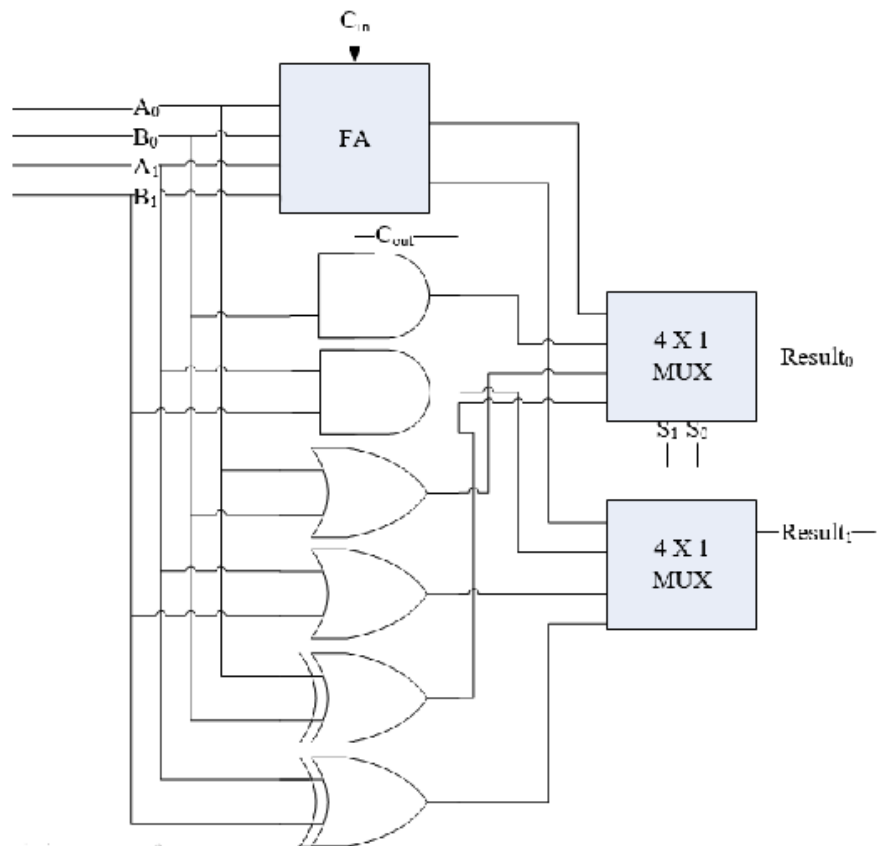
4. 可编程逻辑阵列(PLA)



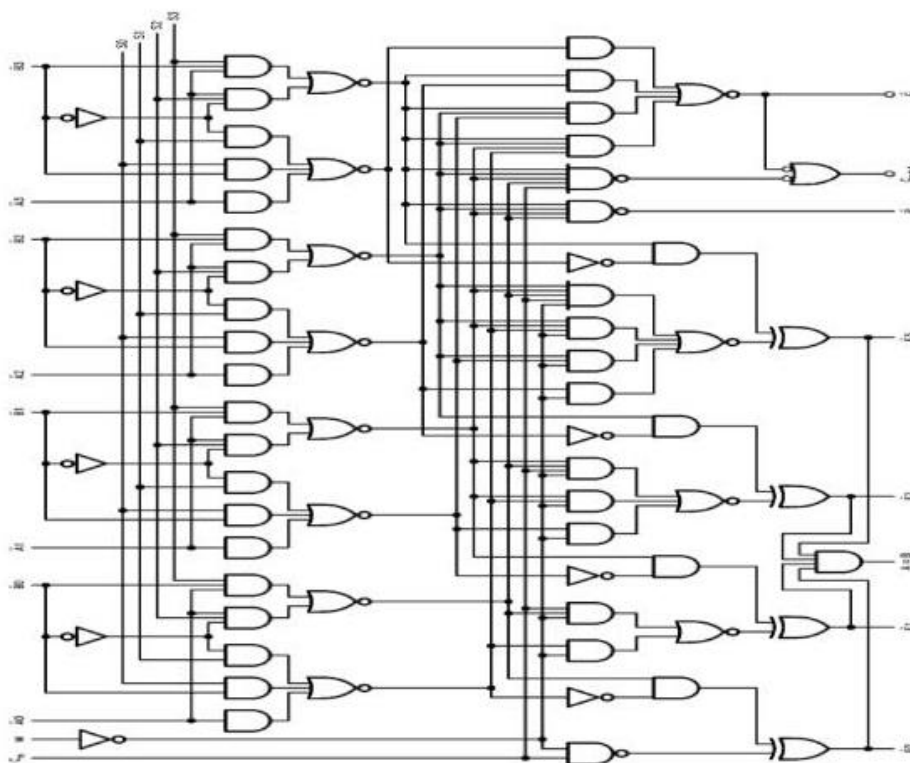
5. 算术逻辑运算单元 (ALU)



两比特加法、与、或、异或逻辑 ALU。



4-BIT ALU。



6. 时钟同步电路

时钟同步电路，基于时钟信号上升沿或下降沿同步改变电路状态的时序电路。

整个电路基于触发器，方便实现；计算时间确定(Deterministic)，易于实现实时功能；可避免竞态问题(Race Condition)；便于测试、调试。

问题，同步时时钟偏差(Clock Skew)、时序抖动(Timing Jitter)，不利于存在大规模状态迁移的电路设计，相对异步电路功耗大。

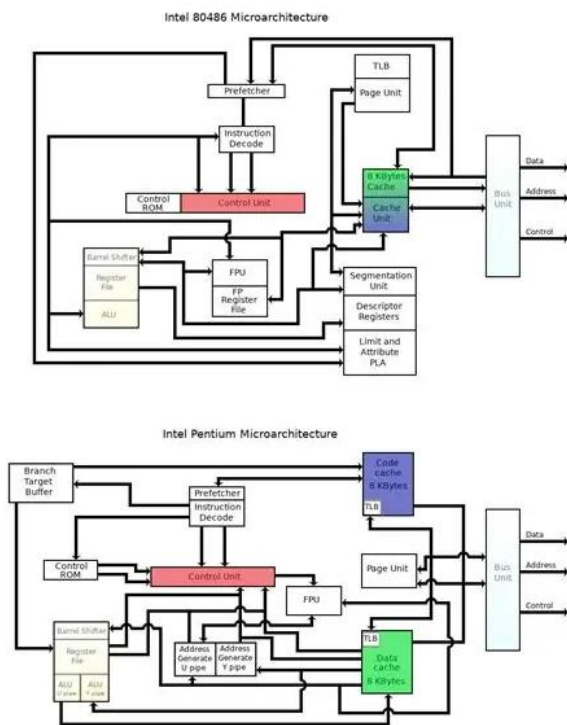
第四章 系统电路

本篇系统电路，基本对应半导体电路的第六层。主要分析不同的系统电路。

计算机性能是 CPU、存储、总线等整体能力的反映。大模型的训练、推理等更加剧了对这些系统综合性能的要求。

一、 计算系统

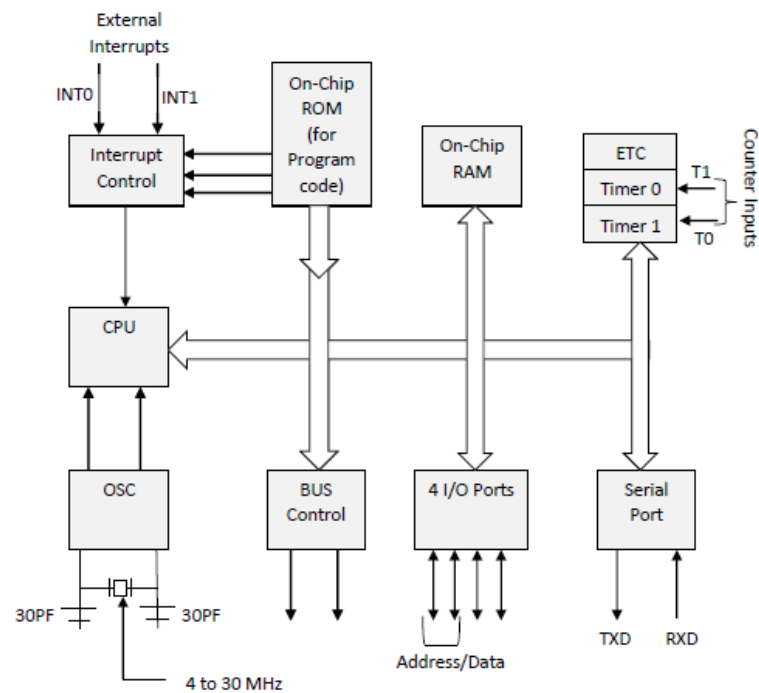
Intel Pentium 架构。



1. MCU

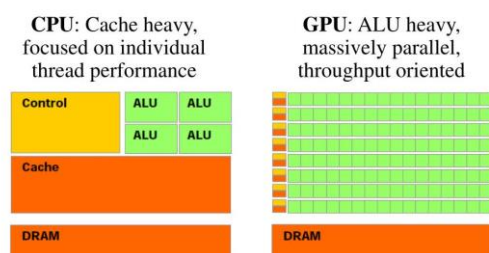
MCU，单芯片上集成了 CPU、存储、I/O 等模块的紧凑型集成电路系统，可用于汽车、机器

人、穿戴设备等领域。



框架图，51 MCU，基于哈佛架构(Harvard Architecture)，程序与数据分离存储。精简指令集(RISC)。支持 5 个软硬中断，定时器 Timer0、Timer1 中断，外部硬中断 INT0、INT1，串口通信中断。片上 ROM 存储程序，RAM 存储数据。地址总线用于定位存储地址，数据总线用于传输数据。振荡器(Oscillator)用于 CPU 时钟脉冲、内部计数器及定时器。4 个双向端口，用于连接外设。内部两个可编程计数器、定时器。

2. GPU



CPU 侧重于控制及缓存，GPU 侧重于大数据量及并行计算，因此 ALU 模块远多于 cpu。不过，计算单元依然是 ALU。

在 Nvidia G80 GPU 中，存在一系列多处理器(SM, streaming multiprocessors)，每个多处理器包括一系列 32bit 的 SIMD(Single Instruction Multiple Data)指令的处理器(SPs, scalar processors；每个 SP 对应一个 CUDA Core，执行一个线程)；每个时钟周期，每个多处理器对应的线程组(group of threads)执行同一指令。这组线程称为一个封装包(warp)，线程数量称为包的大小(warp size)。

Grid 的每个 Block 分为多个 Warps，每个 warp 通过一个多处理器(SM, streaming multiprocessor) 执行。也就是说每个 Block 的线程组可以划分为多个 warps。每个线程块通过一个多处理器(SM)执行，一个多处理器也可以同时执行多个 Blocks。

一个 GPU Core 包括一个 fetch/decode 模块，多个 ALU，多组执行环境(execution context)，每个执行环境对应一个指令；每组执行环境包括多个 context，每个 context 对应一个程序片段(fragment)。

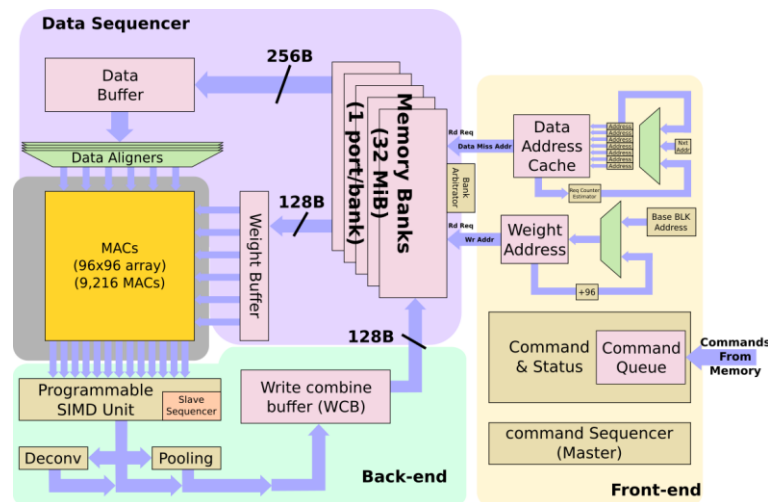
比方，6 个 ALUs, 4 组执行环境，每组执行环境 6 个 context；则每个 GPU Core 可以时分复用或交错并发(concurrent but interleaved)执行 4 条指令流(instruction streams), $4 \times 6 = 24$ 个程序片段。

一个 GPU Core 中的 ALUs 或 SPs 或 Cuda Cores 可以分成多组，每组共享取指令/译码模块，每组称为多处理器(SM)。

GPU 设计，Memory 分成多个 banks，每个 bank 共享地址及数据总线，可并行访问 banks。

3. NPU

Tesla NPU 框架图。



NPU 基于脉动阵列(Systolic Array)流水线并行框架实现大规模乘加(MAC)计算，适用于边缘设备大模型推理等场景。

注，脉动阵列，实质基于矩阵或网面(Mesh)形式布局大量计算单元(PEs, Processing Elements)，形成计算单元阵列；存储空间的数据周期脉动的加载到阵列进行计算，输出到存储空间；计算单元之间在行、列方向流水线计算、传递数据。

注，基于 Flynn 及 Kung 的计算机设计分类。

单指令单数据(SISD, Single Instruction Single Data)，单控制单元(CU, Control Unit)、单数据通路。比方，传统的冯·诺依曼 CPU。

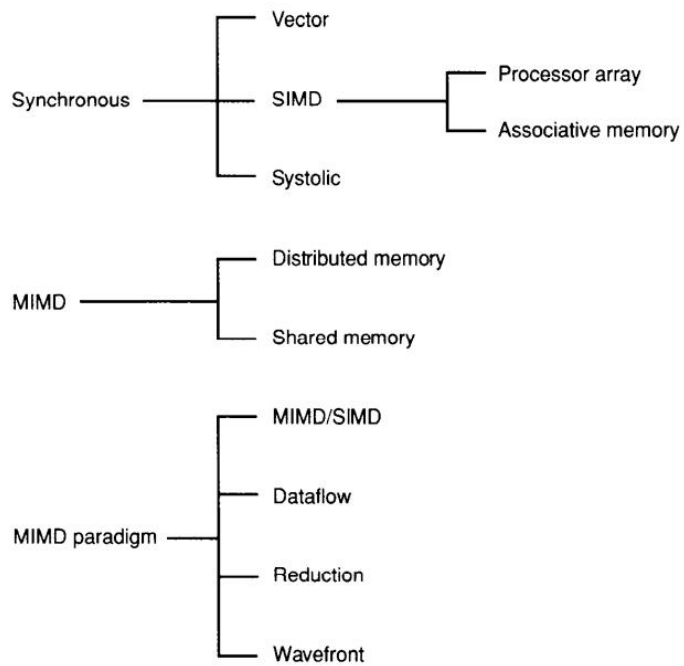
单指令多数据(SIMD, Single Instruction Multiple Data)，单控制单元，多数据通路。比方，Vector Processor, GPU(SIMT, Single Instruction Multiple Threads)。

多指令多数据(Multiple Instruction Multiple Data)，多控制单元，多数据通路。比方，Multi-Core CPU, Multi-CPU Systems。

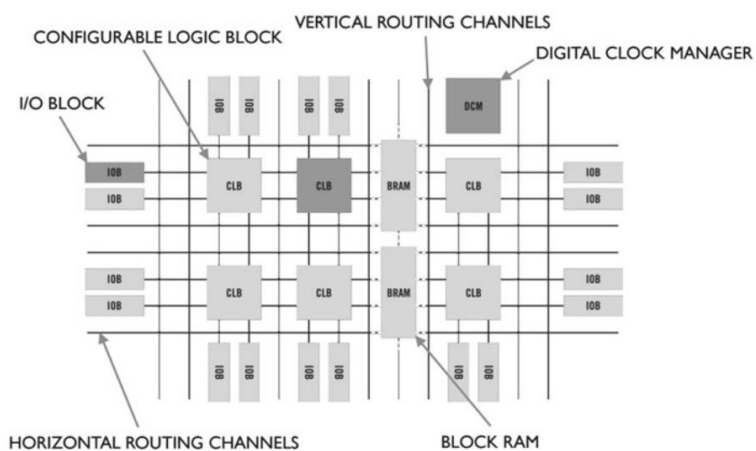
脉动阵列，数据到来时触发计算单元阵列计算。比方，NPU。

波阵列(Wavefront Array)，相当于通用、可编程、异步的脉动阵列。

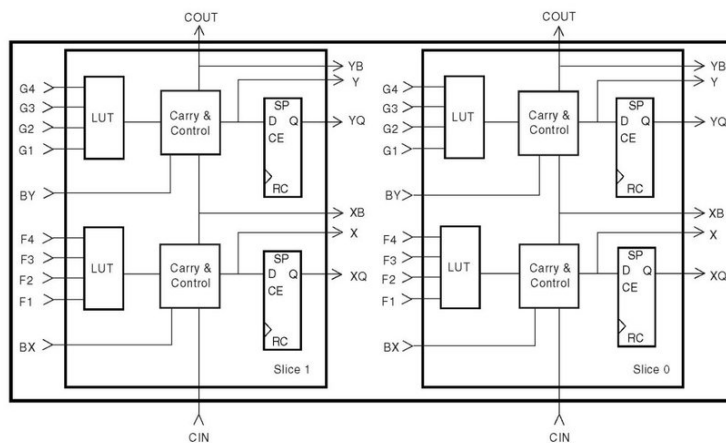
注，并行计算框架分类。



4. FPGA



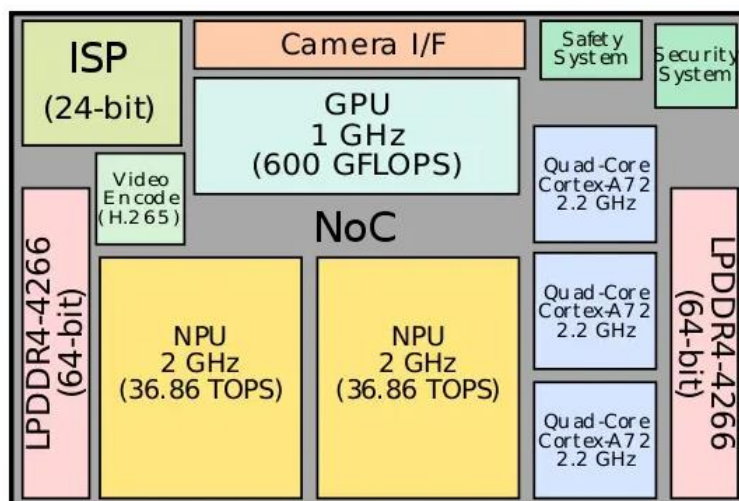
Xilinx FPGA 架构图。



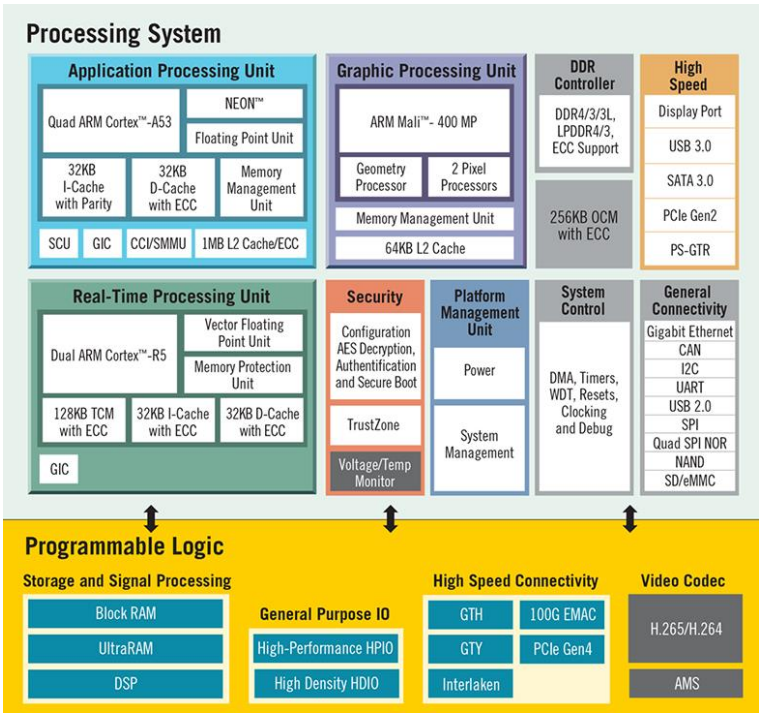
Xilinx Virtex-E FPGA CLB(Configurable Logic Blocks)框架图。

5. SoC

SoC 系统，CPU、GPU、NPU、FPGA 等都可集成在片上。



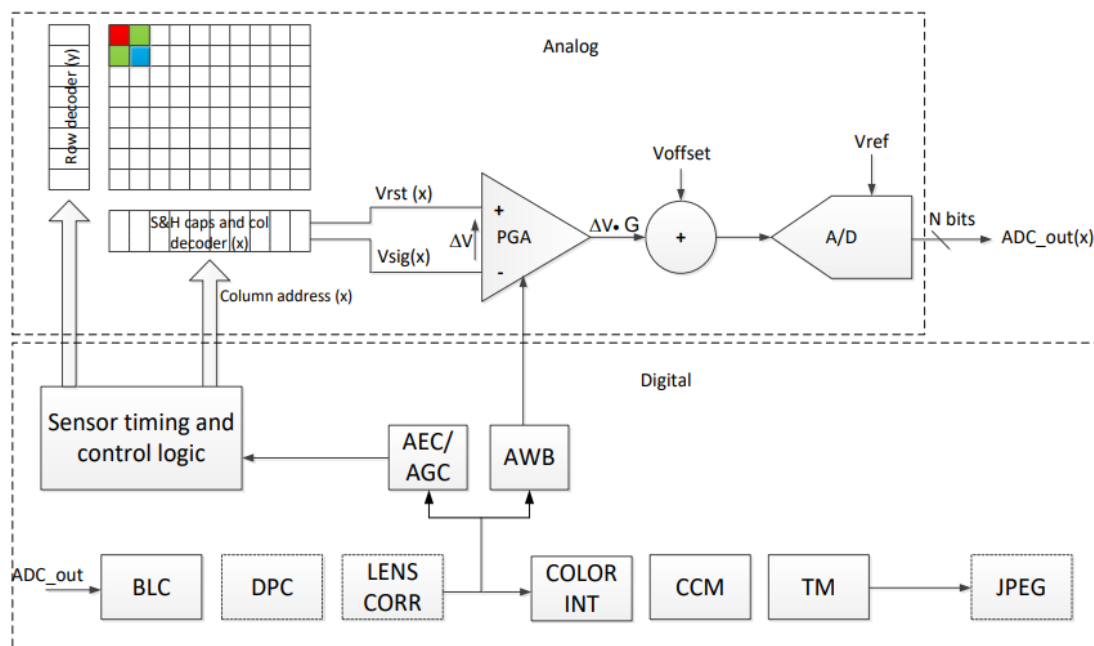
Tesla HW3.0 SoC 架构图。



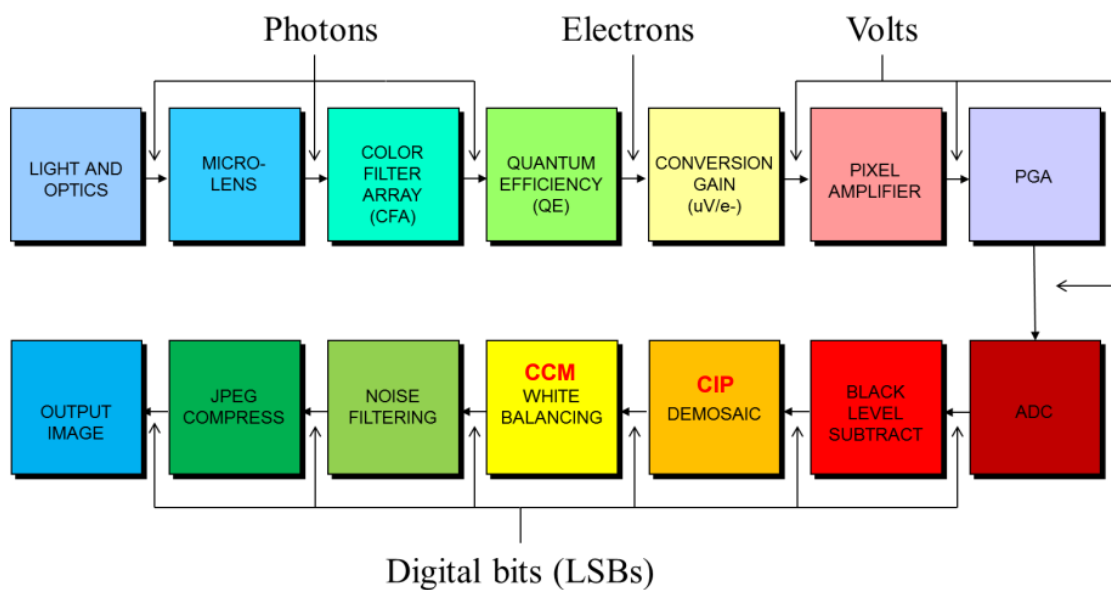
Xilinx Zynq SoC 架构图。

二、 图像信号处理(ISP)

CMOS 图像传感器框图。



数字相机信号链。

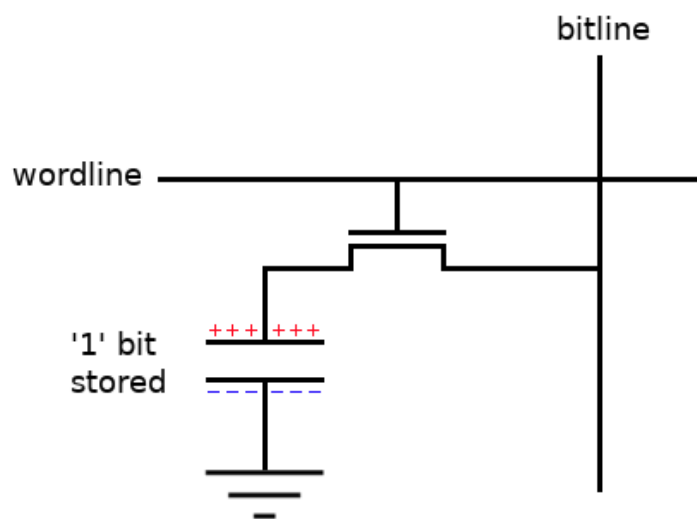


三、 存储系统

存储系统可分寄存器(Register)、缓存(Cache)、内存(Main Memory)、外存等。

1. DRAM

位存储单元(DRAM bit Cell)。



写操作。

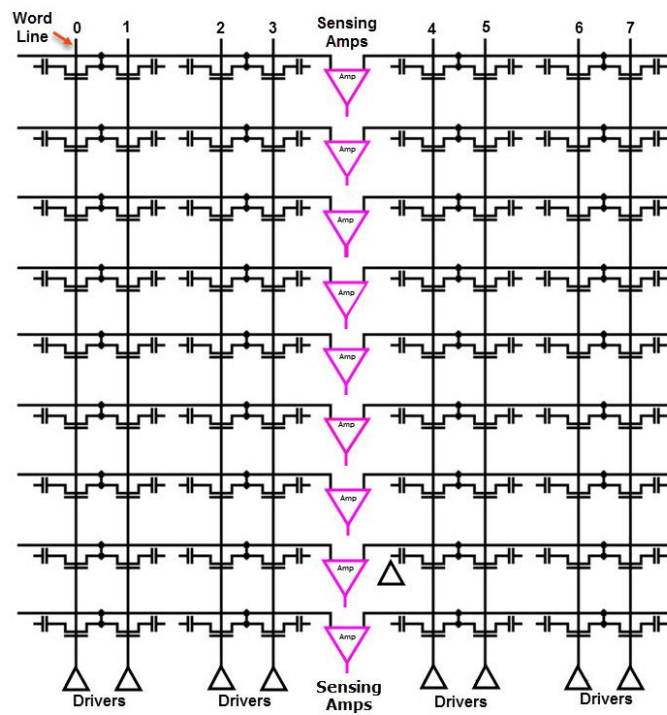
写 1。地址线(Wordline)高电平，晶体管连通；数据线(Bitline)高电平(1)，电容充电到高电平(1)。

写 0。地址线(Wordline)高电平，晶体管连通；数据线(Bitline)低电平(0)，电容放电到低电平(1)。

读操作。

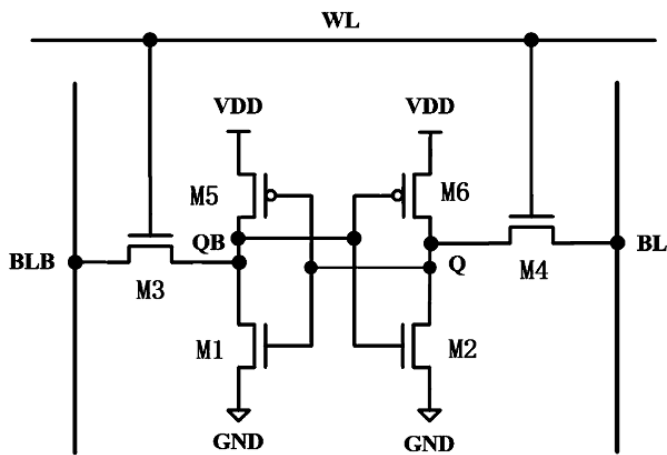
地址线(Wordline)高电平，晶体管连通；数据线(Bitline)保持与电容相同电平。

单元存储阵列(DRAM Cell Array)。



2. SRAM

位存储单元(SRAM bit Cell)。



基于 6 晶体管的 SRAM 位单元存储架构。

假定连接点 QB 低电平(0)，使得晶体管 M6 导通、M2 断开，因此，连接点 Q 高电平(1)。

写操作。

写 1。地址线(WL)高电平，晶体管 M3、M4 分别导通；数据线 BL 高电平(1)、反相数据线 BLB 低电平(0)；因此，连接点 QB、Q 保持不变。

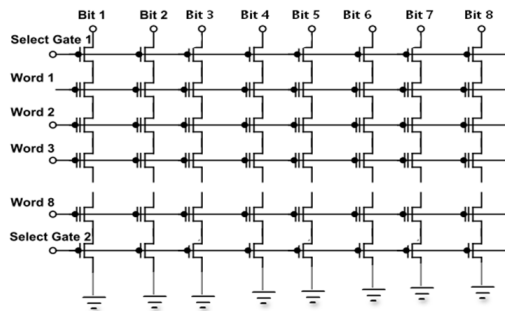
写 0。地址线(WL)高电平，晶体管 M3、M4 分别导通；数据线 BL 低电平(0)，因此，连接点 Q 低电平，使得晶体管 M5 导通、M1 断开，连接点 QB 高电平(1)，使得晶体管 M6 断开、M2 导通，维持连接点 Q 低电平(0)；反相数据线 BLB 高电平(1)，因此，连接点 QB 高电平，使得晶体管 M6 断开、M2 导通，连接点 Q 低电平(0)，使得晶体管 M5 导通、M1 断开，维持连接点 QB 高电平(1)。

读操作。

地址线(WL)高电平，晶体管 M3、M4 分别导通；数据线 BL、反相数据线 BLB 分别同连接点 Q、QB 电平。

3. Flash

1) NOR Flash



2) NAND Flash

在与非门(NAND)门的控制门或控制栅(control gate)极下、沟道上方之间添加一个绝缘体包裹的浮置门或浮置栅极(Floating gate)来实现电荷的存储.

源极及漏极连地, 控制门输入高电压, 基于 F-N 隧道效应(Fowler Nordheim tunneling)或量子隧穿效应, 沟道中的电子将穿过浮置栅极的绝缘层存入浮置栅极.

读取时, 控制门输入较低电压, 该电压与浮置栅极电荷抵消, 则 NAND 门开断, 输出 0, 否则 NAND 门闭合, 输出 1, 基于输出可判断浮置栅极存储状态.

进一步, 通过浮置栅极存储电荷数量的多少, 可将状态分为更多类型, 以表示更多 bit 数, 比方一个浮置栅极可以表示 2bit 信息.

形成 SLC(1bit)、MLC(2bit)、TLC(3bit)、QLC(4bit) 技术.

擦除, 在源极或漏极输入正高电压, 控制栅极输入负电压(或无电压), 通过形成的反向电场将浮置栅极的电荷拉走.

写及擦除时的量子隧穿效应都将损耗浮置栅极与沟道之间的氧化绝缘层, 使得电荷散逸, 数据错误.

四、 通信系统

载波(射频) + 基带(baseband, 信号) , 载波调制基带后的信息一般是带通信号(narrowband, bandpass)。

窄带信号公式(NarrowBand signal formulated)。

$$x(t) = \text{Re}C(t)e^{jw_ct} = A(t)\cos(w_ct + \phi(t))$$

等价形式

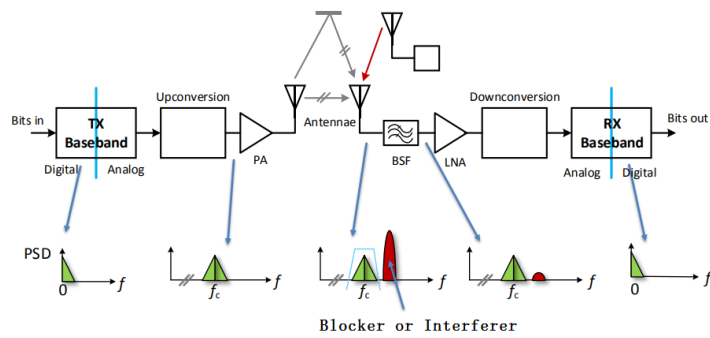
$$x(t) = a_I\cos(w_ct) + a_Q\sin(w_ct)$$

$$a_I(t) = \text{Re}C(t) = A(t)\cos(\phi(t))$$

$$a_Q(t) = \text{Im}C(t) = A(t)\sin(\phi(t))$$

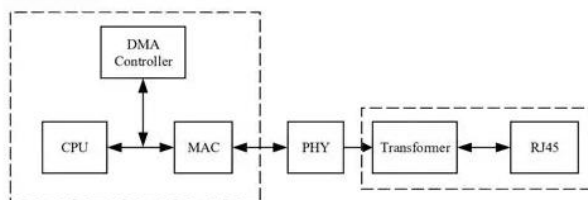
$C(t), A(t), \phi(t), a_I(t), a_Q(t)$ 都是基带信号(baseband signal)

1. 射频

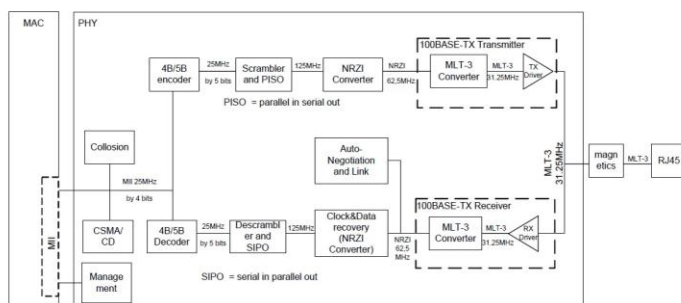


2. 以太网

总体框架。



PHY 框架。



五、 总线系统

总线是 CPU、存储器、I/O 设备等之间进行互连的公共线路集及网络框架。基本越接近 CPU 端，设备通讯速度越高。

1. 总线类型

1) 基于 CPU 连接

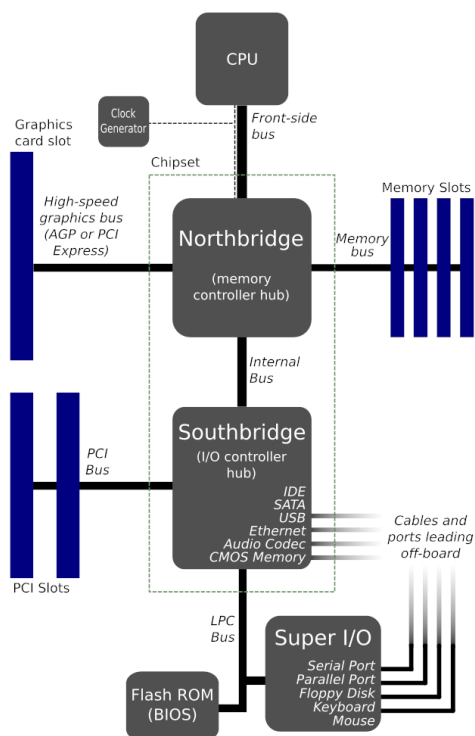
总线可分三层。

第一层，直连 CPU 的前端总线(FSB, Front Side Bus)或系统总线。

第二层，基于集线器(HUB) 仲裁机制，进行设备之间或设备与 CPU 通讯的高速外设总线(Peripheral Bus)。

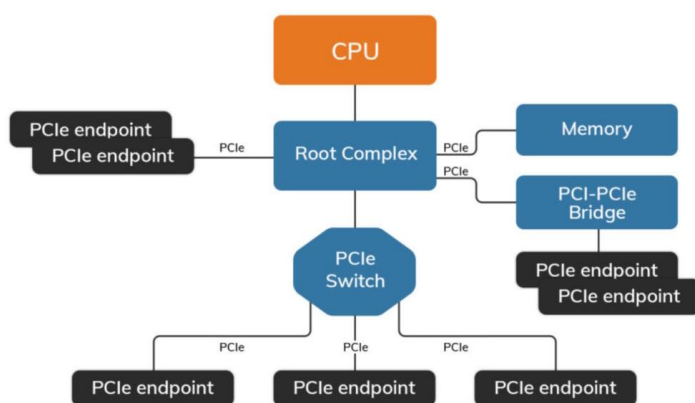
第三层，基于交换机(Switch)机制或主从(Master-Slaves)机制等进行设备通讯的中、低速外设总线。

早期计算机总线框架。



当前，北桥、南桥功能基本已集成到片上系统(SoC)中。

基于 PCIe 网络设计规程的总线框架。



综合节点(Root Complex)，类似北桥的功能。

2) 基于传输方式

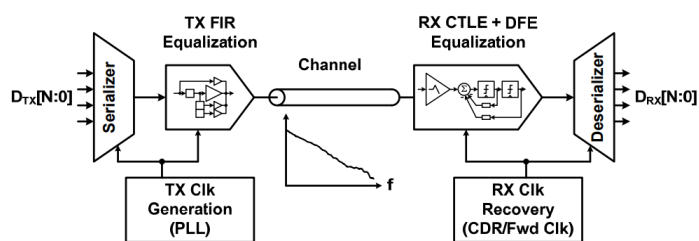
可分为并行总线(Parallel Buses)、串行总线(Serial Buses)。

并行总线传速快，不过多线路占据空间大，电磁干扰等使得不利于保持信号完整性，不适合长距离传输。

串行总线相对基于很少的线路，不过传速慢，实现需多层封装、浪费带宽。

当前，基于空间占用、信号完整性、距离等因素，CPU 内部多用并行总线，周边的多用串行总线。

高速连线路电路(High Speed Link Circuit)。



3) 基于信息形式

可分为数据总线(Data)、地址总线(Address)、控制总线(Control)。

地址总线常与数据总线共享(Shared Or Multiplexed)。

控制总线，控制时钟、总线仲裁(Arbitration)、读/写、中断等。

2. 串行总线

UART、I2C、SPI、USB 等。

3. 并行总线

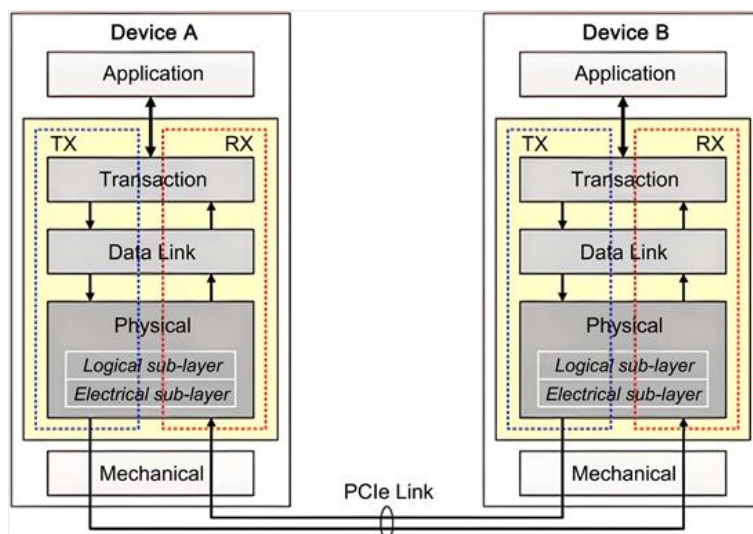
CPU 总线、ISA 等扩展总线、视频总线等。

4. PCIe

PCIe(Peripheral Component Interconnect Express), 实质类似 OSI 网络通信框架, 基于包交换分层模型, 实现端到端定义链路(Link)通讯。区别在于, 物理层基于多路全双工线路, 每路(Lane, 4 Wires)对应两对差分线(Differential Serial), 每对差分线(Two Wires)专用于读(Receive)或写(Transmit)。多个 PCIe 设备之间基于电路交换器(Switch)通讯。就差分方式来说, PCIe 与 USB 或双绞线 Ethernet 相似。

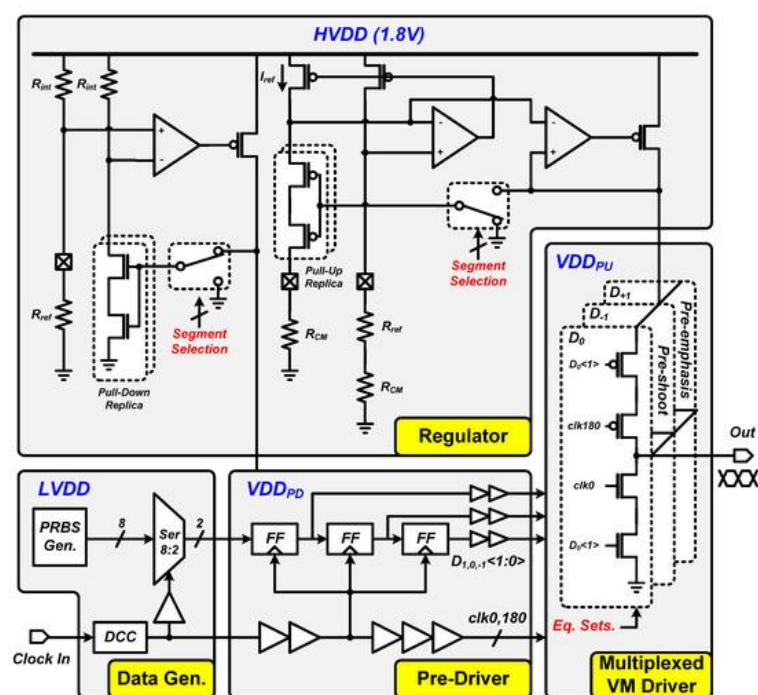
注, Lane, 类似高速路的一对双向车道。

相对于 PCI 基于总线共享(Shared Bus)、控制器仲裁机制, PCIe 基于包交换机制, 在软件层封装、拆包、识别, 物理层多路差分串行交换数据包, 实现组件点对点(Point To Point)高速互连, 具备更便利的扩展性。



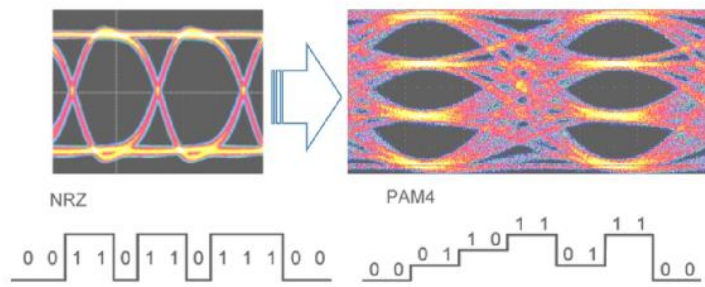
PCIe 应用层(Application), 自定义, 比方 Ethernet、SATA 协议等; 事务层(Transaction), 比方, 内存读写、外设读写、配置设备以及中断或功耗管理消息等; 数据链路层(Data Link), 进行数据包管理, 比方应答(ACK)机制、误差检测(CRC)、数据纠错(FEC, Forward Error Correction)等; 物理层, 模拟信号电平、时序, 线路、接头(Connectors)等。

PCIe Gen4 物理层传输器架构(PHY Transmitter Architecture)。



高速 PCIe PHY Transmitter 设计要注意高速率、兼容(Backward)、多抽头有限脉冲响应均衡(Multi-Tap FIR Equalization)实现、抑制码间串扰(Intersymbol Interference)、阻抗匹配防反射、电压域(Voltage Swing)可调等问题。

基于 NRZ(Non-Return-To-Zero)调制的 PCIe5 与基于 PAM(Pulse Amplitude Modulation)调制的 PCIe6 的数据信号比较。

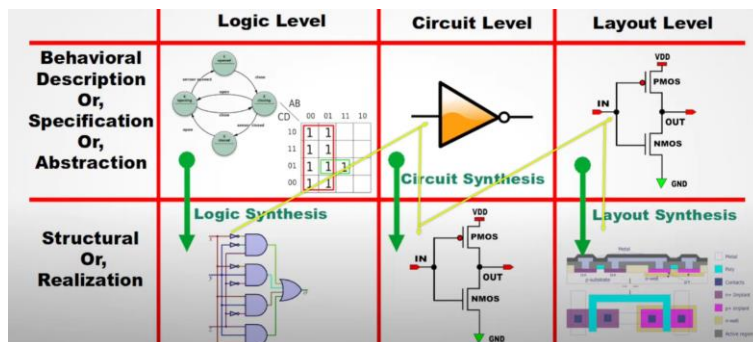
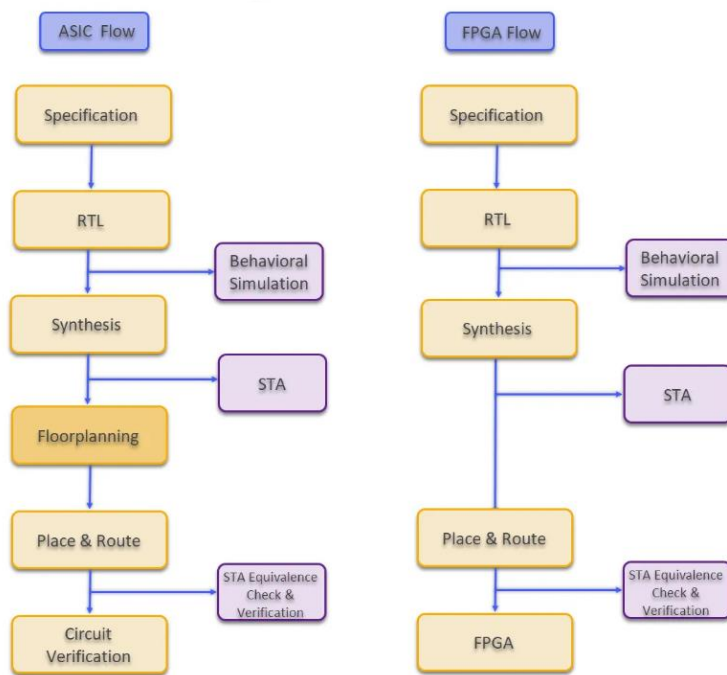


注，NRZ-Level，基于电平高低表示 BIT(1/0)时，连续相同 BITS 之间不用进行电平变换。
NRZ-Inverted，基于电平变换表示 1，不变表示 0。

PCIe5, 单时钟周期，1bit, 双电平模拟信号。

PCIe6, 单时钟周期，2bits, 四电平模拟信号。

第五章 芯片设计



一、 前端设计

1、定义架构

包括指令集数量、寄存器数量、内存地址空间大小等;

2、设计指令集架构(ISA, Instruction Set Architecture)

包括算术计算、逻辑计算、跳转(branch)、内存访问指令等;

3、设计微架构(microarchitecture)

包括设计执行单元、寄存器文件、数据缓存、内存层级体系(memory hierarchy)等;

4、RTL 设计

可利用 Cadence Stratus or Siemens' Catapult 等高级综合工具(HLS, high-level synthesis)将系统的 c/c++语言描述转成 RTL 描述;

基于 verilog 等编程写 RTL(Register Transfer Level)程序, 实现寄存器层级的抽象架构设计;RTL 编程工具, 比方, VHDL, Verilog or SystemVerilog.

利用 VCS (VLSI Designer's Choice) from Synopsys or ModelSim from Siemens or Xcelium from Cadence 等仿真器工具测试及验证 verilog 编程实现的 CPU;

利用 Cadence's JasperFormal/JasperGold or Synopsys' VC Formal/SpyGlass, or Siemens LedaRTL/Questa-Formal-Verification 等形式化验证工具进行形式验证(formal verification).

二、 后端设计

1、综合(synthesis)

利用综合(synthesis)工具将 verilog 程序转成门级表示或描述(gate-level representation or description)或门级网表(gate-level netlist);综合用的各类 EDA 工具, 比方, Synopsys Design Compiler/Fusion Compiler or Cadence Genus. or Synopsys Precision RTL(entry-level FPGA synthesis) or Oasys RTL (For IP Qualification).

网表质量检查、逻辑等效检查、时序检查等(Netlist Quality Checks, Logic Equivalence Checks

(LEC) & Timing Checks)

时序分析(timing analysis)，确保处理器以期望的时钟频率运行;

仿真验证(simulation and validation)，对综合后的设计进行仿真，确保门级运行正确;

2、Place and Route

利用布局布线工具(place and route)将门级网表映射到 FPGA 器件进行原型验证或生成用于制造流片的 GDSII 文件，确定芯片的物理布局(physical layout);

3、制造及测试(prototyping and testing)

发现仿真过程未发现的问题;

三、 智能设计

Google DeepMind Alphacore 基于类似 Alpha Go、Alpha Zero 的方法，在空间栅格中布局、布放电路元件，利用边图神经网络(Edge-Based Graph Neural Network)预测元件之间的互联关系。

1. Alpha Zero

基于蒙特卡洛树搜索(MCTS, Monte Carlo Tree Search)强化学习算法，自驱动(Self-Play)训练神经网络决策模型。

蒙特卡洛树搜索算法，基于叶节点可扩展的状态树，利用决策模型预测当前状态动作概率，选高概率动作对应的状态分支仿真，计算的奖励值反馈回当前状态。过程步骤，选择(Selection)-扩展(Expansion)-仿真(Simulation)-反向传播(Backpropagation)。

2. 图神经网络

图神经网络是对具有置换不变性的图(graph)的所有属性(attributes)的最优变换。

图像(image, 像素作节点, 像素与周边相邻像素的连接作边)、文本(text, token 作节点, 相邻 tokens 之间连接作边)、分子结构(原子作节点, 共价键(covalent bond)作边)、社交网络、机器学习模型、程序代码、数学方程(变量是节点, 运算是边)都可以看作图。

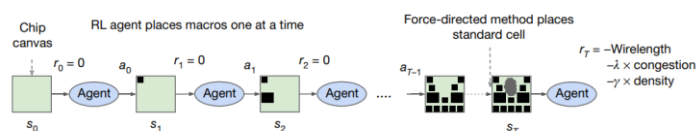
基于图(graph)数据的预测任务可分三类, 1、预测整个图(graph)的特性(property) graph-level, 比方, 图(graph)中多少个闭环; 2、预测图(graph)中每个节点的特性 node-level, 比方, 图(graph)中节点对应的标签; 3、预测图(graph)中每条边的特性 edge-level, 比方图(graph)中人或物节点之间的关系。

基本的图神经网络, 比方, 多个多层感知器(MLP); 每个多层感知器分别应用于图、节点、边向量(V,E,U)以学习得到相应的图嵌量、节点嵌量、边嵌量, 这些嵌量定义生成新的图。

对于节点分类任务, 基于节点的嵌量, 应用线性分类器(Linear Classifier)进行预测。节点中缺乏信息, 信息存在于边中的情况, 一般需通过池化技术(pooling operation)从边搜集信息, 以提供给节点, 再进行预测。

类似的, 对于边或整图的分类预测, 基于节点信息进行池化计算(与 CNN 中的全局平均池化层相似, Global Average Pooling layers), 得到边信息或图信息, 再进行分类预测。

3. AlphaChip



框架图, 左侧, 基于芯片画布(Canvas)状态(S_0), 决策模型(Agent)预测动作(a_0), 放置模拟块, 依此执行; 右侧, 基于力导向方法(Force-Directed Method)放置真实元件(Cell), 每次更新(Iteration)基于线长(Wirelength)、电路拥塞(Congestion)、密度的加权计算奖励(R), 以优化模

型。

注，力导向算法，或弹簧放置器(Spring Embedders)，基于图框架，利用物理力(Physical Forces)排布图中的节点及边，节点之间相互排斥，连接节点的边等价作弹簧，生成对称、无交叉布局(Cross-Free Layout)的平面图(Planar Graph)。

第六章 量子计算

量子理论尽管存在争议，不过基于数学逻辑分析、工程实践验证的量子计算已在飞快发展。

当前，不只是专用量子计算在迅速发展，通用量子计算也在展现极大的可能。

量子力学(Quantum Mechanics)原理，利用微观粒子(电子、光子、离子等)的量子态叠加、量子纠缠、量子退相干(Decoherence)等量子现象实现对数据的存储、传输及运算的量子计算机在解决复杂问题时有着极大的优势，可极大减少计算时间。

基于计算通用性，量子计算机分为通用型与非通用型。基于计算模型，量子计算机可分为基于门的(Gate-based)、绝热(Adiabatic)、拓扑(Topological)、模拟(Analog)量子计算机及基于量子退火(Quantum Annealing)等类型。

量子计算机中，数据通过量子比特存储、传递，量子比特可以是 0、1 状态的叠加，数学形式可表示为 0 与 1 的概率的线性组合或列向量。

量子比特可通过核磁共振(NMR)、超导、离子阱(Ion Trap)、线性光学(Linear Optics)等方法进行物理实现。

目前，通用量子计算机很多基于门模型(Gate-Model)，利用超导(IBM、Google、Rigetti、Quantum Circuits 等)与离子阱(霍尼韦尔、IonQ 等)等方法实现。

量子计算应用可通过专有量子计算机或 Microsoft Azure、AWS 云等获取量子计算服务。

一、量子定义

量子具有叠加态(Superposition)、干涉(Interference)、纠缠(Entanglement)、不确定(Non-Determinism)、不可克隆(Non-Clonability)等特性。

基于量子叠加态(Superposition)，量子状态同时存在 0、1 状态。

量子干涉(quantum interference)决定了量子状态塌缩概率。

纠缠态，量子状态彼此相关，无法单独表示。

不确定，量子状态不可测定。

不可克隆，量子状态不可复制(No-Cloning Theorem)。

1. 量子比特定义

量子比特信息可通过微观粒子的两个正交状态(Spin、Polarization、Position、Energy Level 等)的编码进行定义。

量子比特状态定义。

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \{\alpha, \beta \in \mathbb{C}\}$$

" $| \rangle$ "是量子机制中用于标示量子状态的标准符号，在量子力学狄拉克符号(Dirac notation)中称为列向量(column vector)。

标准正交基(Orthonormal Basis)，" $|0\rangle$ "与" $|1\rangle$ "，

$$|0\rangle = [1,0]^T, \quad |1\rangle = [0,1]^T$$

复数 α 与 β 的模的平方和等于 1，表示量子比特状态概率和等于 1。

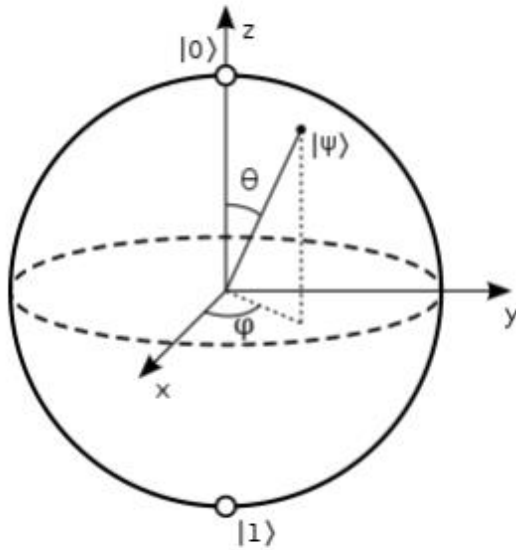
利用复数的极坐标定义，对量子比特状态定义等式进行变换

$$|\psi\rangle = r_0 e^{j\varphi_0} \cdot |0\rangle + r_1 e^{j\varphi_1} \cdot |1\rangle$$

$$e^{-j\varphi_0} \cdot |\psi\rangle = e^{-j\varphi_0} \cdot (r_0 e^{j\varphi_0} \cdot |0\rangle + r_1 e^{j\varphi_1} \cdot |1\rangle) = r_0 \cdot |0\rangle + r_1 e^{j(\varphi_1 - \varphi_0)} \cdot |1\rangle$$

$$|\psi\rangle = \cos \theta \cdot |0\rangle + e^{j\varphi} \sin \theta \cdot |1\rangle$$

因此，单量子比特的几何示意可通过 Bloch Sphere 定义。



$\theta \in [0, \pi)$ ，对应 Bloch Sphere 纬度、量子比特状态塌缩概率。 $\phi \in [0, 2\pi)$ ，对应 Bloch Sphere 经度、量子比特两个状态之间的相对相位，与量子比特状态向量的旋转相关。

基于 Bloch Sphere 可较直观的理解单量子比特状态。

当测量量子比特状态时，量子状态 $|\psi\rangle$ 塌缩向北极或南极，塌缩的概率与 θ 相关。 θ 是直角、使得量子状态箭头位于赤道时，量子状态塌缩向北极与南极的概率是 50%。

双量子比特状态定义，张量积运算。

$$\begin{aligned} |\psi_1\rangle \otimes |\psi_2\rangle &= [\psi_{1,0}, \psi_{1,1}]^T \otimes [\psi_{2,0}, \psi_{2,1}]^T \\ &= [\psi_{1,0}\psi_{2,0}, \psi_{1,0}\psi_{2,1}, \psi_{1,1}\psi_{2,0}, \psi_{1,1}\psi_{2,1}]^T \end{aligned}$$

$$|\psi_i\rangle = \alpha|0\rangle + \beta|1\rangle = \alpha[1,0]^T + \beta[0,1]^T = [\alpha, \beta]^T = [\psi_{i,0}, \psi_{i,1}]^T$$

量子状态张量积运算的等式说明，n 个量子比特状态(n-qubit state)等价于一个具有 2^n 个元素的列向量。

2. 量子叠加态(Superposition State)

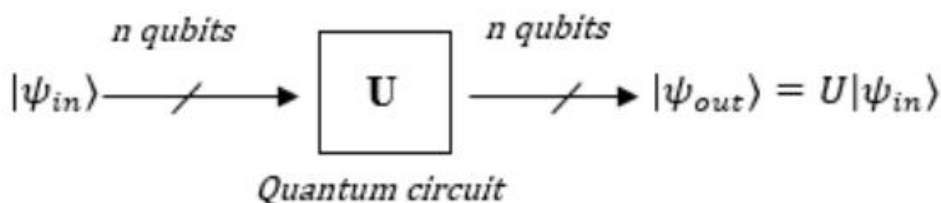
单量子叠加态。

$$|\psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

双量子叠加态。

$$\begin{aligned}
|\psi_1\rangle|\psi_2\rangle &= \left\{ \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle \right\} \left\{ \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle \right\} \\
&= \frac{1}{2} \{ [1,1]^T \otimes [1,1]^T \} = \frac{1}{2} [1,1,1,1]^T \\
&= \frac{1}{2} \{ [1,0,0,0]^T + [0,1,0,0]^T + [0,0,1,0]^T + [0,0,0,1]^T \} \\
&= \frac{1}{2} |0\rangle|0\rangle + \frac{1}{2} |0\rangle|1\rangle + \frac{1}{2} |1\rangle|0\rangle + \frac{1}{2} |1\rangle|1\rangle
\end{aligned}$$

3. 量子运算框架



二、 量子电路

量子通过基于量子门(Quantum Gate)与量子线路(Quantum Wires)互连定义的量子电路实现运算。

1. 单量子比特门(Single Qubit Gate)

包括 X 门, Z 门, Hadamard 门等。

X 门, 比特非, 类似经典计算机非门, 支持线性运算, 要求表示该门的矩阵是么正矩阵(Unitary Matrix)

$$\begin{array}{c} \alpha|0\rangle + \beta|1\rangle \\ \hline \boxed{X} \\ \hline \beta|0\rangle + \alpha|1\rangle \end{array} \quad X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Z 门, 相位非, $|0\rangle$ 不变, 将 $|1\rangle$ 变为 $-|1\rangle$ 。

$$\begin{array}{c} \alpha|0\rangle + \beta|1\rangle \\ \hline \boxed{Z} \\ \hline \alpha|0\rangle - \beta|1\rangle \end{array} \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

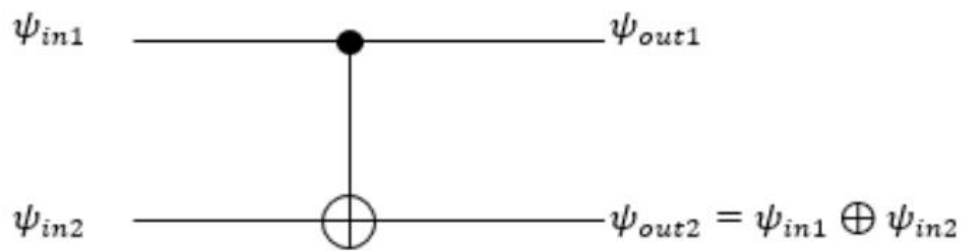
H(Hadamard Gate)门, 基于 X 门与 Z 门的组合, $(X+Z)/\sqrt{2}$, 将 $|0\rangle$ 转换为 $(|0\rangle + |1\rangle)/\sqrt{2}$, 将 $|1\rangle$ 转换为 $(|0\rangle - |1\rangle)/\sqrt{2}$, 对量子态运行两次 Hadamard 门运算等于原状态。

$$\begin{array}{c} \alpha|0\rangle + \beta|1\rangle \\ \hline \boxed{H} \\ \hline \frac{\alpha+\beta}{\sqrt{2}}|0\rangle + \frac{\alpha-\beta}{\sqrt{2}}|1\rangle \end{array} \quad H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

2. 多量子比特门(Multiple Qubit Gate)

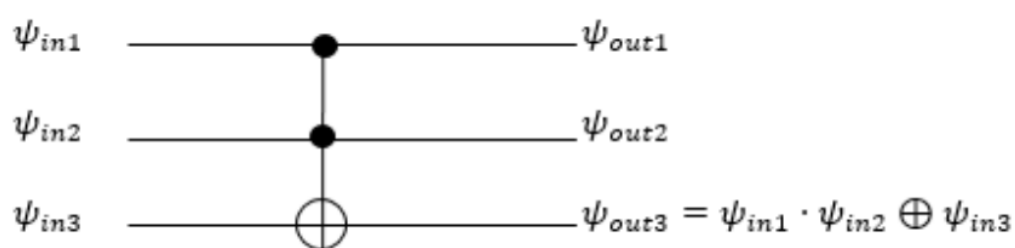
控制非门(cNOT, Controlled-NOT Gate), 类似异或门(XOR Gate), 第一个比特状态 $|1\rangle$ 时, 第二个比特取非转换。

ψ_{in1}	ψ_{in2}	ψ_{out1}	ψ_{out2}
0	0	0	0
0	1	0	1
1	0	1	1
1	1	1	0



可控控制非门(ccNOT ， Controlled-Controlled-NOT Gate, Toffoli Gate)，类似与非门(NAND Gate)，作为输入的第 3 个量子比特状态等于 1 时，作为输出的第 3 个量子比特状态等于输入的第 1 个量子比特状态与第 2 个量子比特状态相与的非计算。

可控控制非门及真值表。



ψ_{in1}	ψ_{in2}	ψ_{in3}	ψ_{out1}	ψ_{out2}	ψ_{out3}
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	1
1	1	1	1	1	0

三、 量子算法

量子应用算法，包括 Deutsch 算法，Deutsch-Jozsa 算法，判别常函数与平衡函数，离散傅立叶变换算法(Discrete Fourier Transform Algorithms)-Shor 分解算法。量子搜索算法-Grover 算法，量子模拟算法等。

四、 量子计算机

印刷电路板篇

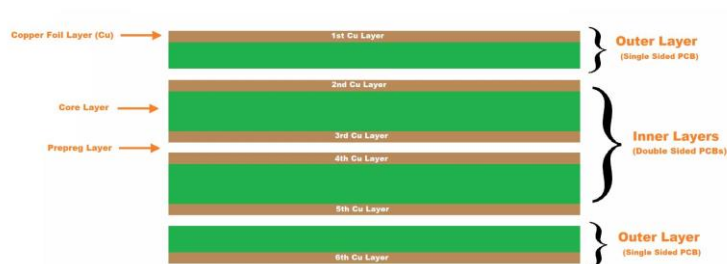
印刷电路板的实质，基于多层电路形成系统电路，利用三维空间便于布局布线的同时，尽量减少空间占用、防止电磁干扰、维护信号完整性。

开发过程可分为原理设计、布局布线、生产制造等。

信号电源完整性主要涉及电路板的信号完整性、电源完整性、电磁兼容/干扰、时序、标准兼容、热管理等。

第一章 印刷电路板

多层电路板。



一、 原理设计

人形机器人电路模块，电池管理系统、感知规划决策交互系统、控制系统、通讯系统等。

电池管理系统，管理电池密集阵，对系统各模块供电。

感知规划决策交互系统，基于多模态大模型及视觉语言动作模型进行交互、推理、规划、决策等。

控制系统，控制机器人关机执行器等。

通讯系统，基于射频电路连接云计算平台、进行在线更新(OTA)等。

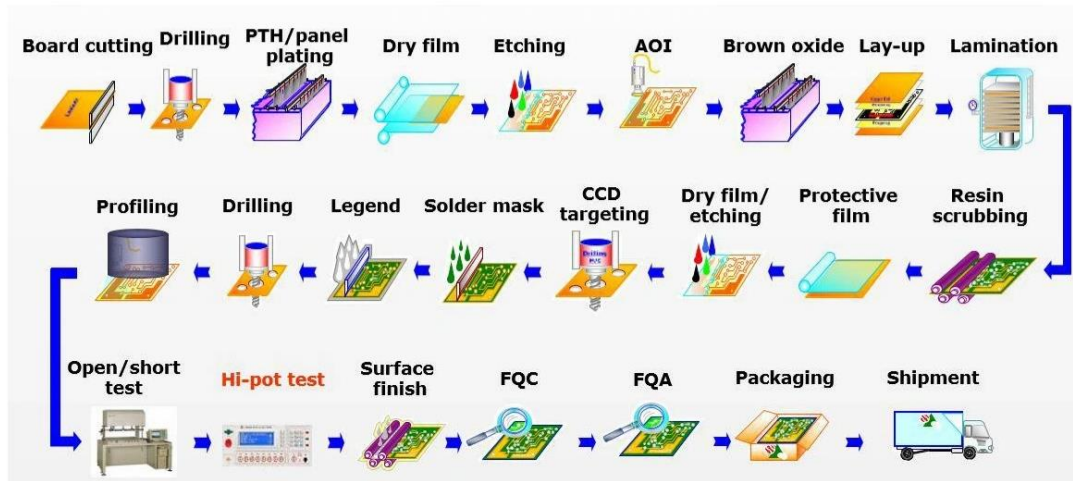
基于机器人平台环境(Embedded)及未来发展，决策交互系统应尽量选用集成 GPU、NPU，算力达到 Tesla HW3.0 级别的 ARM SoC。或暂时用 Intel I7 + Nvidia Jetson Orion 系列等。

注，Tesla HW5 算力预估 2000TOPS。

二、 布局布线

基于元器件数量、不同过孔(VIA)对电路板的作用、成本等因素，电路板一般 10 层左右便可。

三、 生产制造



生产制造主要步骤，内层干膜(Inner Dry Film)，利用贴膜、曝光显影、蚀刻(Etching)等工艺转移电路图象到铜电路板；压合(Lamination)；钻孔(Drilling)，通孔(Through Vias)、盲孔(Blind Vias)、埋孔(Buried Vias)、微孔(Micro Vias)等；外层干膜(Outer Dry Film)；阻焊(Solder Plating/Welding Resistance)；丝印(Silk Screen)；测量、测试等。

第二章 信号完整性

信号完整性，确保源信号通过传输线或印刷电路板到达接收端时的质量，以便可进行解析。是集成电路、集成电路封装、印刷电路板设计的重要部分。

信号完整性，实质是基于对信号时频、导体阻抗、非导媒介、电磁场效应等特性的分析，设计电路，避免信号传输时损失、变形等。

一、 物理基础

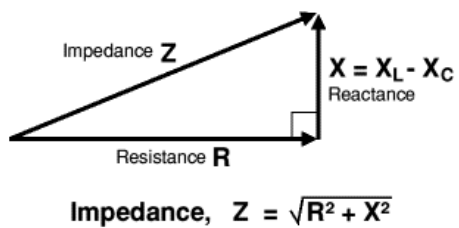
1. 信号模型

时域信号，不同频率正弦、余弦函数的和形式。

频域信号，基于拉普拉斯变换信号到频域，使时域信号的卷积(Convolution)计算转变成频率的乘运算。

2. 阻抗模型

阻抗(Impedance)，电阻(Resistance)及电抗(Reactance)的组合。



Z , 阻抗。

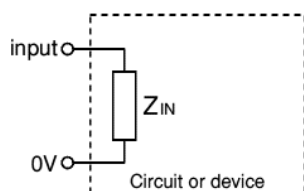
R , 电阻。对直流、交流电进行能量耗散(Energy Dissipation)。

X , 电抗。对交流电，特别是高频电流进行能量存储及释放(Energy Storage And Release)。

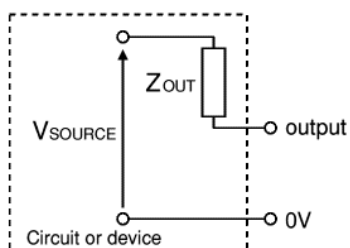
X_L , 电感(Inductor)的电抗。

X_C , 电容(Capacitor)的电抗。

输入阻抗，电路输入端提供电源，电路整体作电阻或阻抗。



输出阻抗，电路整体视作电源与阻抗的串联，对外部载荷(Load)供电。



3. 电磁模型

Maxwell 方程(微分、积分)。

高斯电定律(Electric Gauss's Law)。

$$\oiint_S \mathbf{E} \cdot \hat{\mathbf{n}} dA = \frac{1}{\epsilon_0} \iiint_V \rho dV$$

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\epsilon_0}$$

积分式, 封闭曲面(Closed Surface)二维流形内的电通量(Electric Flux)正比于该封闭体内的电荷密度(Electric Charge Density)。

微分式, 电向量场的散度(Divergence), 或者电场各方向变化的和正比于电荷密度。

高斯磁定律(Magnetic Gauss's Law)。

$$\oiint_S \mathbf{B} \cdot \hat{\mathbf{n}} dA = 0$$

$$\nabla \cdot \mathbf{B} = 0$$

积分式， 封闭曲面内的磁通量(Magnetic Flux)等于零。

微分式， 磁向量场的散度， 或者磁场各方向变化的和等于 0。

法拉第定律(Faraday's Law)。

$$\oint_C \mathbf{E} \cdot d\mathbf{s} = -\frac{d}{dt} \iint_S \mathbf{B} \cdot \mathbf{n} dA$$

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t}$$

积分式， 截面磁通量的变化可在封闭线圈内感应电场(Induced Electric Field)。

微分式， 电向量场的旋度(Curl)， 或者旋转电场的轴向变化正比于磁向量场变化。

安培定律(Maxwell-Ampere's Law)。

$$\oint_C \mathbf{B} \cdot d\mathbf{s} = \mu_0 \iint_S \mathbf{J} \cdot \mathbf{n} dA + \mu_0 \epsilon_0 \frac{d}{dt} \iint_S \mathbf{E} \cdot \mathbf{n} dA$$

$$\nabla \times \mathbf{B} = \mu_0 \left(\mathbf{J} + \epsilon_0 \frac{\partial \mathbf{E}}{\partial t} \right)$$

积分式， 截面电流及截面电场变化的加权等于产生的封闭线内的磁场。

微分式， 磁向量场的旋度， 或者环形磁场的轴向变化正比于传导电流密度及位移电流密度和或电场变化的加权。

注， 散度， 向量形式的函数， 各项偏导的和。

$$\operatorname{div}(\mathbf{F}) = \nabla \cdot \mathbf{F} = \frac{\partial f_1}{\partial x_1} + \frac{\partial f_2}{\partial x_2} + \cdots + \frac{\partial f_n}{\partial x_n}$$

$$\mathbf{F}(x_1, x_2, \cdots, x_n) : \mathbb{R}^n \rightarrow \mathbb{R}^n = (f_1, f_2, \cdots, f_n)$$

旋度，向量形式的函数，偏导算子向量与向量函数的乘积(Cross Product)。

4. 传输线

传输线相关概念，特征阻抗(Characteristic Impedance)、传播延迟(Propagation Delay)、反射系数(Reflection Coefficient)、时域反射(Time domain reflectometry)等。

反射损失(Reflection/Return Loss)，反射回的功率。

$$R_L = -10 \log_{10}(P_R/P_I) = -20 \log_{10}(V_R/V_I) = -20 \log_{10}(\Gamma)$$

P， 功率。

V， 电压。

R， 表示反射(Reflection)。

I， 表示入射(Incident)。

Γ ， 表示反射系数。

$$|\Gamma| = |V_R/V_I| = |Z_L - Z_0|/|Z_L + Z_0|$$

Z_L, Z_0 , 不同介质的阻抗。

驻波比(VSWR, Voltage Standing Wave Ratio), $VSWR = (1 + |\Gamma|) / (1 - |\Gamma|)$, 衡量功率传输性能。

路插损耗(Insertion Loss), 传输线路加入设备后, 信号功率的衰减。

$$IL = -\log_{10} \frac{P_{out}}{P_{in}} = -20 \log_{10} \frac{V_{out}}{V_{in}}$$

5. EMI/EMC

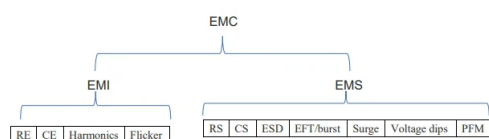
高频信号基于电路走向产生天线效应, 广播的电磁波可能与别的线路、元器件产生干扰, 感应电流、衰减电压信号等。

电磁兼容(EMC), 设备不对外发射电磁场, 可在外部电磁场中工作。

电磁干扰(EMI), 系统内部或外部电磁场引发的信号完整性损坏(Disruption)。

电磁敏感性(EMS), 设备对电磁场的抗扰度(Immunity)。

电磁兼容测试。



注, 浪涌(Surge), 电压持续提高三毫微秒。尖峰, 电压持续提高不到三毫微秒。

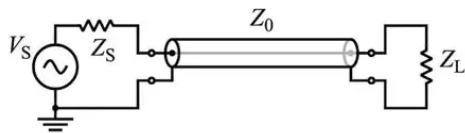
二、 问题类型

电路中，基于阻抗不匹配、电感电容寄生(Parasitic)、电路回路不连续、电磁干扰等因素可能产生反射、串扰、衰减、抖动、开关噪声、地弹等信号完整性问题。

1. 反射(Reflections)

电路信号基于多个不同阻抗的媒介传输时产生反射。反射振荡形成多次反射。

电路阻抗。



Z_S , 源阻抗(Source Impedance)。

Z_0 , 传输线的特征阻抗(Characteristic Impedance)。

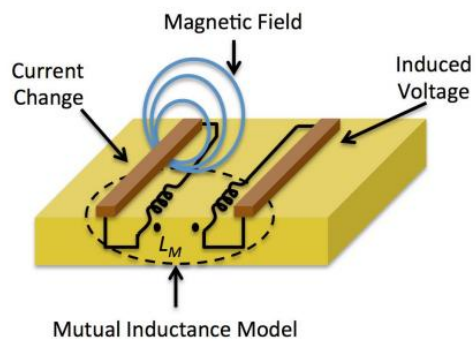
Z_L , 载荷阻抗(Load Impedance)。

当 $Z_S=Z_0$ 时，无反射； $Z_L=\infty$ ，全反射； $Z_L=0$ ，反相(Inverted)全反射； $Z_L \neq Z_0$ ，部分反射。

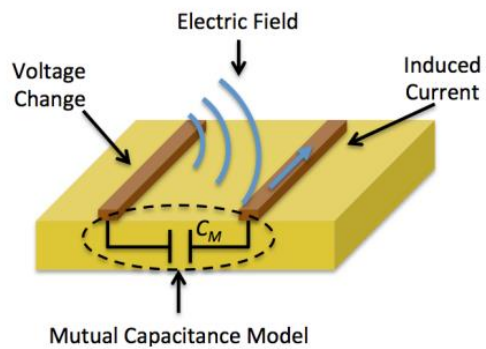
2. 串扰(Crosstalk)

串扰主要基于电路互感及互容问题，可分近端传输串扰(NEXT, Near End CrossTalk)、远端接收串扰(FEXT, Far End CrossTalk)。

互感。



互容。

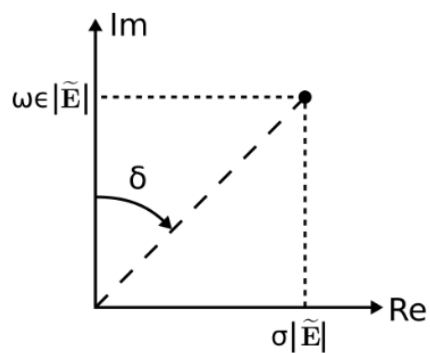


3. 衰减/损耗(Attenuation/Losses)

当阻抗不匹配、线路设置额外元件、信号频率提高时，基于趋表效应(Skin Effect)的导体损失(Conductor Losses)、基于损失正切(Loss Tangent)定义的介质的损失(Dielectric Losses)、辐射损失(Radiation Losses)等引发的信号大小(Amplitude)衰减、脉冲波形(Pulse Shape)及升降边沿变形等问题。

注，损失正切，基于安培定律微分方程，右侧电流密度、位移电流密度转换到相位量域(Phasor Domain)，形成实部、虚部，在虚坐标系表示，可计算感应电流相对于电流的损失比例，损失正切。

$$\tan \delta \triangleq \frac{\sigma}{\omega \epsilon}$$



4. 抖动(Jitter)

信号上升、下降沿无法对齐的时序错误。热噪声、电源噪声、反射、串扰等都可能引发信号抖动。

5. 开关噪声

地弹引发开关噪声。不共信号地(Signal Ground)的多个电路同时发生电平切换,使得地平面电压提高,低电平信号可能变成高电平。在球栅阵列封装(BGA, Ball Grid Array)中,特别注意。

三、 测量分析

电路设计时,可基于 SPICE 等仿真器、时频测量工具,通过建模、仿真、测量等方法防止信号完整性问题。

1. 眼图分析

眼图,基于不同电平时序的重合图形。可分析信号电平、形状、边沿变形等问题。抖动等会使眼图窗口变小。

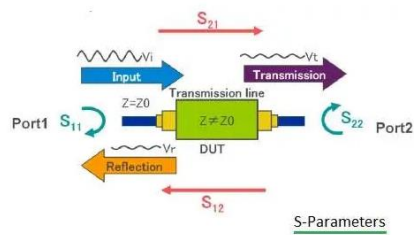
2. 误码率测试

误码率,错误接收到的 bits 数相对于传输的 bit 数的比例。

3. 散射参数

散射参数(Scattering Parameters),实质基于电路模块输入、输出端口的信号在频域的关系矩阵,分析表示电路的增益、相位延迟、传输功率、阻抗、反射损失等。

比方，双端口电路。



同端口测量电路反射特性。不同端口测量电路传输特性。

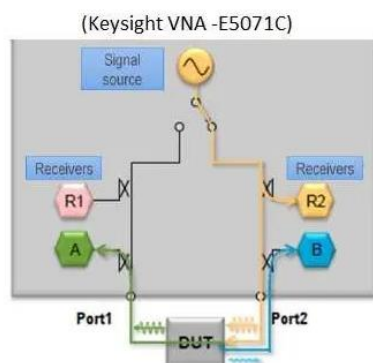
S_{11} ，端口 1 的输入信号与反射信号的关系。

S_{22} ，端口 2 的输入信号与反射信号的关系。

S_{12} ，端口 1 的输出信号相对于端口 2 的输入信号的关系。

S_{21} ，端口 2 的输出信号相对于端口 1 的输入信号的关系。

测量方法。



$$S_{11}=A/R_1$$

$$S_{22}=B/R_2$$

$$S_{12}=A/R_2$$

$$S_{21}=B/R_1$$

4. 时域反射

时域反射(TDR, Time Domain Reflectometry)，通过发送脉冲，测量反射信号来分析、定位传输线路缺陷的技术。

四、 解决方法

1. 阻抗匹配

基于走线宽度、与参考平面的介质高度连续等方法，确保信号线特征阻抗相同。

2. 差分共模

确保差分线长度匹配，控制阻抗误差，对称走线，避免不用的线头(Stubs)、直角走线(Sharp Bends)、过孔不均衡(Unbalanced Via Usage)等。

3. 优化回路

在参考平面(Reference Plane)临近布局高频信号回路，以提供低阻抗、连续性。避免跨分割平面(Split Plane)。

4. 优化堆层(Layer Stackup)

信号层基于微带线(Microstrip Line)或带状线(Stripline)布局，利于高频信号传输、抑制射频；信号层及地层交替(Interleave)布局，以避免回路电感(Loop Inductance)；在两整块平面(Solid Planes)之间布放带状线传输高速信号，以进行最佳屏蔽(Confinement)；对称堆层，以免弯曲(Warping)；相邻层信号正交走线，以免串扰。

5. 防止电磁干扰

基于滤波的方法，比方，在电源端加去耦电容，串联电感；在信号端或信号传输中串联磁珠或电感，并联电容等。

基于隔离的方法，调整布局使干扰源远离设备或弱化干扰能量；加电磁屏蔽进行整体屏蔽、功能块屏蔽、数据线屏蔽等。

第三章 电源完整性(Power Integrity)

电源完整性，确保系统中电源分布网络(PDN, Power Distribution Network)的质量，以使所有元件接收到的额定电压(Required Voltage Levels)是干净、稳定的。电源分布网络，主要基于电源面、地平面、走线。

一、 问题类型

1. 电压掉落

电压掉落(Voltage Drops)，基于电路电阻、电感等因素，在突然大电流时，电压掉落。

2. 电源噪声

电源噪声或电压纹波(Power Supply Noise/Voltage Ripple/Fluctuations)，基于不充分的滤波、解耦等，使得直流电压信号中产生交流成分。

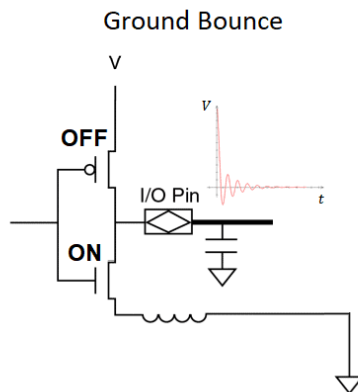
3. 共振

共振(Resonances in the PDN)，电路电感、电容元件在特定频率时的高阻抗状况。放大电源波动状况。

4. 地弹及轨道塌陷

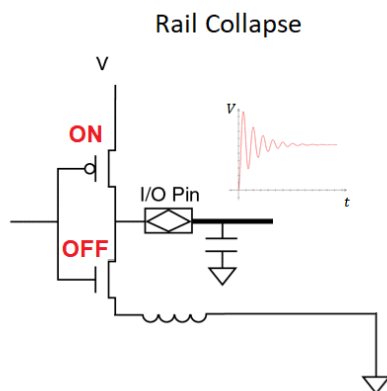
地弹(Ground bounce), 瞬时电流经过电源分配网络(PDN, Power Distribution Network), 网络阻抗在地参考平面引发的电压波动。

CMOS 反相器电路地弹示例。



电路中, 当切换开关, 顶部场效应管断开, 底部场效应管连通, 瞬时电流基于接地线(Ground Lead)与信号线的电感电抗产生电压涟漪。电压, 等于返回地的局部自感去掉信号线对返回地的互感后的电感所产生的感应电压。

轨道塌陷(Rail Collapse), 瞬时电流经过电源分配网络, 网络阻抗在正电压轨(Positive Rail)引发的电压波动。



电路中, 当切换开关, 顶部场效应管连通, 底部场效应管断开, 瞬时电流基于电源轨道(Power

Rail)及信号线电感电抗产生电压涟漪。

二、 测量分析

主要测量电源分配网络的阻抗、电压噪声、电热等。

1. 网络阻抗分析

基于时域瞬态响应(Transient Response)、频域阻抗频谱(Impedance Spectrum)分析电源分配网络的阻抗。高阻抗引发电压波动及电磁干扰。

2. 电压噪声测量

基于示波器(Oscilloscopes)及功率分析仪(Power Analyzer)等测量电压波动、噪声等。

3. 电热损失

基于热成像摄像热图及仿真测量。

三、 解决方法

1. 去耦电容(Decoupling Capacitors)

去耦电容，主要作用，供电及滤波。可在电源稳压模块(VRM, Voltage Regulation Module)芯片引脚加去耦电容，提供电流、形成地回路。低频信号时，电容阻抗小则压降小，轨道塌陷小；高频信号时，回路电感成主要因素，电感大，轨道塌陷大。

2. 正确电源面设计

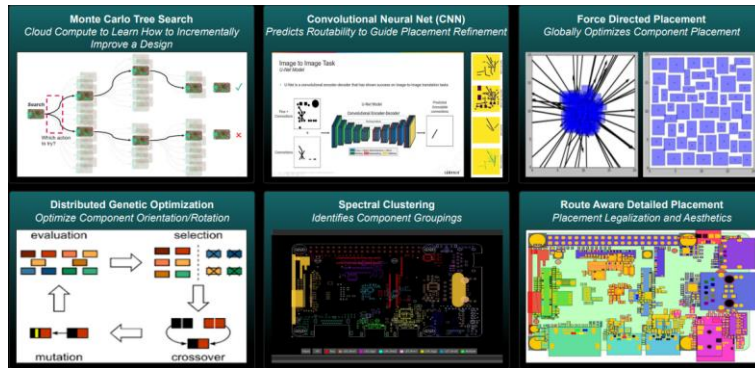
使用电源面及专用电源层。不同电压域，用不同电源面。

3. 低电感布局

信号平面与地平面接近可利用互感抵消返回地的局部自感。

第四章 智能设计

Allegro X AI，可进行元件布局、电源面设计、信号完整性分析、电源完整性分析、热分析等。



基于蒙特卡洛树搜索强化学习方法改进 PCB 设计。

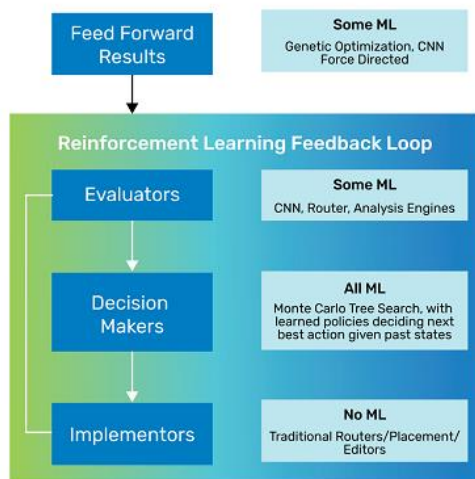
基于 U-Net 神经网络模型预测未连接的布线(Unroutable Connections)。

基于力导向图优化元件布放。

基于基因算法优化元件位姿。

基于谱聚类算法识别元件群。

一、 Allegro X AI 框架



当前，Allegro X AI 似乎只是在进行板级局部智能设计。

二、 端到端设计

基于系统设计需求直接生成印刷电路板图。

模仿人类思维函数，基于设计需求、元器件规程(Specs)，利用可进行物理定律推理的大模型，结合思维链原理图对齐，通过强化学习人工反馈(RLHF)机制，实现端到端板级设计应是可能的。

关节执行器篇

机器人关节执行器，分类，线性执行器、旋转执行器。

第一章 线性执行器(Linear Actuator)

线性执行器(Linear Actuator)可将电动机(Motor)的旋转力转换成直线运动、来回运动等形式。基于无框力矩电机、滚柱丝杠、力矩传感器、编码器、轴承等实现。基于滚柱丝杠传动。

线性执行器实现线性运动,用于人形机器人时，通过撑开机器人大臂、小臂、大腿、小腿连杆来实现关节的部分旋转，可承力远大于用于肩膀、腰、胯的旋转执行器(Rotary Actuator)，功耗当然也小。

丝杠可作线性执行器的传动部分。分类，滑动丝杠、滚动丝杠、静压丝杠。滚动丝杠分滚珠丝杠、行星滚柱丝杠。

厂家，瑞士 GSA、ROLLVIS 等。

一、行星滚珠丝杠

行星滚柱丝杠是主螺丝的周边围绕着几个从螺丝。

第二章 旋转执行器(Rotatory Actuator)

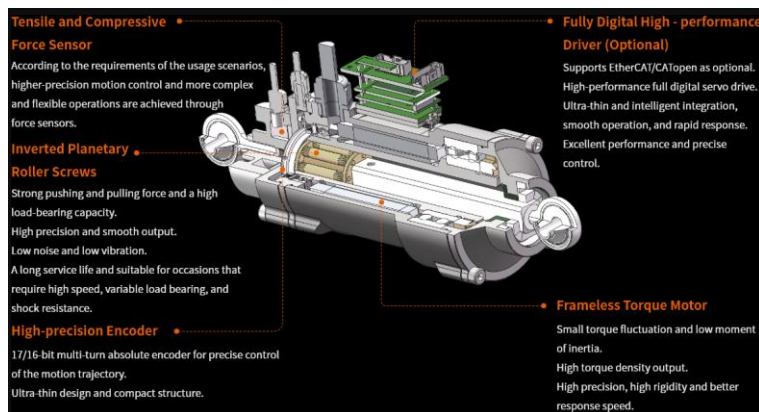
旋转执行器基于电机、减速器、编码器、驱动器、制动器等实现。基于减速器传动。

减速器分为轮减速器、行星齿轮减速器、蜗杆减速器，谐波减速器和 RV 减速器。谐波减速器厂家，日本哈默纳科，中国绿的等。

第三章 特斯拉执行器

Optimus 第一代执行器。

一、 线性执行器



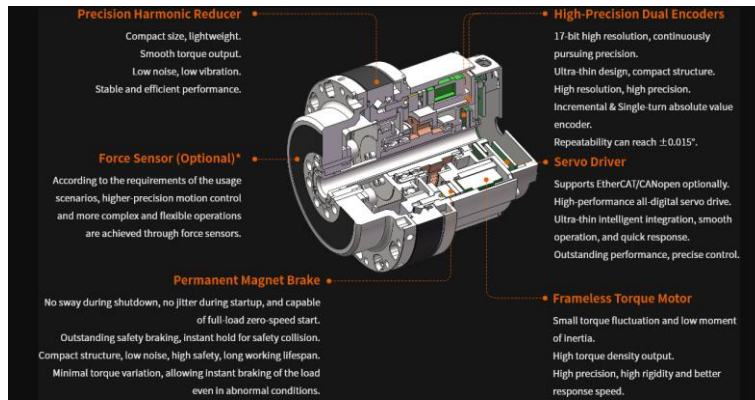
拉压力传感器(Tensile And Compressive Force Sensor)，测量拉力及压力的传感器，转换输出力大小信号。可进行高精度运动控制及灵活动作。

反向行星滚柱丝杠(IPRS, Inverted Planetary Roller Screws)，旋转转换成线性运动。

高精度编码器(High Precision Encoder)，控制运动轨迹。

无框力矩电机(Frameless Torque Motor)，高扭矩、直流电机，便于集成。

二、 旋转执行器



精确谐波减速器(Precision Harmonic Reducer)，精确传输力矩。

永磁制动(Permanent Magnet Brake)，基于永磁体磁场制动。不同于弹簧制动(Spring Engaged Brake)。

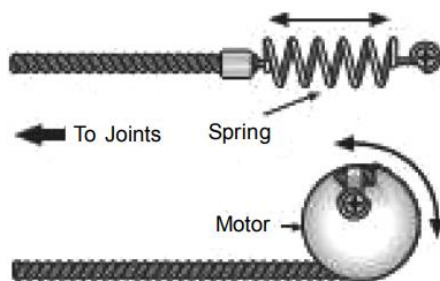
高精度双编码器(High Precision Dual Encoders)，可测量电机端转子及输出端转子角度。

伺服驱动器(Servo Driver)，控制伺服电机的控制器。可支持基于 CAN 总线的通信协议(CANopen)或基于以太网的控制协议(EtherCat)。

第四章 机器人灵巧手

一、 肌腱驱动(Tendon Driven)

用非弹性绳(ropes)模仿手肌腱(tendons).



二、 电缆驱动(Cable Driven)

基于电机控制静电离合器(Electrostatic Clutches)进行路由。

三、 特斯拉机器人手

线缆驱动。6 个执行器，拇指对应两个执行器，别的每个手指一个执行器。

第三代(Tesla Gen3. Optimus hand)，小型线性丝杠执行器及旋转谐波执行器在小前臂，利用金属电缆模仿肌腱驱动手指。

当前，基于视觉语言动作模型，利用模仿学习控制机器人实现灵活任务的前提，可能在于使

机器人手至少达到人类手的自由度,以免在改进机器人手的过程中使得模型不断进行重训练或迁移。

操作系统篇

一、 系统实质

1. 系统实质

操作系统实质，基于 CPU 权限模式对计算资源进行时间划分，基于物理地址区间定义对内存、外设控制器等 CPU 外部资源进行划分，基于虚拟地址空间到物理地址空间的映射隔离用户任务对硬件资源的直接访问、支持资源虚拟化，基于虚拟地址区间隔离用户及系统态事务，以实现单机多系统、多任务独立运行。

比方，ARM CPU 基于架构定义的不同优先级的权限模式实现不同权限程序对 CPU 计算资源的独立访问，依据中断/异常信号及查询表进行不同权限模式的转换，通过高优先级权限模式对低优先级权限模式的事务进行场景(Context)配置、切换管理；基于物理地址定义区分 UART 等控制器地址；基于 MMU 映射隔离虚拟地址与物理地址等。

2. 基本逻辑

操作系统基本逻辑示例。

基于物理地址区分物理内存、串口控制器、以太网控制器、固盘控制器等 CPU 外部资源；利用 CPU 系统模式隔离用户模式对资源的直接访问；用户模式程序发资源调用信号，CPU 发现该信号，找到信号对应的系统程序，转换到系统模式执行；通过系统模式的程序对用户模式的进程或通过虚拟管理模式的程序对系统模式及用户模式的事务进行上下文配置及切换等。

基于这些逻辑以及应用独立、任务并行、云计算等需求，派生定义进程、内存、文件、设备、通信管理以及内核态、用户态、系统调用、中断/异常、虚拟化等概念或方法。

3. 运行分层

操作系统运行分层。

第一层，时钟周期层。CPU 基于时钟周期执行机器指令。每执行一条指令，巡查中断/异常信号。

第二层，内核异常层。CPU 基于中断/异常信号及相应的程序跳转表，找到信号相应程序，转换到相应模式执行。

第三层，内核进程层。CPU 在系统模式中，进行内存、进程、文件系统、外设等资源的配置、访问、切换等。

第四层，用户任务层。CPU 在用户模式中，执行用户任务。

当前，基于虚拟机技术的发展及 CPU 对虚拟化技术的支持的扩展，可在一、二层或二、三层之间，添加虚拟管理层。CPU 在虚拟管理模式，对不同操作系统的地址空间进行转译、对外设中断信号进行配置转发、对不同操作系统进行上下文切换等。

4. 中断异常

内核异常机制

不同权限、模式之间的跳转，总的基于中断/异常机制。CPU 通过中断描述表(IDT, Interrupt Descriptor Table)定位不同中断/异常对应的程序片段、实现不同模式转换。

比方，用户态应用程序执行系统调用引发软中断，CPU 基于中断描述表定位到该软中断对应的内核程序；外设信号引发系统中断，CPU 基于中断描述表定位到该中断信号对应的服务例程(ISR, Interrupt Service Routine)；应用程序执行内存调用引发缺页异常，CPU 基于中断描述表定位到该异常对应的内核程序等。

二、 ARM 架构

不同 CPU 架构，定义不同的权限级别(Privilege Levels)/保护级别(Protection Levels)、运行模式。

在 ARM、Intel、AMD 等 CPU 中，基于精简指令集(RISC, Reduced Instruction Set Computer)、低功耗、版权开放等特点，ARM SoC 在边缘设备、数据中心等场景都有着越来越广的应用。

因此，这里的操作系统主要以 ARM CPU 作阐述。

ARMv6 前的权限、模式。

Mode	Function	Privilege
User (USR)	Mode in which most programs and applications run	Unprivileged
FIQ	Entered on an FIQ interrupt exception	
IRQ	Entered on an IRQ interrupt exception	
Supervisor (SVC)	Entered on reset or when a Supervisor Call instruction (SVC) is executed	Privileged
Abort (ABT)	Entered on a memory access exception	
Undef (UND)	Entered when an undefined instruction executed	
System (SYS)	Mode in which the OS runs, sharing the register view with User mode	

ARMv7 支持虚拟机管理器(Hypervisor)后的权限、模式。

Mode	Encoding	Function	Security state	Privilege level
User (USR)	10000	Unprivileged mode in which most applications run	Both	PL0
FIQ	10001	Entered on an FIQ interrupt exception	Both	PL1
IRQ	10010	Entered on an IRQ interrupt exception	Both	PL1
Supervisor (SVC)	10011	Entered on reset or when a Supervisor Call instruction (SVC) is executed	Both	PL1
Monitor (MON)	10110	Implemented with Security Extensions. See Chapter 21	Secure only	PL1
Abort (ABT)	10111	Entered on a memory access exception	Both	PL1
Hyp (HYP)	11010	Implemented with Virtualization Extensions. See Chapter 22	Non-secure	PL2
Undef (UND)	11011	Entered when an undefined instruction executed	Both	PL1
System (SYS)	11111	Privileged mode, sharing the register view with User mode	Both	PL1

在更高版本的 AARCH64 ARM 芯片中，用异常级别(EL0-3, Exception Level)替代权限级别(PL0-2)。Monitor 模式单作最高级别(EL3)。

ARM 不同模式对应的寄存器组。

R0	R0	R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7	R7	R7
R8	R8	R8_fiq	R8	R8	R8	R8	R8	R8
R9	R9	R9_fiq	R9	R9	R9	R9	R9	R9
R10	R10	R10_fiq	R10	R10	R10	R10	R10	R10
R11	R11	R11_fiq	R11	R11	R11	R11	R11	R11
R12	R12	R12_fiq	R12	R12	R12	R12	R12	R12
R13 (sp)	R13 (sp)	SP_fiq	SP_svc	SP_abt	SP_svc	SP_und	SP_mon	SP_hyp
R14 (lr)	R14 (lr)	LR_fiq	LR_svc	LR_abt	LR_svc	LR_und	LR_mon	R14 (lr)
R15 (pc)	R15 (pc)	R15 (pc)	R15 (pc)	R15 (pc)	R15 (pc)	R15 (pc)	R15 (pc)	R15 (pc)
(A/C)PSR	CPSR	CPSR SPSR_fiq	CPSR SPSR_irq	CPSR SPSR_abt	CPSR SPSR_svc	CPSR SPSR_und	CPSR SPSR_mon	CPSR SPSR_hyp ELR_hyp
User	Sys	FIQ	IRQ	ABT	SVC	UND	MON	HYP
Banked								

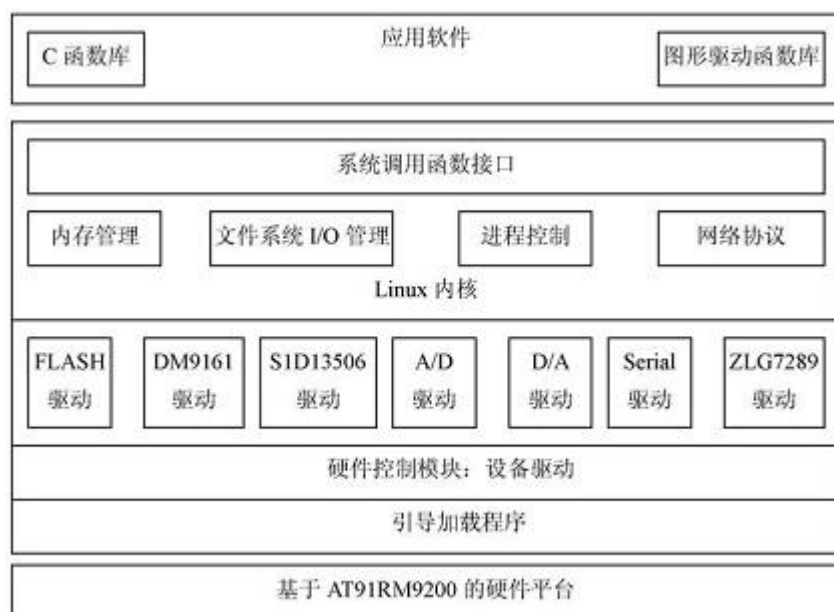
用户(User)、系统(Sys)模式共用相同的寄存器组。FIQ 等模式重用大部分寄存器，R13(SP, Stack Pointer)等个别寄存器对应各模式专有寄存器(Banked Registers)。

注，FIQ 相对于 IRQ，执行延迟低，优先级更高，确定中断到开始执行中断例程之间的时间固定，更多的暂存寄存器，适合实时应用。

在 AARCH64 ARM 芯片中，基本不再有专有寄存器，所有寄存器在任何异常级别都可访问。

三、 操作系统框架

1. 分层框架



2. 列式框架

已知操作系统及应用的执行，实质是 CPU 在时钟信号周期作用下，基于不同级别权限 (Privilege/Level/Ring)、不同模式、不同寄存器组，视察中断/异常信号，执行不同程序片段对应的指令，因此，基于权限级别、模式、寄存器组定义操作系统框架似更合实际。

基于 ARMv7 架构定义操作系统框架。

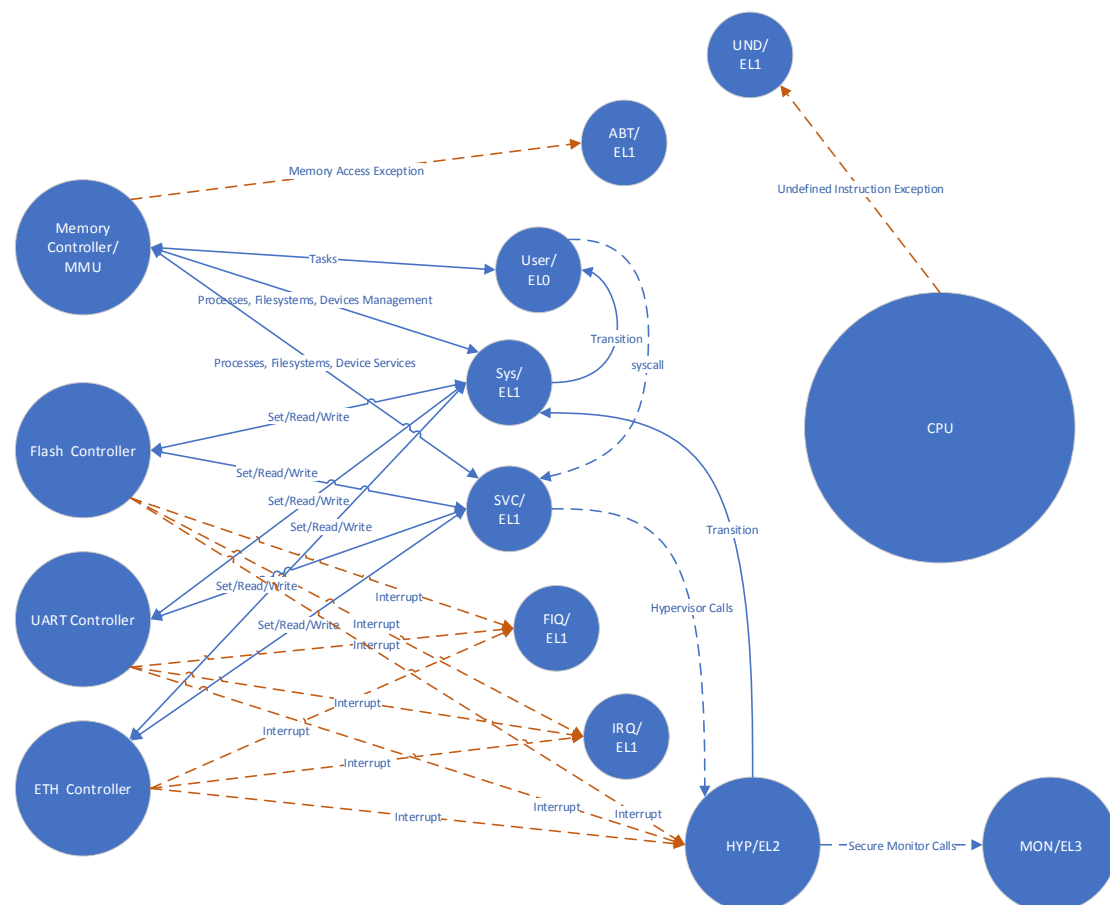
功 能 作用	应用程序 (Shell、仿 真器、 Webserver 等)	常规内 核程序 (进程、 内存、 文件系 统管理 及 IP 网络协 议等)	快 中 断 内 核 例 程	(外设) 中 断 内 核 例程	(MMU 缺 页 等) 异 常 终 止 内 核 例 程	重启、 系 统 调 用 等 对 应 的 内 核 程 序 (设 备 驱 动 等)	未 知 指 令 引 发 的 异 常 程 序 (程 序 指 针 异 常)	安 全 监 控 模 式 例程	虚 拟 机 管 理 内 核 例 程 (KVM 等)
寄 存 器 组	R0-R15 寄 存器组	R0-R15 寄存器 组	专 用 寄 存 器 部 分 重 用 的 寄 存 器 组	专 用 寄 存 器 部 分 重 用 的 寄 存 器 组	专 用 寄 存 器 部 分 重 用 的 寄 存 器 组	专 用 寄 存 器 部 分 重 用 的 寄 存 器 组	专 用 寄 存 器 部 分 重 用 的 寄 存 器 组	专 用 寄 存 器 部 分 重 用 的 寄 存 器 组	专 用 寄 存 器 部 分 重 用 的 寄 存 器 组
模式	User	Sys	FIQ	IRQ	ABT	SVC	UND	MON	HYP
权级	PL0	PL1	PL1	PL1	PL1	PL1	PL1	PL1	PL2

基于任务特性，以及权限越高时间占用应越少以免影响低权限工作执行的原则，大部分执行时间在用户及系统模式。

不同权限级别、运行模式、寄存器空间，执行不同权限的程序片段、访问不同权限的资源，实现不同权限、模式之间的跳转。

可以看到，基于不同的权限及模式可实现对计算及资源的隔离。进程、内存、文件系统等在此基础实现更高层级的隔离。这些为虚拟机、容器虚拟化的实现作了准备。

3. 环形框架图



基于 ARM Arch64 权限模式，高权限模式过渡到低权限模式，低权限模式通过调用(Call)陷落(Trap)到相邻的高权限模式。

中断优先级越高，执行的任务工作量越应该少，同时更应该快。以便让出对 CPU 的使用权，运行一般任务，同时，继续侦听重要事件的发生。

第一章 板级支持系统

板级支持系统(BSP, Board Support Package), 加载操作系统的 Bootloader 程序, 以及使操作系统适配不同硬件平台的支持程序或模块等。

一、 Bootloader-UBoot

Config.mk 与体系无关的一些配置

Include/config.mk 指定硬件体系和 cpu 等

Include/configs/Vendor_Product_XXX.h 功能开关定义及重要的常量定义

U-Boot 示例。无 relocate 功能。_TEXT_BASE 地址内容, 链接前指定的 TEX_BASE 传给寄存器 r0, 进行 Stack 配置。

汇编片段分析。

Start.S

```
.globl _start
_start:
    b    reset
    ldr  pc, _undefined_instruction
    ldr  pc, _software_interrupt
    ldr  pc, _prefetch_abort
    ldr  pc, _data_abort
    ldr  pc, _not_used
    ldr  pc, _irq
    ldr  pc, _fiq

_undefined_instruction:
    .word undefined_instruction
_software_interrupt:
```



```

        .word software_interrupt
_undef_instruction:
        .word prefetch_abort
_data_abort:
        .word data_abort
_not_used:
        .word not_used
_irq:
        .word irq
_fiq:
        .word fiq

        .balignl 16,0xdeadbeef

```

程序中，第一条指令将跳转到 `reset` 标识执行。

第二条指令 `ldr pc, _undefined_instruction`，准备执行 `_undefined_instruction` 位置指令。

接下来的 `ldr` 指令依此类推。

```

stack_setup:
    ldr    r0, _TEXT_BASE
    sub    r0, r0, #CFG_MALLOC_LEN
    sub    r0, r0, #CFG_GBL_DATA_SIZE
    sub    sp, r0, #12

clear_bss:
    ldr    r0, _bss_start
    ldr    r1, _bss_end
    mov    r2, #0x00000000

clbss_l:
    str    r2, [r0]
    add    r0, r0, #4
    cmp    r0, r1
    ble    clbss_l

    bl    coloured_LED_init
    bl    red_LED_on

    ldr    pc, _start_armboot

```

```
_start_armboot:  
    .word start_armboot
```

1. Start_armboot()

```
{  
    初始化;
```

```
    init_fnc_t *init_sequence[] = {  
        cpu_init,  
#if defined(CONFIG_SKIP_RELOCATE_UBOOT)  
        reloc_init,  
#endif  
        board_init,  
        interrupt_init,  
        env_init,  
        init_baudrate,  
        serial_init,  
        console_init_f,  
        display_banner,  
#if defined(CONFIG_DISPLAY_CPUINFO)  
        print_cpuinfo,  
#endif  
#if defined(CONFIG_DISPLAY_BOARDINFO)  
        checkboard,  
#endif  
#if defined(CONFIG_HARD_I2C) || defined(CONFIG_SOFT_I2C)  
        init_func_i2c,  
#endif  
        dram_init,  
        display_dram_config,  
        NULL,  
    };
```

NAND 初始化。

基于 driver/mtd/nand.c 中的 nand_init()。

设备初始化及控制台初始化。

基于 common/devices.c 中的 devices_init(), 进行各设备的初始化, 包括网络控制台设备的初始化 drv_nc_init()。在初始化过程中将 dev 注册到全局链表 devlist 中。

基于 common/console.c 中的 console_init_r()。设备的读写接口挂在控制台的读写接口即标准输入/输出上, 以实现对不同设备的适配。具体, 通过 console_setfile(), 使 dev 挂入 stdio_devices[file]数组。

各类杂项初始化, 包括对 eth 的配置。

misc_init_r()

进行 ether 的 MII 和 PHY 进行初始化。

使用 net/eth.c 中的 eth_initialize()。

main_loop

等待控制台命令, 该命令可通过键盘输入或者网络输入。

}

2. main_loop()

{
readline()

基于 getc 最终会调用设备读接口[stdio_devices[file]->getc ()], 因此此前控制台适配的设备, 便是当前通信设备。

run_command()

U_BOOT_CMD 宏被链接器搜集到 .u_boot_cmd 所在的 __u_boot_cmd_start 与 __u_boot_cmd_end 区间(board/Vendor_Product_XXX/dm365/u-boot.lds)。

Find_cmd 中, 从 __u_boot_cmd_start 开始查找对应命令结构体变量。比方, tftp 等网络命

```
令，执行 do_tftp()->netboot_common()->NetLoop();  
  
}
```

3. netloop()

在 for (;;)循环中，使用 eth_rx()接收数据，同时通过状态机判定是否处理完毕。

4. eth_rx()

这个函数实际上实现了数据由网络到上层协议栈再到应用层解析的整个过程，应用解析后会更改状态机状态。

对于 tftp 则是在 tftp_start 时，因利用 NetSetHandler()指定全局量 packetHandler=TftpHandler；因此 eth_rx()->net_receive()->packetHandler()对应 tftphandler，进行数据处理并修改 Netstate 状态值。

5. nand

对于 nand 的 write 操作需注意。Nand 基于 “与”操作才能写入，也就是擦除后，写入前 nand 中的 bit 值必须是 1。这意味着所写是 1，不改变 nand 的 bit。

在 uboot nand 的驱动中对于 oob 的 fill 也正是利用了这一点。看似 memory 中的 oob 数据写到了 nand oob 中，但 memory 中的 oob 各 bit 是 1，因此对 nand oob 值不产生影响。

二、 系统内核

设备树(Device Tree)、内核配置(Kernel Configuration)、电源管理等。

三、 设备驱动

字符设备、块设备、中断响应(Interrupt Handling)、DMA 等。

四、 测试调试

单元、系统测试、JTAG 调试等。

五、 性能优化

内存管理、缓存优化、中断优化、电源优化(Power Consumption)等。

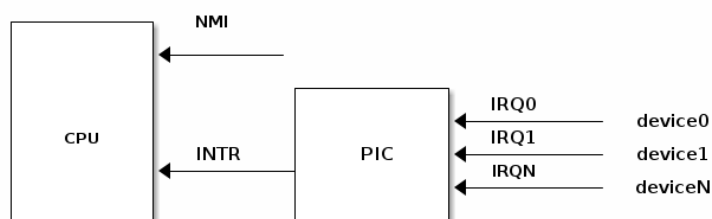
第二章 操作系统-Linux

一、 内核启动

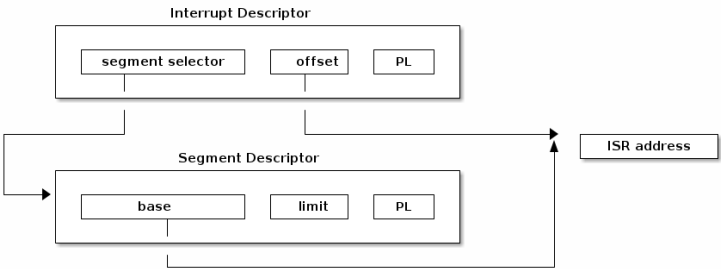
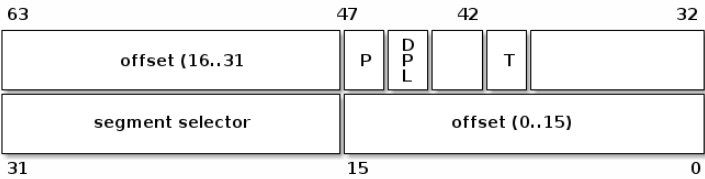
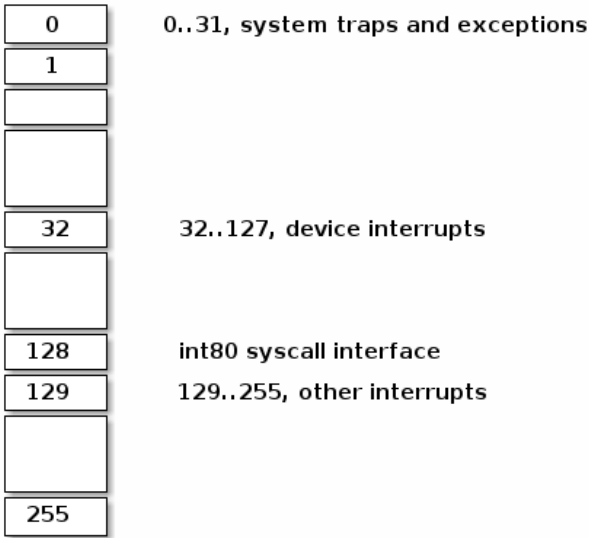
在分析内核启动汇编代码时，要注意以下几个事项：

- 1.熟悉 ARM 指令集；尤其需要注意:地址加载指令 `adr`，地址中的内容加载指令 `ldr`,加载数据后累加栈指针指令 `ldmia` 等；
- 2.了解链接器的基本原理，因为很多地址和数据信息是通过隐性处理的；
- 3.熟悉 `arch/arm/kernel/vmlinux.lds.S` 文件；
- 4.注意内核被加载的物理地址和内核映像中的虚拟地址的区别；物理地址就是硬件平台相关的物理空间，比如以 `0x80000000` 作为内存起始地址，虚拟地址则是编译链接时指定的，比如以 `0xC0000000` 作为内核虚拟空间起始地址；
- 5.在 MMU 打开前后，建立页表时除了要建立内核虚拟地址空间与物理地址的映射关系，还要特别建立内核被加载处的汇编代码的物理地址与物理地址的映射关系；这样做的目的是保证在打开 MMU 后，以虚拟地址方式仍然能执行这些汇编代码，即完成无缝执行；

二、 中断/异常



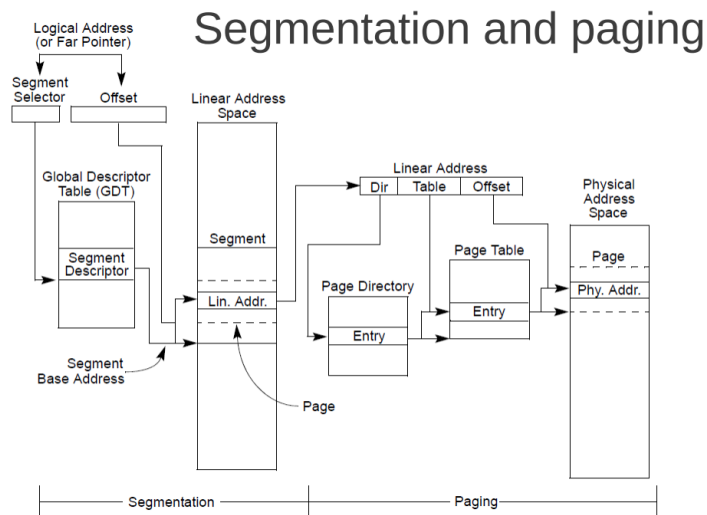
arch/x86/include/asm/irq_vectors.h



GDT,系统唯一，存储所有进程的 LDT 及 TSS；可存在内存任何位置；GDT 地址存在 GDTR，，可用 lgdt 指令更改；

LDT, 每个进程对应一个 LDT; 是 segment descriptor ,定位相应进程的 segment ; 当前 LDT 地址存在 LDTR 寄存器;

TSS, 每个进程对应一个 TSS(Task State Segment); 地址存在 GDT 中的 LDT 相邻的 segment descriptor 中; 用于存当前进程的寄存器信息, 可恢复上下文信息;



传统的中断定义,是指从内核及应用执行的角度,因外设触发了中断请求而导致当前执行的事务被中断。从cpu 角度运行的角度,或者说从系统整体运行的角度,特别是硬件系统运行的角度,cpu 始终是轮询的方式在处理来自内存的指令以及来自外设的请求,也就是cpu 每执行一条指令就要轮询是否有外部中断请求,而不管是否实际发生了中断,反过来cpu 每次轮询外设是否发生中断请求后,都要轮询内存中是否有指令要执行。

两者的分歧主要在于传统方式,是从内存指令的执行角度出发,而不是从操作系统的角度出发来看待中断,或者说没有站在硬件和软件的交界处来看待问题。

异常和中断是两个易混淆的概念,在 arm 体系中,异常是有特殊的意义的,即它是包含了中断和快速中断的;中断号是由硬件给出的,具体而言就是中断控制器所提供的,cpu 通过读取该控制器的相应寄存器即可获取是哪类中断发生了;同时通过硬件约定知道到哪个地址找到要执行的指令,比如对于 ARM 而言,FIQ 中断发生时跳转到 0x0000001c 处执行一跳转指令,跳到要执行的程序处开始执行;

如果不考虑系统的演变,最初的考虑是一个中断发生时根据中断向量找到中断处理程序处理中断即可,但由于外设的发展越来越复杂,中断向量不够用了,因此中断描述符表或中断向量表或中断向量数组是对中断进行分类处理的依据,该表的首地址理论上可以在内存中的任何位置,因为cpu 发展到现在基本上都允许软件来配置该地址;中断描述符表,系统唯一,

地址存在 IDTR 寄存器，可用 `lidt` 指令更改。

中断向量即中断号是由硬件提供的，但一般而言，软件上为了方便起见，会将它们与中断向量表的索引对应，这样利用中断向量表的首地址和中断号相加就可以得到对应的表项；中断向量表中将表项划分为不同类型，比如陷阱门，系统调用门和中断门；而实际上这些表项是一个结构体，不仅仅包含门类型信息，还包括中断处理的入口，注意该中断处理程序不是中断服务例程 `ISR`；

跟踪流程 1:

`start_kernel()->trap_init()->__trap_init()`，该函数在 `/arch/arm/kernel/entry-armv.S` 中定义，会将 `__start_start` 和 `__start_end` 之间定义的指令赋给中断向量表的 `cpu` 常用表项；当中断发生时即根据对应的这些指令跳转到对应的程序入口，比如 `vector_irq()`，在该函数中将进行 `__irq_usr` 或 `__irq_svc` 函数的调用，然后最终调用 `asm_do_irq()`；在 `asm_do_irq()` 中会根据向量号调用 `irqdesc` 表中相应的表项的 `handler`；

跟踪流程 2:

`start_kernel()->init_irq()->init_arch_irq()`，其中 `init_arch_irq` 在 `arch/arm/kernel/setup.c` 中的 `setup_arch()` 中被赋值为 `mdesc->init_irq`；在 `davinci dm355` 中通过 `arch/arm/mach-davinci/board-dm355-evm.c` 中的 `MACHINE_START` 定义的 `davinci_irq_init` 及 `davinci_init`，可跟踪到这个入口程序可能是 `do_edge_IRQ` 或 `do_level_IRQ` 或 `do_simple_IRQ`

linux 的中断向量表是由 linux 本身设置的

三、 进程管理

内核基于进程控制块(PCB, Process Control Block)或任务结构(Task_Structure)直接管理进程，基于进程树进行整体管理。

管理的核心在创立、销毁、运行、挂起、切换等。运行及挂起主要基于进程调度策略，比方，先进先出(FIFO)、轮询 (Round Robin)、时间片等。进程切换主要基于 CPU 不同的权限模式及进程块执行。

实时操作系统基于优先级调度策略进行内部实时响应、基于中断处理策略进行外部实时响应。

四、 内存管理

每个进程，虚拟空间划分为多个隔离的区域来使用，每个区域基于 `vm_area_struct` 维护。

虚拟区域到物理空间的映射是逐步完成的，而不是一次到位，这样便于将工作量分散，以带来系统的整体流畅性。这与组件加载等思想类似。

Linux 2.6 以后的进程调度是分散的计算各进程的优先级，对过时的进程逐步清出就绪队列，不是一次计算所有进程的调度优先级，然后进行切换。

申请内存时，系统理论上是可以零散的物理内存区拼成一个连续的虚拟区间，但是在很多情况下，系统更倾向用连续的物理空间构成一个连续的虚拟区间，因为这样增加了 TLB 的命中率，减少了 TLB 的刷新率，提高 cpu 访问数据的速度。

为了尽可能的提供连续的物理空间，系统一般采用伙伴算法，基于 `mem_map` 数组和 `free_area` 数组来管理物理页面。

`mem_map` 每个成员项对应一物理页，每个 `free_area` 成员维护一类具有连续物理页的物理区，连续数为 2 的指数。

用户函数 `malloc` 或系统调用 `brk` 都是在进程的用户虚拟空间进行准确分配。系统在物理区分配上采用了以页为单位的扩量式分配。这与 c++ 等实现 `Vector` 向量类似，比方，当空间不足时，以原空间的倍数来申请新的空间。

基于 `buddy` 算法的内存方式在某些情况下，显得太笨重。比方，`unix` 中的 `mem_buf` 的申请或进程 `task_struct` 的申请等，这些数据体占用空间不大，但却会频繁使用。因此系统采用内存池方式来解决可能的降低数据访问速度的问题。

另一方面，这些数据体占用空间不大，但数量不少，因此采用按大小划分内存，且按链表组织这些内存块的方式来解决因小内存块的

申请和释放带来的 `buddy` 算法管理的物理内存中连续页区减少的问题。基于此，产生了 `slab` 方式；因此 `slab` 是基于 `buddy` 算法的内存管理方式；

内核中内存申请方式即可通过 `slab` 相关接口申请，比方，`kmem_cache_alloc`，也可通过 `buddy` 接口申请，如 `get_free_pages`；使用两者的原则是一般小于一页的内存用 `kmem_cache_alloc`。

`slab` 和 `buddy` 接口分配的都是连续物理内存，都是预分配思想，即先准备好可用的以待使用。当系统中发生了，很多割裂的内存页，可基于页面映射表管理使用。内存分配接口 `vmalloc`，因为要更新页表，因此效率会低一些；`vmalloc` 也是基于 `buddy` 算法。

以上三类内存申请接口所申请的内存所在的内核区域是相互隔离的，不会重合。

基于 `get_free_page` 或 `Kmalloc` 函数所分配的连续内存都陷于物理映射区域，所以它们返回的内核虚拟地址和实际物理地址仅仅是相差一个偏移量 (`PAGE_OFFSET`)。

`vmalloc` 分配的地址则限于 `vmalloc_start` 与 `vmalloc_end` 之间。这些虚拟地址可与物理内存没有简单的位移关系，必须通过内核页表才可转换为物理地址或物理页。

管理进程虚拟空间的结构体是 `vm_area_struct`，管理内核的则是 `vm_struct`。

DMA 传输中使用的内存，不涉及页机制，因此必须连续分配。

Mmap，内核空间映射到一设备文件，用户在创建该设备文件时，已经获得了该文件的用户虚拟地址，再通过指定长度，就可以获得该虚拟空间。

在内核中利用 `vmalloc` 申请一内核空间，要使其与用户虚拟空间建立联系，切入点就是在构造内核虚拟空间和物理页面映射的地方进行构造，即构造页表的地方。

在这里就是构造 `vm_operations` 的 `nopage` 函数。这样只要真正要对设备文件读写操作时，就会发生 `nopage` 调用，也就会执行构造的程序来实现用户虚拟空间到内核空间的页表项的填充。

五、 文件系统

嵌入式设备文件系统列表示例。

```
nodev    sysfs
nodev    rootfs
nodev    bdev
nodev    proc
nodev    tmpfs
nodev    debugfs
nodev    sockfs
nodev    usbfs
nodev    pipefs
nodev    anon_inodefs
nodev    rpc_pipefs
nodev    inotifyfs
nodev    devpts
          cramfs
nodev    ramfs
          vfat
          msdos
nodev    nfs
nodev    mqueue
nodev    ubifs
```

PC 设备文件系统列表示例。

```

nodev    sysfs
nodev    rootfs
nodev    bdev
nodev    proc
nodev    cgroup
nodev    cpuset
nodev    tmpfs
nodev    devtmpfs
nodev    debugfs
nodev    securityfs
nodev    sockfs
nodev    pipefs
nodev    anon_inodefs
nodev    inotifyfs
nodev    devpts
nodev    ext3
nodev    ext2
nodev    ext4
nodev    ramfs
nodev    hugetlbfs
nodev    ecryptfs
nodev    fuse
nodev    fuseblk
nodev    fusectl
nodev    mqueue
nodev    vmhgfs
nodev    binfmt_misc

```

首先要注册文件系统类型结构体数据，关键是要设置该结构体中的 `read_super` 函数指针。

这相当于告诉内核遵循的报文协议。这个报文协议就是物理或虚拟设备上的文件系统数据的格式，比如一个 `fat32` 或 `ext3` 文件系统。

`Super_block` 面向的是文件系统，`inode` 面向的是文件或目录的物理性质，`dentry` 面向的是文件或目录的抽象描述。一个 `dentry` 对应一个 `inode`，但一个 `inode` 可以对应多个 `dentry`。

```

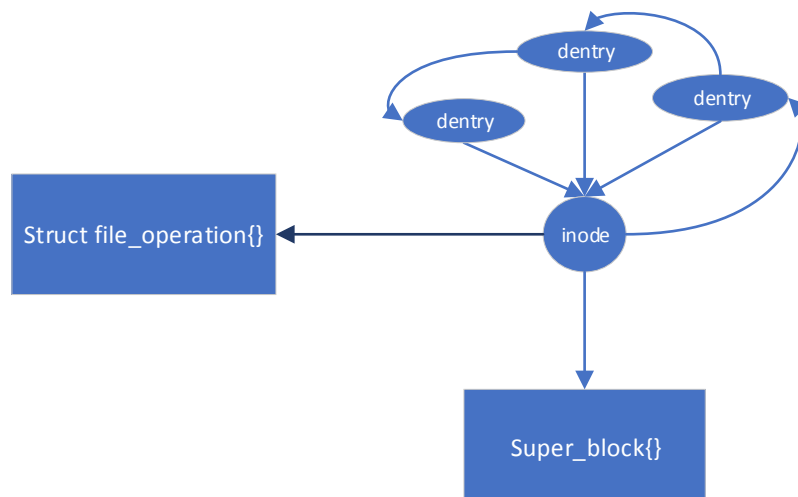
Struct file_system_type{
    struct super_block
    *(*read_super) (struct
    super_block *, void *,
    int);}

```

```

Struct file_system_type{
    struct super_block
    *(*read_super) (struct
    super_block *, void *,
    int);}

```



无论设备中是否存储了文件系统信息，首先都要在 linux 虚拟的文件系统树上添加抽象的 dentry 和 inode 信息，因为通过这些 dentry 和 inode 信息，linux 才能统一管理设备。

比方，设备都在/dev 目录下有相关的文件名或设备节点。这些信息一般基于相应的设备驱动来填写。示例，/dev/video0，这个 video0 是驱动添加到/dev/目录。

当应用层要访问这些设备时，先从系统调用开始，比方，open。利用 linux 虚拟的文件系统接口按照目录来查找这个设备，当找到的是目录，比方，/dev，则利用 dentry 中的 d_inode 来找下一级的文件。因为 d_inode 中保存了下一级的文件的 dentry 信息。

当找到了符合需要的设备名的 d_inode，则利用这个 d_inode 中保存的 i_fop 接口中的函数，比方，file_operations 接口中 open 来对设备进行操作。这个 file_operations 接口是设备驱动注册时添加到对应的抽象的 inode 中去的。

当设备中存储了文件系统，这个文件系统安装到系统中的虚拟文件系统的目录节点上或者说安装点。

基本过程，文件系统通过 vfstmount 结构来安装的。vfsmount 的作用就是将安装点和目标文件系统的信息关联起来。具体，vfsmount 结构中间接的保存了安装目录或安装节点信息，也保存了目标文件系统的根目录信息，这些根目录信息和设备节点名是不同的概念。

安装点信息，比方，操作系统内部构建了一个根目录/。

目标文件系统的目录树建立步骤。

因为不同的文件系统都有一套该文件系统特定的 inode 访问接口，比方，ext3 的 inode_operations 接口。

在这些接口中主要的就是利用对设备的访问操作，获取设备上的数据，然后基于指定的文件

系统格式解析这些数据。至于完全解析还是只解析一部分，这个不重要，主要的是找到了设备文件系统数据中的根目录，并构造相应的 `dentry` 和 `inode`。根目录下的其他文件也可以依此方法构建相应的 `dentry` 和 `inode`。

综上所述，正规文件/目录文件/符号链接文件使用其所属文件系统的操作接口，比方，`ext3_file_operations`, `ext3_dir_operations`。当是设备时，使用设备驱动注册的操作接口，`file_operation`。

应用层对于设备节点的访问关系。

system call ——> vfs 的 `file_operations` 接口 ——> 经过对 `dentry` 和 `inode` 树的查找，找到设备的 `inode` ——> 利用该 `inode` 中已通过设备驱动注册的 `file_operations` 接口 ——> 访问设备。

应用层对于正规文件，即设备中存储的文件的访问关系。

system ——> vfs 的 `file_operations` 接口 ——> 经过对 `dentry` 和 `inode` 树的查找，找到文件的 `inode` ——> 如果是正规文件，则利用正规文件接口的 `file_operations` 中的函数来访问该文件 ——> 如果该文件不在内存中则利用 `inode` 中注册的设备访问接口，`file_operations`，来获取文件数据。如果存在，则基于指定的文件系统的 `file_operations` 接口修改该文件以及对应的 `dentry` 和 `inode` 信息，然后通过设备访问接口将这些修改都同步到设备上。注，在修改和同步方面实际中往往利用 `slab cache` 机制以避免设备访问造成系统读写瓶颈。

`Sysfs`，提供分类和统计的文件系统。设备模块注册后，利用其内嵌的 `kobject` 结构建立各式关联，比方，划归到 `block subsystem` 下，`sysfs` 的设备目录下，`sysfs` 的 `class` 目录下等。

一定意义，`sysfs` 是另一 `devfs`，各模块注册时多了额外的步骤去基于 `sysfs` 的规范进行登记。

面向应用的抽象的文件操作 `vfs`，它体现的是对抽象后的文件的管理，它把文件作为一个文档。Linux 通过它所抽象出来的 `dentry` 和 `inode` 信息管理文件/目录。`dentry` 和 `Inode` 共同体现了 linux 抽象出来的文件系统结构。

真正的文件系统结构提供了构造 linux 标准的 `dentry` 和 `inode` 信息，但是它们直观上不一定都有 `dentry` 和 `inode` 的概念。因此需要通过具体文件系统的 `inode_ops` 来在抽象的 `dentry`，`inode` 和真实的文件系统之间建立关系。

当然真实的文件系统所能提供的 `dentry` 和 `inode` 信息作为设备上的数据是要通过设备驱动来访问的。

六、 设备驱动

当 device 主设备号不为0，还会调用 `devtmpfs_create_node(dev)`函数在 `/dev` 目录下自动生成一个设备节点。在 Linux 早期，`/dev` 下的设备节点是通过 `mknod` 命令手动添加的，现在通过 `devtmpfs` 文件系统，就可以在调用 `device_register` 函数注册设备时自动向 `/dev` 目录下添加设备节点，该节点的名字就是 `dev->init_name`。

七、 系统调用

基于 ARM 指令的系统调用示例。

```
'''

Assembly
push {r7,lr} /*函数后的下一条指令的地址存到 lr 寄存器*/

/* syscall write(int fd, const void *buf, size_t count) */
    mov     r0, 1      /* fd -> stdout */
    ldr     r1, =msg    /* buf -> msg */
    ldr     r2, =len    /* count -> len(msg) */
    mov     r7, 4      /* write is syscall #4 系统调用号 4, 存到 r7 寄存器; 内核利用该号在 IDT
中查找相应的执行程序*/
    svc     0          /* invoke syscall 触发 svc 模式系统调用*/

pop {r7,pc} /*函数下一条指令地址存到 pc 寄存器，准备取指令指令；注，内核已利用 lr 寄
存器的函数后地址，存到 r7 寄存器*/

.data

msg:
    .ascii  "Hello, ARM!\n"
len = . - msg

'''
```

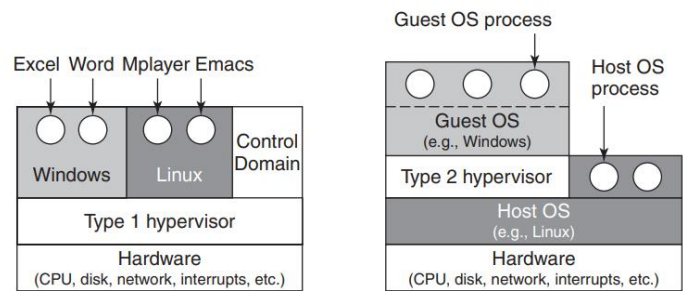

八、 电源管理

基于内核级(Kernel-Level)直接管理硬件，基于用户级(User-Level)访问内核设置。

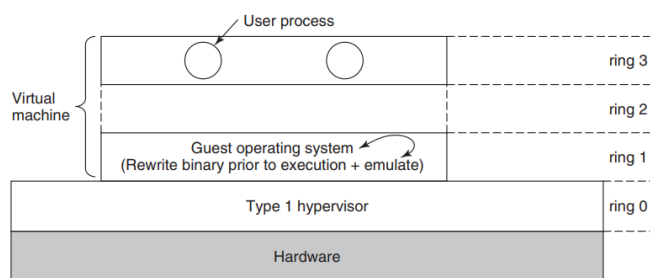
系统挂起、继续，实时电源管理，ACPI(Advanced Configuration And Power Interface)等。

九、 虚拟化

虚拟机管理器(VMM, Virtual Machine Monitor; Hypervisor)类型。



一、二进制转译(Binary Translation)。早期，在不支持硬件虚拟技术 (VT, Virtualization Technology)时，可基于解释器(Interpreters)替换运行在非内核态的虚拟机客户应用及客户操作系统中特定指令。



比方，基于 X86 芯片的四类保护模式(Protection Modes)或环(Rings)。使管理器(Hypervisor)运行在原本运行内核的最高权限模式(Ring 0)，虚拟机客户应用运行在最低权限模式(Ring 3)，客户操作系统运行在中等权限模式(Ring 1)。

在执行客户应用或客户操作系统指令前，利用 Bochs 等解释器令这些程序中基本指令块(Basic Block)内的敏性指令(Sensitive Instructions)、特权指令(Privileged Instructions)替换为对管理器过程(Hypervisor Procedure)的调用(Call)。

注，客户操作系统中特权指令(Privileged Instructions)不替换亦可，执行时会引发陷落异常(Trap Exception)，转到管理器执行。

注，敏性指令，在内核模式、用户模式执行产生不同行为的指令。比方，I/O 访问指令、MMU 配置指令等。

特权指令，在用户模式执行，会引发陷落异常的指令。

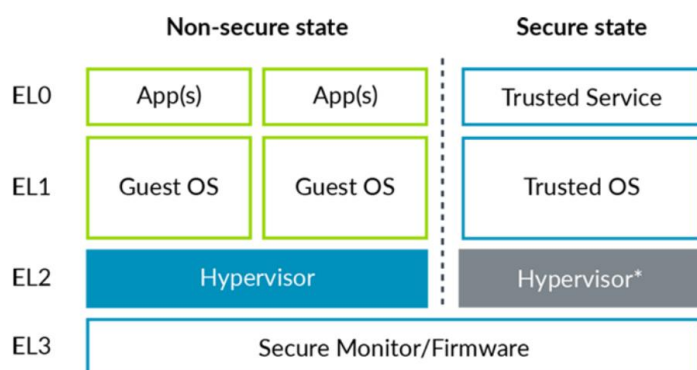
二、半虚拟化(Paravirtualization)。客户操作系统程序中的敏性指令替换为管理器 API 调用(Hypercalls)，以便对 I/O 等的访问转到管理器执行。

三、异常仿真法(Hypervisor Trap And Emulate)。虚拟机客户应用与客户操作系统程序不用修改，敏性指令、特权指令引发陷落异常。基于硬件虚拟技术支持，管理器(Hypervisor)拦截这些异常，仿真执行。

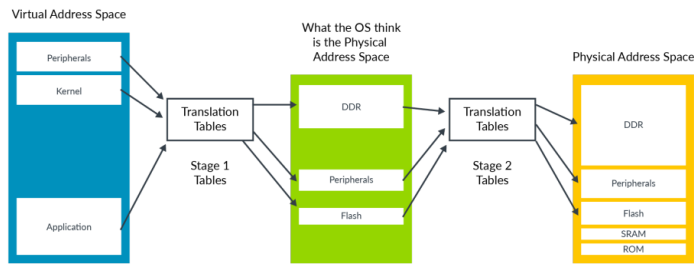
虚拟化方案。

Virtualizaton method	Type 1 hypervisor	Type 2 hypervisor
Virtualization without HW support	ESX Server 1.0	VMware Workstation 1
Paravirtualization	Xen 1.0	
Virtualization with HW support	vSphere, Xen, Hyper-V	VMware Fusion, KVM, Parallels
Process virtualization		Wine

1. ARM 虚拟化



2. 虚拟地址二级转译框架



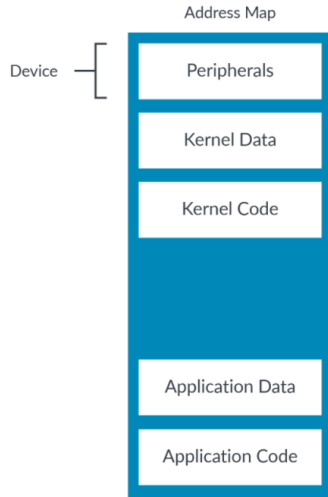
框架图，左侧，客户操作系统虚拟地址基于客户操作系统控制的第一级转译表(Stage1 Translation Tables)转换成客户操作系统所认知的物理地址；右侧，客户操作系统认知的物理地址基于虚拟机管理器(Hypervisor)控制的第二级转译表(Stage2 Translation Tables)转译成真实的物理地址。

注，客户操作系统认知的物理地址空间可称作中间物理地址空间(IPA, Intermediate Physical Address Space)。

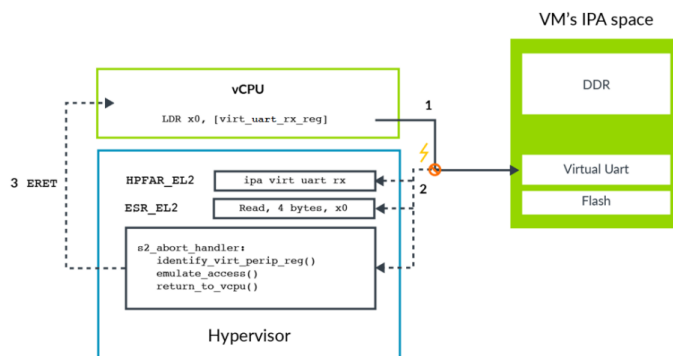
第二级转译表，基本相似于第一级转译表。区别在于地址类型信息直接编码到表项(Table Entry)中；旁路转译表(Translation Lookaside Buffer)表项基于虚拟机标记(VMID, Virtual Machine Identifier)、地址空间标记(Address Space Identifier)区别不同虚拟机中的不同应用；内存管理单元(MMU)基于第一、第二级转译表表项中的类型、访问权限等属性，选最苛刻的(Restrict)的转译表作映射。

比方，第一级转译表表项内存访问类型是设备(Device)，第二级转译表表项内存访问类型是常规(Normal)，则内存管理单元选设备类型进行映射访问。

注，常规类型表示访问的是内存地址，设备类型表示访问的内存地址空间映射到外设(Memory-Mapped Input/Output)，外设的寄存器空间是有限的、不存指令等信息，处理器(Processor)不允许进行推测访问。



3. 内存映射 I/O 仿真

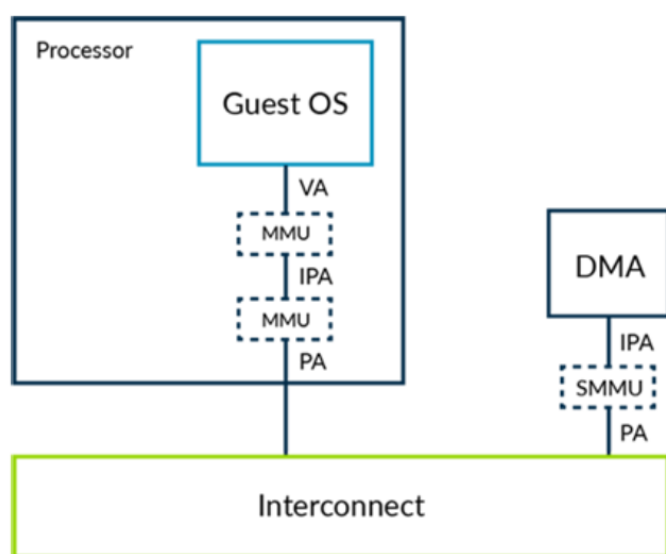


当基于虚拟 CPU 线程的虚拟机中的指令访问的内存地址对应于中间物理地址空间的虚拟地址时，比方，虚拟 UART 设备的接收 FIFO，第一级转译表触发异常(Fault)，跳转到 EL2 权限、Hypervisor 模式，要访问的外设目标寄存器、字节数、读存方式保存在 ESR_EL2 寄存器中；在 Hypervisor 模式，基于第二级转译表触发异常(Fault)，虚拟地址对应的中间物理地址 HPFAR_EL2 寄存器中；Hypervisor 基于这些寄存器信息，仿真虚拟外设运行，利用 ERET 指令跳回到虚拟 CPU

4. 系统内存管理单元(System MMU 或 IOMMU)

当虚拟机使用 DMA 时，基于系统内存管理单元的设置使 DMA 控制器访问的内存地址与虚拟机内的访问地址相同。

注，系统内存管理单元的用途不限于虚拟化支持。

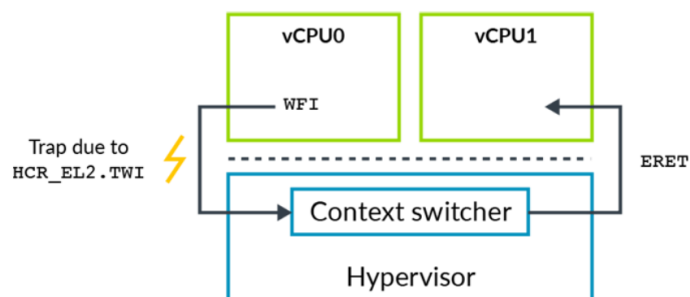


Hypervisor 编写 SMMU，使得中间物理地址到真实物理地址的映射与客户操作系统的中间物理地址空间到真实物理地址的映射相同。

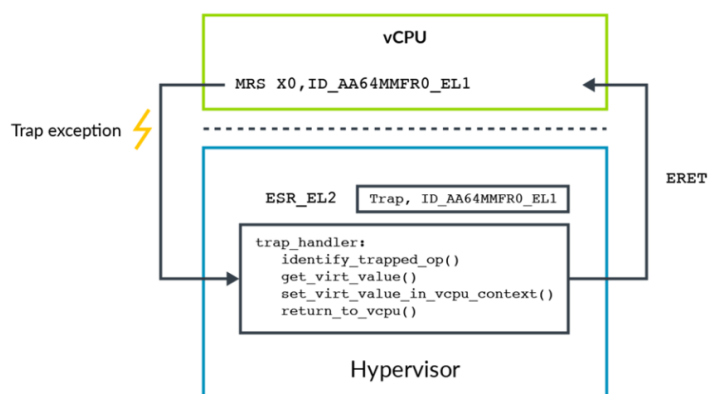
5. 陷落仿真指令

基于特定指令，引发异常到 Hypervisor 模式，以便仿真虚拟机中的指令运行，隔离对底层的设置。

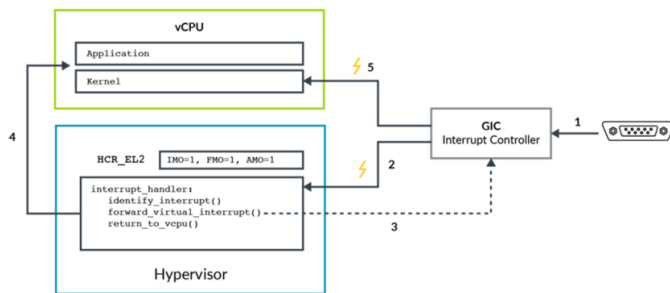
比方，虚拟机中执行等待中断指令，以便使 CPU 居于低功耗状态，该指令实际引发异常到 Hypervisor 模式，Hypervisor 调度别的虚拟 CPU。



比方，Hypervisor 基于 Trap 异常拦截对寄存器的读取指令，提供虚拟值。



6. 虚拟中断



外设激发中断信号到中断控制器(GIC, Generic Interrupt Controller); 中断控制器生成真实中断异常(Physical Interrupt Exception), 跳到 EL2 权限、Hypervisor 模式; Hypervisor 识别该中断对应的外设及虚拟机、虚拟 CPU, 设置中断控制器以便真实中断作为虚拟中断转发到虚拟 CPU; 中断控制器激发虚拟中断信号; Hypervisor 让渡控制权给虚拟 CPU, 虚拟 CPU 侦听到虚拟中断信号, 进行相应处理。

7. 虚拟化通用定时器(Generic Timers)

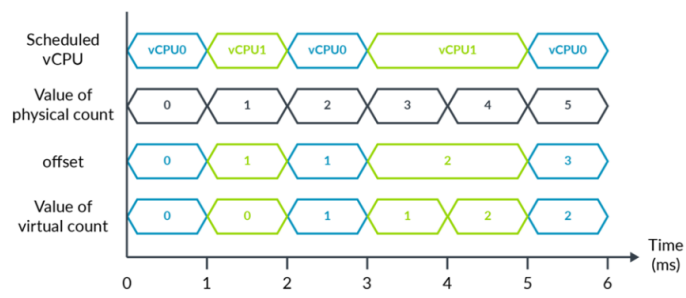
通用定时器基于一系列比较器(Comparators), 当比较器的值等于、小于系统计数(System Count)时, 生成中断。



虚拟机可以访问真实定时器或虚拟定时器。真实定时器基于真实计数器生成中断, 虚拟定时器基于虚拟计数器生成中断。

虚拟计数器值(Virtual Count) = 真实计数器值(Physical Count) – 虚拟偏移值(Virtual Offset)

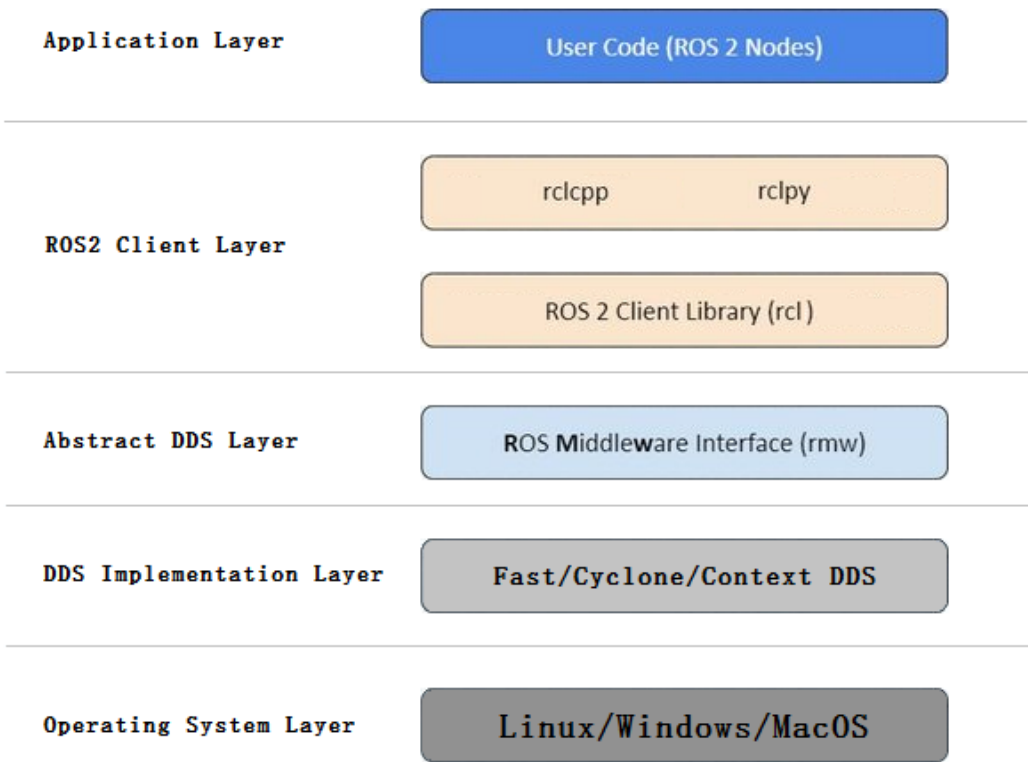
虚拟定时器时序图。



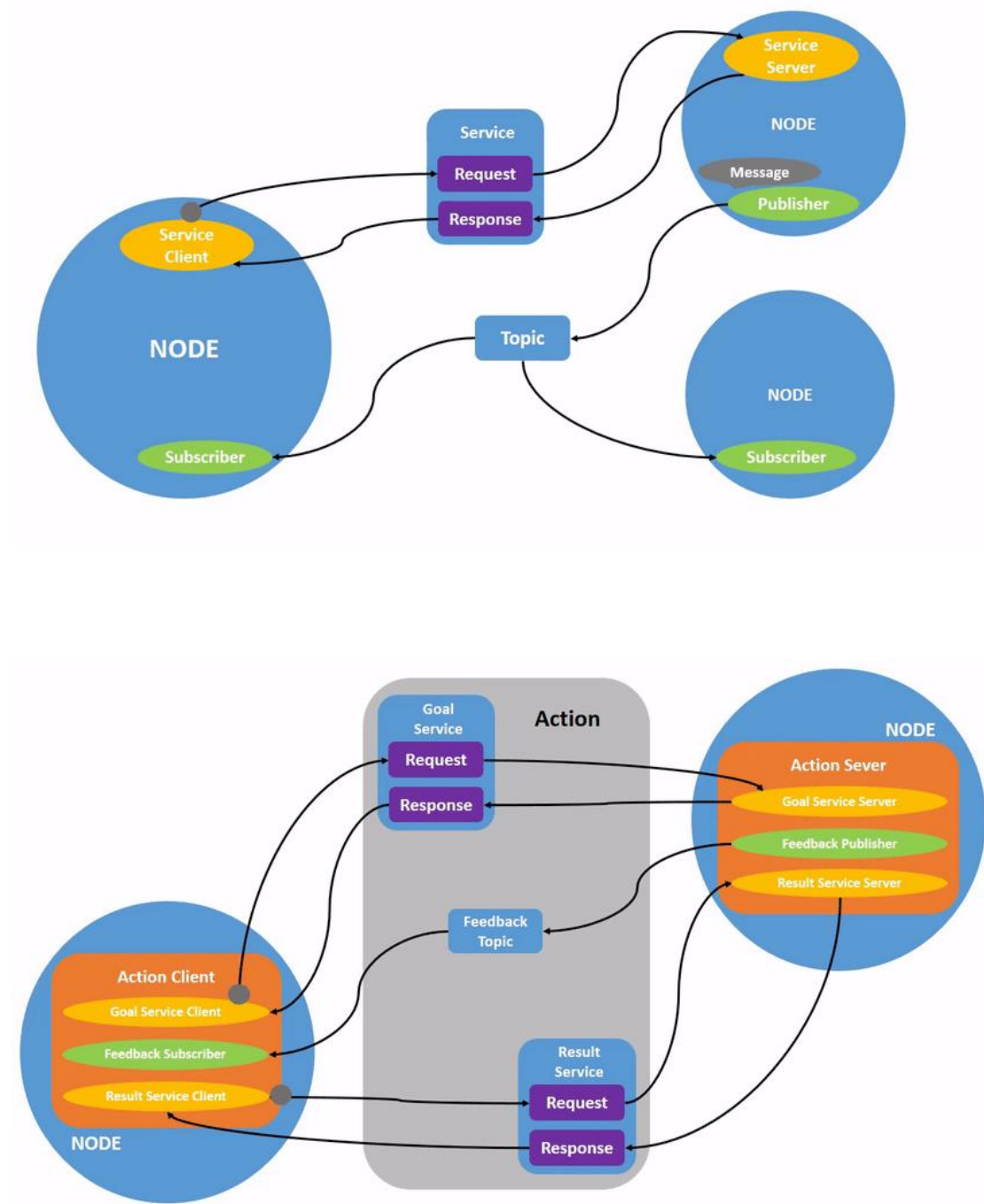
虚拟 CPU 当前的虚拟计数 = 真实计数 - Hypervisor 指定的当前虚拟 CPU 的偏移值

第三章 机器人系统-ROS2

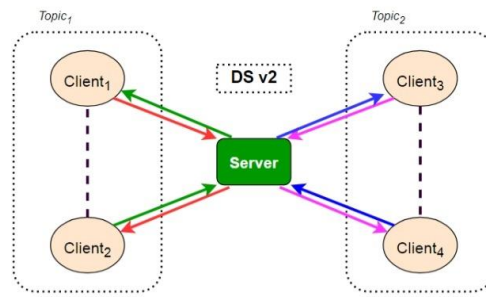
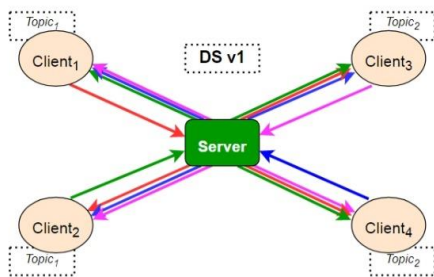
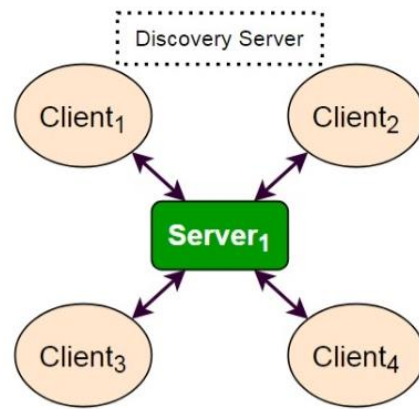
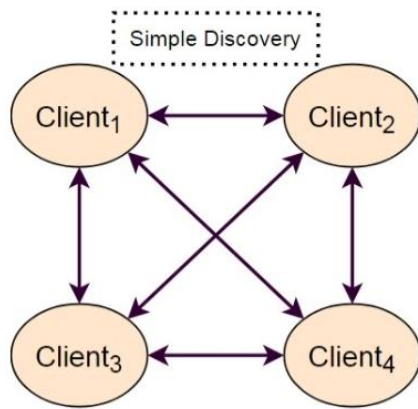
基于 DDS 通信机制的应用层机器人实时操作系统。



ROS2 每个节点具备独立的功能；节点之间通过 Topics , Services , Actions , Parameters 通信。



节点发现基于组播去中心化或集中化。



模型篇

人工智能的实质，在数学层面是基于异构基函数拟合人类思维函数。

基函数是定义及分析未知函数的普遍方法。比方，信号处理中，傅里叶级数(Series)或积分(Integral)基于正弦函数、余弦函数或指数函数(Complex Exponentials)表示周期或非周期函数；语音识别中，高斯混合模型(GMM, Gaussian Mixed Model)基于加权高斯基函数表示隐状态到音频帧的发射概率；机器人轨迹规划中，样条曲线(B-Spline)基于基函数线性组合定义轨迹等。

一、 模型要素

机器学习的要素，表征、模型架构、目标函数、优化算法。数据及算力次之。

表征，数据集尽量转换到正交或更广的高维空间进行表示，以提炼尽可能多的特征，同时去冗余。比方，傅里叶级数中，函数基于一系列正交的三角函数变换；支持向量机(SVM, Support Vector Machine)算法中的基函数(Base Function)；LLaMA 模型中旋转位置编码(RoPE, Rotary Positional Encoding)等。

模型架构，基于数据表征进行一系列的高维或低维空间转换，以计算表征在不同维度空间的特征；在同维度空间计算特征的拓扑关系；两者独立计算或交替进行，以实现目标的预测。比方，支持向量机，数据基于基函数转换到高维空间，再基于核函数(Kernel Function)计算关系；图像分类，图像基于卷积核提炼图像特征、池化特征，再分类；大模型语言翻译，分词表征基于不同权重矩阵转换到特征空间，进行注意力计算，再基于新的不同权重矩阵转换到新的特征空间，再进行注意力计算等。

目标函数，对预测的数据进行对齐或比对，同时防止过拟合或欠拟合。比方，变分自编码器(VAE, Variational Auto Encoder)基于编码图像到一个隐变量分布函数，再采样该分布函数来生成图像，生成的图像分布尽量与原始图像分布相似的同时，应使得编码后的隐变量分布与真实分布相似，以便采样是平滑的，或者说采样点不容易聚集于训练数据所对应的隐变量点，防止推理时图像生成缺乏泛化性。

优化算法，以有效、实用的方法计算目标函数关于模型参数的最优解，以便模型预测尽可能接近真相。比方，随机梯度下降法，基于函数的链式规则反向传播(BP, Backpropagation)计算目标函数的参数变量梯度，以更新参数。

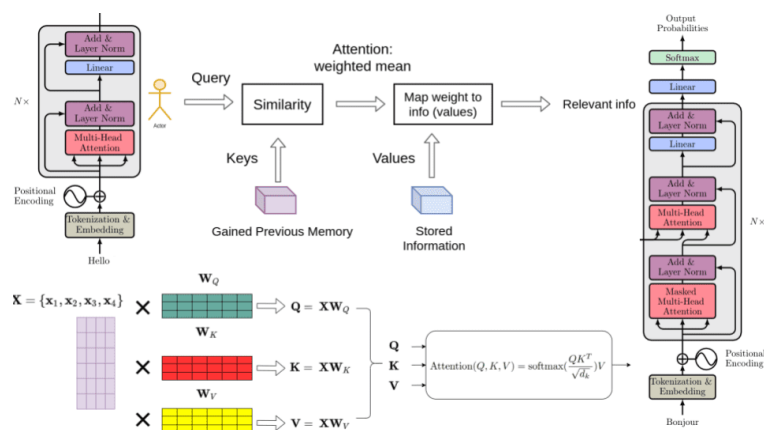
二、大模型

传统机器学习利用有限的训练拟合形式基本确定的函数，容易过拟合或缺乏泛化能力。

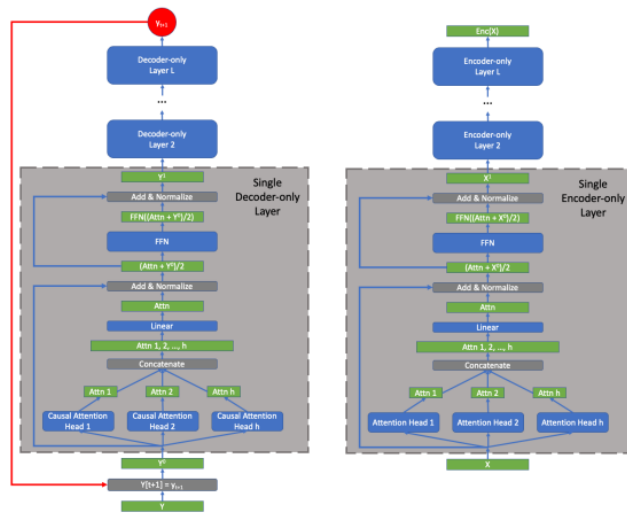
深度学习基于对神经网络的模仿，利用海量数据拟合具有泛化能力的函数模型。

基于 Transformer 及规模定律(Scaling Law)的大模型，利用多模态数据表征、高维张量注意力机制、专家混合路由模式、扩散或流匹配生成、梯度优化等算法，结合模仿学习、强化学习、GPU/NPU 集群、分布训练拟合可与现实对齐(Ground)、同时具有泛化能力的函数模型。

Transformer 框架示例。



Transformer Decoder-Only 及 Encoder-Only 框架。



注，Transformer 核心实质类似多个支持向量机(SVM)的测量加权。输入嵌量基于不同的基函数变换到相同维度空间，进行注意力测量计算(Attention)；多个测量结果加权计算综合注意力(Multi-Head Attention)；注意力基于前馈网络进行激活表征。

当前，存在着模仿人类情感价值观作奖励函数以及新的框架实现世界模型的趋势。

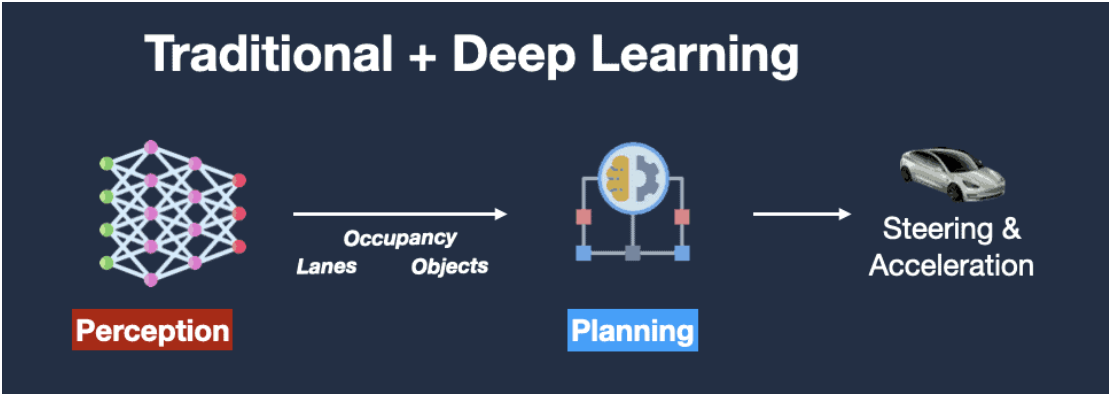
三、 机器人模型

机器人模型的发展有着独有的演进逻辑，同时，在感知、规划、决策等传统机器人三要素方面与自动驾驶也有着一定的相似度。

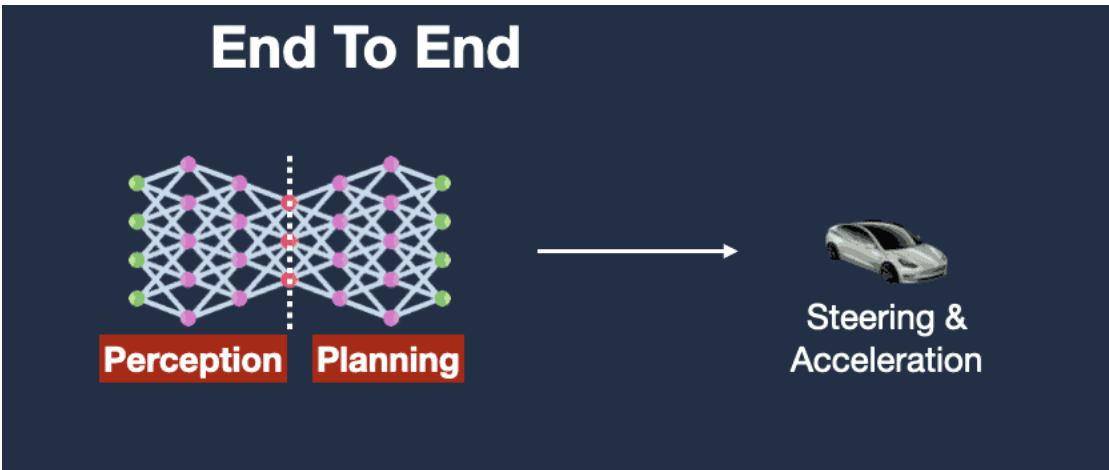
以 Tesla FSD 演进举例。

早期，FSD 的感知、规划、决策分别作为独立模块。基于 HydraNet 神经网络模型实现感知模块，检测目标、信号标记(Signs)、边线(Lane Lines)等环境信息。基于传统 A*算法、导航路由(Navigation Route)增强的 A*算法或神经网络增强的蒙特卡洛树搜索算法(Monte-Carlo Tree Search)实现规划模块。

之后，演变为在感知模块中添加占据网络(Occupancy Network)以增强感知模块对场景特征的理解；规划模块的蒙特卡罗树搜索算法利用感知模块生成的占据信息及检测信息进行路径规划。



再之后，利用神经网络实现规划模块，同时与感知模块的神经网络进行连接，形成端到端的感知、规划、决策架构。



未来，机器人模型可能模仿人类大脑、小脑等形成不同作用的模型，再通过类似神经纤维束的方法实现连通。

当前，通用机器人在环境感知、三维重建、语言交互、路径规划、步态平衡等方面都已有很大的商用可能，难点更多在于通过灵活机械手实现精细任务的实时操控。

深度学习模型的实质在于通过参数未知的异构基函数的组合来表示实际函数，利用因果或关系数据集对该组合函数的估计误差进行优化，以确定合适的基函数参数，形成实际函数的最可能近似定义。

因此，机器人要实现任务的灵活操控，至少要解决函数模型表示、因果数据集及机械感知问

题。

假定，基于已有的基函数可表示实际函数，利用肌腱线缆混合及光电传感可解决机械手的灵活控制及触觉感知，则在环境视觉信息、关节状态、语言指令等数据已知的情况下，找到机器人实现任务的结果数据就很关键。

通常的做法是人类遥操作机器人实现任务，以便机器人模型模仿学习轨迹等。不过，深度学习的问题在于通过海量不同数据训练以使模型具备泛化能力。因此，要收集海量数据，是不容易通过遥操作方法实现的。

强化学习是具有利用及探索(Exploit And Explore)能力的机器学习方法，可以利用合适的奖励函数使策略函数探索对应的动作轨迹尽快收敛到实际的轨迹。

深度强化学习利用深度学习模型拟合强化学习模型，可使得策略函数具备探索及泛化能力。

假定，可基于深度模型确定合适的奖励函数，再通过深度奖励函数指导深度策略模型，使得深度策略模型的训练完全自动化。

这就是说，不用收集海量的因果数据，机器人基于深度策略模型随机探索动作轨迹，只要使得奖励最大，便可找到可实现灵活任务、具备泛化能力的策略模型。

比方，任务，让机器人拿起桌面上的一瓶水，放到桌面另一位置。

基于多模态大模型作奖励函数，判断瓶到目标位置、瓶与手相对位姿、瓶不掉落、水不洒落、不碰到别的物体、动作平滑、用时最短、功耗最小，给予得分奖励。

基于视觉触觉语言动作生成大模型作策略函数，生成的关节电机力矩等序列，使得奖励最大，则对应策略函数作机器人操控模型。

同时，基于机器学习物理仿真环境，用模仿学习的机器人模型作起始策略模型，通过大规模并发训练，尽快实现商用化的机器人灵活控制模型是可能的。

总的来说，综合多模态世界模型、深度强化学习、模仿学习、深度物理引擎等技术的发展，加快通用机器人模型的可用性应是可能的。

第一章 理解交互

基于大语言模型，利用多模态实现自然语言理解及交互，目前已达可用的水平。

当然，在幻觉、实时性、逻辑的本质理解等方面还需很大的改进，甚至架构革新。

不过，就一般的场景，工程化、商业化应用是可能的。

主流的大语言模型，Gemini、Grok、GPT、Claude、LLaMA、DeepSeek 等。

一、 语音识别

ChatGPT 的语音识别(Whisper)基于 Web 收集的大规模、多样化数据进行多语言(Multilingual)、多任务(Multitask)弱监督(Weak Supervision)训练，不需微调(Fine Tuned)，具备更好的健壮性、通用性、更少的错误率。

Whisper 架构，基于 Encoder-Decoder Transformer 端到端系统。输入音频文件划分为 30 秒的片段(Chunk)，进行分帧(帧长 25ms，步长 10ms)及短时傅立叶变换得到梅尔频谱图(Mel Spectrogram)，取对数，传给 Encoder；Decoder 进行相应的文本预测，同时基于特定的标记(Tokens)执行语言识别(Language Identification)、多语言语音转录识别(Multilingual Speech Transcription)及语音翻译(Speech Translation)等任务。

1. 数据处理

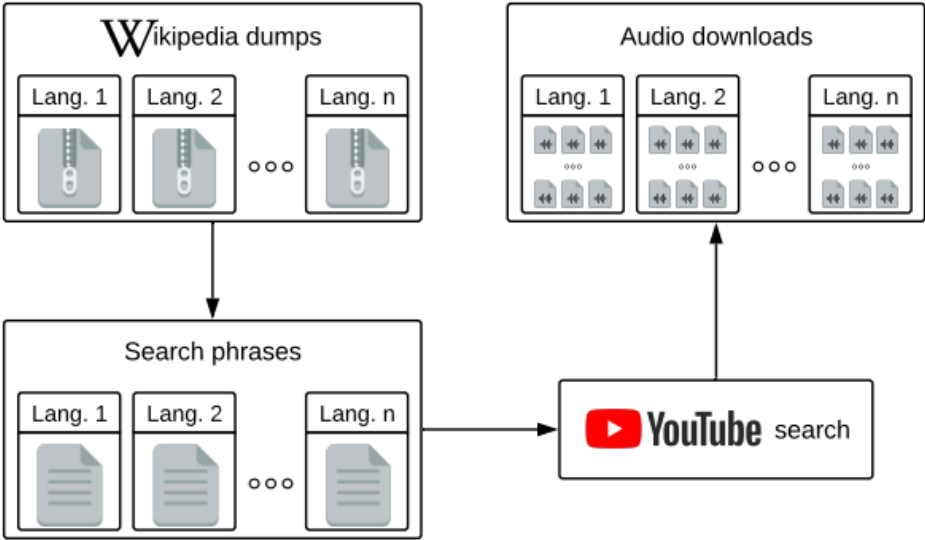
进行大规模、多语言、翻译集数据的收集、过滤及语音时频图谱转换。

1)、数据收集

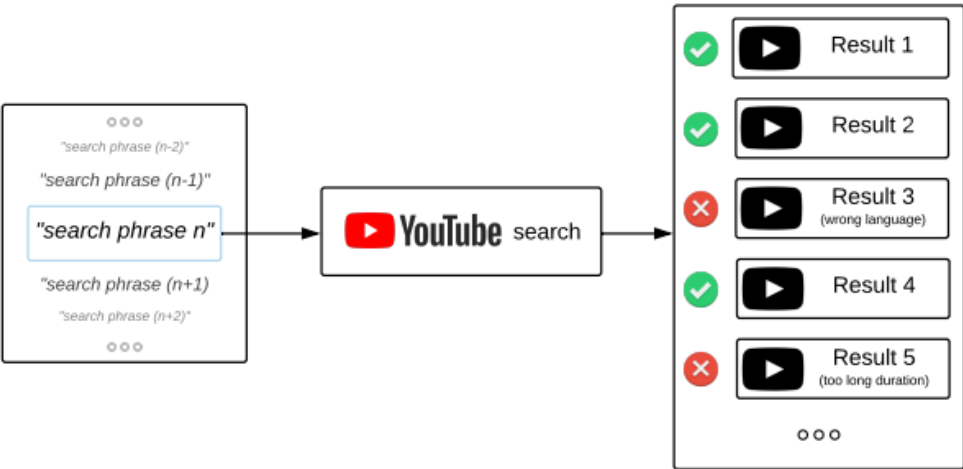
基于 Web 搜索、下载对应的不同语言的语音及文本。

数据收集示例，从 Wikipedia 下载不同语言的 Web 页面文本信息，基于 TF-IDF(Term Frequency Inverse Document Frequency)提炼文本短语语句，基于文本语言识别模型(Language

Identification Model, Python Polyglot)过滤得到期望的短语，在 YouTube 平台进行短语搜索下载与文本语句相对应的视频文件。



YouTube 平台搜索到的视频，基于文本语言识别模型对视频的标题、描述等信息进行过滤得到期望的视频文件。



从视频文件中提取音频文件，将音频文件划分为多个音频片段，基于 LIUM SpkDiarization(Java) 过滤掉噪声、音乐或静音的片段。然后基于众包(Crowd Sourcing)及生成式分类器(RoG, Robust Generative Classifier)进一步过滤掉不期望的片段。

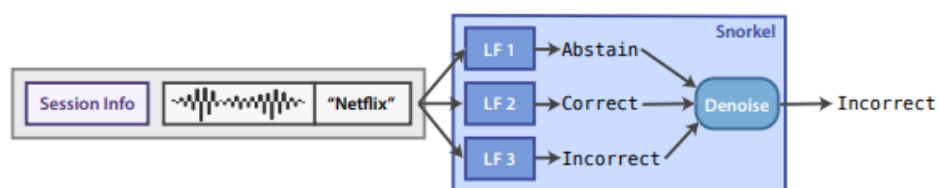
2)、文本质量提炼

基于启发式等方法过滤掉机器生成的质量不佳的文本。

基于优调的音频语言检测器(Audio Language Detector)确保语音与文本匹配。

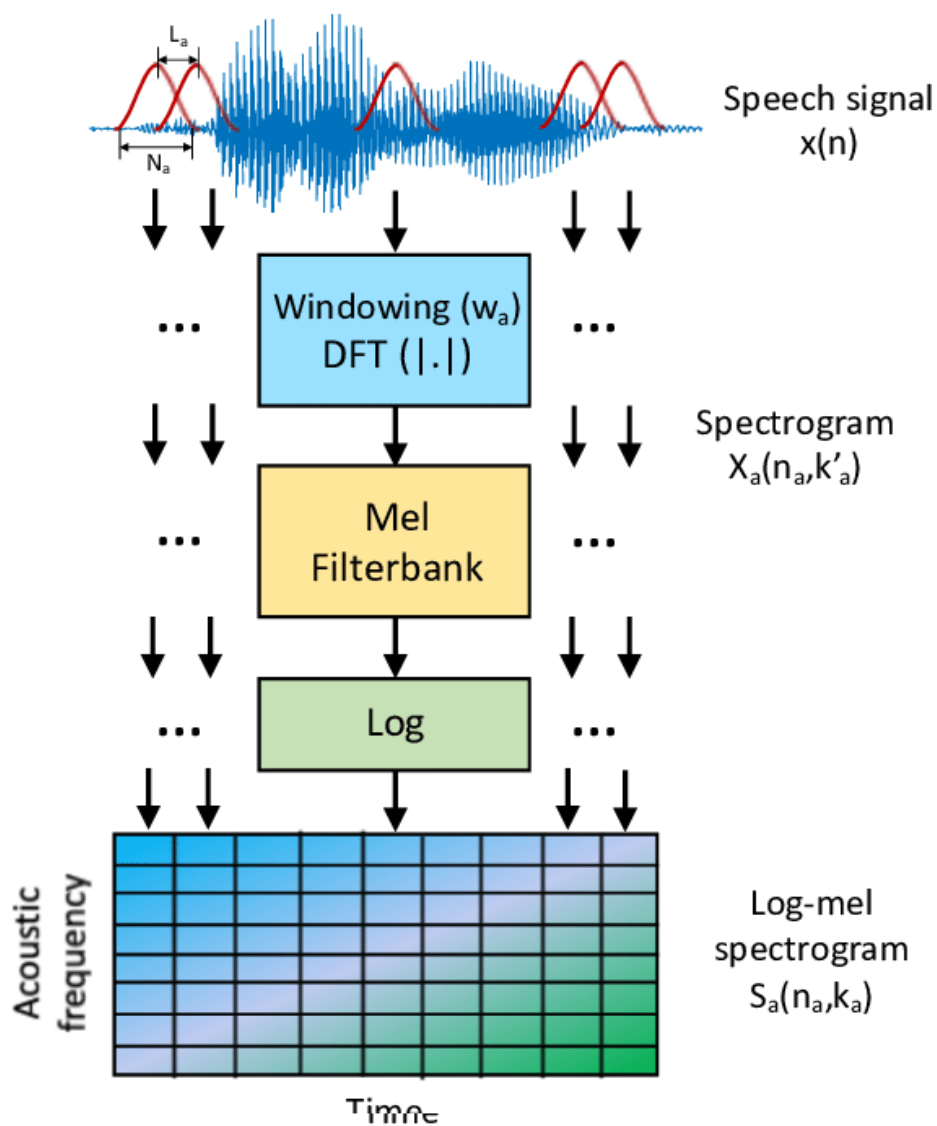
基于模糊去重方法去掉重复的文本。

基于标签函数(LF, Labeling Functions)过滤掉不期望的文本(Whisper 可能未用该方法)。语音片段及文本提供给标签函数，标签函数提示弃权(Abstain)等信息，去噪后，丢弃标记为弃权或不匹配的文本。



3)、梅尔频谱图(Mel Spectrogram)

对语音片段进行预加重、分帧、加窗及傅立叶变换，Mel 滤波后，取对数得到提供给模型的对数梅尔频谱图(Log-Mel Spectrogram)。

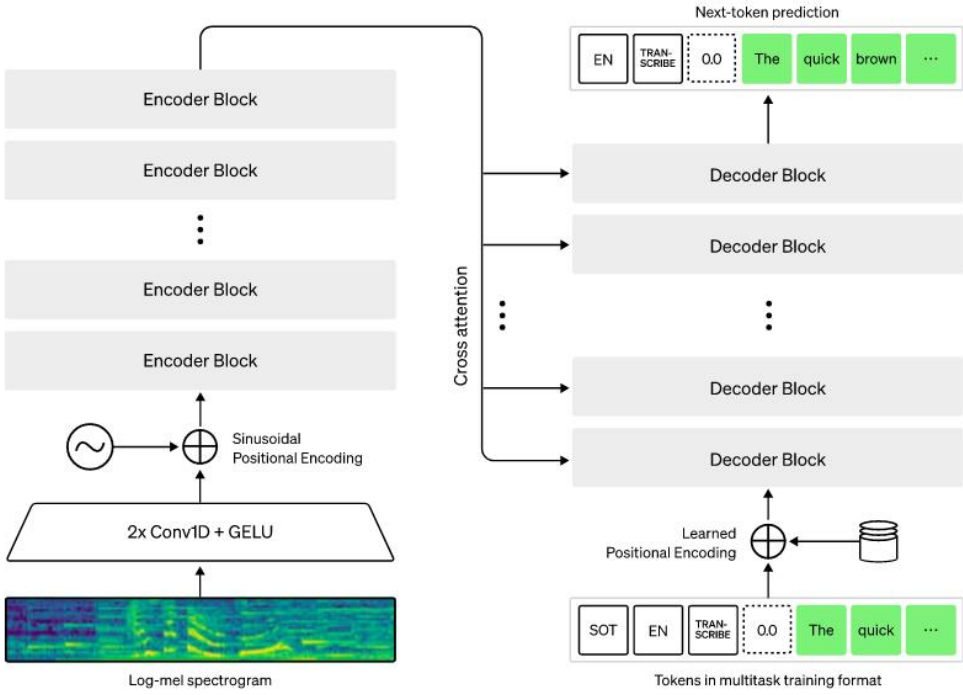


2. 模型分析

1)、模型框架

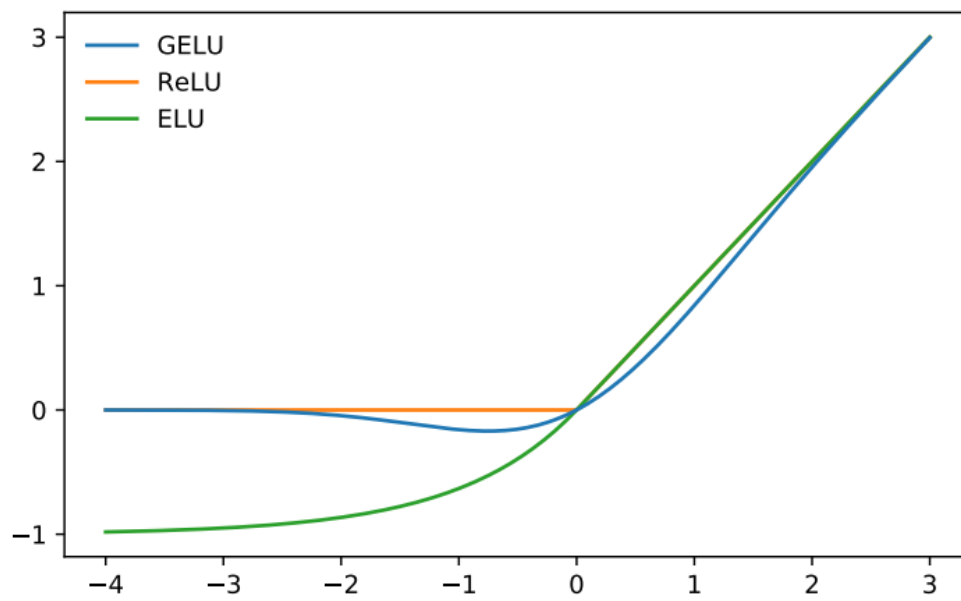
对频谱信息进行卷积(2 个卷积层，卷积核宽度 3)及高斯误差线性单元激活(GELU， Gaussian

Error Linear Unit), 通过位置编码(Sinusoidal Position Encoding)后提供给 Transformer 的编码栈 (Encoder Blocks)。解码栈基于学习的位置嵌量及编码栈的归一化输出进行预测。



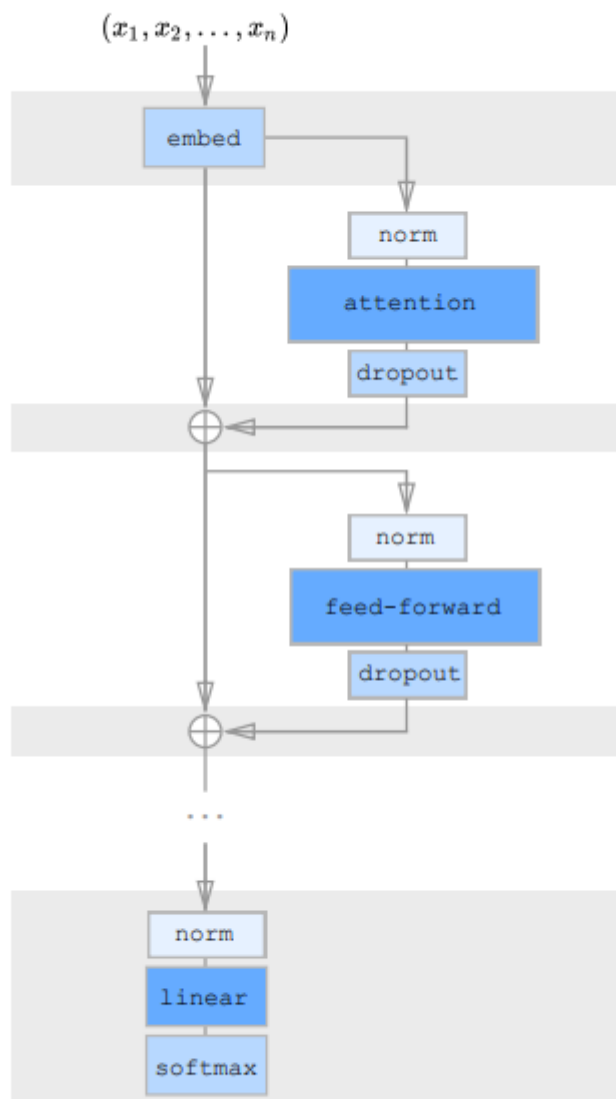
2)、GELU

基于高斯累积分布函数(Gaussian Cumulative Distribution Function)对输入进行非线性加权的高性能神经网络激活函数。



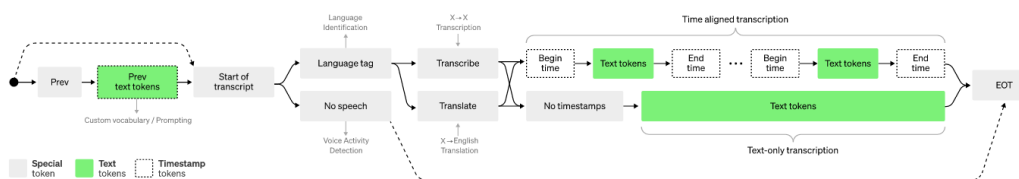
3)、Residual Block

模型 Transformer 基于残差块(Residual Block)进行改进，以提高性能、减少内存的使用。



3. 多任务

基于同一模型对同一音频信号进行文本转录识别(Transcription)、翻译(Translation)、声音激活检测(Voice Activity Detection)、文本语音对齐(Alignment)、语言识别(Language identification)等不同目的的训练及预测任务，利用不同的标记(Token)类型对任务进行定义或选择。



多任务流程，当前语音片段可前缀此前识别的文本作为解码器的上下文(Context)，开始识别(START OF TRANSCRIPT)后选择进行语言识别(LANGUAGE TAG)或声音激活检测，语言识别后选择进行文本转录识别(TRANSCRIBE)或翻译，选择是否进行时间戳(Timestamps)预测对齐，预测开始，预测语音帧相对于语音片段的开始时间标记(Start Time Token)、文本及结束时间标记(End Time Token)或只是文本预测，结束(EOT，END OF TRANSCRIPT)。

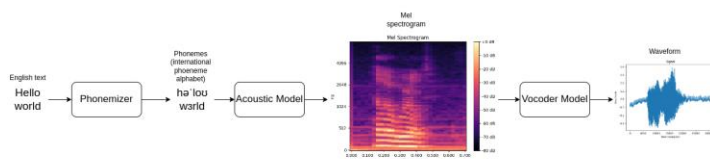
二、 语音合成

语音合成实质是离散信号转连续信号、数字信号转模拟信号，相似于语音识别的逆过程。

语音合成方法可分为，基于拼接合成(Concatenative Synthesis)、参数合成(Parametric Synthesis)及神经网络方法。

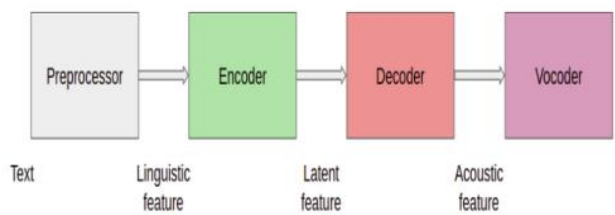
当前，基于神经网络的语音合成框架可分为多级管路(Multi-Stage Pipeline)及端到端(End-To-End)形式。

1. 多级管路框架



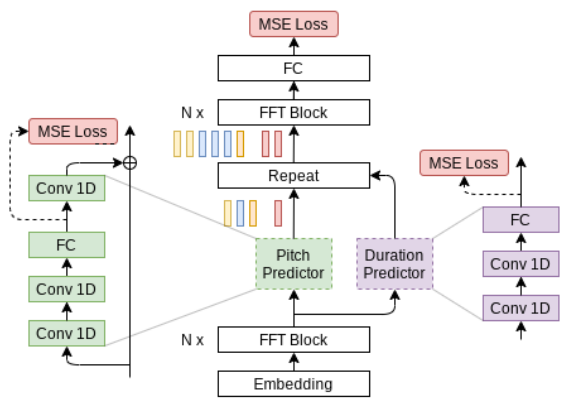
框架图，左侧，文本转换成音素(Phoneme)；居中，基于声学模型(Acoustic Model)，比方，FastPitch, Radtts 等，使音素转换成频谱图(Mel-Spectrogram)；右侧，基于声码器(Vocoder)，比方，自回归的 WaveNet、WaveRNN，非自回归的 Parallel WaveNet、WaveGlow 等，对抗生成类型的 MelGAN 等，使频谱图转换成音频波形(Waveform)。

声学模型可基于编解码框架定义。



框架图，预处理提炼文本的语言特征(Linguistic Feature)，基于编码器计算隐特征(Latent Feature)，经过解码器获得声学特征(Acoustic Feature)，传给声码器。

FastPitch 框架。



框架图，底部，文本词(Lexical Units)标记序列， $x=(x_1,...,x_n)$ ，基于 N 个前馈变换器(FFT, Feedfoward Transformer)块编码计算注意力隐表征 $h(R^{n \times d})$ ；居中，基于 1 维卷积的音高预

测器(Pitch Predictor)及时长预测器(Duration Predictor)预测隐表征音高(R^n)及时长

(N^n); 对音高投影, 加隐表征 $\mathbf{h}$, 基于时长进行离散上采样; 顶部, 基于前馈变换器计算上采样隐表征的注意力, 利用全连接计算频谱特征。

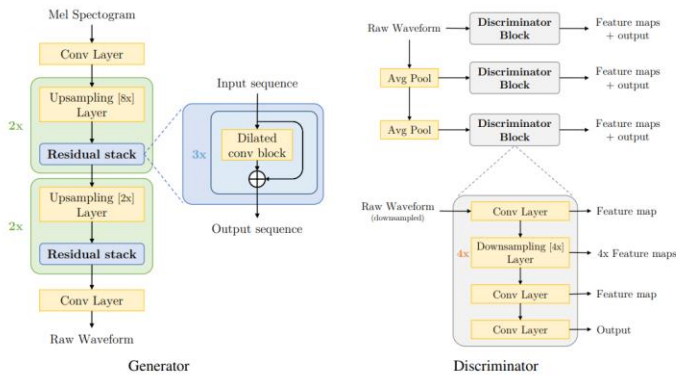
注, 前馈变换器, 标准变换器编码器块中的前馈神经网络替换成 1 维卷积。

上采样过程, 合成的隐表征 $\mathbf{s} = s^1, \dots, s^n$, 隐表征相应时长

$t = t_1, \dots, t_n$, 基于时长元素值, 对隐表征相应元素进行克隆,

$$\mathbf{s}' = s_1, \dots, s_{t_1}, \dots, s_1^n, \dots, s_{t_n}^n$$

MeIGAN 框架。



框架图, 左侧, 基于转置卷积上采样及膨胀卷积等进行波形生成; 右侧, 波形降采样到不同分辨率, 基于不同判别器(Discrminators)计算特征及标量值, 以定义生成器的目标函数。

注, 生成器目标函数, 不同判别器对生成器计算结果的和的期望 + 真值频谱与生成频谱基于不同判别器不同层特征图的匹配。

$$\min_G \left(\mathbb{E}_{s,z} \left[\sum_{k=1,2,3} -D_k(G(s,z)) \right] + \lambda \sum_{k=1}^3 \mathcal{L}_{\text{FM}}(G, D_k) \right)$$

$$\mathcal{L}_{\text{FM}}(G, D_k) = \mathbb{E}_{x,s \sim p_{\text{data}}} \left[\sum_{i=1}^T \frac{1}{N_i} \|D_k^{(i)}(x) - D_k^{(i)}(G(s))\|_1 \right]$$

D_k , 判别器函数。

$G(s,z)$, 生成器函数。

s , 生成的频谱，用于调控生成器。

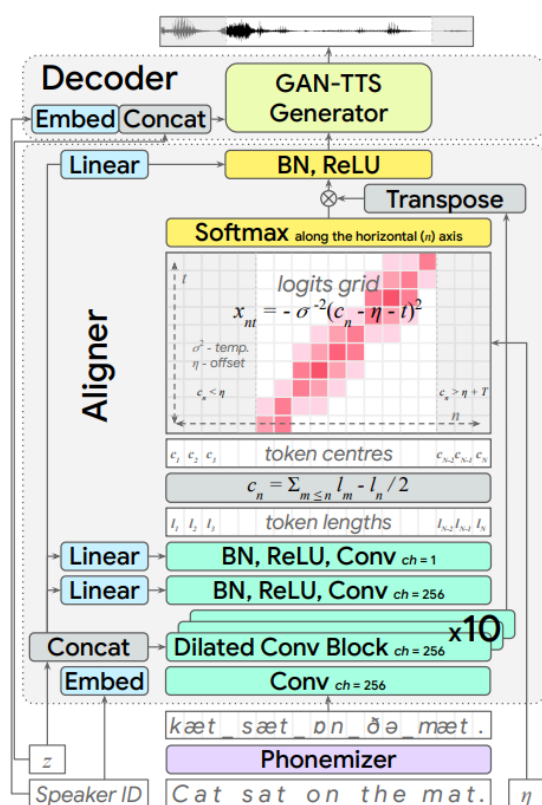
z , 高斯噪声。

$\mathcal{L}_{\text{FM}}$, 生成器、判别器的损失函数。

$D_k^{(i)}$, 第 k 个判别器的第 i 层特征图。

2. 端到端框架

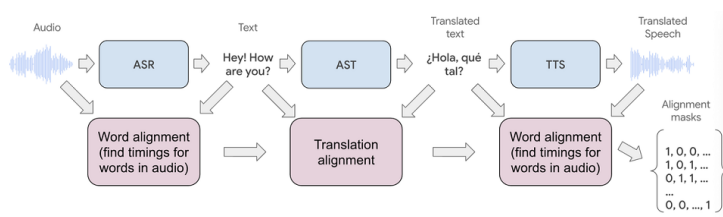
Google Deepmind EATS 框架。



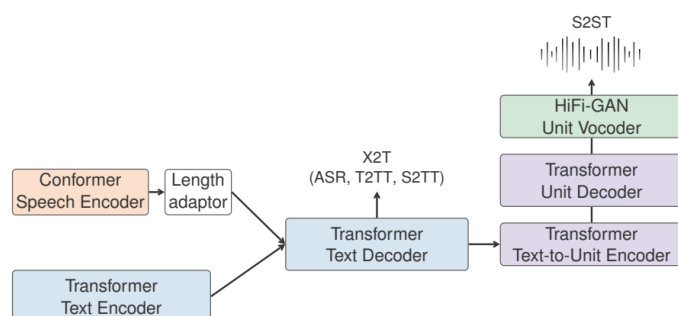
注，基于语音信号模拟连续的特征及因果关系，利用变换器(Transformer)、变分或扩散模型、流匹配(Flow Matching)实现端到端语音合成应是发展的方向。

三、 语音翻译

Google Real Time Speech To Speech Translation 级联框架。



SeamlessM4T 端到端框架。



框架图，左侧，基于卷积变换器(Conformer)进行语音编码；居中，基于文本解码器(Text Decoder)，利用亚词(Subwords)对语言序列(Linguistic Sequence)建模；右侧，底部，基于文本单元编码器(Text-To-Unit Encoder)对语言表征编码，传给单元解码器(Unit Decoder)计算声学表征(Acoustic Representation)；右侧，顶部，基于单元声码器(Unit Vocoder)计算波形。

注，语言编码器基于卷积变换器，文本、单元解码器、编码器都基于变换器(Transformer)。

四、 自然语言

基于深度学习模型的自然语言处理早期基于循环神经网络(RNN)、长短期记忆网络(LSTM)等，大发展始于基于 Transformer 框架的 GPT。

因此，这里主要以 ChatGPT 来说明自然语言处理模型。

ChatGPT 是基于 GPT(Generative Pre-Trained Transformer)自然语言处理模型的多模态聊天机器人。

ChatGPT 核心是基于 RNN、LSTM、Attention、Transformer 模型演进发展来的，通过自监督学习(SSL, Self Supervised Learning)预训练、监督学习微调(Fine Tuning)、强化学习实现非监督学习训练的自然语言处理的大语言模型(LLM, Large Language Model)。

RNN(Recurrent Neural Network)、LSTM(Long Short Term)是早期的循环网络模型，Attention 是机器翻译、问答等 Seq2Seq 场景下用于对输入与输出 Tokens 进行对齐的方法，Transformer 是基于 Self-Attention 机制，通过对输入 Tokens 进行语境关联改进 Seq2Seq 实现的模型。

自监督学习基于对部分数据自动生成标签来训练模型学习另一部分数据的特征,可用于机器视觉、自然语言处理模型预训练。

随着多模态(Multi-Modal)技术的发展, ChatGPT 自然发展为多模态大语言模型 (MLLM, Multi-modal Large Language Model), 该模型可以通过文字、图片等形式实现与外部的交互。

在作为大规模语言模型的训练时, GPT 先基于自监督学习自动预训练模型,再基于监督学习利用人工方法实现对预训练后的 GPT 模型的优调(Fine-Tuning); 然后用优调后的模型预测答案,再经过人工后打分训练强化学习中的奖励模型(Reward Model); 之后基于 GPT 模型的策略模型生成答案,用训练的奖励模型作为价值函数对答案打分,然后,再基于打分更新策略模型参数。基于第二及第三步,循环更新模型。

1. Transformer

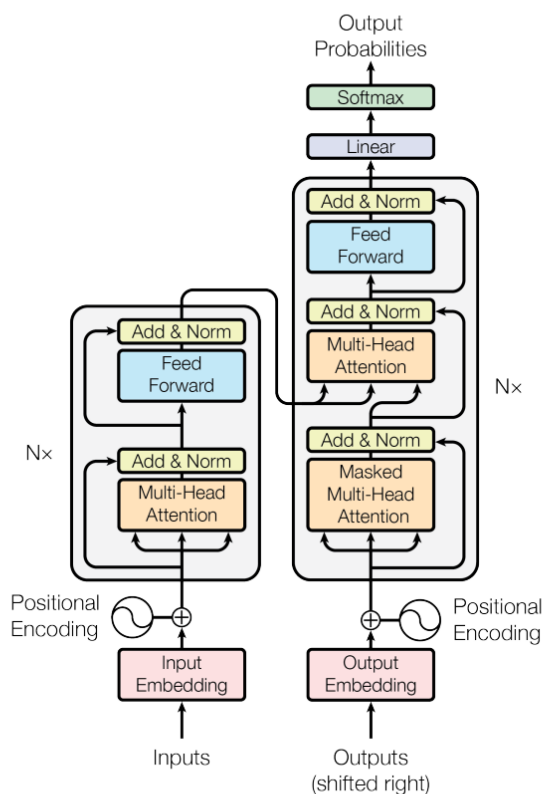
Transformer 基于 Encoders-Decoders 框架实现。

Encoders, 基于 N 个 Encoder 级联形成的 Encoder 栈。Encoders 的输入为包括输入序列元素语义及位置信息的嵌向量,输出为表示输入序列高级语义及位置特征的 Key 及 Value 向量。

Decoders, 基于 N 个 Decoder 级联形成的 Decoder 栈。Decoders 的输入为预测输出序列嵌向量位置编码后的向量,输出为表示输出元素位置的向量,经过 Linear 及 Softmax 层转换为输出元素概率向量。

Encoder 主要由自注意力层或多项注意力模块(Multi-Head Attention)与前馈模块(Feed Forward)组成。

Decoder 相对于 Encoder, 主要增加了一个基于多项注意力模块的、综合 Encoders 的输出组成输入信息的编解码层。



Encoder 的作用主要在于获取输入序列各元素之间语义与位置的关联信息。

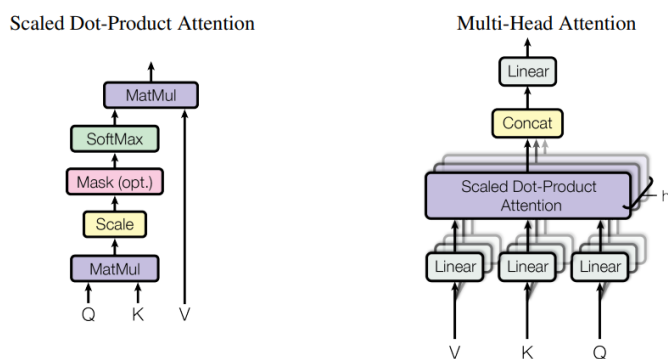
自注意力层(Self-Attention Layer)，或多项注意力模块(Multi-Head Attention)，用于计算 Encoder 输入向量序列之间的相关性。前馈层(Feed Forward Layer)用于提取注意力信息中的特征。

第一级 Encoder 的输入为基于语义及位置信息的嵌向量(Embedding Vector)。

嵌向量经过自注意力层生成的向量分别提供给不同的前馈神经网络生成语义及位置信息向量作为当前 Encoder 的输出及下一级 Encoder 的输入。

自注意力层，或多项注意力模块(Multi-Head Attention)，基于多个调整点积注意力模块(Scaled Dot-Product Attention)并联实现。

调整点积注意力模块(Scaled Dot-Product Attention)，对表示输入向量特征的 Query、Key、Value 向量进行点积等运算以获取输入向量之间的语义及位置关联信息。



基于 Q、K、V 的权重矩阵(W)计算 Encoder 输入向量对应的 Query、Key、Value 向量。

对于第一级 Encoder，基于权重矩阵计算不同嵌向量的 Query、Key、Value 向量。

基于 Query 与 Key 向量点积计算单词之间的相关性，进行归一化。

归一化的权重信息与相应的 Value 向量进行乘加运算作为 Encoder 自注意力层(Self-Attention Layer)的输出(z)。

基于不同的权重集 W 计算得到不同的 Q、K、V 信息，多个权重集对应多个 Q、K、V 信息集。

对多个注意力信息拼接后的信息进行降维，以提供给前馈层(Feed Forward Layer)。

Decoder 框架与 Encoder 框架基本一致，主要增加了基于多项注意力模块(Multi-Head Attention)的编解码注意力层(Encoder-Decoder Attention Layer)。

编解码注意力层基于 Encoders 输出的 Key、Value 信息及 Decoder 中的自注意力层的 Query 信息计算对应于输入序列语义及位置信息的输出信息。

Decoders 输出的向量基于 Linear 及 Softmax 层转换为词汇表概率向量。

2. 模型优调(Fine-Tuning)

第一步，基于监督学习方法，用手工编辑的问题及答案优调(Fine-Tuning) GPT-3.5 模型。

从人工编辑的提示数据(训练)集中随机选一些提示(prompt, commands or questions);

人为写下这些提示对应的答案，将这些答案作为调试 GPT 模型的标注答案(annotated answers);

用这些对应的提示与回答对(prompt-answer pair)来调试 GPT 模型;

第二阶段

用 GPT 模型生成的答案来训练奖励模型(RM, reward model)。

从人工编辑的提示数据(验证)集中随机选一些提示(prompt, commands or questions)，提供给优调后的 GPT 模型来生成答案;

每个提示可以对应 k 个答案;

标注人员可以基于一系列标准(相关性，信息量，危害性等)对这 k 个答案进行排名/排序;

基于常用的成对学习排名法，将这些排序后的答案作为标注数据用于训练奖励模型(RM, Reward Model);

将提示与答案对提供给奖励模型，由奖励模型输出一个打分(score)，对于答案排名高的，奖励模型输出的打分相应高。

第三阶段

基于策略模型与奖励模型的迭代训练更新策略模型参数。

用冷启动模型初始化 PPO 策略模型;

从人工编辑的提示数据(预测)集中随机选一些提示(prompt, commands or questions)，提供给策略模型，生成答案;

由第二阶段训练的奖励模型对 PPO 模型生成的答案进行打分;

基于强化学习策略梯度算法更新 PPO 参数;

重复第二步与第三步可以不断增强大语言模型的能力。

3. 多模态

多模态大规模语言模型(MLLM)通过对文本、图像等进行不同维度、不同层级的特征提取，隐状态语义匹配，再经过逆运算等变换生成相应的文本、图像等信息。

图像转文本。比方，Meshed-Memory Transformer 框架，基于不同 Encoder 层提取的不同层级的图像特征信息，进行不同层级的解码实现图像转文本预测。

文本转图像。基于不同的模型框架，生成与文本信息相应的图像。框架包括基于生成对抗网络的框架、基于变自编码器(VAE, Variational Autoencoder)的框架，基于扩散模型(Diffusion Models)的框架等。

在多模态模型训练时，可基于语言图像比对预训练(CLIP, Contrastive Language Image Pretraining)模型提取对齐的图像或文本特征。

第二章 感知导航

基于当前工业界自动驾驶发展水平，端到端实现通用人形机器人感知导航、规划决策是可能的。

当然，人形机器人是欠驱动(UnderActuated)的多自由度关节机器，步态平衡是最基本要解决的问题。

三维重建与 SLAM 定位建图有很大的相似度。

早期 SLAM 侧重于轨迹追踪定位，建立稀疏三维图像。随着深度学习的发展，两者已越来越融合。

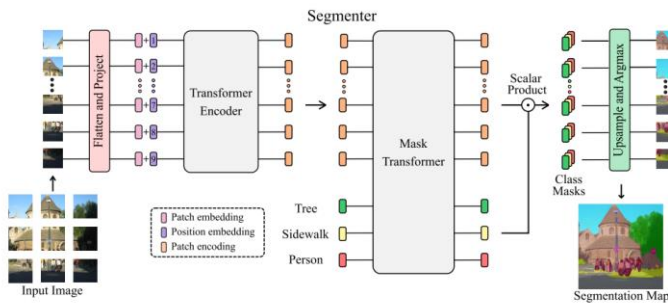
传统三维重建基于 TSDF 等方法实现重建，SLAM 基于光束平差、卡尔曼滤波、扩展卡尔曼滤波、图优化等方法实现机器人位姿优化、稀疏地图重建。

当前，基于神经辐射场(Neural Radio Field)、三维高斯泼溅(3D Gaussian Splatting)在实现机器人轨迹追踪的同时，可实时进行几何及光学准确度的稠密三维地图重建。

一、目标识别

分类、目标检测、语义分割、实例分割等。

早期 VGG、Inception、YOLO、FCN、Unet 等。当前，基于 Transformer 的 ViT、DETR、Segmenter 等。



Segementer 框架图，左侧，图像分片序列(Patch Sequence)基于编码器(Transformer Encoder, ViT Variants)计算分片之间的注意力或关联嵌量；右侧，分片注意力嵌量与分类嵌量(Learnable Class Embeddings)基于遮变换器(Mask Transformer)计算嵌量之间的注意力，再进行标量乘积(Scalar Product)，双线性插值到图像分辨率，与真实值对齐优化。

相对于早期方法，Segementer 同样是降采样、上采样，主要区别在于用 Transformer 的注意力嵌量计算替代 FCN、Unet 等框架中的加或拼接(Concatenation)上采样计算。

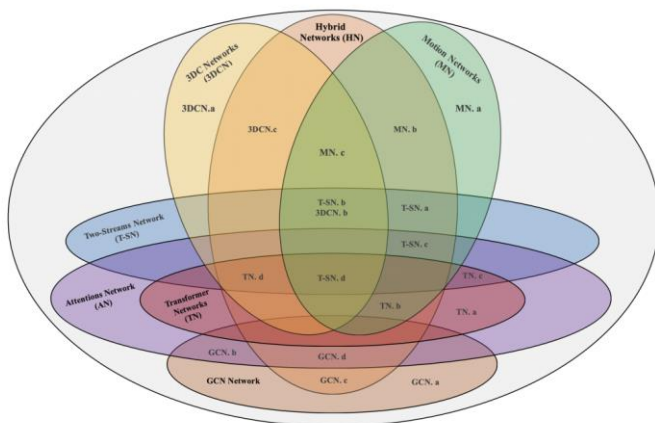
注，基于 Transformer 的自注意力、交叉注意力机制，似可直接像素嵌量化，结合分类注意力预测语义。

注，基于扩散模型等变分方法，利用分类信息调控似可能更利于进行语义预测。

二、 行为识别

人类行为识别(Human Action Recognition)，基于视频序列中目标运动的时域、空域动力学信息进行行为分析。

行为识别方法分类。



框架图，水平方向，双路网络(T-SN, Two-Stream Network)，注意力网络(AN, Attention Network)及变换器网络(TN, Transformer Network)，图卷积网络(GCN, Graph Convolutional Networks)；垂直方向，三维卷积网络(3DCN, 3D Convolutional Networks)，运动网络(MN, Motion Networks)，以及基于不同方法的混合网络(HN, Hybrid Networks)。

双路网络，基于空域及时域卷积网络，利用多帧稠密光流(Dense Optical Flow)训练预测行为。

注，光流，连续视频帧之间的图像目标亮度模式(Brightness Patterns)的表观移动(Apparent Motion)速度向量。

光流前提假设，连续帧中对象的像素强度不变及相邻像素存在相似移动。

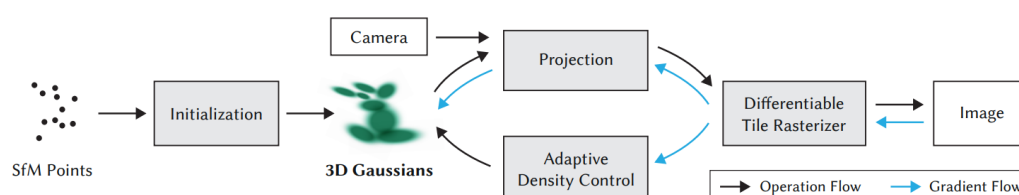
三、 三维重建

三维重建方法包括 TSDF、NeRF、3D Gaussian Splatting 等。以 3D Gaussian Splatting 示例。

3D Gaussian Splatting，基于辐射场原理、三维高斯函数及瓦片栅格化(Tile-Based Rasterization)方法实现三维场景椭球体素显式表示与实时渲染。

模型框架

基于三维场景空间稀疏点云实例化三维高斯椭球体(Ellipsoids)，利用图像瓦片投影原理(Tile-Based Splatting)及相机光线方向椭球体光辐射量计算图像像素颜色，通过误差优化计算更新三维高斯参数，同时调控三维场景椭球体数量及形状。



框架图，左侧，基于移动重建(SfM，Structure from Motion)算法、利用相机图像序列生成三维场景空间的稀疏点云，实例化以点云点位置作均值的三维高斯；居中，基于相机视锥区域及光辐射场原理计算相机光线方向三维高斯椭球体到相机图像空间的投影；右侧，基于瓦片渲染器(Differentiable Tile Rasterizer)计算投影像素对应的颜色。

优化控制(蓝线)，基于图像像素颜色误差及图像相似度误差进行优化计算，更新三维高斯参数，同时基于视域空间位置梯度(View Space Positional Gradients)等信息对三维高斯椭球体进行调控。

模型定义

一、三维高斯定义

$$G(x) = e^{-\frac{1}{2}(x)^T \Sigma^{-1}(x)}$$

x ，三维场景空间稀疏点云点位置向量。

Σ ，协方差矩阵。

二、高斯投影

1、三维高斯协方差定义

三维高斯的协方差矩阵定义，基于椭球体轴缩放(Scaling)及旋转(Rotation)，以确保矩阵的物理意义及半正定性(Positive Semi-Definition)。

$$\Sigma = RSS^T R^T$$

R ，旋转矩阵(Rotation Matrix)，四元数(Quaternion)表示。

S ，缩放矩阵(Scaling Matrix)，三维向量(3D Vector)表示。

2、投影的协方差定义

三维高斯椭球体投影到相机二维图像空间椭圆的协方差。

$$\Sigma' = JW \Sigma W^T J^T$$

Σ' , 二维图像空间椭圆的协方差。

J , 相机投影变换的仿射近似的雅可比(Jacobian)。

W , 相机视域变换(Viewing Transformation)。

三、像素颜色定义

$$C = \sum_{i \in N} \prod_{j=1}^{i-1} (1 - a_j)$$

c_i , 相机光线方向三维空间高斯椭球体颜色, 基于球谐函数(SH, Spherical Harmonics)计算。

a_i , 相机光线方向三维空间椭球体投影的不透明度(opacity), 基于二维图像空间投影椭圆高斯与三维空间高斯椭球体的不透明度的乘积计算。

四、损失函数定义

$$L = (1 - \lambda)L_1 + \lambda L_{D-SSIM}$$

L_1 , 均值误差(MAE, Mean Absolute Error)。

L_{D-SSIM} , 图像相似度(D-SSIM, Displacement Structural Similarity Index Measure)。

优化计算时, 基于链式规则分别计算 Σ' 对三维缩放向量及旋转四元数的导数。

优化控制

一、优化控制

Algorithm 1 Optimization and Densification
 w, h : width and height of the training images

```
 $M \leftarrow \text{SfM Points}$  ▷ Positions  
 $S, C, A \leftarrow \text{InitAttributes}()$  ▷ Covariances, Colors, Opacities  
 $i \leftarrow 0$  ▷ Iteration Count  
while not converged do  
   $V, \hat{I} \leftarrow \text{SampleTrainingView}()$  ▷ Camera  $V$  and Image  
   $I \leftarrow \text{Rasterize}(M, S, C, A, V)$  ▷ Alg. 2  
   $L \leftarrow \text{Loss}(I, \hat{I})$  ▷ Loss  
   $M, S, C, A \leftarrow \text{Adam}(\nabla L)$  ▷ Backprop & Step  
  if  $\text{IsRefinementIteration}(i)$  then  
    for all Gaussians  $(\mu, \Sigma, c, \alpha)$  in  $(M, S, C, A)$  do  
      if  $\alpha < \epsilon$  or  $\text{IsTooLarge}(\mu, \Sigma)$  then ▷ Pruning  
         $\text{RemoveGaussian}()$   
      end if  
      if  $\nabla_p L > \tau_p$  then ▷ Densification  
        if  $\|S\| > \tau_S$  then ▷ Over-reconstruction  
           $\text{SplitGaussian}(\mu, \Sigma, c, \alpha)$   
        else ▷ Under-reconstruction  
           $\text{CloneGaussian}(\mu, \Sigma, c, \alpha)$   
        end if  
      end if  
    end for  
  end if  
   $i \leftarrow i + 1$   
end while
```

1、三维高斯实例化

定义三维高斯均值参数集(**M**)，基于相机图像序列与移动重建(**SfM**)生成三维场景稀疏点云。

定义协方差(**S**)、颜色(**C**)及不透明度(**A**)参数集，基于稀疏点云实例化三维高斯集。

2、优化计算

选择相机视域(**V**)及样本图像。

基于三维高斯及相机视域进行栅格化渲染得到投影图像(**I**)。

基于投影图像与样本图像的损失函数进行优化计算,通过随机梯度下降法反向传播更新三维高斯参数。

3、稠密控制

对三维高斯椭球体集进行裁剪调控。

三维高斯椭球体不透明度太小或椭球体太大，则去掉(Prune)该三维高斯椭球体。

损失函数位置梯度大于阈值，过重建(Over-Reconstruction)，协方差大于阈值，用两个小的三维高斯进行替换；欠重建(Under-Reconstruction)，协方差小于等于阈值，在位置梯度方向对三维高斯进行复制。

二、栅格渲染

基于图像瓦片化与基数排序(Radix Sort)算法，利用近似图层混合(α -Blending)方法实现三维场景高斯各向投影(Gaussian Anisotropic Splats)的栅格化渲染。

Algorithm 2 GPU software rasterization of 3D Gaussians
 w, h : width and height of the image to rasterize
 M, S : Gaussian means and covariances in world space
 C, A : Gaussian colors and opacities
 V : view configuration of current camera

```
function RASTERIZE( $w, h, M, S, C, A, V$ )
  CullGaussian( $p, V$ )                                ▶ Frustum Culling
   $M', S' \leftarrow$  ScreenspaceGaussians( $M, S, V$ )    ▶ Transform
   $T \leftarrow$  CreateTiles( $w, h$ )
   $L, K \leftarrow$  DuplicateWithKeys( $M', T$ )           ▶ Indices and Keys
  SortByKeys( $K, L$ )                                   ▶ Globally Sort
   $R \leftarrow$  IdentifyTileRanges( $T, K$ )
   $I \leftarrow 0$                                        ▶ Init Canvas
  for all Tiles  $t$  in  $I$  do
    for all Pixels  $i$  in  $t$  do
       $r \leftarrow$  GetTileRange( $R, t$ )
       $I[i] \leftarrow$  BlendInOrder( $i, L, r, K, M', S', C, A$ )
    end for
  end for
  return  $I$ 
end function
```

去掉在相机视锥体区域(Frustum)远端、近端边界及外部的三维高斯。

基于相机视域信息、三维高斯均值集与协方差集，计算三维高斯投影到二维图像椭圆位置的均值与协方差集。

基于需栅格化图像的宽、高，定义瓦片集(T)，瓦片(16*16 Pixel)。

基于三维高斯投影均值集与瓦片集定义高斯投影对应的瓦片索引与键值列表。每个瓦片对应一个投影序列(Splat List)；每个投影实例对应一个键值；每个键值(Key, 64bits)，高位，瓦片索引，低位，投影深度。

基于瓦片索引，利用基数排序算法对键值列表进行排序。

基于瓦片集与排序的键值列表定义瓦片对应的键值区间。

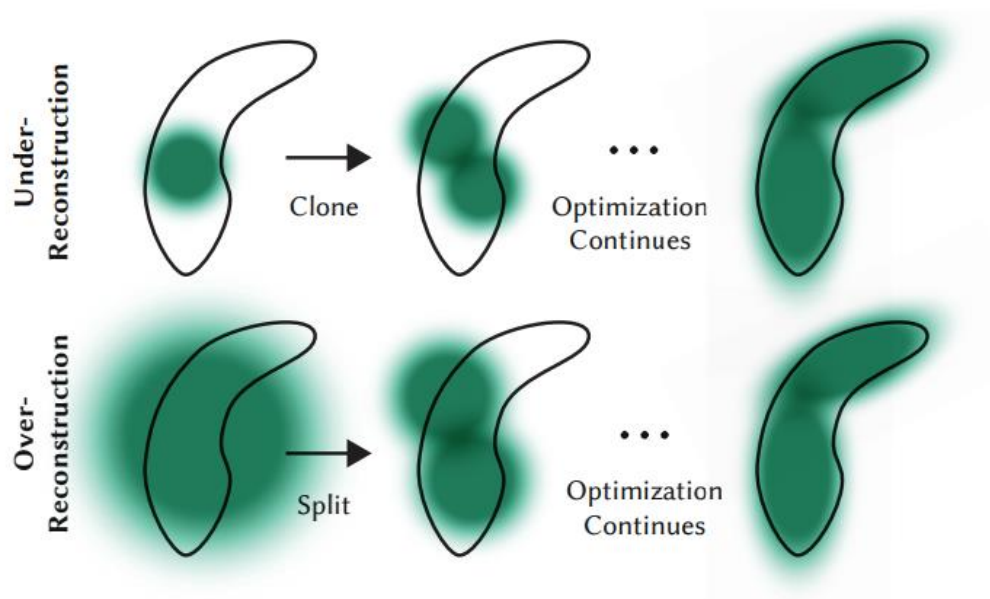
实例化画布。

基于瓦片索引获得瓦片对应的键值区间。

基于像素索引、瓦片索引集、键值区间、投影均值、投影协方差及三维场景高斯颜色等信息，利用图层混合方法计算像素颜色。实际计算时，可通过基于共享内存的线程块(Thread Block)实现并行化的栅格渲染。

生成图像。

三、稠密控制



高斯投影小于小尺度几何(Small-Scale Geometry)时，复制三维高斯；大于时，拆分三维高斯。

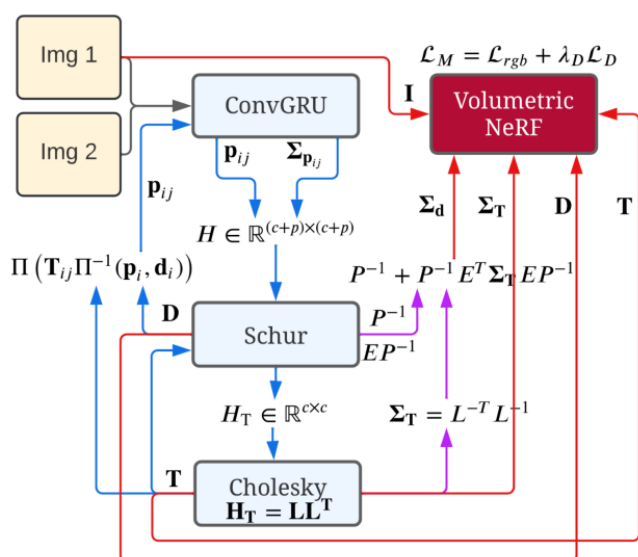
四、 SLAM

当前基于深度学习的 SLAM 有 DROID-SLAM、NeRF-SLAM、Gaussian Splatting SLAM 等。以 NeRF-SLAM 示例。

NeRF-SLAM，基于单目稠密 SLAM 系统估计的位姿及深度图，通过置信加权，监督训练哈希层级表征的神经辐射场，实现定位追踪及光学、几何准确的三维重建。

总体框架

NeRF-SLAM 基于 Instant-NGP 的哈希层级表征方法拟合神经辐射场，利用 DROID-SLAM 系统估计的相机位姿、像素深度图及深度置信(Uncertainty)进行色彩、深度对齐，优化模型。实际实现时，分别基于追踪线程(Tracking Thread)生成关键帧，优化光束平差，计算位姿、深度及置信；基于建图线程(Mapping Thread)进行建图、渲染及展示。



框架图，左侧，类似于 DROID-SLAM 方法，基于起始(Init)或预测的相机位姿、像素深度，利用逆投影、投影变换计算视频帧 i 到 j 的光流 $\mathbf{p}_{ij}$ ；再结合 RAFT 的卷积门循环单元(ConvGRU, Convolutional GRU)预测更新光流 $\mathbf{p}_{ij}$ 及相应的权重 $\Sigma_{\mathbf{p}_{ij}}$ ；基于视频帧图(Frame Graph)的光束平差法代价函数利用高斯-牛顿法进行一阶泰勒展开，得到包括相机位姿矩阵及像素深度矩阵的稀疏块状的海森矩阵(Block-Sparse Hessian) $\mathbf{H}$ ；利用块状矩阵的互补方法(Schur Complement)计算相机位姿矩阵 $\mathbf{H}_{\mathbf{T}}$ ；矩阵分解(Cholesky factorization)为下三角矩阵

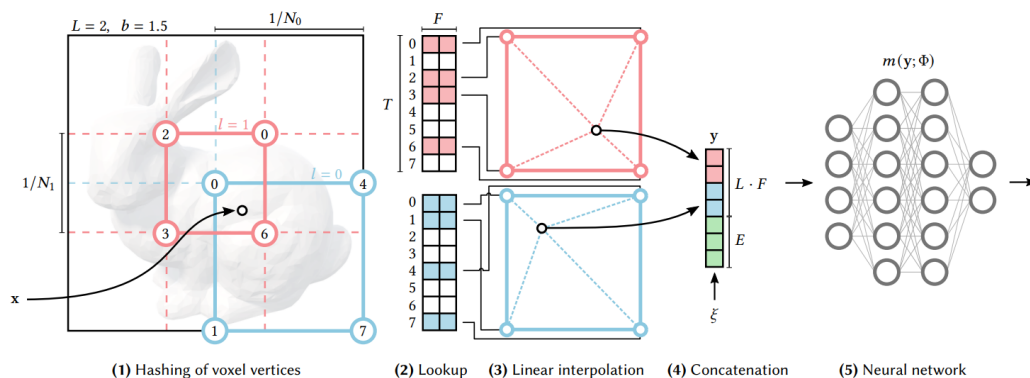
(Lower-Triangular Matrix)与上三角矩阵(Upper-Triangular Matrix)积; 利用前代、回代算法(Front And Back Substitution)分别计算下、上三角矩阵, 得到相机位姿; 基于相机位姿计算像素深度;

右侧, 下方, 基于矩阵互补及分解计算, 可分别得到像素深度、相机位姿的边缘协方差(Marginal Covariances) Σd 、 ΣT ;

右侧, 上方, 基于协方差及预测的像素深度 D 、相机位姿 T 定义哈希层级表征的神经辐射场的损失函数, 进行优化训练;

表征

相对于 NeRF 原有的对三维空间采样点方位每个坐标分量进行位置编码, 通过神经网络实现坐标到色彩、密度映射的方法, 哈希层级表征坐标有利于减少多层感知器网络的大小, 提高计算性能。



基于二维图像示例, 左侧, 图像可划分为 L 个级别、不同分辨率的栅格; 坐标 x 存在于不同级别的栅格的单元(Cell)中; 对 x 的坐标分量分别计算上边界、下边界, 形成 4 个坐标对(Pairs); 每个坐标对的分量进行加权异或、模 T 计算, 得到 4 个独立或重合的哈希表索引;

居中, 基于索引找到 4 个维度 F 的特征量表项; 基于特征量, 进行双线性插值(Bi-Linear Interpolation), 计算坐标 x 的特征量坐标; 每个坐标 x 对应的 L 个特征量坐标与额外的维度 E 的视野方向、纹理等信息进行拼接, 形成维度 $L \cdot F + E$ 的编码;

右侧, 编码传给多层感知器网络计算; 训练优化时, 损失梯度反向传播过多层感知器、拼接、线性插值, 基于特征量(Looked-Up Feature Vectors)进行累加(Accumulated)。

注, 三维空间进行体格划分; 相应的坐标对数乘倍 (2^d); 基于三线性插值(Trilinear Interpolation)计算特征量坐标。

追踪

一、光流更新

基于视频帧对关系、投影计算的光流，计算更新的光流及权重。

$$\sum p_{ij}$$

光束平差法的最小方差代价函数基于高斯牛顿法展开为一阶泰勒级数，得到稀疏块状海森矩阵。

$$H \in R^{(C+P) \times (C+P)}$$

海森矩阵分区(Block-Partition)展开，利用互补(Schur Complement)、矩阵分解(Cholesky Decomposition)等方法计算位姿、深度更新。

基于 DROID-SLAM 的迭代框架计算更新光流及后序的位姿、深度。

二、更新计算

计算相机位姿、像素深度更新。

海森矩阵进行分区。

$$H\mathbf{x} = \mathbf{b}, \quad i.e. \quad \begin{bmatrix} C & E \\ E^T & P \end{bmatrix} \begin{bmatrix} \Delta\boldsymbol{\xi} \\ \Delta\mathbf{d} \end{bmatrix} = \begin{bmatrix} \mathbf{v} \\ \mathbf{w} \end{bmatrix}$$

H, 海森矩阵。

b, 残差。

C, 相机位姿矩阵。

P, 像素逆深度对角矩阵。

E, 相机/深度的非对角块矩阵。

$\Delta\boldsymbol{\xi}$, 相机位姿特定欧几里德群 SE(3)的李代数更新(Delta Updates)。

$\Delta\mathbf{d}$, 像素逆深度的更新。

$\mathbf{v}$, 位姿残差。

$\mathbf{w}$, 深度残差。

海森矩阵 H 关于 P 的互补(Schur Complement)H/P。

$$(H/P)\Delta\boldsymbol{\xi} = [C - EP^{-1}E^T] \Delta\boldsymbol{\xi} = (\mathbf{v} - EP^{-1}\mathbf{w})$$

基于矩阵分解(Cholesky Decomposition)相机位姿矩阵, $H^T = LL^T$ 。

通过前向、反向替代计算位姿更新 $\Delta\boldsymbol{\xi}$ 。

基于位姿更新计算深度更新 $\Delta\mathbf{d}$ 。

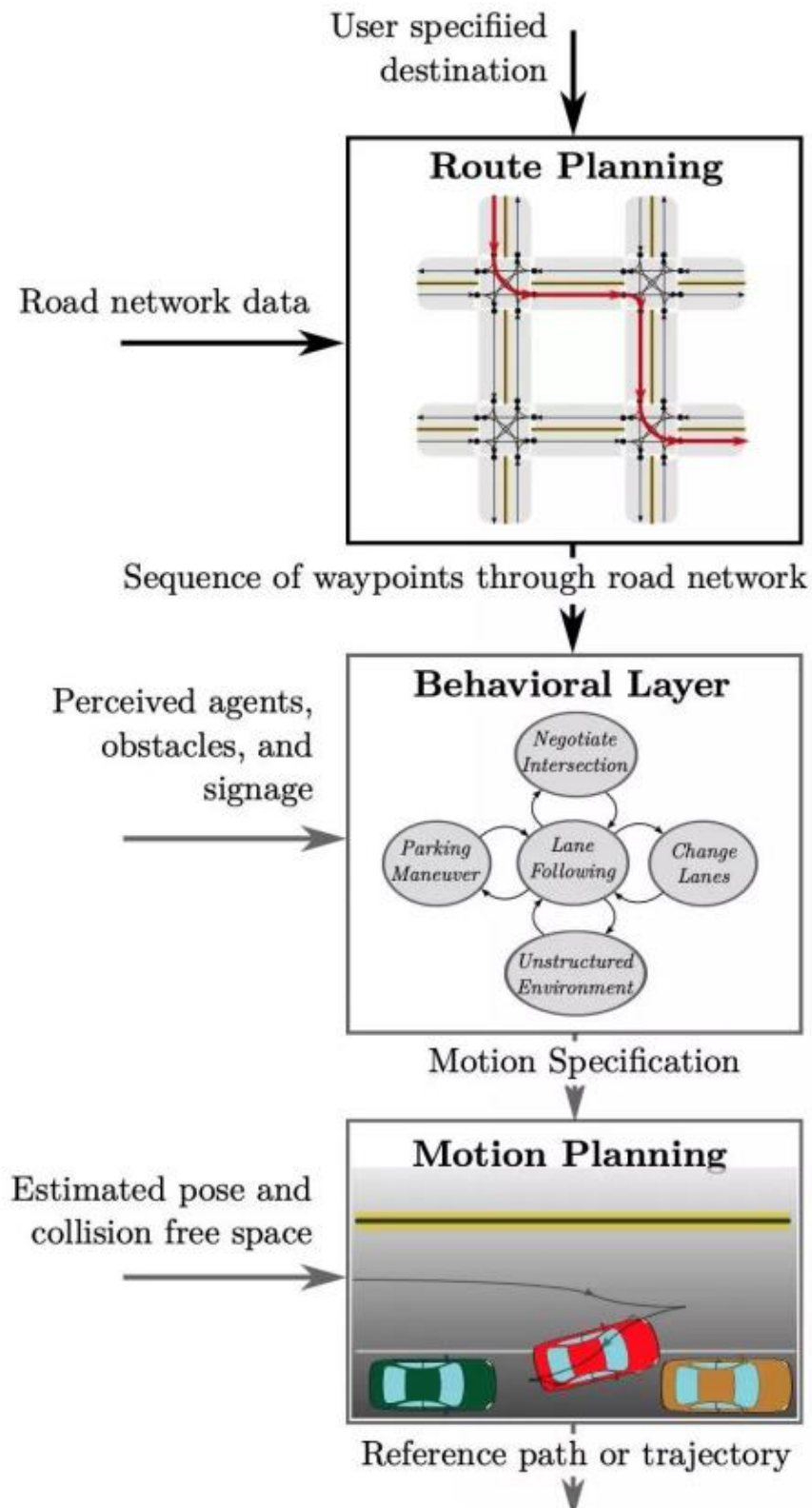
三、置信计算

基于海森块状矩阵方程，利用同样的互补、分解法，计算位姿、深度图的边缘协方差(Marginal Covariances)或不确定性(Uncertainty)、置信度(Confidence)。

$$\begin{aligned}\Sigma_{\mathbf{d}} &= P^{-1} + P^{-T} E^T \Sigma_{\mathbf{T}} E P^{-1} \\ \Sigma_{\mathbf{T}} &= (L L^T)^{-1}\end{aligned}$$

第三章 规划决策

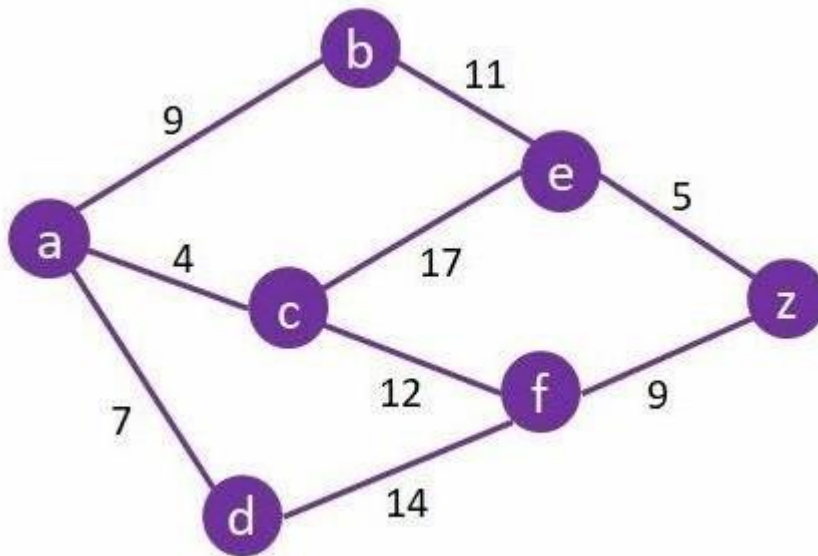
规划包括路径规划(Route Planning, Path Planning)及运动规划(Motion Planning)。路径规划在于进行全局、高层级的规划，运动规划在于进行具体的、实时的规划。运动规划一般依照路径规划进行，也可能改变路径规划。



一、 路径规划

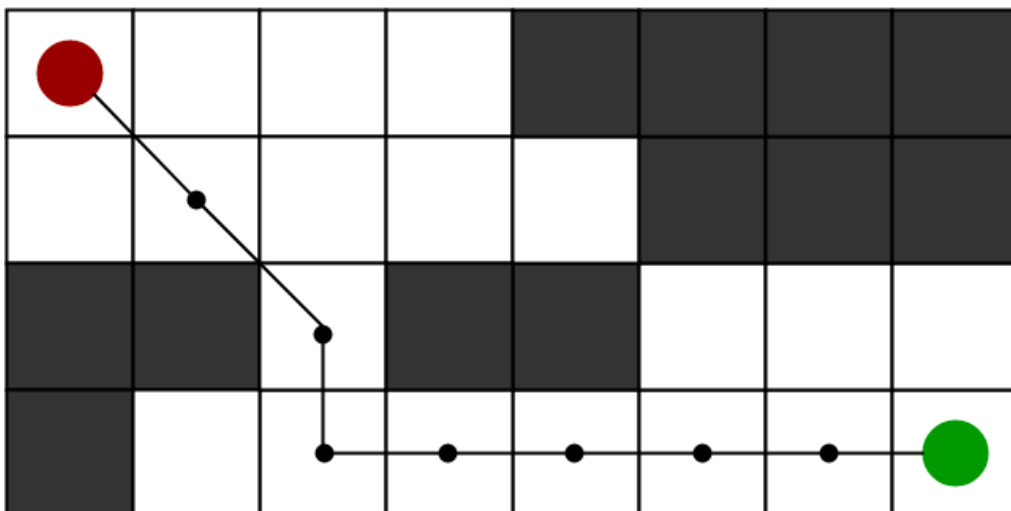
Dijkstra 算法

基于加权 Grid 图拓扑结构的广度优先算法。遍历当前节点所有相邻节点，再遍历相邻节点的相邻节点，依此类推直到遍历所有节点，找到当前节点到目标节点的最短路径。



A*算法

基于加权 Grid 图拓扑结构，以当前节点到起始节点及目标节点距离作为遍历依据。本质是以当前节点到目标节点连线作为 Grid 图遍历方向。



二、 运动规划

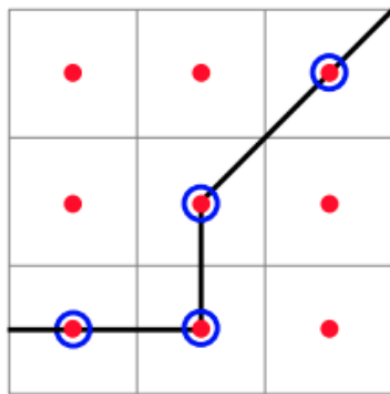
基于搜索的规划算法、基于采样的规划算法、直接优化法等。

D*(Dynamic A*)

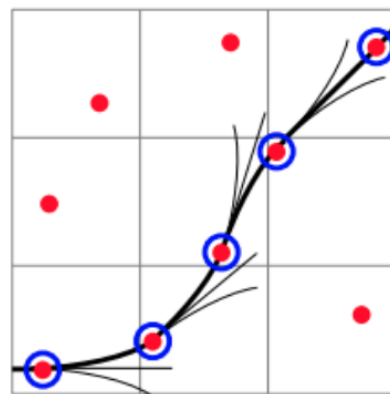
基于 Dijkstra 算法计算所有节点到目标点的最短路径，形成路径链表并保存；当起始点与目标点最短路径中的一个节点变更为障碍点时，将该节点之前节点作为临时起始点，并在该障碍节点周边重新计算到临时起始点的最短路径。

Hybrid A*

基于 Reed_Shepp、Dubins 曲线规划更符合车辆运动学及环境限制的路线。相对于 A* 算法，Hybrid A* 增加车辆运行角度限制，邻近节点位置任意；

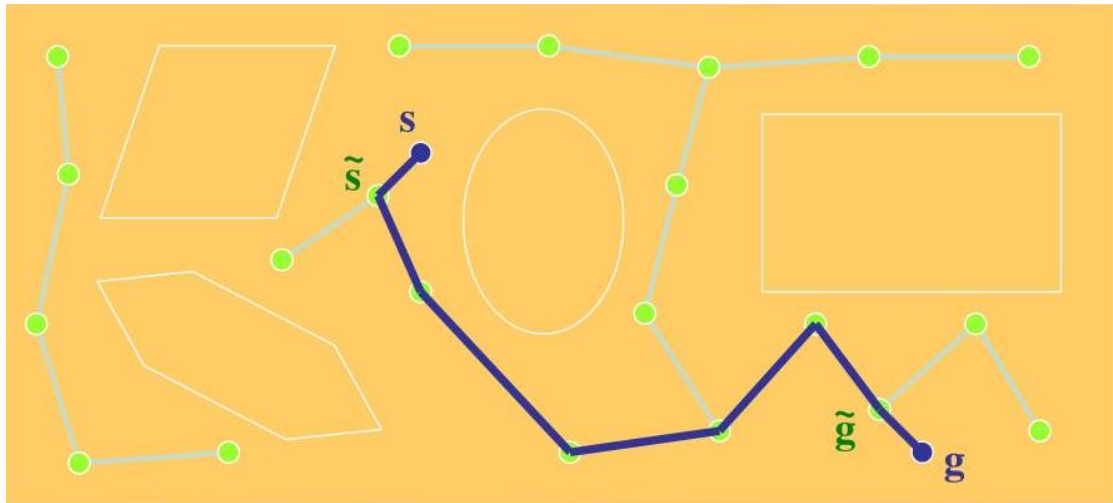


A*



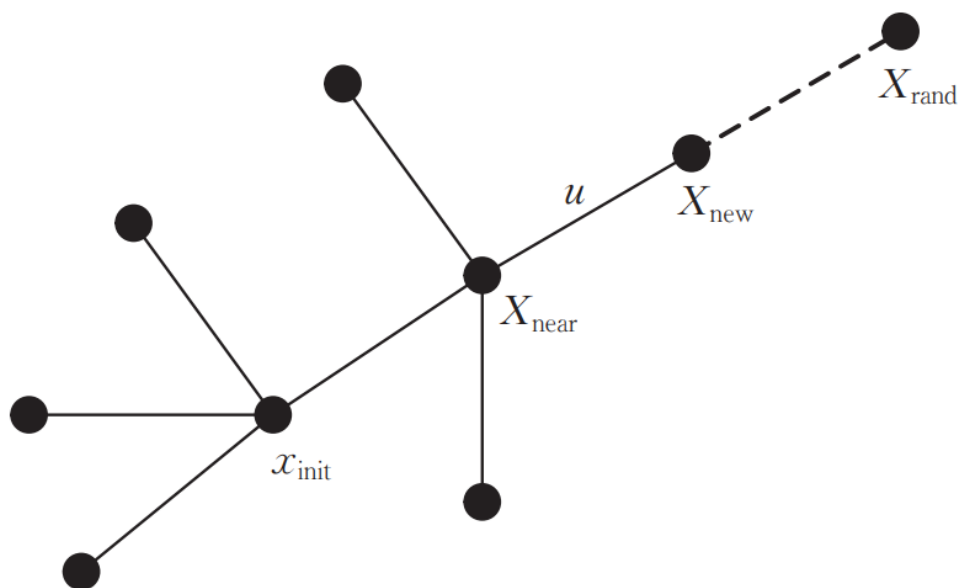
Hybrid A*

PRM(Probabilistic Road Map Planner)基于随机采样形成 Grid 图拓扑结构，然后利用 Dijkstra 等算法找最短路径。



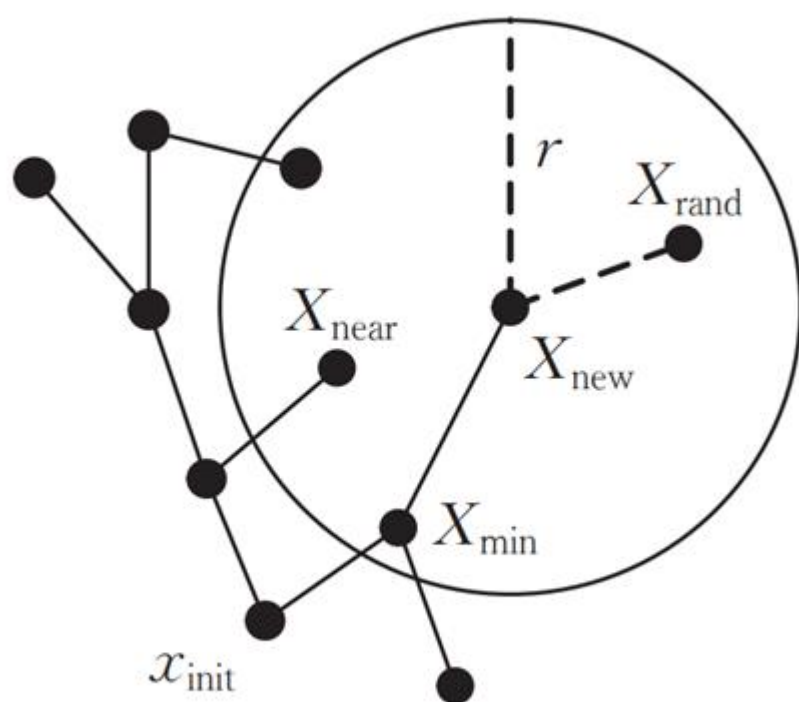
RRT(Rapidly Explore Random Tree)

基于随机采样机制，利用随机采样点与已知路径节点确定路径新节点。



RRT*

基于随机采样机制，确定新节点的同时，确定令整个路径最短的已知节点。



第四章 运动控制

传统机械臂基于运动学(Kinematics)及动力学(Dynamics)计算关节力矩及终端执行器位置、速度等，适用于有限特定场景及任务。

传统机器人基于倒立摆、整体模型等，利用线性二次调整(LQR, Linear Quadratic Regulator)、模型预测控制(MPC, Model Preview Control)及整体控制(WBC, Whole Body Control)等优化计算、控制机器人运动。

通用机器人，基于神经网络拟合人类思维函数。不同于传统机器人基于物理力学、动能分析定义变换函数。

注，传统机械臂、机器人、SLAM 等与神经网络方法表象似都是基于目标函数、优化算法进行计算，实质迥异。传统方法以机器人**本体物理模型**为中心，简化模型的同时，简化状态，基于任务优化**计算物理模型控制量**，无法主动适用多样化环境；神经网络方法以**状态变换数学模型**为中心，不限制状态，基于大规模数据**优化拟合变换函数**，预测控制量，可具备主动及通用性。

基于机器人运动控制过程来看，神经网络方法是基于整体运动变换、更本质的机制。跳离对机器人本体模型的定义，模仿人类思维，直接计算状态变换函数。

当前，通用机器人基于视觉语言动作模型进行运动控制，模型有 RT-1、RT-2、Gemini-Robotic、Phi、SmolVLA、GR00T 等。

RT-1，基于 FiLM EfficientNet 及 标准 Transformer Decoder-only 的视觉语言动作模型。基于 Transformer 框架 不过不是 Diffusion Transformer.

RT-2，基于 PaLI-X 或 PaLM-E 视觉语言模型，直接生成动作的视觉语言动作模型(RT-2-PaLI-X and RT-2-PaLM-E)。

注，当前通用机器人的控制挑战应在于步态平衡及灵活手控制。

一、机械臂

机械臂一般基于基座、连杆(Link)、关节(Joint)及终端执行器(End Effector)组成。关节可以分为 Linear、Rotational 等类型。不同类型的机械臂用不同类型的机械关节(Mechanical Joints)。在工业制造中,机械臂用于将终端执行器移动到目标位置进行相应的操作。这一移动过程主要涉及路径与轨迹规划、运动学(Kinematics)、动力学(Dynamics)与控制。

基于路径规划算法确定终端执行器从原始位置移动到目标位置的路径,形成 Waypoints 序列;基于时间约束,利用多项式、样条函数等轨迹函数实现对整个路径的轨迹优化;基于 Waypoints 进行逆向运动学计算,获得相邻 Waypoints 机械臂各关节应该调整的角度/偏移等信息;将相邻 Waypoints 的时间划分为多个时刻,基于轨迹函数计算每一时刻,机械臂各关节的角度/角加速度等信息;基于逆向动力学计算每一时刻机械臂各关节应该设置的力/力矩等信息;用计算得到的力/力矩等信息驱动电机使机械臂移动;基于反馈控制控制电机使机械臂运动轨迹尽可能符合期望轨迹;测量机械臂实际运行轨迹与期望轨迹的区别,进行调整。

动力学的必要性在于,尽管对于 Pick and Place 等简单应用场景,机械臂可以基于路径规划与运动学控制机械臂实现终端执行器从原始位置到目标点的运动,但是在大多数情况下,这不仅要求在路径规划的每个 Waypoint 机械臂需要进行逆向运动学计算,导致计算量较大,而且容易出现卡顿,能量消耗大,对于某些曲线轨迹出现难以控制的问题,更重要的是计算获得的关节位置可能不符合关节空间,同时在大功率情况下,不可控的机械臂会带来很大的危险。

基于轨迹规划与动力学则可以在符合时间约束、关节约束、能耗限制等约束条件的情况下,控制机械臂关节依照轨迹拟合函数更平滑、更准确地运动,并且可以控制机械臂与外界交互时的作用力。

1. 机械臂类型

串联机械臂。比方,多关节型机械臂(Articulated), 扭关节连接基座,别的关节为旋转关节。

笛卡尔机械臂(Cartesian), 连杆之间线性关节连接,终端执行器可以用旋转关节。

圆柱形机械臂(Cylindrical), 旋转关节连接基座,连杆之间线性关节连接。

极坐标机械臂(Polar), 扭关节连接基座,连杆之间用旋转关节及线性关节。

并联机械臂。

一个基座,多个机械臂;连杆之间用旋转关节及线性关节连接。

2. 运动学

运动学(kinematics)可分为正向运动学，逆向运动学。

正向运动学，基于关节的角度、角速度等信息计算终端执行器的位置、速度等信息；逆向运动学，基于终端执行器位置计算各关节的角度信息。

1) 正向运动学

常用计算方法，DH 法。

在相邻关节 Z 轴不平行的前提下，可基于 DH 法快速计算机械臂基座到终端执行器位置的变换矩阵。DH 法的本质是获取相邻关节坐标系的 X 轴、Z 轴的旋转、平移信息，然后基于关节坐标变换矩阵的乘积关系计算终端执行器的旋转与位置信息。

DH 参数：d，X 轴平移； θ ，X 轴的旋转；a(r)，Z 轴的平移； α ，Z 轴的旋转。

计算得到每个关节的 DH 参数后，可计算之前关节到该关节的变换矩阵，将所有关节的变换矩阵相乘，得到的矩阵为终端执行器位置及旋转信息。得到的 T 矩阵中，T 向量为终端执行器的位置信息，R 向量为旋转角度信息。

$$T = \left[\begin{array}{ccc|c} R & & & T \\ \hline 0 & 0 & 0 & 1 \end{array} \right] = \left[\begin{array}{ccc|c} \cos \theta & -\sin \theta \cos \alpha & \sin \theta \sin \alpha & r \cos \theta \\ \sin \theta & \cos \theta \cos \alpha & -\cos \theta \sin \alpha & r \sin \theta \\ 0 & \sin \alpha & \cos \alpha & d \\ \hline 0 & 0 & 0 & 1 \end{array} \right]$$

2) 逆向运动学

逆向运动学的传统计算方法分为解析法(Analytical)与数值法(Numerical)。

解析法有代数法、几何法等；传统数值法有迭代法、优化法等。迭代法则有雅可比求逆法、牛顿迭代法等。解析法运算更快，数值法更适合高自由度机械臂计算。

代数法(Algebraic)

对于二维平面，三个旋转关节的机械臂的逆向运动学计算，就是基于正向运动学基座到目标点的矩阵变换关系公式形成目标点位置 x, y 的方程组；解该方程组可知前两个关节的旋转角度，目标点关节即第三个关节旋转角度等于基准点与目标点角度减去前两个关节旋转角度的和。

几何法(Geometrical)

基于机械臂的俯视图、侧视图的三角形，利用三角形定理计算关节角度。

雅可比(Jacobian)求逆法

不同于代数法基于各关节坐标系的旋转、平移的变换进行解方程组的方法，雅可比求逆法基于雅可比矩阵计算关节角度信息。雅可比矩阵描述的是机械臂各关节速度与终端执行器速度之间的关系。

$$\dot{X} = J\dot{q}$$

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ \dot{\alpha} \\ \dot{\beta} \\ \dot{\gamma} \end{bmatrix}_{6 \times 1} = \begin{bmatrix} J_{11} & J_{12} & \cdot & \cdot & \cdot & J_{1n} \\ J_{21} & J_{22} & \cdot & \cdot & \cdot & J_{2n} \\ \vdots & \ddots & \vdots & \vdots & \vdots & \vdots \\ \cdot & \dots & \cdot & \cdot & \cdot & \cdot \\ J_{61} & \dots & \cdot & \cdot & \cdot & J_{6n} \end{bmatrix}_{6 \times n} * \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \\ \vdots \\ \vdots \\ \dot{q}_n \end{bmatrix}_{n \times 1}$$

雅可比矩阵的列数目等于机械臂关节数目，每一列表示每一个关节的速度对机械臂的终端执行器的速度的影响。雅可比矩阵的前三行与机械臂终端执行器的线速度相关，后三行与终端执行器的角速度相关。

雅可比矩阵的角速度成员形式逻辑意义是各关节轴相对于机械臂基座坐标系的旋转变化,实际可由正向运动学中机械臂基座到各关节及终端执行器的变换矩阵的前三列的某一系列成员获得。

机械臂从原始位置移动到目标位置计算过程。

- 1、通过电机编码器获取机械臂各关节角度,基于正向运动学计算 Jacobian 矩阵;
- 2、以目前位置到目标位置的距离的百分比作为趋近距离,以 Jacobian 矩阵的逆乘以该趋近距离,计算得到各关节应设置的角度;
- 3、依据计算获得的关节角度驱动机械臂关节电机;
- 4、获取机械臂终端执行器位置,与目标位置不一致,则循环步骤 1-3;

注:为确保机械臂各关节在有效范围移动,比方, Jacobian 矩阵非方形矩阵或是行列式等于零的奇异矩阵,不可逆。一般用 Jacobian Pseudo Inverse 矩阵替代 Jacobian 逆矩阵。

3. 动力学

动力学的目的就是建立刚体的运动的数学模型。机械臂的动力学就是建立机械臂的力/力矩与加速度/角速度的数学模型。动力学可分为正向动力学与逆向动力学。

正向动力学,基于关节的力(线性移动)/力矩(旋转)预测连杆的运动;逆向动力学,基于物体的运动及惯性特性计算力/力矩。

动力学的传统计算方法可基于牛顿-欧拉方法、欧拉-拉格朗日方法等,近年则出现了基于深度强化学习的方法。

牛顿-欧拉方法从牛顿第二定律发展而来。牛顿第二定律基于粒子动力学,分析的是力与加速度的关系。牛顿-欧拉方法则建立了适用于由粒子所组成的刚体的动力学的力矩与角加速度的模型。

欧拉-拉格朗日方法更多是从刚体能量的角度分析动力关系。欧拉-拉格朗日方法不仅建立了力矩与角加速度的模型,而且使力矩的计算更简单。

1) 牛顿-欧拉法(Newton-Euler)

基于牛顿第二运动定律。

$$\Sigma F_x = ma_x$$

$$\Sigma F_y = ma_y$$

$$\Sigma F_z = ma_z$$

$$\Sigma M = I\alpha$$

M 是力矩， I 是转动惯量， α 是角加速度。

2) 欧拉-拉格朗日法(Euler-Lagrange)

欧拉-拉格朗日法实质是基于动能与动量、力的关系，势能与功、力的关系建立速度、加速度与力的关系的等式。

欧拉-拉格朗日法的计算过程：

基于拉格朗日函数(T ，动能； U ，势能)

$$\mathcal{L}(q_i, \dot{q}_i) = T - U$$

计算拉格朗日动力学微分等式(Q , 广义力)。

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right) - \frac{\partial L}{\partial q_i} = Q_i = \tau_i$$

拉格朗日动力学多项式形式。

$$\sum_{j=1}^n H_{ij} \ddot{q}_j + \sum_{j=1}^n \sum_{k=1}^n h_{ijk} \dot{q}_j \dot{q}_k + G_i = Q_i$$

$$h_{ijk} = \frac{\partial H_{ij}}{\partial q_k} - \frac{1}{2} \frac{\partial H_{jk}}{\partial q_i}$$

$$G_i = \sum_{j=1}^n m_j \mathbf{g}^T \mathbf{J}_{Li}^{(j)}$$

多项式等式左侧第一项，惯性相关；第二项，科氏力与离心力相关；第三项，重力相关。基本只需计算机械臂各连杆的雅可比矩阵即可计算多项式等式左侧的每一项，由此可计算等式右侧的广义力。

3) 正向动力学

两连杆旋转机械臂，基于基座原点与第 2 个关节的力与力矩等式计算关节线加速度、角加

速度。

4) 逆向动力学

两连杆旋转机械臂。

- 1、基于正向运动学可知机械臂每个连杆的线速度、角速度对应的 **Jacobian** 矩阵。
- 2、基于动能等式计算所有连杆的动能。
- 3、基于获取的矩阵项 **H** 的元素项计算 **h**。
- 4、基于 **Jacobian** 矩阵列项计算重力项 **G**。

4. 路径规划

基于 PRM(Probabilistic Road Map)、RRT(Rapidly Exploring Random Tree)采样等路径规划算法确定机械臂终端执行器的运动路径。终端执行器的运动路径由一系列 **Waypoints** 组成。

5. 轨迹规划

轨迹规划是在符合机械臂移动时间限制、关节运动范围限制等条件下，对规划的机械臂路径进行优化，实现更平滑、更准确的轨迹。传统的轨迹规划可基于直线、抛物线、多项式曲线、样条曲线等方法来拟合位置轨迹，基于球面线性插值(Slerp, Spherical Linear Interpolation)等方法来拟合终端执行器旋转轨迹。

基于终端执行器的笛卡儿空间进行轨迹规划，直观，但是耗费能量，可能出现 **Singularity** 点，并需大计算量，因为每一个路径点都需要进行逆运动学计算。基于各关节的关节空间进行轨迹规划，便于基于关节的约束实现机械臂的移动，省能量。

二、 传统机器人

基于模型的机器人步态生成与控制方法对应的模型主要包括线性倒立摆模型(LIPM, Linear Inverted Pendulum Model)、机器人整体动力学模型(Whole Body Model)等; 对应的控制框架主要包括基于二次调整器的预览控制(LQR Preview Control)、模型预测控制(MPC, Model Predictive Control)或混合零动态(HZD, Hybrid Zero Dynamics)等。

线性倒立摆模型(LIPM, Linear Inverted Pendulum Model), 主要通过忽略肢体质量、集中质量于质心、降低机器人状态维度来简化动力学计算, 适合平整地面的移动, 但是存在要求机器人质心(CoM)轨迹在同一平面、低速移动、躯干无旋转等限制。

机器人整体模型(Whole Body Model), 主要通过分析机器人各连杆、关节以及与环境的动力学, 来实现在不平整地面上的移动、通过上肢摆动实现平衡、行走的同时实现多个任务等需求。

1. 线性倒立摆模型(LIPM, Linear Inverted Pendulum Model)

机器人线性倒立摆模型, 假定机器人质量集中于质心, 并且在腿部移动时, 质心的高度始终符合约束平面, 机器人的移动就像是移动中的倒立的钟摆。

通过线性倒立摆模型(LIPM)可以建立零力矩点(ZMP)与质心(CoM)的关系, 并使基于零力矩点轨迹计算获得的质心(CoM)轨迹符合零力矩点稳定性判断准则。

2. 整体模型(Whole Body Model)

机器人视作一个树形结构的机械臂, 在腰部是机械臂的基座, 这样机器人可以看作是浮动基座机器人。由于仿人机器人质心(CoM)存在 6 个自由度(线性/角度)是没有电机直接控制实现的, 因此仿人机器人又是欠驱动的。

整体模型可以简单认为是分析这个浮动的机械臂整体各部分及与外部环境的作用力的模型。基于机器人连杆、关节、质心及与环境作用部分的(线/角)速度/力矩可以建立机器人整体模型的动力学等式。有时为了简化复杂性，只分析机器人质心与腿足的动力学。

基于机械臂动力学欧拉-拉格朗日方程计算力矩的等式，可以将机器人力矩分为状态(关节)空间的力矩及任务(扭力)空间的力矩。

机器人单只脚连接地面时，添加 6 个约束；双脚都连接地面，则会形成过驱动，使得计算逆向动力学方程时，方程相对于未知量存在冗余。因此，浮动基座的动力学计算是随着机器人与地面的接触情况在变化的。

为实现多个不同的任务，力矩可通过任务零空间(Null-Space)投影方式、加权方式或层级(Hierarchical)方式实现对不同目标任务力矩进行优先级控制。

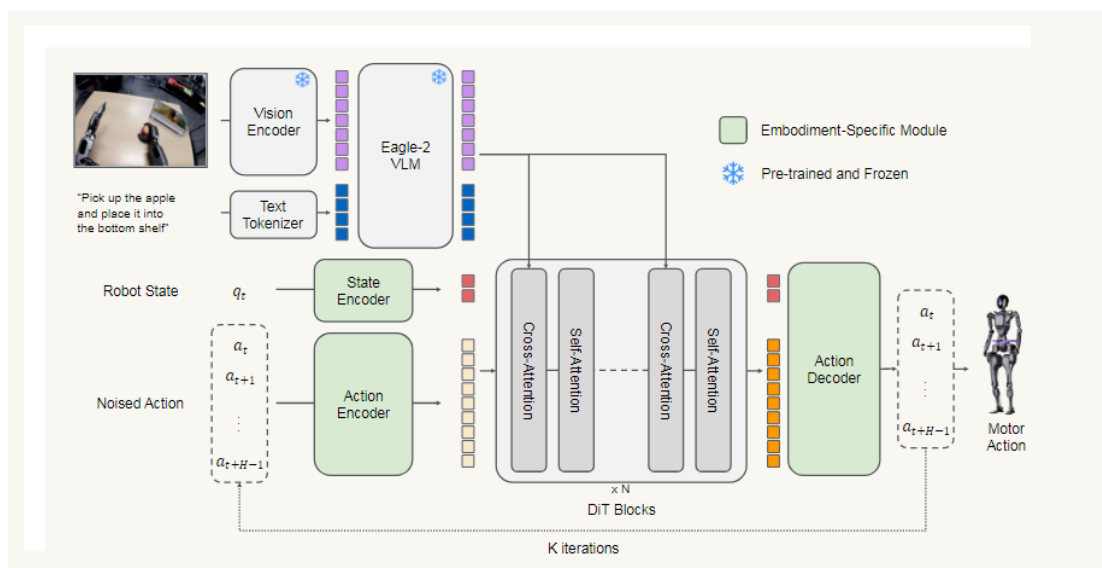
3. 运动控制

基于线性二次调整器(LQR, Linear Quadratic Regulator)的预览控制(Preview Control)是基于固定时间窗，通过黎卡提方程(Riccati Equation)进行优化计算的控制框架；模型预测控制(Model Prediction Control, MPC)是基于回退时域控制(Receding Horizon Control)适合线性/非线性模型及多约束，可通过二次规划(Quadratic Programming)进行优化计算、实时计算非周期步态的控制框架；混合零动态(HZD, Hybrid Zero Dynamics)是基于传递函数判断机器人周期步态稳定性的适用于离线计算的控制框架。

三、 Nvidia GR00T

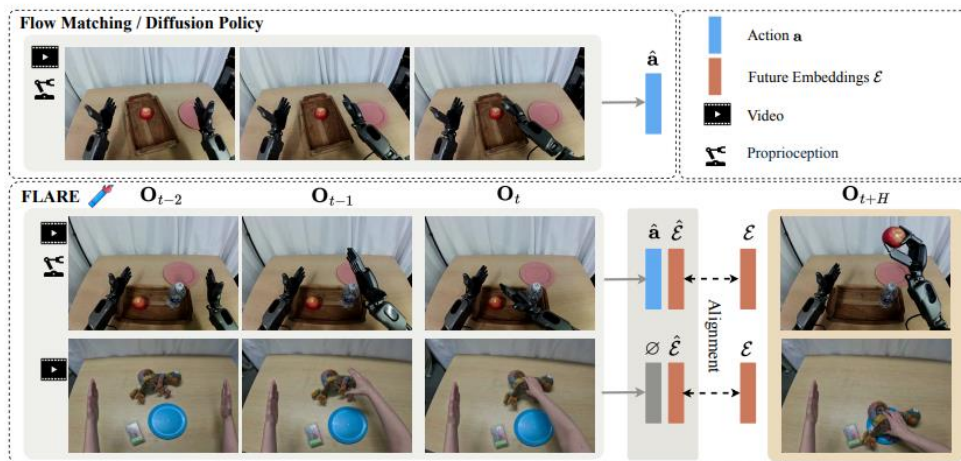
Nvidia 发布的视觉语言动作模型 N1.5，基于 GR00T N1 基础模型，通过简化视觉语言模型的多层感知器、对视觉及文本标记嵌量进行层归一化，提高语言指令遵循及泛化能力(Language Following And Generalization)；基于流匹配及未来隐表征对齐(FLARE, Future LATent Representation Alignment)目标函数改进机器人关节策略模型性能，实现视频学习；利用 DreamGen 合成机器人数据训练扩大模型泛化能力。

1. 总体框架

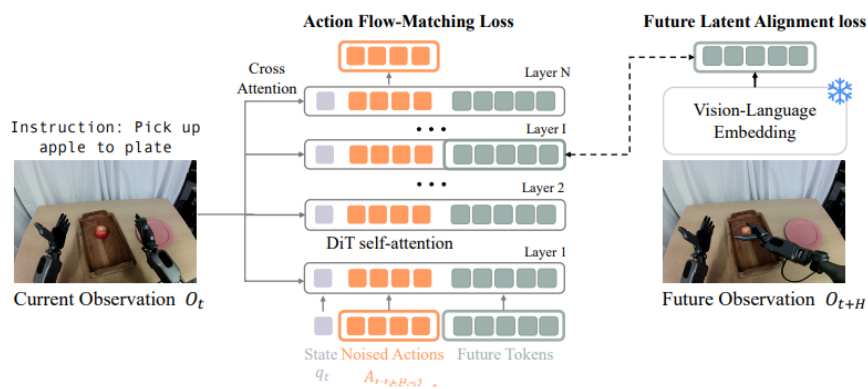


框架图，基本与 **GR00T N1** 相同。区别，预训练及优调时，冻结视觉语言模型；简化连接视觉编码器与视觉语言模型的多层感知器；对视觉及文本标记嵌量进行层归一化；基于流匹配及未来隐表征对齐损失训练动作生成模型；利用 **DreamGen** 合成数据训练扩展模型泛化能力。

2. 未来隐表征对齐(FLARE)



图，顶部，基于流匹配或扩散模型预测动作状态；居中，基于动作流匹配及未来隐表征对齐目标函数训练模型；底部，基于未来隐表征对齐目标函数训练模型。



框架图，左侧，计算视觉语言嵌量；居中，基于流匹配损失函数的扩散变换器(DiT，Diffusion Transformer)对机器人状态标记嵌量、动作标记嵌量及未来标记嵌量的拼接序列进行自注意力计算，通过视觉语言嵌量交叉注意力计算调控动作生成；右侧，扩散变换器中间层，相应于未来标器嵌量的激活特征与观测的未来视觉语言嵌量，基于余弦相似度(Cosine Similarity)计算未来隐特征对齐损失。

一、损失函数

1、流匹配

损失函数定义。

$$L_{fm}(\theta) = E_{\tau}[\|V_{\theta}(\phi_t, A_t^{\tau}, q_t) - (\epsilon - A_t)\|^2]$$

动作序列推理定义。

基于前向欧拉积分(Forward Euler Integration)方程，通过 K 步去噪生成动作序列。

$$A_t^{\frac{\tau+1}{K}} = A_t^{\tau} + \frac{1}{K} V_{\theta}(\phi_t, A_t^{\tau}, q_t)$$

时步(Timestep) τ 的分布定义， $s = 0.999$ 。

$$p(\tau) = \text{Beta}(\frac{s-\tau}{s}; 1.5, 1)$$

2、隐表征对齐

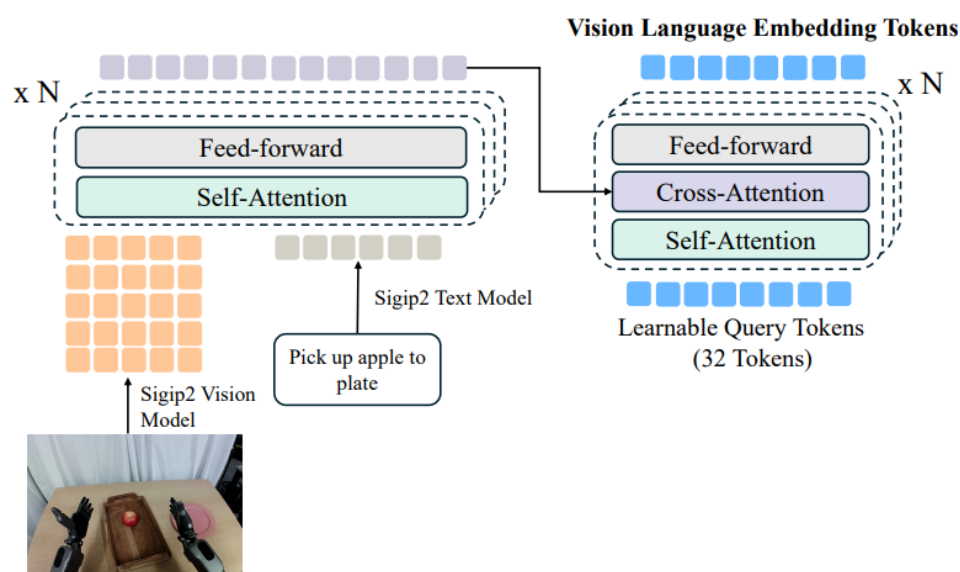
隐表征对齐损失函数定义。

$$L_{align}(\theta) = -E_{\tau}[\cos(f_{\theta}(\phi_t, A_t^{\tau}, q_t), g(\phi_t + H))]$$

3、模型损失函数

$$L=L_{fm}+\lambda L_{align}$$

二、视觉语言模型

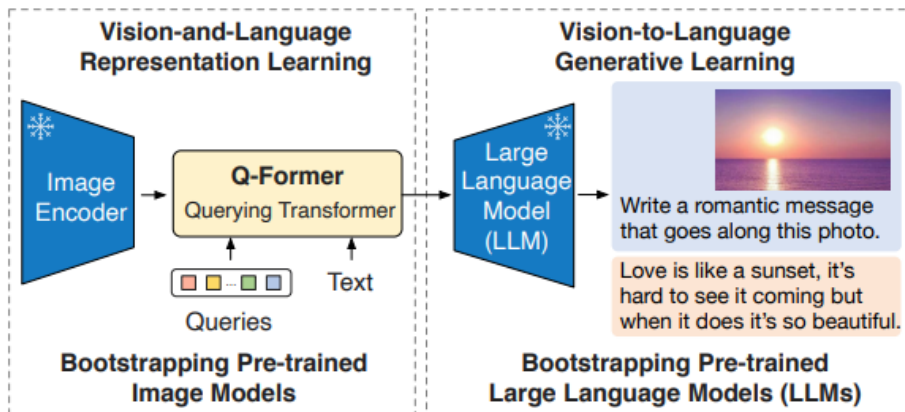


框架图，左侧，基于 SigLIP-2 编码视觉及语言嵌量，通过多个自注意力变换器块(Self-Attention Transformer Blocks)计算视觉语言注意力(Cross-modal Dependencies)；右侧，视觉语言注意力及查询标记(32 Learnable Query Tokens, Randomly Initialized)基于 Q-former 模块生成压缩的视觉语言嵌量标记(32 Compressed Vision-Language Tokens)。

三、查询变换器(Q-Former, Querying Transformer)

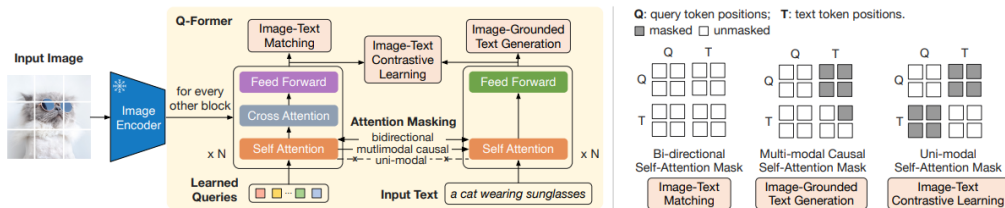
改进语言图像预训练(BLIP-2, Bootstrapping Language-Image Pre-training)框架，基于已有固参图像编码器及大语言模型改进视觉语言预训练，利用查询变换器提取文本最相关的视觉特征、弥合模态间隙(Modality Gap)。

查询变换器预训练分两阶段。



框架图，左侧，第一阶段，通过固参图像编码器(frozen image encoder)改进视觉语言表征学习(Bootstraps Vision-Language Representation Learning)；右侧，第二阶段，通过固参大语言模型改进视觉转语言生成学习(Bootstraps Vision-To-Language Generative Learning)。

查询变换器框架及预训练第一阶段。



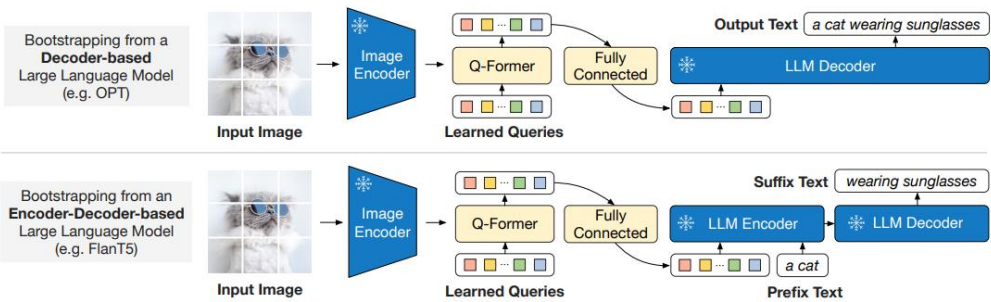
框架图，左侧，查询变换器，基于共享自注意力层的图像变换器及文本变换器亚模块(Transformer Submodules)实现，利用多个目标函数进行预训练；图像变换器通过交叉注意力与图像编码器交互，提取视觉特征；文本变换器作文本编解码器，提取文本表征；自注意力计算时基于不同的预训练任务及自注意力遮码(Self-Attention Masks)策略进行查询量与文本的交互。

右侧，预训练目标函数相应的自注意力遮码策略，图文匹配(ITM, Image-Text Matching) 基于双向自注意力遮码(Bi-Directional Self-Attention Mask)，图生文(ITG, Image-Grounded Text Generation)基于多模态因果自注意力遮码(Multimodal Causal Self-Attention Mask)，图文比对学习(ITC, Image-Text Contrastive Learning)基于单模态自注意力遮码(Unimodal Self-Attention

Mask)。

预训练第二阶段。

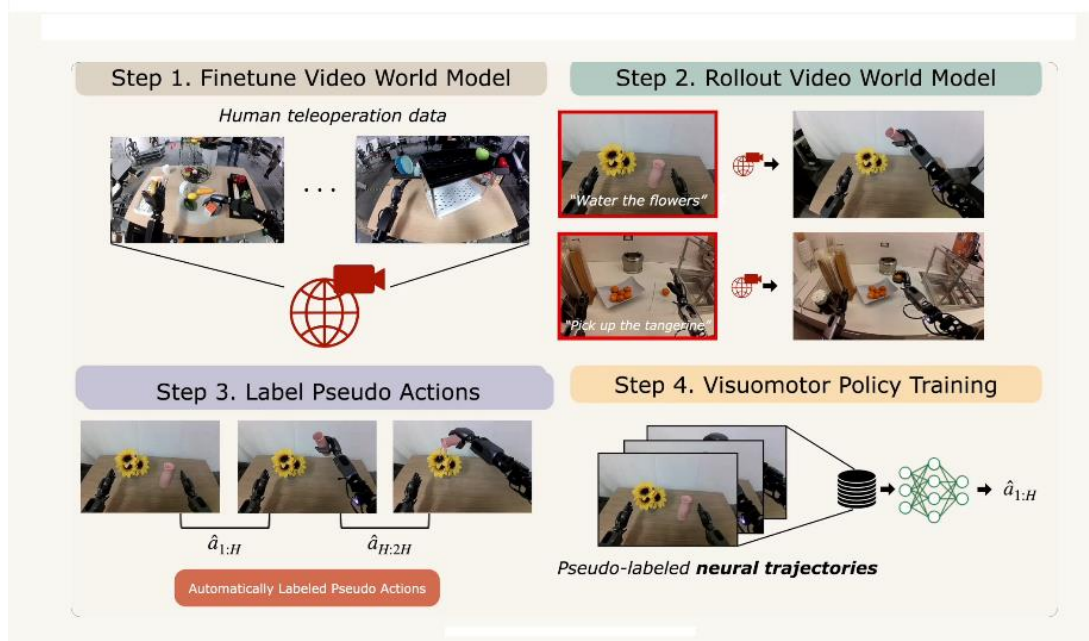
基于全连接层对查询变换器计算的查询注意力嵌量进行投影，得到与大语言模型的文本嵌量等维度的嵌量，前置(Prepended)到文本嵌量前，调控语言模型生成文本。



3. DreamGen

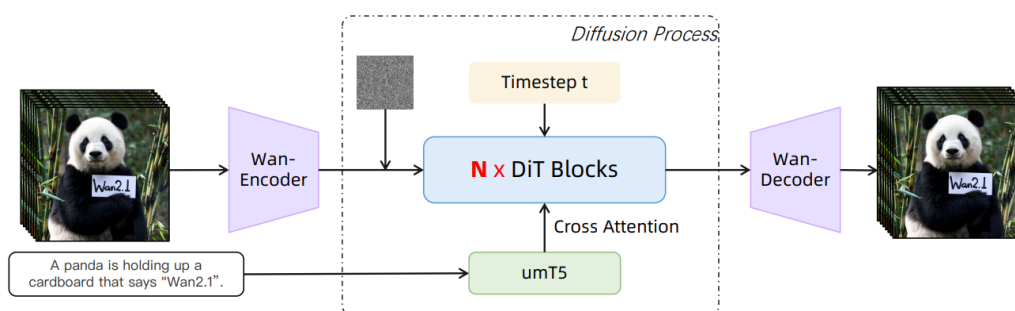
基于图生视频模型合成适用于目标机器人具身的各类视频(Photorealistic Synthetic Videos)，再利用隐动作模型(Latent Action Model)或逆动力学模型(IDM， Inverse-Dynamics Model)还原动作序列(Pseudo-Action Sequences)。

一、总体框架

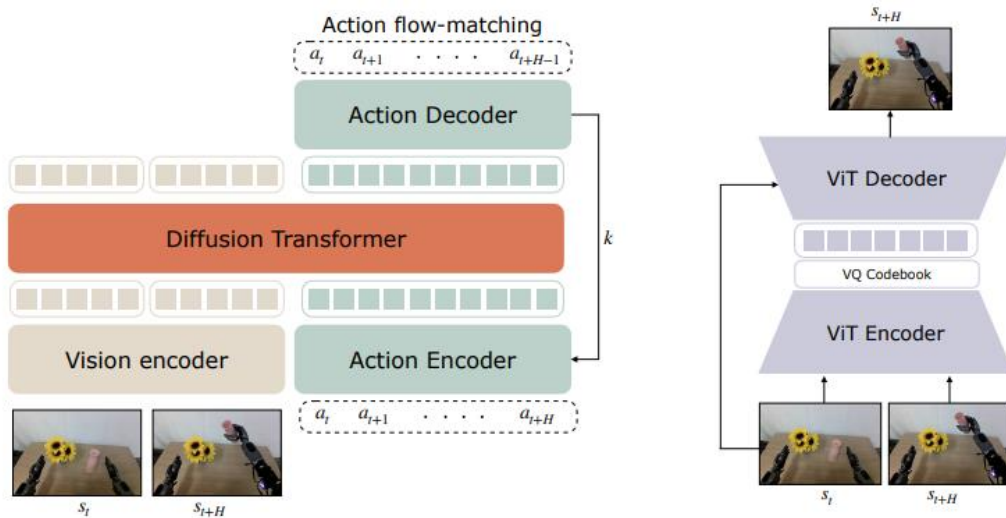


框架图，基于遥操作机器人轨迹微调视频世界模型(Video World Model)，比方，WAN；基于给定的起始视频帧及语言指令，生成执行指令意图的视频；基于隐动作模型或逆动力学模型标注视频中的动作，形成神经轨迹(Neural Trajectories)；基于神经轨迹训练视觉机器人策略(Visuomotor Robot Policies)。

二、视频世界模型

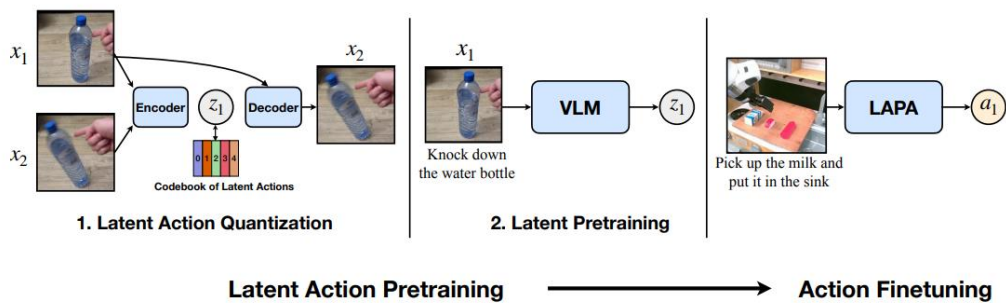


三、还原动作模型



框架图，左侧，逆动力学模型框架，不同时刻的两个视频帧通过 SigLIP-2 编码器计算嵌量，基于流匹配损失函数训练，调控扩散变换器生成两个视频帧之间的动作序列；右侧，基于视频隐动作预训练模型(LAPA, Latent Action Pretraining for General Action Models)，利用 VQ-VAE 损失函数训练，生成隐动作。

隐动作预训练。



框架图，左侧，基于噪声替代(NSVQ, Noise Substitution In Vector Quantization)方法的向量量化-变分自编码器模型学习连续视频帧所示的量化隐动作；居中，利用学习的隐动作训练视频语言模型，实现行为克隆(Behavior Cloning)；右侧，基于少量机器人动作数据集微调视觉语言动作模型，实现隐动作到机器人动作的映射。

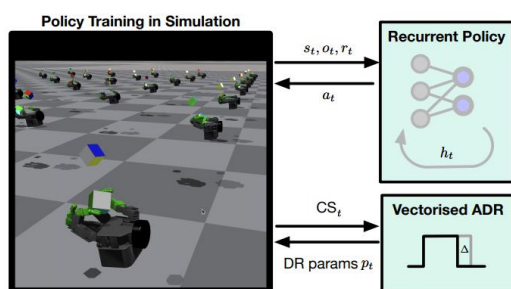
Nvidia GR00T 模型框架与 Physical Intelligence 的机器人视觉语言动作模型相似。模型当前主要侧重于短程(Short-Horizon)控制任务，要实现长程运动控制(Long-Horizon Loco-Manipulation)可能需机器人硬件、模型架构及训练数据的改进。

基于隐动作学习方法，利用云端计算优调模型应可实现机器人实时模仿学习。基于多模态动作模型，利用推理、三维空间信息、步骤对齐及强化训练，应该可实现机器人长程灵活运动控制。

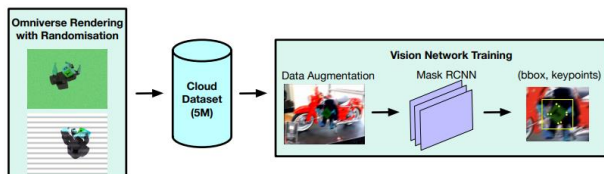
四、 Nvidia Allegro Hand

Nvidia Allegro 机器人手运动控制，仿照 OpenAI Shadow Hand 方法，基于 PPO 强化学习算法框架与视觉追踪系统，在 Isaac Gym 仿真环境下，利用向量化自动场景域随机化(Vectorised Automatic Domain Randomisation)机制生成多样化的训练数据、尽可能扩大域随机化区间，保持策略性能的同时，提高策略的泛化能力及仿真到现实的迁移能力。

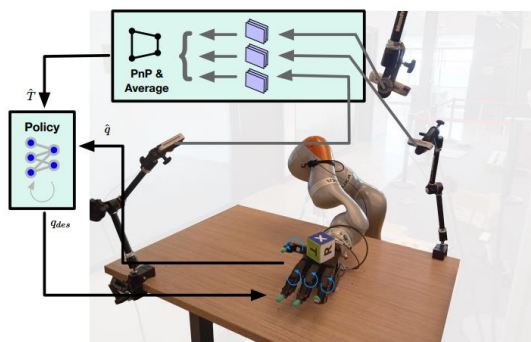
总体框架，Allegro 机器人手，基于 RGB 相机与 Mask R-CNN 网络实现对机器人手心中目标物的位姿的预测；基于 PPO 算法训练的策略指导机器人手指的运动。PPO 算法的策略(Actor)与价值函数(Value Function, Critic)基于长短期记忆网络(LSTM)。



(a) Policy training.



(b) Vision data generation and training pipeline.



(c) Functioning in the real world.

1. 任务定义

通过对 Allegro 机器人手指的控制，将随机放置在机器人手心、带颜色及标记的立方体目标物不断地重定向到指定的目标方向。

步骤，

- 1、立方体随机放置在机器人手心；
- 2、基于特定正交矩阵群(SO(3))采样，随机指定期望的立方体目标方向；
- 3、基于策略模型指挥机器人手指运动，将立方体调整到期望的目标方向；
- 4、立方体调整后与目标方向偏离小于阈值度(比方，0.4 弧度)，则采样指定新的目标方向；

依此循环，不断控制机器人手指将立方体从当前方向调整到新的目标方向；

在立方体不掉落的情况下，连续将立方体调整到目标方向的次数或者保持立方体在同一个状态下的时长可作判断机器人手控制成功与否的标准；

2. 环境定义

机器人手，Allegro Hand + 机械臂。

视觉追踪系统，3 个基于 Intel D415 的 RGB 相机，相机相对于机器人手心连杆(Palm Link of the Hand)进行外部校准；

目标物，各面不同颜色及标记的 6.5 厘米的立方体，基于机器人手心表示立方体的位姿；

仿真器，Isaac Gym；

训练平台，8 块 Nvidia A40 GPU + 32 Nvidia V100 GPU；

通过相机-相机外部校准、相机-机器人校准可实现相机参考坐标框(Reference Frame)到机器人掌心坐标系进行变换。

3. 策略学习

机器人手通过运动控制调整立方体到期望方向，是一个串行决策问题。

策略代理与环境交互实现最大化折扣奖励和。

基于 PPO 算法框架学习随机策略(Actor)与价值函数(Critic)。

1、策略框架

LSTM 网络，层归一化隐单元(Hidden Units of Layer Normalization)的*1024 个；多层感知器(MLP)*2 个(512 units)；激活函数(ELU, Exponential Linear Units)；

输入，观测量(50 维度)及此前的隐状态；

输出,动作,通过指数移动均值(EMA, Exponential Moving Average)平滑因子(Smoothing Factor)进行低通过滤(Low-Pass Filtered)以防电机线缆等损坏;

动作空间,基于机器人手 16 个关节对应的比例差分控制器的目的位置(PD Controller Target)定义;

2、价值函数

LSTM 网络,层归一化的隐单元*2048 个; 多层感知器(1024 units, 512 units); ELU 激活*2 个;

输入,观测量(265 维度);

输出,价值;

3、训练方法

策略,采用用线性调度学习率(Linear Scheduling of the Learning Rate);

价值函数,采用固定学习率(Fixed Learning Rate);

优化计算,时序反向传播(BPTT, Backpropagation Through Time),时窗截取长度 16;

4、输入定义

Input	Dimensionality	Actor	Critic
Object position with noise	3D	✓	✓
Object orientation with noise	4D (quaternion)	✓	✓
Target position	3D	✓	✓
Target orientation	4D (quaternion)	✓	✓
Relative target orientation	4D (quaternion)	✓	✓
Last actions	16D	✓	✓
Hand joints angles	16D	✓	✓
Stochastic delays	4D	×	✓
Fingertip positions	12D	×	✓
Fingertip rotations	16D (quaternions)	×	✓
Fingertip velocities	24D	×	✓
Fingertip forces and torques	24D	×	✓
Hand joints velocities	16D	×	✓
Hand joints generalised forces	16D	×	✓
object scale, mass, friction	3D	×	✓
Object linear velocity	3D	×	✓
Object angular velocity	3D	×	✓
Object position	3D	×	✓
Object rotation	4D	×	✓
Random forces on object	3D	×	✓
Domain randomisation params	78D	×	✓
Gravity vector	3D	×	✓
Rotation distances	2D	×	✓
Hand scale	1D	×	✓

4. 奖励定义

Reward	Formula	Weight	Justification
Rotation Close to Goal	$1/(d + 0.1)$	1.0	Shaped reward to bring cube close to goal
Position Close to Fixed Target	$\ p_{\text{object}} - p_{\text{goal}}\ $	-10.0	Encourage the cube to stay in the hand
Action Penalty	$\ a\ ^2$	-0.001	Prevent actions that are too large
Action Delta Penalty	$\ targ_{\text{curr}} - targ_{\text{prev}}\ ^2$	-0.25	Prevent rapid changes in joint target
Joint Velocity Penalty	$\ v_{\text{joints}}\ ^2$	-0.003	Stop fingers from moving too quickly
Reset Reward	Condition	Value	
Reach Goal Bonus	$d < 0.1$	250.0	Large reward for getting the cube to the target

奖励，奖励项加权后的和。

d ，立方体当前方向到指定方向的旋转距离；

p_{object} ，立方体位置；

p_{goal} ，目标位置；

a ，当前动作；

$targ_{\text{curr}}$ ，当前关节位置目标；

$targ_{\text{prev}}$ ，此前关节位置目标；

vjoints，当前关节速度向量；

5. 仿真

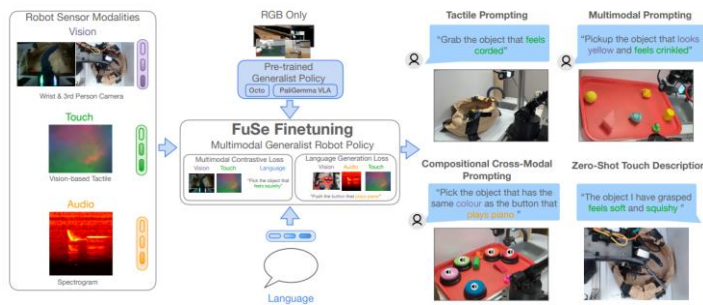
基于 Isaac Gym 物理仿真器，对连接力进行建模；基于单个 GPU 进行并行仿真。

Nvidia Allegro 基于向量化自动域随机化机制生成的数据训练近端策略优化(PPO)模型与视觉追踪系统，降低了训练成本，改进了机器人手控制目标进行重定向的泛化能力。

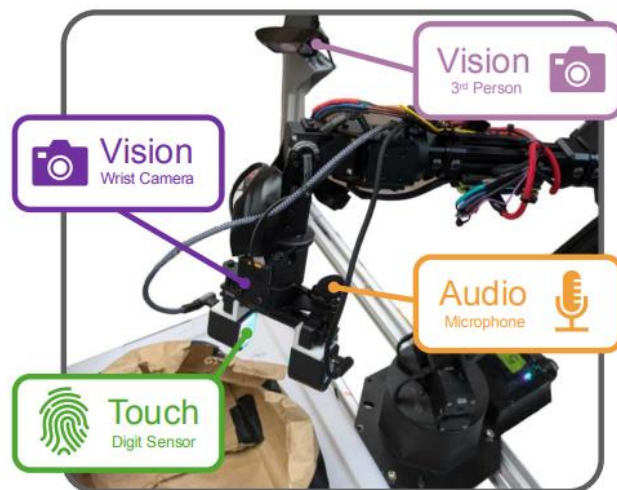
五、超视觉模态模型

加州伯克利发布的超视觉多感知模态融合(FuSe, Fuse Heterogeneous Sensory Data)模型，基于视觉、触觉、听觉、本体及语言等模态，利用自然语言跨模态对齐(Cross-Modal Grounding)优调视觉语言动作等通用模型，提高模型任务成功率。

总体框架，基于预训练的 Octo 模型或 PaliGemma 视觉语言动作模型，利用机器人第三方视角相机及腕部相机图像，触觉感知图像，麦克风语谱图像(Spectrogram)及自然语言指令等，通过多模态对比损失、语言生成损失及动作损失，进行模型优调，实现自然语言指令或图像目标任务。



1. 硬件环境



机械臂， WidowX 250 6-DoF。

第三方视角相机， RGB 相机。

腕部相机， RGB 相机。

触觉传感器， DIGIT， 2 个， 分置于终端钳。

麦克风， 标准型。

惯性测量单元， 9-DoF IMU。

2. 训练环境及数据

TPU， Google v5e-128 TPU pod。

机器人任务轨迹收集器， Meta Quest 2 VR headset。

任务数据集， 基于遥操作收集大概 3 万条轨迹。

每条轨迹通过模板化的语言指令(Templated Language Instruction)标记。

三类任务， 两类抓取任务具备视觉、触觉及动作数据， 一类具备声音的按钮任务。触觉观测信息去掉静态背景图。音频包括最近 1 秒的麦克风采样数据， 频率 44K。

3. 机器学习框架

Google Research 发布的 JAX，基于自动梯度(Autograd)及线代加速器(XLA, Accelerated Linear Algebra)、自动向量实现自动微分(Automatic Differentiation)、实时编译(JIT, Just-In-Time)及并行计算，提高基于 TPU 等的大规模、高性能计算。不过易用性似不好。

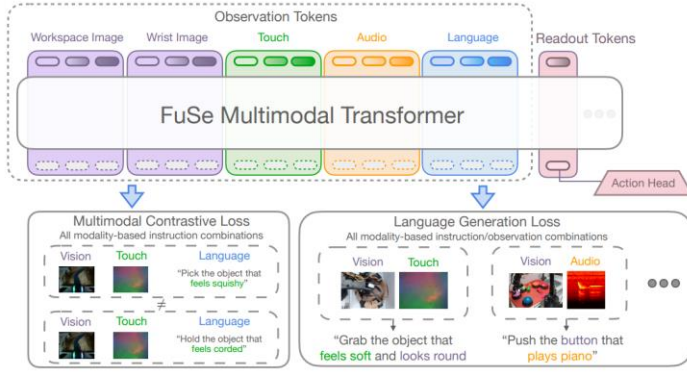
4. 触觉传感器(DIGIT)

GelSight DIGIT 基于视觉的触觉传感器，利用相机捕获弹性材料的形变图像来测量接触力(Contact Forces)。



拆解图，左至右，弹性材料(Elastomer)，窗(Acrylic Window)，卡座(Snap-Fit Holder)，照明电路板(Lighting PCB)，外壳(Plastic Housing)，相机电路板(Camera PCB)，背壳(Back Housing)。

5. 模型框架



框架图，顶部，自然语言标记化作任务标记(原图似未正确表示)；相机图像、触觉图像、语音谱图标记化作观测标记(Observation Tokens)；取读标记(Readout Tokens)添加到变换器输入，用于注意此前任务及观测标记；任务标记、观测标记及取读标记基于遮挡码方法实现注意力规则，通过多模态编码变换器生成注意力嵌量；底部，基于动作头(Action Head)生成动作；基于多模态对比损失对齐视觉触觉与语言指令，基于语言生成损失对齐多模态与语言语义，基于动作损失对齐多模态与动作；基于注意力嵌量，利用累加损失训练模型。

取读标记(Readout Tokens)作用类似基于变换器的双向编码器表征模型(BERT, Bidirectional Encoders Representations From Transformers)中的 [CLS] 标记(Token)。可存在多个取读(Readout)，一个取读可对应多个标记(Tokens)。取读标记实例化时等价于位置嵌量。

动作头(Action Head)基于标准去噪扩散概率模型(DDPM, Denoising Diffusion Probabilistic Models)，利用变换器生成的动作取读(Action Readout)注意力嵌量等信息学习去噪神经网络，通过对标准高斯噪声多步去噪预测动作。

6. 损失函数

$$\mathcal{L} = \mathcal{L}_{BC} + \beta \mathcal{L}_{gen} + \lambda \mathcal{L}_{contrast}$$

Nvidia GR00T 模型及 Physical Intelligence 模型，基于动作状态，通过扩散变换器或流匹配生成未来动作；利用视觉语言模态注意力嵌量直接调控动作生成；利用行为克隆进行总对齐。

超视觉模态模型基于多模态图像及语言，通过编码变换器(Transformer Encoder)及标准扩散模型进行嵌量化及生成动作；基于行为克隆对齐的同时，利用对比损失及生成损失进行嵌量对齐，间接调控动作生成。

超视觉模态模型框架似过冗，多模态注意力嵌量直接调控扩散变换器或流匹配模型可能是更合适的方法。

基于触觉、听觉等模态信息及遮挡码注意力规则可提供更全面的语义对齐能力；同时，利于基于多样化的环境反馈实现机器人操作控制约束条件。

新发布的触觉传感器 DIGIT360 可实现全向感知、近千万级感素(Taxels)，空间特征分辨率可到 7 微米，法向力(Normal Forces)及切向力(Shear Forces)分辨率可到近 1 毫牛，可感知振动、热，甚至异味。

基于 U-Net 骨干神经网络的标准去噪扩散概率模型应可替换成扩散变换器(DiT, Diffusion Transformer)或流匹配模型，以提高性能或平滑性。

人形机器人基于惯性测量单元的超视觉模态注意力对齐，似有利于实现多任务操作时的整机平衡。

第五章 世界模型

世界模型是机器人理解真实世界的基石。当前世界模型的五+1 类实现方法。

- 1、基于图像、视频理解的扩散生成式模型。比方，Google Genie3 等。
- 2、基于生成式 AI、侧重于内部世界(Internal world)的世界模型。比方，Li fei fei world lab 实验室的, Marble。
- 3、基于联合嵌量预测(Joint Embedding Predictive Architecture)。比方，Yann LeCun, LeJEPa。
- 4、基于主动推理及半符号的形式。对象动力学等属性基于主动推理进行建模，混合了半象征方法。比方，Karl Friston 等，AXIOM，场景表示作对象的合成(Compositions)，基于逐像素线性投影建模，以获得对象间关联。
- 5、神经象征世界模型(Neuro-symbolic World Models)。
- 6、VAE + MDN-RNN + Control Model + Reinforcement Learning . Jürgen Schmidhuber.

第六章 机器人仿真

机器人仿真实质，基于机器人几何模型及环境状态，利用运动学及动力学模型进行机器人运动仿真，以训练、测试、验证机器人控制模型或策略。

注，传统机器人控制中的运动学、动力学作用在于模型状态预测，以便与真值形成目标函数，优化计算力矩等控制量。

一、 仿真器

机器人仿真器，Gazebo、MuJoCo、Pybullet、Webots、V-REP、Nvidia Isaac Sim、Agility Robotics AR-Sim 等。

仿真器主要由渲染引擎、物理引擎组成，基于物理引擎进行物理建模及运动仿真，通过渲染引擎进行二维、三维显示，界面可基于 Qt 等实现。

渲染引擎，OpenGL、DirectX、Vulkan、Nvidia OptiX 光线追踪器(Ray Tracer)、路线追踪器(Path Tracer)等。

物理引擎，可看作基于当前位置、速度、控制力(τ)及惯性特性(μ)计算下一时刻位置及速度的函数，动力学状态可基于广义坐标系(Generalized Coordinate；最小坐标系，Minimal Coordinates；关节坐标系，Joint Coordinates)，或笛卡尔坐标系(Cartesian Coordinate；最大坐标系，Maximal Coordinates，六自由度；XYZ 坐标系)。

$$P(\mathbf{q}_t, \dot{\mathbf{q}}_t, \boldsymbol{\tau}, \boldsymbol{\mu}) = [\mathbf{q}_{t+1}, \dot{\mathbf{q}}_{t+1}]$$

物理引擎类型可分为不可微分引擎(Non-Differentiable Physics Engines；基于梯度的，Gradient-Based)及可微分引擎(Differentiable Physics Engines；非梯度的，Gradient-Free)类型，ODE、Bullet、PhysX(Unreal、Unity3D)、MuJoCo、Havok、DART、Simbody、Vortex 等。

Engine	Application	Integrator	State	Contact Type	Contact Solver	Gradients
MuJoCo	robotics	RK4	minimal	soft	Newton	finite difference
Drake	robotics	implicit Euler	minimal	soft/hard	LCP	randomized smoothing
Bullet	graphics	implicit Euler	minimal	soft/hard	LCP	subgradient
DART	robotics	implicit Euler	minimal	hard	LCP	subgradient
PhysX	graphics	explicit	minimal	soft	iterative	finite difference
Brax	graphics	explicit	maximal	soft	iterative	subgradient

动力学

人形机器人关节刚体系统的动力学方程推理。

一、粒子动力学

假定人形机器人由一系列刚性粒子组成，每个粒子 $\mathbf{r}_i$ 由一系列对应于机器人关节设置的广义坐标定义。

$$\mathbf{r}_i = \mathbf{r}_i(q_1, q_2, \dots, q_{n_j}, t)$$

粒子 $\mathbf{r}_i$ 的极小变化等于 $\mathbf{r}_i$ 在广义坐标轴 $\mathbf{q}_j$ 方向上的变化的矢量和。

$$\delta \mathbf{r}_i = \sum_j \frac{\partial \mathbf{r}_i}{\partial q_j} \delta q_j$$

基于惯性力的虚功计算等式、惯性力在广义坐标轴 $\mathbf{q}_j$ 方向上的分量的计算等式、刚性粒子的动能计算等式，可以推理得到刚性粒子的拉格朗日方程，基于刚性粒子动能和推理得到

多刚体系统的拉格朗日方程。

惯性力的虚功。

$$\mathbf{f}_i \cdot \delta \mathbf{r}_i = \mathbf{f}_i \cdot \sum_j \frac{\partial \mathbf{r}_i}{\partial q_j} \delta q_j \equiv \sum_j Q_{ij} \delta q_j = \mathbf{Q}_i \cdot \delta \mathbf{q}$$

惯性力在广义坐标轴 $\mathbf{q}_j$ 方向上的分量。

$$\mu_i \ddot{\mathbf{r}}_i \cdot \frac{\partial \mathbf{r}_i}{\partial q_j} = \frac{d}{dt} \left(\frac{\partial}{\partial \dot{q}_j} \left(\frac{1}{2} \mu_i \dot{\mathbf{r}}_i^T \dot{\mathbf{r}}_i \right) \right) - \frac{\partial}{\partial q_j} \left(\frac{1}{2} \mu_i \dot{\mathbf{r}}_i^T \dot{\mathbf{r}}_i \right)$$

刚性粒子的动能。

$$T_i = \frac{1}{2} \mu_i \dot{\mathbf{r}}_i^T \dot{\mathbf{r}}_i$$

多刚体系统的刚性粒子对应的拉格朗日动力学方程。

$$\frac{d}{dt} \left(\frac{\partial T_i}{\partial \dot{q}_j} \right) - \frac{\partial T_i}{\partial q_j} - Q_{ij} = 0$$

多刚体粒子系统的拉格朗日动力学方程。

$$\frac{d}{dt} \frac{\partial T}{\partial \dot{\mathbf{q}}} - \frac{\partial T}{\partial \mathbf{q}} = M\ddot{\mathbf{q}} + \dot{M}\dot{\mathbf{q}} - \frac{1}{2}\dot{\mathbf{q}}^T \left(\frac{\partial M}{\partial \mathbf{q}} \right)^T \dot{\mathbf{q}} \equiv M\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})$$

二、刚体动力学

基于线动量、角动量计算刚体动能等式，再利用质心线速度、角速度化简动能等式，然后基于刚性粒子的拉格朗日动力学等式计算刚体拉格朗日动力学等式，与基于牛顿-欧拉等式计算获得的刚体拉格朗日动力学等式比对，可得质量矩阵、科氏力与离心力、力矩的计算等式。

基于线动量、角动量计算获得的动能等式。

$$\begin{aligned} T &= \frac{1}{2} m \mathbf{v}^T \mathbf{v} + \frac{1}{2} \boldsymbol{\omega}^T \left(\sum_i -\mu [\mathbf{r}'_i] [\mathbf{r}'_i] \right) \boldsymbol{\omega} \\ &= \frac{1}{2} m \mathbf{v}^T \mathbf{v} + \frac{1}{2} \boldsymbol{\omega}^T I_c \boldsymbol{\omega} \end{aligned}$$

通过线速度、角速度化简后的动能等式。

$$T = \frac{1}{2} \dot{\mathbf{q}}^T J^T M_c J \dot{\mathbf{q}}$$

刚体拉格朗日动力学等式。

$$\frac{d}{dt} \left(\frac{\partial T}{\partial \dot{\mathbf{q}}} \right) - \frac{\partial T}{\partial \mathbf{q}} = (J^T M_c J) \ddot{\mathbf{q}} + (J^T M_c \dot{J} + J^T [\tilde{\omega}] M_c J) \dot{\mathbf{q}}$$

基于牛顿-欧拉等式计算得到的刚体动力学等式。

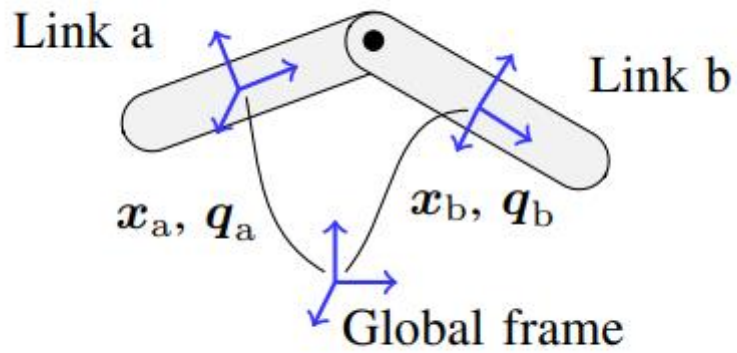
$$(J^T M_c J) \ddot{\mathbf{q}} + (J^T M_c \dot{J} + J^T [\tilde{\omega}] M_c J) \dot{\mathbf{q}} = J_v^T \mathbf{f} + J_\omega^T \boldsymbol{\tau}$$

可知质量矩阵、科氏力与离心力、力矩等式。

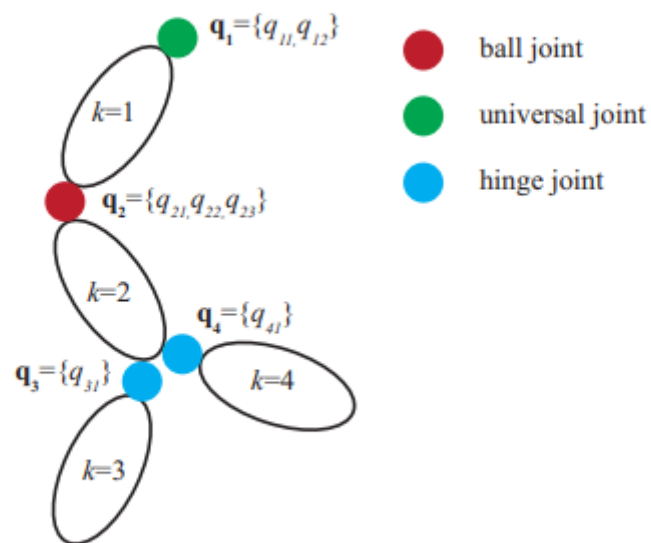
$$\begin{aligned} M(\mathbf{q}) &= J^T M_c J \\ C(\mathbf{q}, \dot{\mathbf{q}}) &= (J^T M_c \dot{J} + J^T [\tilde{\omega}] M_c J) \dot{\mathbf{q}} \\ \mathbf{Q} &= J_v^T \mathbf{f} + J_\omega^T \boldsymbol{\tau} \end{aligned}$$

三、关节刚体动力学

基于旋转关节的两连杆。



基于不同类型关节的多连杆刚体系统。



基于机械臂原理连杆之间的线性、旋转变换雅可比矩阵，再基于刚体拉格朗日动力学等式可知关节刚体系统的运动计算等式。

笛卡尔坐标系物理引擎

对于基于笛卡尔坐标系的物理引擎，仿真器仿真时基于广义坐标系计算控制力，然后转换为笛卡尔坐标系下的笛卡尔力(Cartesian Forces)，再基于物理引擎进行前向仿真(Forward Simulation)。

一、速度转换

$$\mathbf{V} \equiv \begin{pmatrix} \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_m \\ \boldsymbol{\omega}_1 \\ \vdots \\ \boldsymbol{\omega}_m \end{pmatrix} = \begin{pmatrix} J_{v1} \\ \vdots \\ J_{vm} \\ J_{\omega 1} \\ \vdots \\ J_{\omega m} \end{pmatrix} \begin{pmatrix} \dot{\mathbf{q}}_1 \\ \vdots \\ \dot{\mathbf{q}}_m \end{pmatrix} \equiv \begin{pmatrix} J_v \\ J_\omega \end{pmatrix} \dot{\mathbf{q}} \equiv J \dot{\mathbf{q}}$$

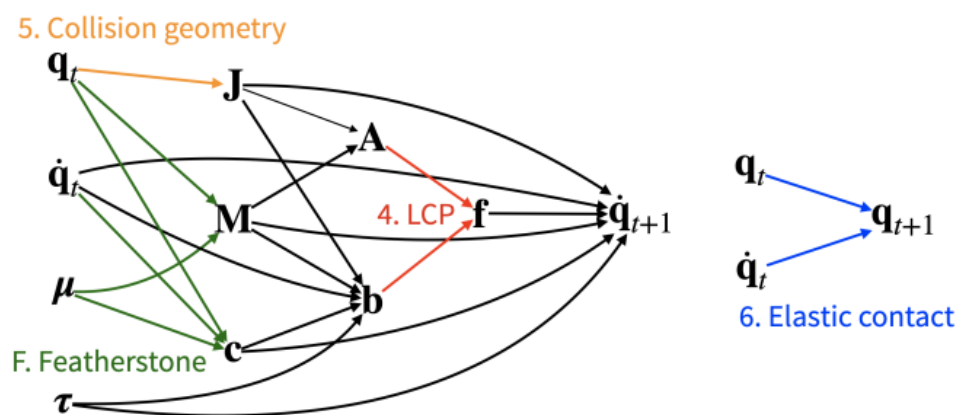
二、力转换

$$\mathbf{Q} = \sum_k J_{vk}'^T \mathbf{f}_k + J_{\omega k}^T \boldsymbol{\tau}_k' = \begin{pmatrix} J_v'^T & J_{\omega}^T \end{pmatrix} \begin{pmatrix} \mathbf{f}' \\ \boldsymbol{\tau}' \end{pmatrix}$$

广义坐标系物理引擎

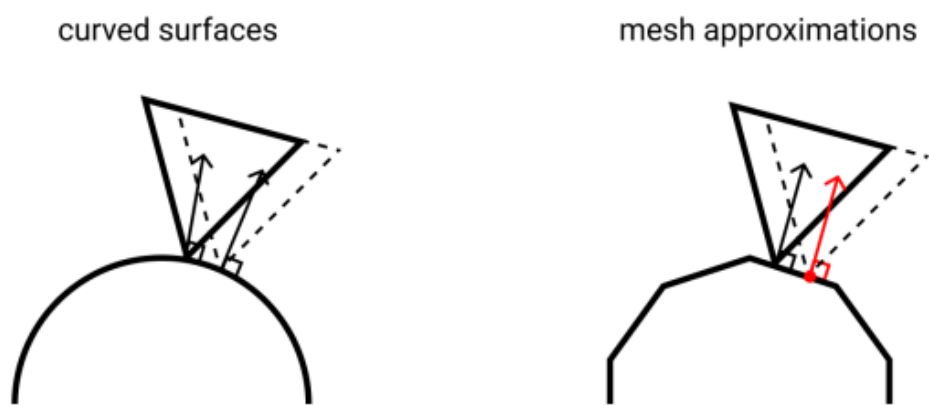
对于基于广义坐标系的物理引擎，前向仿真时输出下一时刻广义坐标系位置及速度。

可微分广义坐标系物理引擎前向仿真示例，基于当前位置、速度、控制力($\boldsymbol{\tau}$)及惯性特性(μ)计算连接力雅可比矩阵($\mathbf{J}$)、质量矩阵($\mathbf{M}$)及科氏力与重力($\mathbf{c}$)，基于线性互补问题(LCP， Linear Complementarity Problem)计算连接力。



一、碰撞计算

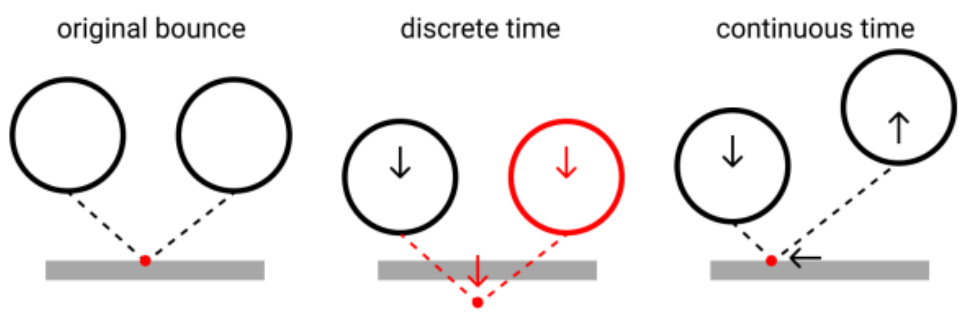
碰撞示例。



曲面碰撞计算不同于多面体。

二、弹性碰撞计算

弹性碰撞示例。



基于连续时间定义弹性碰撞的动力学，离散时间雅可比不足以定义弹性碰撞的动力学。

二、 MuJoCo

MuJoCo(Multi-Joint Dynamics with Contact)，基于广义坐标(Generalized Coordinates)、利用递归算法计算机器人动力学状态，基于速度相关(Velocity-Stepping)算法仿真连接点力。

MuJoCo 特性

传统基于笛卡尔坐标的物理引擎(ODE、PhysX、Bullet 等)在仿真人形机器人等多关节系统时，准确度与效率都不高；在应用线性互补问题(LCP, Linear Complementarity Problem)计算连接动力学(Contact Dynamics)时，需手动调节及很小的时间步长(time-steps)来获得稳定的移动。

基于广义坐标的物理引擎(SD/FAST、OpenSim)或者忽略连接力，或者利用弹簧阻尼(Spring Damper)方法导致碰撞变成重合(Large Penetration)或使得连接过于刚性(Stiff Dynamics)。

MuJoCo 改进了这些问题，同时，集成了很多仿真可能不需要却便于实际应用的特性。比方，可基于共享内存或分布架构等并行计算不同的状态及控制量；可进行逆向运动学计算以便分析记录的数据或需计算力矩的应用；可对执行器动力学(液压、气压动力，生物肌肉等)建模；可利用整个计算流程或选择部分流程进行目标计算；模型支持 XML 格式及 C++ API 形式等。

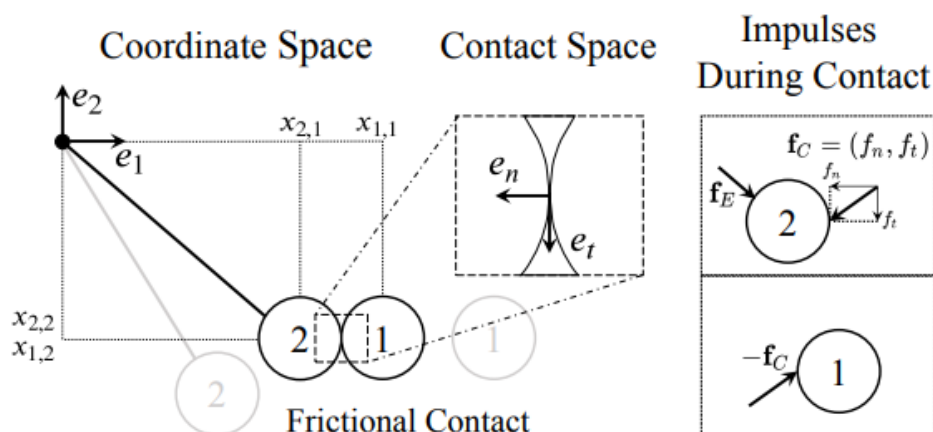
动力学解算

基于微分形式的机器人连续时间的运动动力学等式。

$$M(\mathbf{q}) d\mathbf{v} = (\mathbf{b}(\mathbf{q}, \mathbf{v}) + \boldsymbol{\tau}) dt + J_E(\mathbf{q})^T \mathbf{f}_E(\mathbf{q}, \mathbf{v}, \boldsymbol{\tau}) + J_C(\mathbf{q})^T \mathbf{f}_C(\mathbf{q}, \mathbf{v}, \boldsymbol{\tau})$$

M ，广义坐标惯性矩阵； b ，偏制力，科氏力、离心力、重力、弹簧力； J_E ，等式约束的雅可比矩阵； J_C ，连接力雅可比矩阵； f_E ，等式约束对应的冲力； f_C ，连接对应的冲力。

碰撞冲力示例说明。



运动等式的解算过程。

- 1、计算所有刚体的笛卡尔位置(Positions)及方向(Orientations)，检测潜在碰撞(Potential Collisions)，建立 J_E 及 J_C ；
- 2、利用复合刚体算法(CRB, Composite Rigid Body Algorithm)计算惯性矩阵 M ，利用递归牛顿-欧拉算法(RNE, Recursive Newton Euler Algorithm)计算偏置力 b 等；
- 3、建立 f_E 与 f_C 的函数关系式，将碰撞时连接力在切线方向的切线(Subspace Tangent)力投影到机器人等式约束空间(Equality Constraint Manifold)；

由于冲力(约束力、连接力)不连续，用 $v_{t+h}-v_t$ 替代 dv ，用 h 替代 dt ，可得机器人离散时间运动动力学等式，

$$M(q_t) v_{t+h} = s(q_t, v_t, \tau_t) + J_E(q_t)^T f_E + J_C(q_t)^T f_C$$

$$\mathbf{s}(\mathbf{q}_t, \mathbf{v}_t, \tau_t) = M(\mathbf{q}_t) \mathbf{v}_t + h\mathbf{b}(\mathbf{q}_t, \mathbf{v}_t) + h\tau_t$$

机器人期望的等式约束速度与下一时刻广义坐标速度的雅可比变换等式。

$$J_E(\mathbf{q}_t) \mathbf{v}_{t+h} = \mathbf{v}_E^*(\mathbf{q}_t)$$

机器人连接冲速度与下一时刻广义坐标速度的雅可比变换等式。

$$J_C(\mathbf{q}_t) \mathbf{v}_{t+h} = \mathbf{v}_C$$

上述方程联立可得 $\mathbf{f}_E$ 与 $\mathbf{f}_C$ 的函数关系式。

$$\mathbf{f}_E = (J_E M^{-1} J_E^T)^{-1} (\mathbf{v}_E^* - J_E M^{-1} (\mathbf{s} + J_C^T \mathbf{f}_C))$$

将 $\mathbf{f}_E$ 代入机器人离散时间运动动力学等式，可得机器人速度($\mathbf{v}$)计算等式。

$$\mathbf{v} = P J_C^T \mathbf{f}_C + \mathbf{r}$$

$$P = M^{-1} - M^{-1} J_E^T (J_E M^{-1} J_E^T)^{-1} J_E M^{-1}$$

$$\mathbf{r} = P \mathbf{s} + M^{-1} J_E^T (J_E M^{-1} J_E^T)^{-1} \mathbf{v}_E^*$$

4、投影连接力(到机器人等式约束空间)，计算连接冲力 $\mathbf{f}_C$ 及连接冲速度 $\mathbf{v}_C$;

机器人速度($\mathbf{v}$)计算等式与机器人连接冲速度雅可比变换等式联立可得连接坐标系下的连接冲力($\mathbf{f}_C$)与冲速度($\mathbf{v}_C$)计算等式。

$$A\mathbf{f}_C + \mathbf{v}_0 = \mathbf{v}_C$$

$$A = J_C P J_C^T$$

$$\mathbf{v}_0 = J_C \mathbf{r}$$

连接冲力法线分量与连接冲速度法线分量形成的约束等式(互补条件)。

$$f^{\mathcal{N}} \geq 0, \quad v^{\mathcal{N}} \geq 0, \quad f^{\mathcal{N}} v^{\mathcal{N}} = 0$$

虽然，约束等式与连接冲力($\mathbf{f}_C$)及冲速度($\mathbf{v}_C$)计算等式联立可形成线性互补问题(LCP, Linear Complementarity Problem)。

但是，连接冲力摩擦力方向分量与连接冲速度摩擦力方向分量方向相反，连接冲力在摩擦力矢量锥体内，不适于线性互补问题。

因此，MuJoCo 提供的是隐式互补、凸解算及对角解算三类不同的解算算法。

5、进行数值积分获得机器人下一刻状态(位置信息等);

基于机器人速度($\mathbf{v}$)计算等式及连接冲力得到下一刻速度，然后利用 Semi-Implicit Euler 或四阶 Runge-Kutta 积分器进行四元数积分得到机器人下一刻位置状态信息等。

连接仿真解算算法

MuJoCo 连接仿真基于数值优化算法，依据解算速度与准度分为隐式互补解算器(Implicit Complementarity Solver)、凸解算器(Convex Solver)、对角解算器(Diagonal Solver)。

一、隐式互补解算器

准确度优先解算器。基于连接冲力与连接冲速互补条件，将两者映射作变量 $\mathbf{x}$ 的函数。基于连接冲力($\mathbf{f}_C$)与冲速度($\mathbf{v}_C$)计算等式可得优化目标函数，由此可得近似解。

$$\min_{\mathbf{x}} \|\mathbf{A}\mathbf{f}_C(\mathbf{x}) + \mathbf{v}_0 - \mathbf{v}_C(\mathbf{x})\|^2$$

二、凸解算器

兼顾速度与准确度的解算器。基于连接冲力总是减缓碰撞表面之间的相对速度，因此可对连接坐标系下的动能进行最小化计算，同时满足在摩擦力矢量锥内及碰撞约束条件，由此利用凸优化问题替代连接冲力与连接冲速度法线及摩擦力方向的约束条件得到近似解。

连接坐标系动能最小时的连接冲力的计算式。

$$\mathbf{f}_C = \arg \min_{\mathbf{f} \in \text{Cone}} \frac{1}{2} \mathbf{f}^T (\mathbf{A} + \mathbf{R}) \mathbf{f} + \mathbf{f}^T \mathbf{v}_0 + q(\mathbf{A}\mathbf{f} + \mathbf{v}_0)$$

三、对角解算器

速度优先解算器。相当于质量-弹簧-阻尼系统，基于惯性相关矩阵 $\mathbf{A}$ 对角元素，调整弹性阻尼器为临界阻尼弹簧，使连接过程不振荡、可积分，由此可计算期望的下一状态的冲速及连接冲力。

第七章 测试台(Benchmark)

一、模型评估

大模型评估方法

1, 多项选择问答(multiple-choice question answering)

比方 MMLU(Massive Multitask Language Understanding) 测试台 ,
<https://huggingface.co/datasets/cais/mmlu>

57 subjects (from high school math to biology) with about 16 thousand multiple-choice questions in total

基于回答的正确率进行评估.

code, 需要加载 MMLU 数据集, 编写 提示函数, 标记化提示, 定义得分函数、检验函数.

2, 检验器(verifiers)方法, 与多项选择回答不同的是, 不限定选项, 允许 LLM 自由回答, 然后提取与答案相关部分进行比较。

只适用于数学及编程(math and code)等容易验证的场景; 同时可能需要用到第三方工具进行比较。

优点在于允许生成无限的数学或编程问题, 同时展示逐步推理过程, 因此, 适合推理模型的评估及开发。

3、基于用户偏好投票或对两两模型进行比较排名(LLM leaderboards),得到包括对回答风格等的整体评价. 不同于多项选择问答、检验器等基于模型准确率的方法.

比方 LM Aren 排行榜

4、基于别的指定评估标准的大模型来对待评估模型进行比较排名.

早期方法是 BLEU , 粗略的比较生成的文本与参考文本. 现在, 用提示词训练 gpt-oss 作 判断模型(judge model).

比方 提示告诉 gpt-oss 要判断的内容的格式及应该具备什么样的回答风格较好.

可在 lap-top 本地机器用 ollama 运行不同权重规模的 gpt-oss 模型进行推理.

命令行运行 比方 ollama run gpt-oss:20b 或 ollama run qwen3:4b

然后 在命令行进行交互 >>> What is 1+2?

二、 测试台

Gemini Robotics-ER 1.5 的学术具身推理测试台 (Academic Embodied Reasoning Benchmarks) : Point-Bench, RefSpatial, RoboSpatial-Pointing, Where2Place, BLINK, CV-Bench, ERQA, EmbSpatial, MindCube, RoboSpatial-VQA , SAT , Cosmos-Reason1, Min Video Pairs, OpenEQA, VSI-Bench.

评估 Robot 语义安全的测试台 :ASIMOV。

网络通讯篇

基于通讯介质的网络分类, 有线, 比方, 双绞线(Copper Twisted Pair Cables)、同轴线缆(Copper Coaxial Calbes)、光纤(Glass Fiber Cables)等; 无线, 比方, Zigbee、蓝牙(2.4GHZ)、WIFI(2.4,5,6GHZ)、蜂窝(Cellular, About 1G-1THZ)、卫星(About 1-40GHZ)等。

基于拓扑, 可分为总线、树形、环形、星形、网面形网等。基于端、中心, 可分为接入、汇聚、骨干网等。

当前, 卫星网络作为星际及人工智能时代的核心要素, 不只在改变传统以地球为主的基站蜂窝通讯、算力中心格局, 更可能在实现未来地球、月球、火星等行星之间计算、通讯的基石。

第一章 卫星网络

卫星网络正成为可与地面网络并驱的通信设施。当前，美国 SpaceX 已部署万颗在轨星链卫星。随着星舰的逐步成功及人工智能算力需求的发展，星链不只在数量、带宽等方面会很快跃升，在人工智能基础设施方面也将会发挥极大作用。

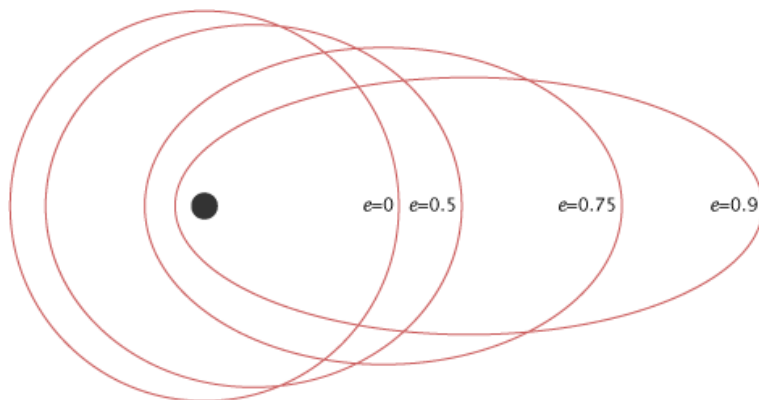
卫星星座似乎在变成综合网络通讯、算力集群、太阳能汲取等多项功能的发展基座。这使得通用机器人等人工智能应用的通讯、智算中心等可能发生深刻的变革。

一、 卫星轨道

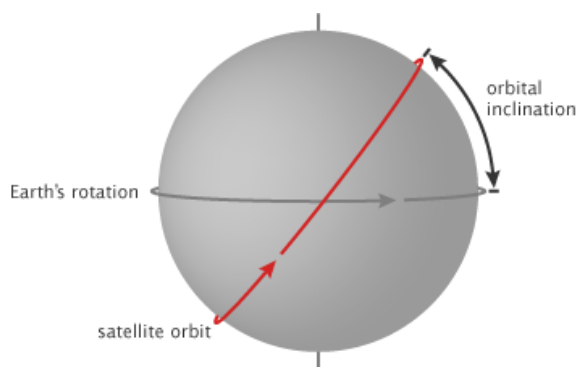
卫星高度不同则环绕地球运行的速度不同。



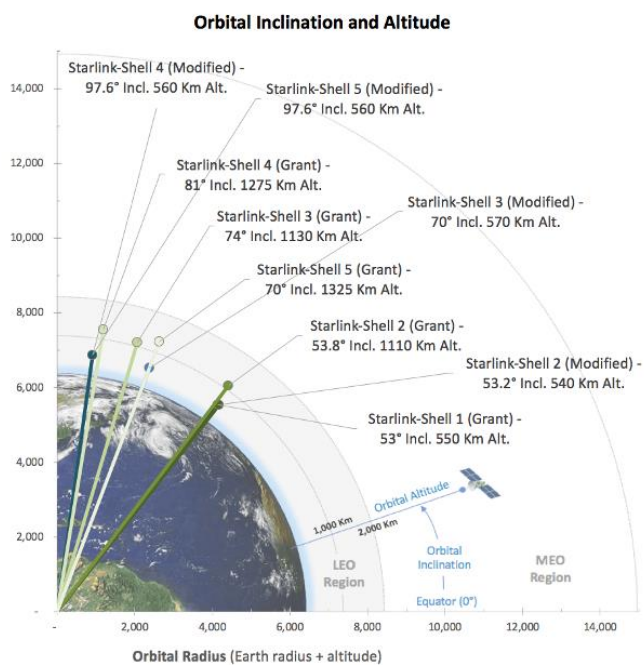
卫星轨道离心率(e , eccentricity)不同则椭圆程度不同。



卫星轨道倾角不同卫星轨道形状不同。

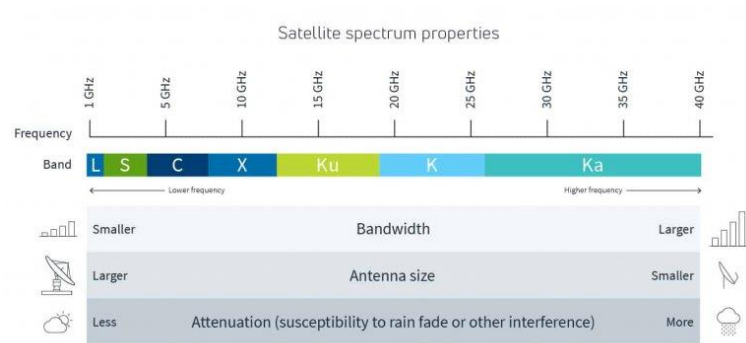


星链轨道平面。

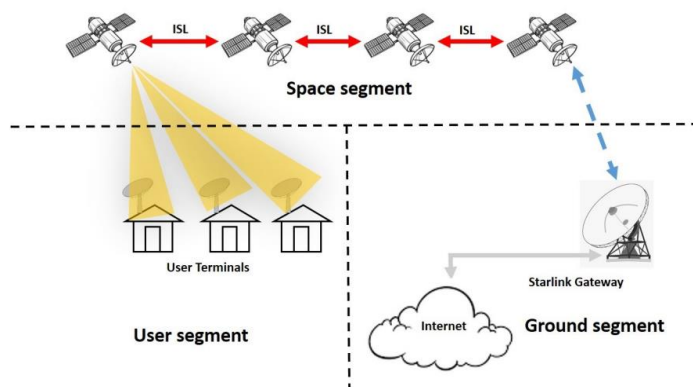


二、 网络架构

卫星通信频谱。



通信网络。



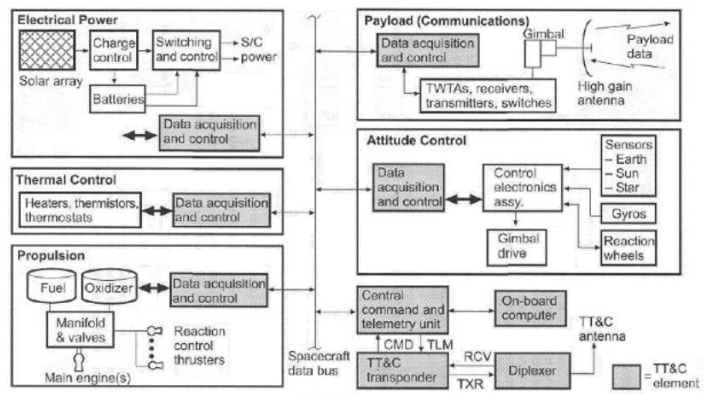
星链卫星运行在不同平面的近地轨道(LEO, Low Earth Orbit), 每个卫星多个相控阵列天线(Phased Array Antenna), 基于 Ka、Ku 等频带利用 OFDM(Orthogonal Frequency-Division Multiplexing)机制与地面通信, 卫星间通过激光建立通信链路。

星链用户终端(UT, User Terminal)同样基于相控阵列天线基于 Ka、Ku 等频带与星链卫星群(Satellite Constellation)通信。用户终端联网初始状态, 基于定位等信息调整相控阵列天线仰角确立终端与卫星通信的基本方位, 然后基于相控阵列天线多波束搜索通信卫星。

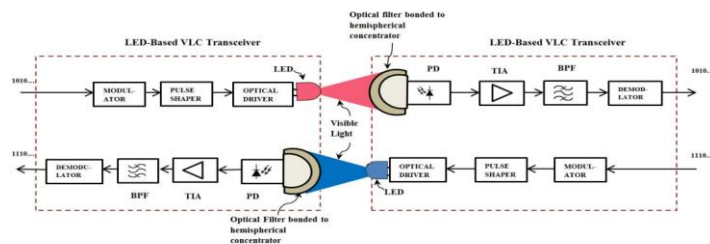
星链用户终端在搜索完成后, 选择信号最强或易于维持连接时长的一个卫星建立上行/下行的通信连接, 同时基于多波束保持对其余卫星的追踪。

当通信卫星远离星链用户终端使通信信号衰落或不可用时, 则从追踪的卫星中选择信号最强或易于维持连接时长的卫星接管通信。

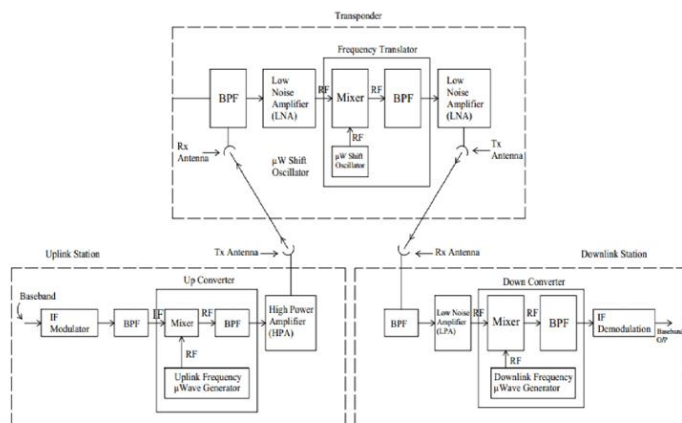
三、 卫星框架



四、 星间通信



五、 星地通信



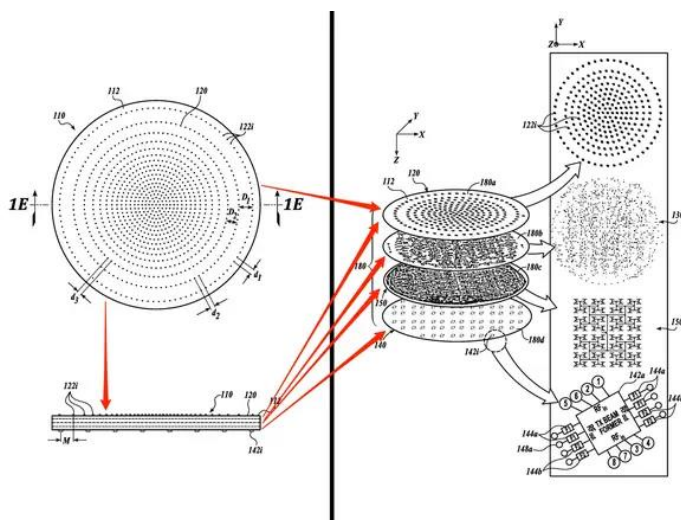
信号接收，通过接收天线接收到来自地面的信号，先经过带通滤波器(BPF, Band Pass Filter)选择特定频率的信号，再通过低噪放大器(LNA, Low Noise Amplifier)获得降噪放大的调制射频信号，然后进行信号解调等处理。

信号发送，将来自指令与数据处理中心(CDH, Command and Data Handling)的信息调制为射频信号，通过发射天线发射。

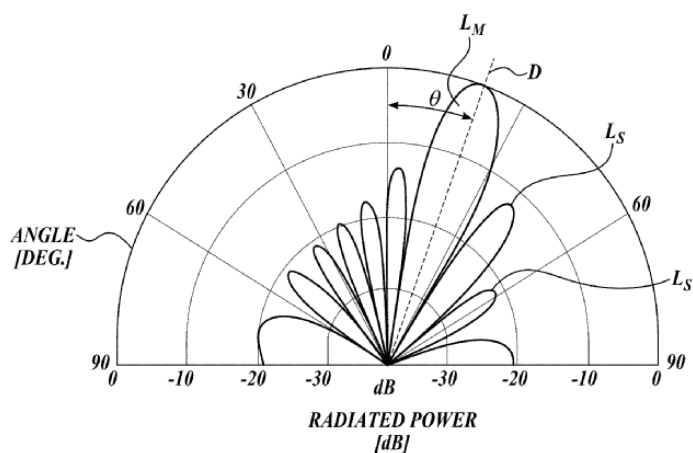
信号转发，卫星转发器(Transponder)对接收天线接收到的来自地面的上行信号进行带通滤波、低噪放大，然后通过频率变换器(Frequency Translator)将射频信号调整为发向地面的下行信号频率，经过低噪放大器放大后，通过发射天线发射。

六、 星链终端

1. 天线阵列



星链专利相控阵列天线基于电磁波干涉原理,通过调整相控阵列天线单元电磁波相位控制电磁波方向及强度(电磁波塑形, Beamforming)形成多波束。



2. 主要芯片

星链用户终端电路部件主要包括应用芯片、射频芯片、内存、存储芯片、电源芯片、安全加密芯片及 GPS Receiver 等，整个系统通过以太网供电(PoE, Power over Ethernet)。

1) 应用芯片

ST GLLCCOCA6BF，芯片处理器为四核 Quad-Core ARM Cortex-A53，各核任务功能独立以增强系统健壮性。

第 0 核(CPU0): 控制进程(control process)。

第 1 核(CPU1): 接收进程(RX Process)。

第 2 核(CPU2): 传输进程(TX Process)。

第 3 核(CPU3): 中断等任务(Interrupts, Auxiliary processes)。

2) 存储芯片

Micron eMMC，基于 Reed Solomon ECC 算法进行检验及纠错以避免数据错误影响系统运行。

3) 射频电路(Radio Frequency Equipment)

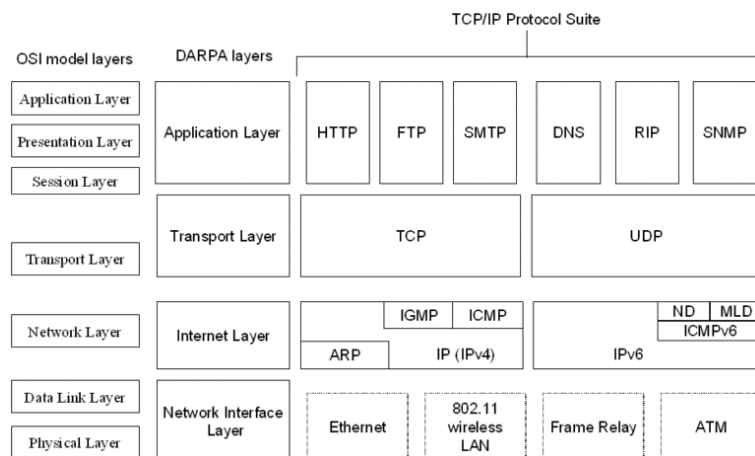
射频前端及末级芯片，贴铝箔的为前端芯片，用于进行调制及混频等，末级芯片用于进行功率放大等。

3. 固件

星链用户终端固件由引导程序、内核、文件系统等组成，基于卫星通信实现远程自动更新。固件运行基本流程，U-Boot 作为引导程序基于 FIT(Flattened ulmage Tree)镜像格式加载 Linux 内核等镜像后，运行卫星通讯等应用程序。

第二章 TCP/IP 网络

TCP/IP 网络是互联网、电信网、广电网等应用网络实现数据通讯的重要支撑。



一、 物理层网络

计算机、路由器物理节点之间的直接通信。

比方，以太网。物理控制器，ETH PHY。

二、 网络层网络

基于 IP 报文封装，可通过路由器等转发节点传递的通信。

比方，通信内容分成多个 IP 报文，在发端发布，跨路由器矩阵分进传输，在接收点合成。路由器基于 IP 报文进行拥塞控制。

三、 传输层网络

基于 TCP 进行可靠连接(Stream)，基于 UDP 进行非可靠连接(Datagram)。

基于源 IP、端口，目的 IP、端口，及协议确定事务对应的连接。

TCP 基于三次握手建立连接、四次握手拆连，利用 ACK 可靠传输数据，通过滑动窗口进行通信速率控制。

UDP 基于报文非可靠通信。

四、 应用层网络

当前，对通信实时性要求不是很高的应用，基本基于 HTTP 的 GET、POST、PUT、DELETE 方法及 JSON 格式定义实现。

第三章 软件定义网络(SDN)

软件定义网络，实质是 OSI 或 TCP/IP 网络层的嵌套，以实现在原网络中架设虚拟网络。这与操作系统嵌套操作系统以支持虚拟机的方法类似。

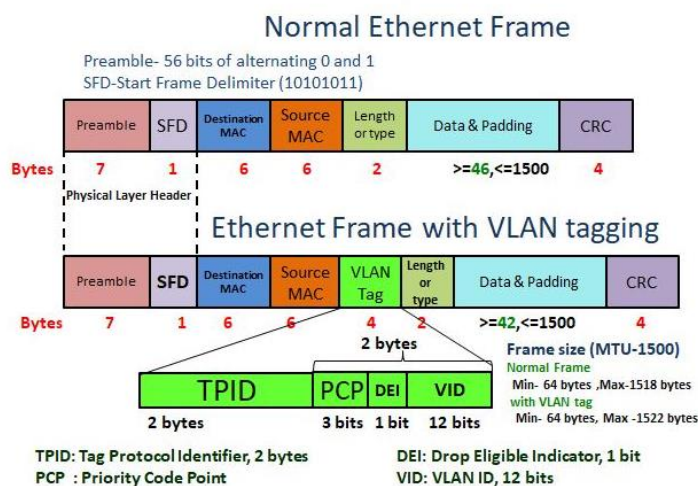
在云设施环境，虚拟网络与虚拟机配套提供相互隔离、独立的环境。

软件定义网络的基石是 VLAN, VxLAN 协议。VLAN 基于以太网层封装，一般需交换机支持，独立性不高。VxLAN 基于 UDP 网络层封装，可完全通过软件定义，性能相对低。

注，网络分层定义，OSI 7 层，TCP/IP 4 层。以第 N 层来说明封装易带来概念混淆。

一、 VLAN

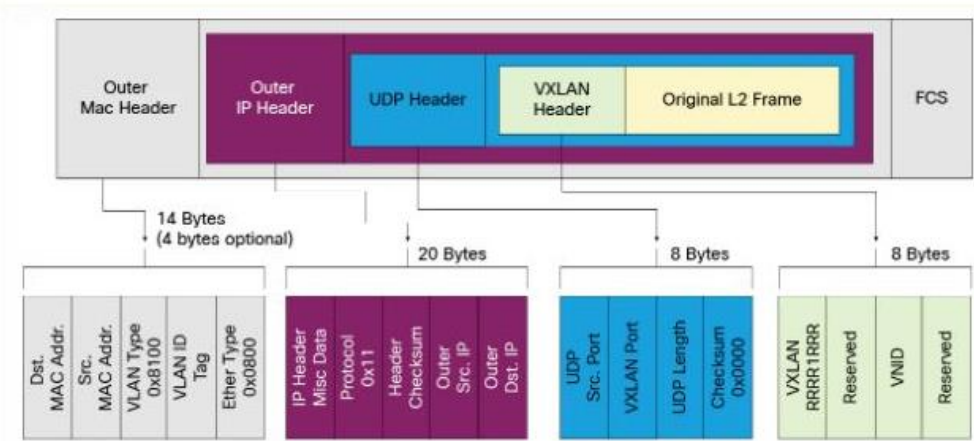
VLAN 帧格式。



框架图，基于以太帧，通过 VLAN 标签扩展。VLAN 标签，协议标识(TPID)，VLAN 标识(VID)等。

二、 VxLAN

VxLAN 帧格式。



框架图，左至右，以太帧头、IP 报头、UDP 报头、VxLAN 帧头及数据。VxLAN 帧头，协议标
记(RRRR1RRR)、VxLAN 标识(VNID, Virtual Network ID)等。

云设施篇

Tesla Colossus 2 超算中心。



当前，主流的云设施基于 ARM、Intel、AMD 等 SoC 芯片的电路板，利用虚拟机及容器等虚拟化技术实现硬件资源的充分使用以尽可能提供更多用户的计算环境独有需求。

同时，量子计算与半导体计算机融合、智能算力太空化部署已展露迹象。

云设施分层。

第一层，芯片虚拟化技术支持。

第二层，操作系统虚拟化及容器化技术支持。

第三层，分布式虚拟机支持。

第四层，分布式容器化支持。

一、 半导体芯片

在指令集架构(ISA, Instruction Set Architecture)层面, ARM 基于精简指令集(RISC), Intel 基于复杂指令集(CISC)。从向量空间的角度来看, RISC、CISC 指令集可看作是对 CPU 指令数据集的不同正交化分解。应该说, ARM RISC 正交性更好。因此, 设计简洁、功耗相对低, 适用于边缘设备、服务器及数据中心。

当前, 边缘计算、云计算芯片是 ARM-Based SoC, Intel SoC, AMD SoC。

intel PC architecture 中,北桥 分别连接内存及 GPU(PCIe link), GPU 有单独的 GPU 显存; AMD PC architecture 中, 北桥分别连接内存及 chipset, chipset 连接 GPU(PCIe link), GPU 有单独的 GPU 显存 GDDR.

二、 量子芯片

当前, 量子计算的主要问题是量子纠错(Quantum Error Correction)及通用性。可能需要重新设计计算架构来避免量子比特(Qubits)错误、实现量子通用计算。

三、 虚拟机

基于 CPU 指令运行模式(Mode)及权限(Privilege Level)架构, 利用运行在高权限虚拟化模式的虚拟机管理器(Hypervisor)作客户虚拟机与主机、客户虚拟机与内存、客户虚拟机与外设、客户虚拟机与外部中断等之间的代理或上下文切换的配置者, 实现单芯片多虚拟机可相互独立、高速运行。

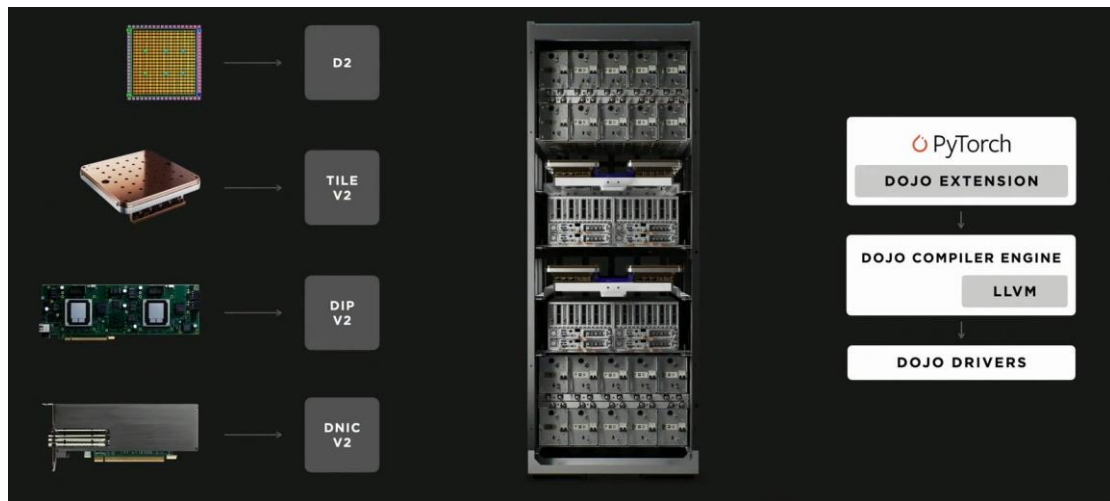
四、 容器虚拟化

基于操作系统的命名空间(Namespace)及控制组(Cgroup)等特性，结合镜像分层等方法，实现容器的创立、封装及运行、部署等。

第一章 超算中心

一、 Tesla Dojo

超级计算机大规模分布式系统，运算单元基本硬件架构由 CPU、GPU、RDMA(Remote Direct Memory Access)组成。相应硬件厂商包括 IBM、Intel、Nvidia、AMD、Mellanox 等。软件架构可以用 OpenHPC。



机柜左侧分别是封装集成电路单元阵列的 D1/D2 晶粒(Die)，封装晶粒阵列的 TILE 模块，连接主机与 GPU 的接口处理器(DIP, Dojo Interface Processor)板，网络接口卡(DNIC, Dojo Network Interface Card)。

机柜右侧是基于 Dojo 驱动及 Dojo 编译引擎的 Pytorch 框架。

超级计算机运算单元主要包括计算(Compute)、通信(Communication)与同步(Synchronization)三部分。

一、 计算(Compute)

TILE 模块，5 行*5 列 Die。

二、通信(Communication)

不同 TILE 相邻 Die 之间、Die 与 Die 之间、集成电路单元之间通过集成电路连接，DIP 与主机之间通过 PCIe 连接实现通信，DIPs 之间通过以太网交换机(Ethernet Switch)相互连接。

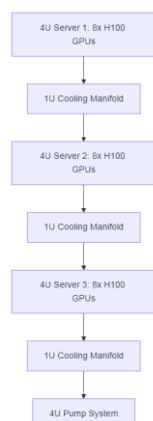
三、同步(Synchronization)

系统通过信号量、Barrier Trees 实现信息同步。

二、 xAI Colossus

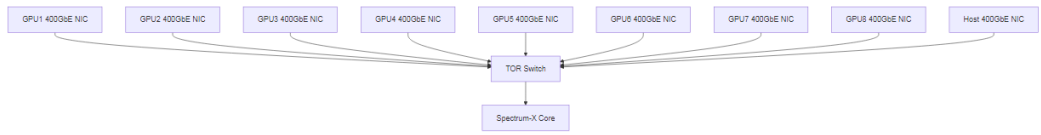
基于超微(Supermicro)GPU(Nvidia H100/H200)系统部署。超微主板集成 4 个 PCIe 交换模块(Switches)。基于 NVMe 进行存储计算通信。基于 NVIDIA BlueField-3 SuperNICs 及 Spectrum-X networking 进行 RDMA 远程内存访问。GPUs 网络独立于集群余部。网络速度 400GbE。

Rack 配置示例。



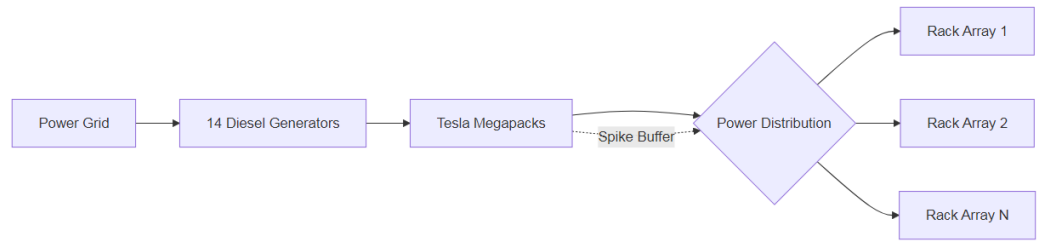
框架图，顶部，Server 单元与液冷管(Manifold)交替摆放；底部，冷泵系统(Pump System)，冷却循环管理(Coolant Circulation Management)等。

网络系统架构。



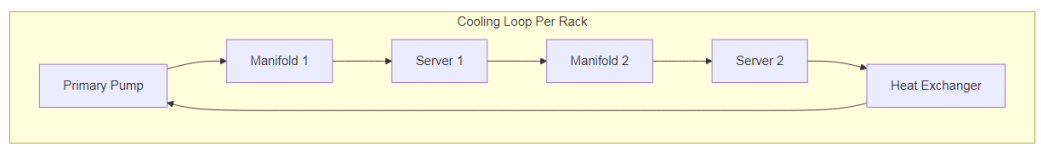
框架图，基于以太(Ethernet-Only)架构；每个 GPU 专用 400GbE 网卡；主机网卡独立。

电源分布系统(Power Distribution System)。



框架图，左侧，Power Grid 主要供电；Diesel Generation 备用供电；Megapacks，缓冲器；右侧，Power Distribution，分布供电。可平滑系统训练时的功率波动(Power Fluctuations)。

液冷系统框架。



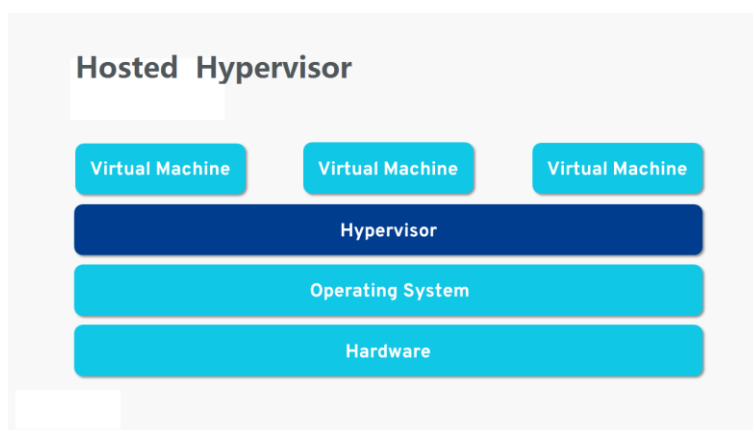
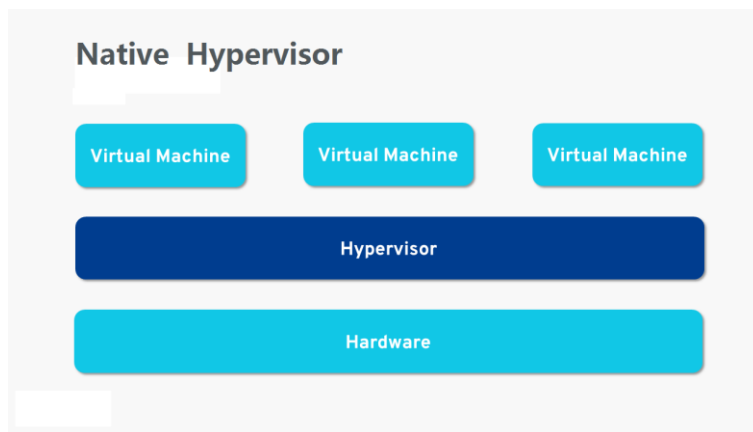
框架图，闭环设计(Closed Loop Design)，主泵可持续循环冷却(Circulates Coolant)；Rack 液冷

循环独立；泵冗余设计以确保系统持续运行；最小化单点风险。液冷管系统，向 GPUs 均压供冷，支持 Servers 热交换等。热交换器(Heat Exchanger)。

第二章 虚拟机

一、 Hypervisor

Hypervisor，虚拟机管理器。可看作一个框架，用来创建隔离及连接物理资源与虚拟机的虚拟层。存在两类 Hypervisor，基于物理资源的 Native Hypervisor ； 基于操作系统的 Hosted Hypervisor。



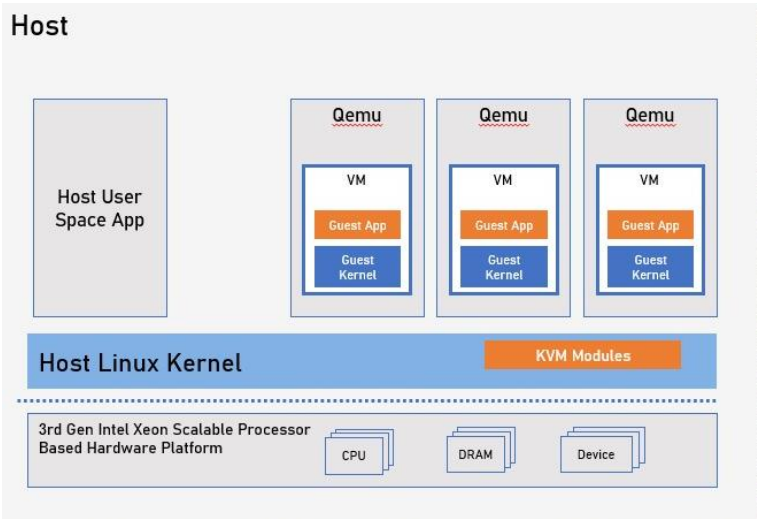
基于硬件资源的虚拟机(Type-1 Hypervisor)，XEN、ESXi。

基于主机的虚拟机(Type-2 Hypervisor)，Virtual Box、Vmware Workstation。

KVM、Hyper-V 等可作 Type-2 类型或介于 Type-1、Type2 类型。比方，KVM 基于 Linux 内核，同时又是内核模块，运行时在内核态，使得 Linux 内核表现作 Type-1 类型。

二、 KVM

KVM(Kernel-Based Virtual Machine)，基于 Linux 内核，结合 libvirt(API and Service Manager)、Virtual-Manager(GUI)、Virsh(CLI)及 QEMU(硬件仿真器)实现的开源硬件虚拟机。可作为独立进程运行，支持虚拟 CPU、内存、网络接口等。支持 OpenStack、AWS 等，单机可支持数千虚拟机。支持 Linux 容器(LXC, Linux Containers)。



KVM Modules， 运行在 Linux 内核态、基于内核调度器(Scheduler)及内存管理系统创建及管理虚拟机的模块，使得 Linux 内核转变成硬件型虚拟机管理器(Type-1 Hypervisor)。可利用硬件虚拟化技术(Intel VT-x, AMD-V)。支持 CPU 虚拟化。

QEMU(Quick Emulator)，运行在用户态，协同 KVM 提供硬件虚拟化的仿真器。支持硬盘、网络、USB 设备等虚拟化。

Libvirt，运行在用户态，支撑 QEMU 的虚拟化，用于简化创建及管理虚拟机的工具库(Toolkit)。

Libvirt 联合 QEMU 作 KVM 虚拟化栈(Virtualization Stack)的用户空间侧的实现。

三、 XEN

早期发布的虚拟化技术，是可直接运行在硬件上的、基于微内核(Microkernel)架构、独立于操作系统的开源虚拟机管理器。基于控制域(Dom0, Control Domain)管理 Hypervisor 及客户虚拟机；基于非特权域(DomU, Unprivileged Domains)定义客户虚拟机(Guest VMs)。可支持全虚拟机(Fully Virtualized Guests)及半虚拟机(Paravirtualized Guests)。

Xen 的 Hypervisor 只实现对硬件的最小支持。基于 Dom0 虚拟机实现对设备驱动的支持。Dom0 虚拟机相当于 DomU 等虚拟机访问设备驱动的代理。

第三章 容器虚拟化

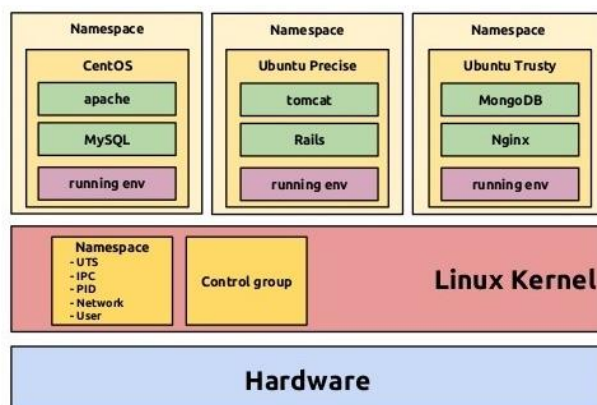
容器虚拟化技术始于早期的 Chroot 可更改进程 Root 目录的功能; 2000 年时, 经 FreeBSD Jails 扩展可隔离网络接口及 IP 地址; Linux-VServer 利用相似机制实现文件系统及内存等资源的划分(Partition); 2005 年, Linux OpenVZ 对操作系统级的虚拟化作了进一步改进; 之后, LXC(Linux Containers)实现完全的容器管理, 可对应用进程树、网络、文件系统等进行完全的隔离。

Docker 基于 LXC 思路, 利用 libcontainer 替代 LXC 作用, 通过简洁化打包应用及依赖到容器, 自动化构建(Building)、版本管理(Versioning)、容器重用等极大的促进了容器虚拟技术的使用。

容器技术核心在于利用 Linux Kernel 的 Cgroups 及 Namespaces 等特性实现进程隔离(Process Isolation)。

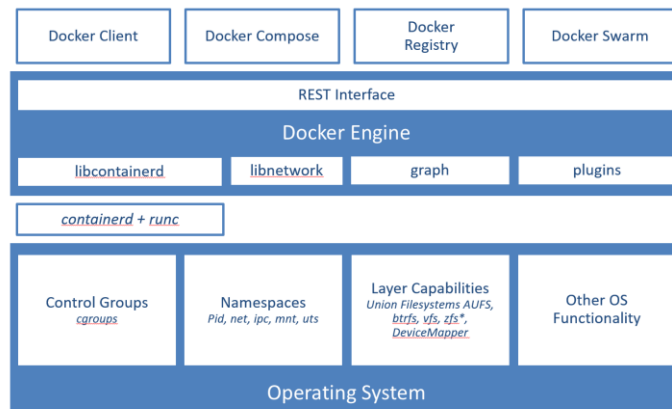
一、 LXC

基于 Linux 的命名空间(Namespace)、进程控制组(Cgroups)等 Linux 内核特性, 实现进程、文件系统、网络资源等的隔离及控制, 以创建、管理操作系统级容器虚拟环境(OS-Level Virtualization)。

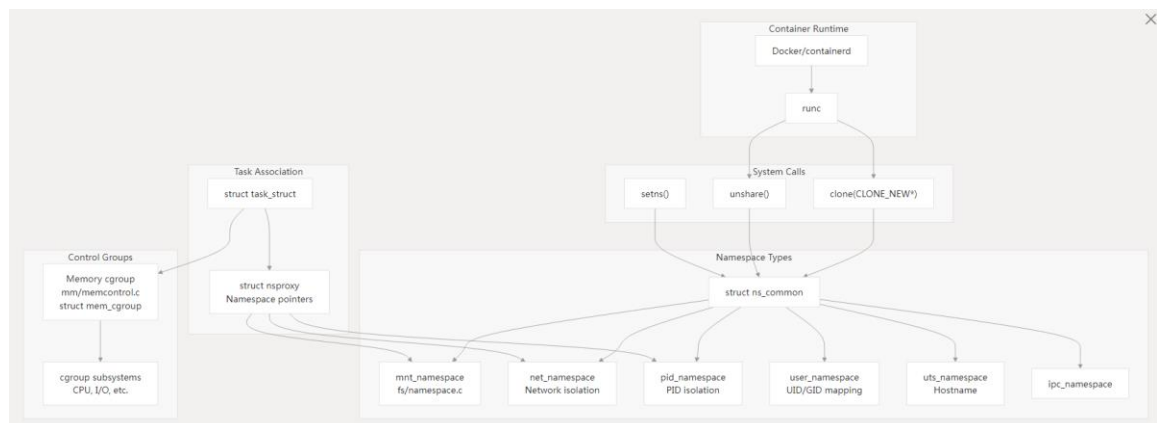


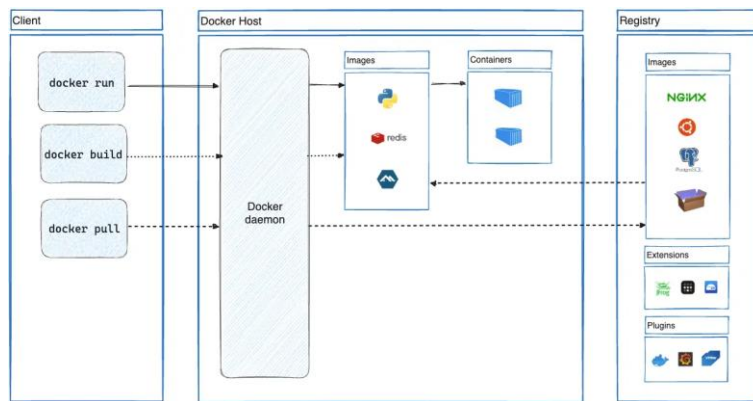
二、 Dockers

1. Docker 架构



Docker 容器实质基于 Linux 的命名空间(Namespace)、进程控制组(Cgroups)、重文件系统(OverlayFS)等实现容器的专用及相互隔离。





Client， Docker 客户端。

Docker Host， Docker 主机端。

Docker daemon， 侦听客户端发来的 API Requests， 对镜像(Images)、容器(Containers)、网络等 Docker 对象(Objects)进行管理。

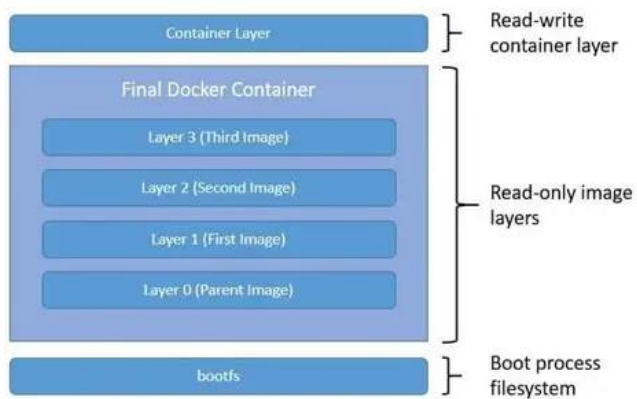
Images， 创建 Docker 容器的指令模板(Read-Only Template with Instructions)。注， 镜像一般可基于别的镜像创建。比方， 基于 Ubuntu 镜像创建特定镜像。同时， 创建指令存放在 Dockerfile 文件中， 每条指令执行时， 只是在镜像中创建文件系统层(Filesystem Layer)。这样便于修改指令时， 只改变相关层信息。

Containers， 可运行的镜像实例。

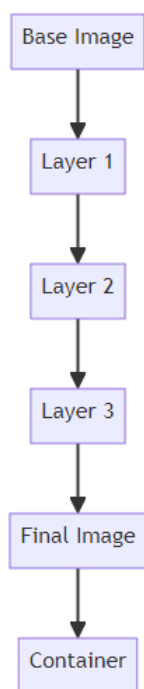
Registry， 镜像仓库。比方， DockerHub。

2. 镜像层(Image Layers)

镜像层实质是文件系统的变化量(Difference)， 对所有层顺序堆栈形成容器运行的文件系统。



创建(Build)容器时的镜像层顺序。



基层(Base Layer)是 Base Image，比方，Ubuntu 镜像；中间层(Intermediate Layers)，Layer1 等；最顶层(Top Layer)，对应 Dockerfile 中的最末指令。

基于 Dockerfile 创建 Docker 镜像示例。

```
FROM ubuntu:20.04
RUN apt-get update
```

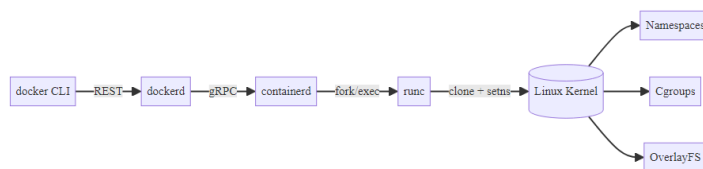
COPY app /app

这些指令对应着不同的镜像层。运行容器时，这些镜像层堆栈(Stack)形成容器的文件系统。

镜像基于层进行模块化设计，便于修改、共享、重用等，可加速创建流程、减少存储空间占用、镜像分发等。

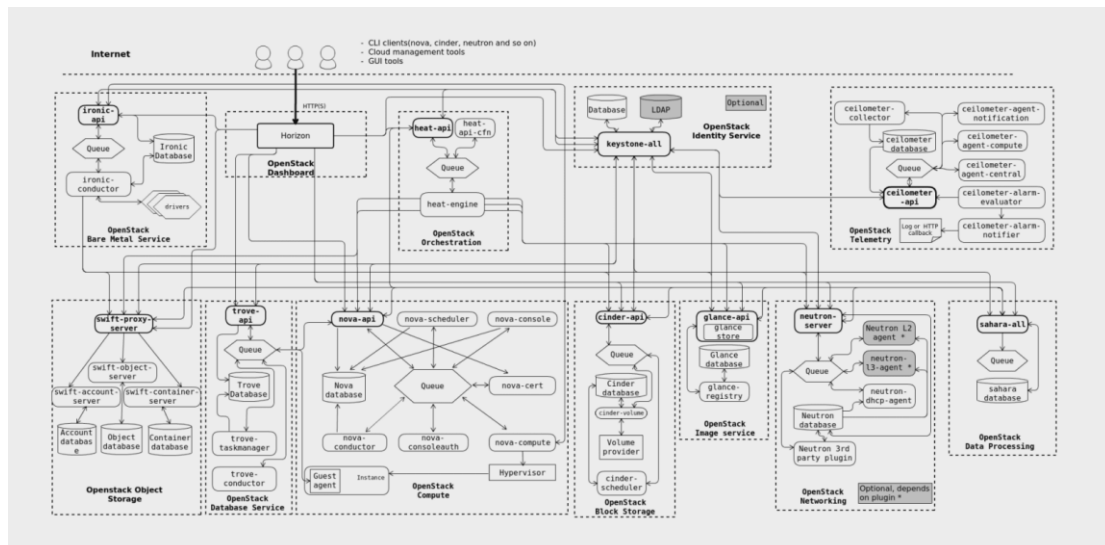
3. 命令调用

命令调用关系。



第四章 虚拟机云设施

Openstack，整体框架核心在于对计算、存储、网络、镜像资源的管理及基于模板自动组装资源。运营部分与一般的互联网运营平台相似，用户交互、认证授权、账单计费。



运营管理。

Dashboard，用户交互管理。支持用户网络访问、管理云资源。

Keystone，认证授权。支持用户名单(LDAP)、单点登录、访问鉴权授权等。

Ceilometer，账单计费管理。支持云模块及数据测控、计费。

资源管理。

Nova，计算资源管理。创建、清理、调度虚拟机等。

Cinder，块存储资源管理。创建、绑定块存储设备等。

Swift，对象存储资源管理。存储非结构化数据等。

Neutron，网络资源管理。支持设备之间建立连接、软件定义网络(SDN)、设置防火墙等。

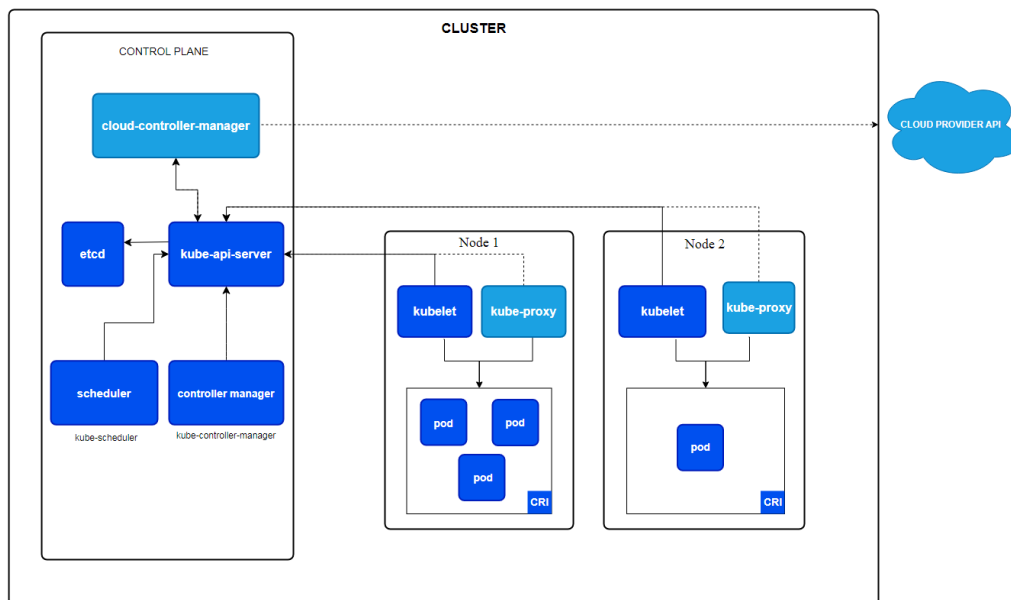
Glance， 虚拟机镜像管理。支持搜索、注册、获取镜像等功能。

Heat， 自动组装资源。基于模板文件配置，自动创立资源模块。

第五章 容器云设施

一、Kubernetes

Kubernetes 实质，基于任务需求，创建、分配、运行、保活(Keepalive)、销毁容器或容器集群。整体框架基于控制及业务分离原则。



控制面板(Control Plane)。

Cloud-Controller-Manager，第三方云设施管理。基于第三方云设施提供商 API 进行通信，确定机器节点是否运行正常、设置云基础设施网络、创建或移去云提供商的均衡器(Load Balancers)。

Kube-API-Server，前端 API。基于该 API 实现用户交互界面，以使用户访问、控制容器集群。

Etcd，键值(Key-Value)存储器。相对于 Redis 数据库，具备高一致性及可用性；基于多版本并发控制机制(MVCC, Multi-Version Concurrency Control)实现对数据库键(Keys)的实时观测(Watch)。存储集群配置、状态及元数据等信息。

Kube-Scheduler，调度器。给新创建的主机群(Pods)分配设备节点(Node)，以便运行。

Kube-Controller-Manager，控制器集。实现对节点、任务、网络地址等的控制。比方，节点控制器(Node Controller)，给节点分配路由、维持节点列表最新、侦测节点运行情况、回收异常节点中的主机群(Pods)等；任务控制器(Job Controller)，为任务创建主机群，完成任务执行；端点分片控制器(EndpointSlice Controller)，管理业务(Service)对应的后端主机群(Pods)的网络信息分片。

注，端点分片(Endpoint Slices)。业务在大量容器中的实例对应大量 IP 地址及端口、协议信息。这些网络信息可分片存储，比方，每 100 对地址、端口等信息进行分片(Slice)，以便在容器状况变化时，随时添加或清理。

节点管理。

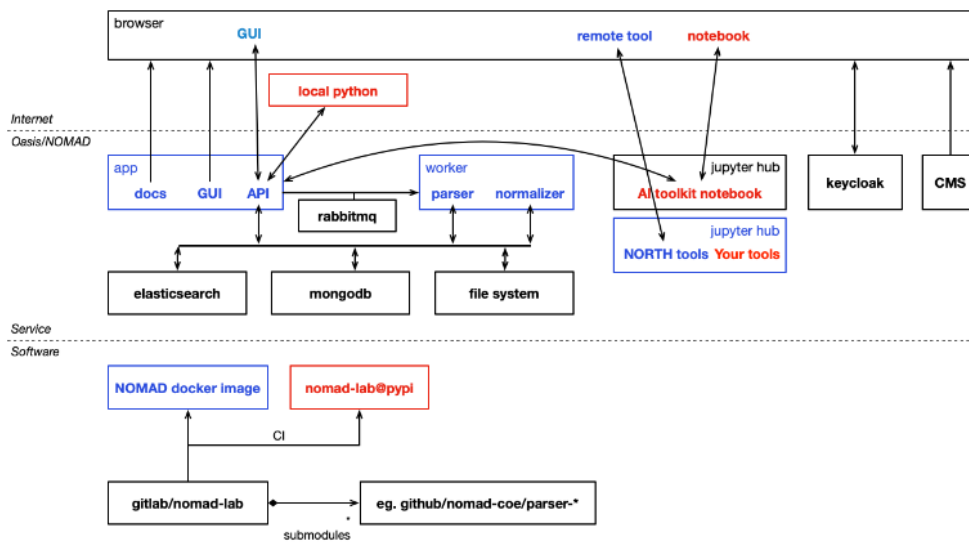
Kubelet，管理 Kubernetes 创建的容器。运行在节点中，确保主机中基于主机规范(PodSpecs)的容器正确运行。

Kube-proxy，网络代理。运行在机器节点中，维护节点网络规则。可选 IPTables 或 IPVS(LVS, Linux Virtual Server)模式。

Container Runtime，容器运行环境。比方，Containerd，管理容器运行周期(Lifecycle)。

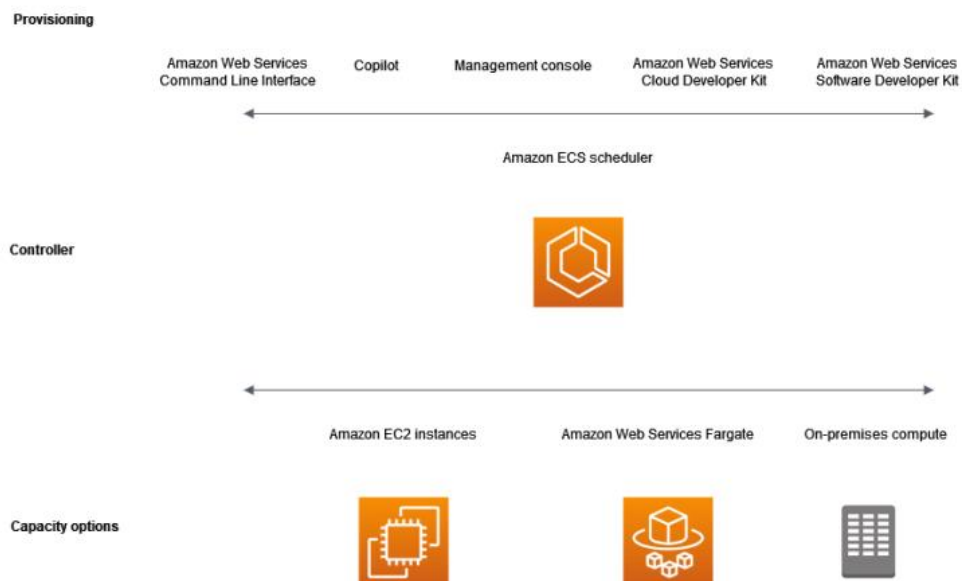
二、 Nomad

相对于 Kubernetes，简洁、易用、可扩展，不过特性少、社区生态不足，存在供应商约束(Vendor Lock-In)问题。



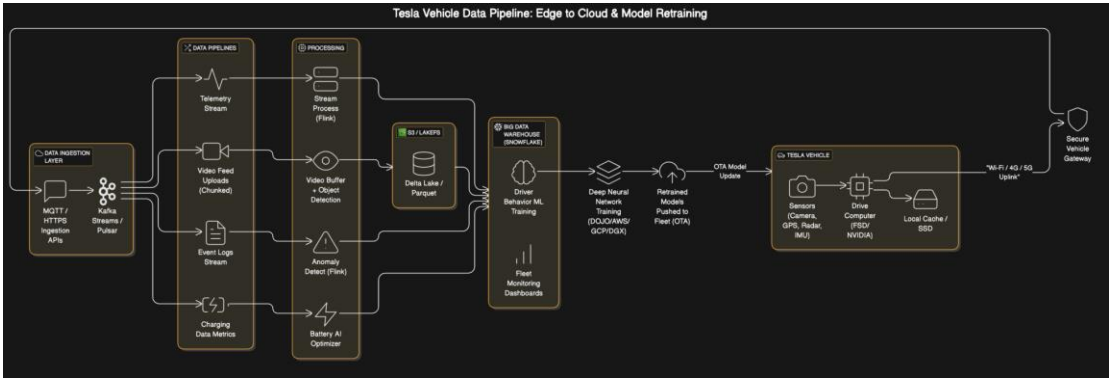
三、 Amazon ECS(Elastic Container Service)

类似于 Nomad 特点，可作了解。



云计算篇

Tesla 云计算框架。



第一章 Dify

Dify，基于大模型应用的整个 web 业务服务器框架。 vue + nginx + Next.js + flask + core services(AccountService, DatasetService, WorkflowService, ToolManager) + LLM + RAG + Data Layer。

数据，关系数据库 PostgreSQL, 内存数据库 Redis, 海量用向量数据库(Vector Database for embedding), 对象存储数据库(Object storage for files)。

第二章 RAG

大模型微调成本太高，RAG(Lewis 2020)算是一个替代或补充方案。

利用数据检索机制(retrieval mechanism)，通过实时访问外部数据库或文档来弥补大模型训练时未能获得当前最新信息的问题。

将最新的外部数据以块(chunks)形式转成嵌量存在向量数据集库(vector database)中。定期以最新数据更新数据库。

当用户访问 LLM 时，先基于用户问题信息在向量数据库中找到最相关嵌量，形成提示(prompt)；然后发给 LLM 进行回答。

实际可有**不同的架构**，比方 先查询再生成；先基于 agentic llm 对查询判断是不是需要查询外部数据，再生成；两个方案的结合。

先查询再生成架构，主要包括两个大模型,一个基于 OpenAI embedding model 等的大模型，用于对数据进行向量化,以便在用户查询时进行向量比对,找到与用户查询信息匹配的内容以增强用户查询信息；一个基于 GPT 等大模型，用于生成回答；向量数据库存储可用 weaviate cloud 或 langchain 的 vector store 等. langchain 可用作对文档分割、分块、加载，嵌量模型、向量数据存储、提示词模板、加载 llm 模型、方案制定、答案解释等作用。

获取器(Retriever): 基于查询量从资料库获取相关信息。比方，基于模型 BERT (Bidirectional Encoder Representations from Transformers)搜索及排列与查询量相关的文档。

生成器(Generator): 基于获取器获得的信息，生成上下文一致的回应(Response)。比方，基于 GPT-3 或 T5 等 Transformer 模型生成回答。

RAG 大概存在 4 个步骤,创立向量数据库, 获取与用户查询相关的信息, 增强用户查询提示语, 更新向量数据库.

创建企业向量数据库， 创建大语言模型训练数据集以外的数据。数据可通过第三方 API、数据库或文档仓库获得。

获取与用户查询相关的信息，用户查询量转换成向量表征(Vector Representation)，然后再向量数据库中进行匹配，以获得相关联文档。

增强大预言模型提示，RAG 模型通过在上下文中添加获取到的相关信息来增强用户输入，以便 LLM 生成准确的回答。

更新外部数据，通过自动化实时进程或周期批处理程序更新文档及相应的嵌量表征 (Embedding Representations) 以保持信息获取时最新。

第三章 分布训练

基本的分布式训练框架，Accelerate(HuggingFace) 、 DeepSpeed(MicroSoft) 、 FSDP(Meta Pytorch) 、 Megatron-LM(Nvidia)。

核心方法，基本在于对模型参数、梯度、更新方程中的优化参数的并行计算或非重计算。

一、 Accelerate

Accelerate 是对 torch.distributed 的封装。可与 DeepSpeed 及 FSDP 联合使用。

二、 DeepSpeed

DeepSpeed 与 FSDP 是相同理念的不同实现，都是对**1、优化器状态、2、梯度及 3、参数**进行跨 GPU 分片(sharding model parameters, gradients, and optimizer states across multiple GPUs.)。同时，都支持卸载训练信息到 CPU 或闪存设备(offload the optimizer state, gradients, and some model weights to CPU memory or NVMe storage.)。

DeepSpeed 的**零冗余优化器 ZeRO(Zero Redundancy Optimizer)**允许跨 GPU 对模型划分(Partition)，**不用每个 GPU 都复制模型参数**。(partition a model across multiple GPUs without having to replicate the model's parameters across each GPU.)。

****ZeRO 三个阶段****。

ZeRO Stage 1 将优化器状态及优化算法公式中的参数及因变量划分到不同 GPU(partitions optimizer states)

ZeRO Stage 2 划分梯度(also partitions gradients)

ZeRO Stage 3 划分模型参数(also partitions parameters)

****优化器状态举例, 12 倍于模型参数规模****。

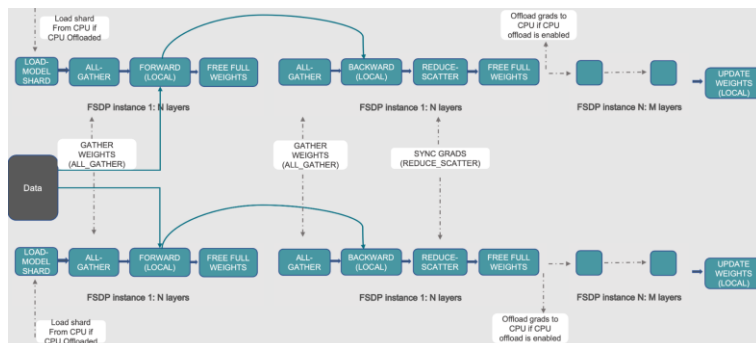
Adam 的 momentum + variance 占 2*4Byte

混合精度(mixed-precision, 比方模型参数 FP16, 优化器参数 FP32 以提高计算精度)的实现中，需要复制一份 fp32 的参数作为被 optimizer 更新的参数， 1*4Byte

DeepSpeed	FSDP	Megatron-LM	Accelerate
支持万亿参数	支持千亿参数		
配置繁多	配置简单		
需额外库以支持集成到Pytorch等框架	Pytorch原生支持，集成方便		
可调整ZeRO级别实现细粒度内存优化控制	可动态分片参数		
性能开销最小(performance overhead minimal)	非常低		
适合最大化内存效率及多个机器学习框架的场景	适合pytorch框架场景及学院派		

三、 FSDP

Pytorch FSDP 框架，通过全切片数据并行技术（Fully Sharded Data Parallel）来处理更多更大的模型，将模型参数、优化器状态、梯度等信息分片到运行着同一模型的不同任务进程，同时基于 Map-Reduce 思想，将批数据分片到这些任务进程分别进行梯度计算，然后对梯度进行加计算(Reduce)，同步更新不同任务进程的模型参数等信息。



第四章 数据存储

数据库早期主要是关系型，比方 MySQL, PostgreSQL, SQLServer, Oracle 等。大数据、云计算的发展促进了列类型、内存型、大文件型等数据库的开发及计算方式，比方，Hadoop, HBase, ElasticSearch, Parquet, Redis, Memcached 等。

一、 Parquet (拼木地板)存储

适用于数据湖、大数据处理框架(比方 Apache 的 hadoop、spark、hive 等)及云平台 AWS Athena, Snowflake, and BigQuery 等。

基本思路，每列看作一个对象， n 列对应 n 个对象，存在一个或多个文件中；通过称为 footer 的 metadata 文件说明这些对象在文件中的位置索引、对象中成员类型及编码等信息。

实质就是将传统的数据库以列对象的形式组织起来，然后通过类似文件描述(file descriptor)的元数据文件来说明这些对象的类型、位置等信息，以便快速查询；同时将数据元素数字化，以便压缩文件大小(比方，苹果的 index 为 0，篮球的 index 为 1 等)。

为了便于对稀疏数据或者跳过与查询目的不相关的行列元素，因此，改进思路，将行数据化分成 M 个行组(row group)，比方第 1、2 行是第 1 行组，第 3、4 行是第 2 行组等。

这样形成 $m * n$ 个块(chunks)对象，这样便于跳过与查询无关的块，实现更快的查询；同时，也容易进行数据扩展、合并等。每个对象中有 n 个亚列对象，每个亚列对象中包含对应的数据元素。当然，对于数据量小的情况，这样的划分及这些元数据信息便显得冗余。这就是文件组织形式称为 Parquet (拼木地板) 的原因。

4-byte magic number "PAR1"

<Column 1 Chunk 1>

<Column 2 Chunk 1>

...

<Column N Chunk 1>

<Column 1 Chunk 2>

<Column 2 Chunk 2>

...

<Column N Chunk 2>

...

<Column 1 Chunk M>

<Column 2 Chunk M>

...

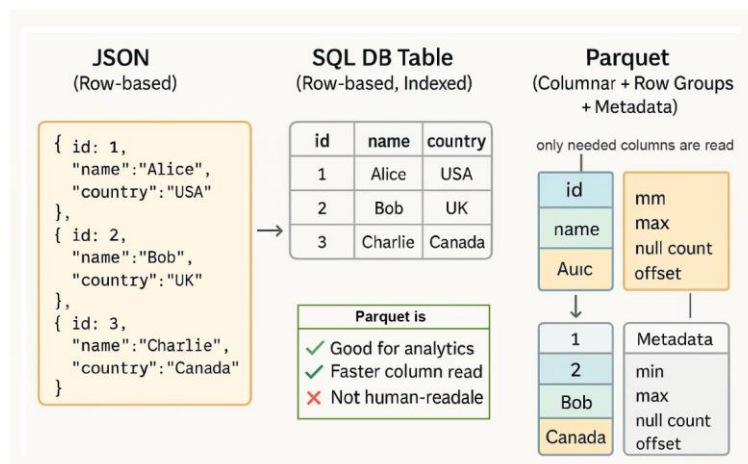
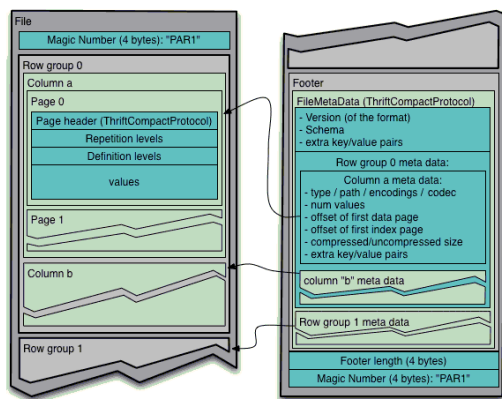
<Column N Chunk M>

File Metadata

4-byte length in bytes of file metadata (little endian)

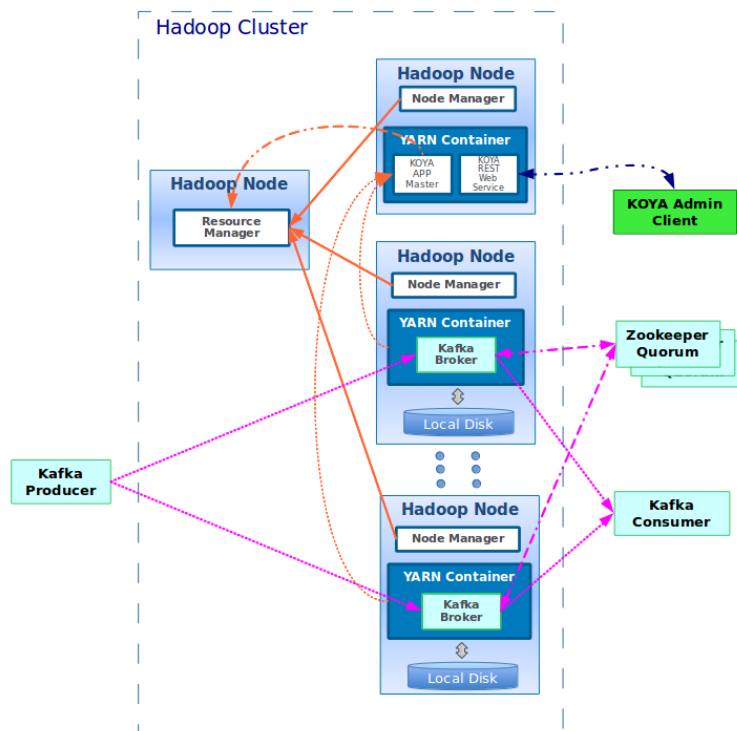
4-byte magic number "PAR1"

Nvidia GROOT 状态、动作数据等便基于 parquet 格式。



二、 Hadoop

Hadoop，基于主从分布存储框架的存算体系。文件系统，HDFS。



Hadoop 基于磁盘的分布式处理框架，可以存储海量数据，并基于集群运行分布式分析。适合离线数据计算。

综合篇

一、物理层面，世界的表象是连续的，通过采样、特征提取等可转换到离散数字世界以便计算。

时与空间可能是共消长的，基底的物理定理可能是相同的。

二、数学层面，世界存在大量的因果运动，很多时候就是找到此状态到彼状态的变换函数。

基于基函数定义变换函数，利用数据拟合、奖励反馈确定人类思维的近似函数是当前人工智能的方法。

多行业、多样化的科技方法的综合产生共性的数学抽象及逻辑演绎。

三、工程层面，通用机器人作为大算力智能机电产品，越发的基于芯片、执行器、大模型及云计算等的整合、融合及设计。

控制与数据分离是计算芯片、存储芯片、网络通讯、云计算中数据管理的基本方法。基于协议栈分组交换是独立通讯的基本方法。

人工智能在替代或改进着不同行业的不同工程环节。

半导体计算技术的发展，使得集群化技术变得可行，极大促进了行业发展。比方，多核 CPU、多线程 GPU、云计算载荷均衡、超算中心、电池密集阵列、可重用火箭矢量发动机群、AI 通讯卫星星座等。这类集群化与深度学习的矩阵体系、人类神经网络神经元体系、社会团体的人才密度等存在着单元活性、高计算性、网络性等共性。

四、商贸层面，科技创新产品的商业化是基于开放的多元权力、多元文化的社会网络，理性、理念、知识、人才、资本、勇气等自由聚合的产物。

五、社会层面，大量科技创新产品及普遍的商业化是提高人类文明的基本方法，同时也是促进社会阶层合理变迁、防止社会存量内乱的引擎。

六、理性哲学，知识是无尽的，理念是多样的，重要的是基于知识的创新发展。哲学是基座，发展是方向，自主、自由是基石，正义、保障是基本，权力普选是前提，商贸开放是要素，科学与工程相互驱动是方法。

参考文献

Pavan H Vora. etc. A Review Paper on CMOS, SOI and FinFET Technology. 2017.

MIT. Modern Processor Architecture. 2022.

Aslam. etc. Design of a Simple Processor.

Lars Ohlsson. Memory Circuits. 2018.

Prithviraj V. etc. Implementation of NAND Flash Memory Using Adiabatic Logic Circuit. 2022.

Rfinsights. Device Physics Explained: Key Concepts for IC Designers. 2025.

Next. Frequency Response Analysis of Amplifiers.

芯片篇

DWARAKAN RAMANATHAN. Unveiling Operational Amplifiers: Design, Applications, Secrets.

Hardwarebee. The Ultimate Guide to Operational Amplifiers.

Peter YK Cheung. Understanding op-amp specifications. 2025.

<https://github.com/MadhuriKadam9/Phase-Locked-Loop-Design-in-Sky130nm>.

<https://www.electronics-tutorials.ws/amplifier/transistor-biasing.html>.

MIT. Operational Amplifier Circuits. 2006.

Sanchez. Filter Design Implementations.

Oregonstate. SECTION 7: ACTIVE FILTERS.

Gabor C. Temes. CMOS Active Filters.

TI. Analysis of the Sallen-Key Architecture. 2002.

Mathforengineers. Low Pass Filter Transfer Function.

系统电路

University of Oslo. CMOS Image Sensor Design.

Sam Palermo. High-Speed Links Circuits and Systems. 2025.

Vishal Saxena. RADIO FREQUENCY IC DESIGN.

H. T. Kung. etc. Systolic Arrays For VLSI. 1978.

Quentin E. QUEN: THE APL WAVEFRONT ARRAY PROCESSOR.

R. Duncan. etc. A Survey Of Parallel Computer Architectures.

芯片设计

Azalia Mirhoseini. etc. A graph placement methodology for fast chip design. 2020.

David Silver. etc. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. 2017.

Stephen G. K. Force-Directed Drawing Algorithms.

印刷电路板

NextPCB. A Step-by-Step Guide to PCB Fabrication Process.

信号完整性

[Signal Integrity vs Power Integrity Engineering - Key Differences and Their Roles in Electronic Design / calledges.com](#). 2025.

Amphenol. Fundamentals of Signal Integrity. 2023.

MenloMicro. Reflection on Return Loss. 2021.

Cadence. How PDN Transient Current Dynamics Affect DC Power.

关节执行器

Tesla.Rocks. OPTIMUS HAND DESIGN. 2025.

操作系统

ANDREW S. TANENBAUM. etc. Modern Operating Systems. 2015.

模型篇

Alexandre Defossez. etc. High Fidelity Neural Audio Compression. 2022.

Seamless Communication. etc. SeamlessM4T: Massively Multilingual & Multimodal Machine Translation. 2023.

Meta. etc. UnitY: Two-pass Direct Speech-to-speech Translation with Discrete Units. 2023.

Kundan K. etc. MelGAN: Generative Adversarial Networks for Conditional Waveform Synthesis. 2019.

2021. Saeed Anwar. etc. A Systematic Evaluation: Fine-Grained CNN vs. Traditional CNN Classifiers.

2021. Robin Strudel. etc. Segmenter: Transformer for Semantic Segmentation.

2020. Nicolas Carion. etc. End-to-End Object Detection with Transformers.

2014. Andrej Karpathy. etc. Large-scale Video Classification with Convolutional Neural Networks.

2014. Karen Simonyan. etc. Two-Stream Convolutional Networks for Action Recognition in Videos.

2025. Ali K. Alshami. etc. SMART-VISION: SURVEY OF MODERN ACTION RECOGNITION TECHNIQUES IN VISION.

3D Gaussian Splatting for Real-Time Radiance Field Rendering.

NeRF-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields.

Instant Neural Graphics Primitives with a Multiresolution Hash Encoding.

Deep Local Shapes: Learning Local SDF Priors for Detailed 3D Reconstruction.

Probabilistic Volumetric Fusion for Dense Monocular SLAM.

Real-time Neural Radiance Caching for Path Tracing.

GR00T N1: An Open Foundation Model for Generalist Humanoid Robots.

FLOW MATCHING FOR GENERATIVE MODELING.

$\pi 0$: A Vision-Language-Action Flow Model for General Robot Control.

LATENT ACTION PRETRAINING FROM VIDEOS.

NSVQ: Noise Substitution in Vector Quantization for Machine Learning.

Neural Discrete Representation Learning.

FLARE, Robot Learning with Implicit World Modeling.

REPRESENTATION ALIGNMENT FOR GENERATION , TRAINING DIFFUSION TRANSFORMERS I S
EASIER THAN YOU THINK.

DREAMGEN, Unlocking Generalization in Robot Learning through Video World Models.

WAN, OPEN AND ADVANCED LARGE-SCALE VIDEO GENERATIVE MODELS.

DeXtreme: Transfer of Agile In-hand Manipulation from Simulation to Reality.

SOLVING RUBIK’S CUBE WITH A ROBOT HAND.

Mask R-CNN.

Beyond Sight: Finetuning Generalist Robot Policies with Heterogeneous Sensors via Language
Grounding.

A Touch, Vision, and Language Dataset for Multimodal Alignment.

Octo: An Open-Source Generalist Robot Policy.

DIGIT: A Novel Design for a Low-Cost Compact High-Resolution Tactile Sensor with Application to
In-Hand Manipulation.

Digitizing Touch with an Artificial Multimodal Fingertip.

Themesis. World Models: Five Competing Approaches – Overview. 2026.

Yasin Y. Immerse Yourself in Reinforcement Learning and Robotics with MuJoCo. 2025.

Emanuel T. MuJoCo: A physics engine for model-based control. 2012.

网络通讯

Starlink : A Solution to the Digital Connectivity Divide in Education in the Global South.

A Knowledge-Based and Model-Driven Requirements Engineering Approach to Conceptual Satellite Design.

Inter-Satellite Communication System based on Visible Light.

A Review of Communications Satellite by Focusing on ‘Bangabandhu Satellite-1’ , the First GEO Communications Satellite of Bangladesh.

云设施

Kumar Chandrakant. Evolution of Docker from Linux Containers. 2020.

Bill Trancois. Know About Linux containers. 2022.

Microsoft. Container platform tools on Windows. 2025.

ARM. Learn the architecture - AArch64 virtualization. 2022.

Christoffer Dall. etc. ARM Virtualization: Performance and Architectural Implications. 2016.

Zserge. KVM HOST IN A FEW LINES OF CODE. 2020.

Anton Burtsev. Operating Systems. 2013.

Chester L. Tesla's Dojo Microarchitecture. 2022.

Patrick K. Inside the 100K GPU xAI Colossus Cluster. 2024.

Jens Weber. Inside Colossus: Technical Deep Dive into World's Largest AI Training Infrastructure. 2024.

云计算

DevOps. Tesla Vehicle Data Pipeline: Architecture of a Smarter Car. 2025.